

**A WEB-BASED CAR RENTAL MANAGEMENT SYSTEM WITH DEMAND FORECASTING
AND CONTEXT-AWARE FLEET ALLOCATION FOR SMALL-SCALE MULTI-BRANCH
BUSINESSES**

A Capstone Project
Presented to the Faculty of the College of Computer and Information Sciences
Polytechnic University of the Philippines
Sta. Mesa, Manila

In Partial Fulfilment of the Requirements for the Degree
Bachelor of Science in Information Technology

by

Sebastian Dane D. Bien
Shane Richmond A. Binuya
Micaela M. Depaur
John Arron P. Mojico
Roi Aldrich S. Santos

May 2026

TABLE OF CONTENTS

	Page
Title Page	i
Table of Contents	ii
List of Tables	iv
List of Figures	vi
1 INTRODUCTION	
Project Context	1
Technical Background	4
Equipment/Hardware	5
Software	6
Peopleware/Manpower	7
Network Infrastructure/Architecture	9
Storage, Backup and Recovery Procedure	9
Security Procedures	10
Policies and Procedures	10
Problem Analysis	16
Fishbone Diagram	20
Problem and Solution Statement	23
Problem-Requirements Matrix	25
Purpose and Description	27
Specific Objectives	28
Scope and Limitations	30
Definition of Terms	34
2 REVIEW OF LITERATURE, STUDIES, AND SYSTEMS	
Car Rental Industry and Operational Context	42

Fleet Management and Operational Decision-Making	56
Context-Aware Transportation and Operational Intelligence	69
Related Systems and Research Gaps	79
Synthesis of the Study	83

3 METHODOLOGY

Requirements Analysis	90
Requirements – Features Matrix	93
Use Case Diagram	95
Use Case Report	121
Design Specifications	154
Activity Diagram	154
Database Schema	184
Data Dictionary	185
Development Methodology	222
Process Model	225
Development Tools	227
Test Methodology/Procedures	232
System Requirements	241
Quality Plan	244
Implementation Plan	246
Evaluation Plan	250
Ethical Considerations	265
Data Analysis (Procedure and Treatment)	266
Statistical Treatments	268

REFERENCES	274
-------------------	------------

LIST OF TABLES

Table No.	Description	Page
1	Existing Hardware of the Business	5
2	Existing Software and Digital Tools Used by the Business	6
3	System Users and Operational Roles of the Business	8
4	Problem – Requirements Matrix	25
5	Requirement – Features Matrix	93
6	Use Case Report - Register / Log in Customer Account	121
7	Use Case Report - Authenticate User / Login	122
8	Use Case Report - Manage Access Permissions	123
9	Use Case Report - Manage Customer Records	125
10	Use Case Report - Manage Vehicle and Branch Records	126
11	Use Case Report - Manage Maintenance Records	127
12	Use Case Report - View Vehicle Catalog	128
13	Use Case Report - View Rental Policy and Agreement	130
14	Use Case Report - Create and Manage Reservation	131
15	Use Case Report - View Booking Calendar	132
16	Use Case Report - Check Vehicle Availability	134
17	Use Case Report - Verify Requirements	135
18	Use Case Report - Validate Payment Record	137
19	Use Case Report - Forecast Demand using Weighted Moving Average	138
20	Use Case Report - Evaluate Context-Aware Factors	139
21	Use Case Report - Recommend Fleet Allocation	141
22	Use Case Report - Assign Vehicle	142
23	Use Case Report - Manage Rental Transaction	143
24	Use Case Report - Track Booking Status	145
25	Use Case Report - Receive Booking Notifications	146
26	Use Case Report - Monitor Fleet Status	147
27	Use Case Report - Process Vehicle Return and Settlement	148
28	Use Case Report - Record Penalties and Damages	150
29	Use Case Report - Manage Notifications and Alerts	151
30	Use Case Report - Generate Reports	152
31	Use Case Report - View Dashboard / Operational Summary	153
32	Data Dictionary – User Profiles	229
33	Data Dictionary – Branches	231

34	Data Dictionary – Vehicle Categories	231
35	Data Dictionary – Vehicles	232
36	Data Dictionary – Vehicle Images	234
37	Data Dictionary – Vehicle Operational State Events	235
38	Data Dictionary – Booking Requests	236
39	Data Dictionary – Booking Finder Context	239
40	Data Dictionary – Booking Creation Idempotency	240
41	Data Dictionary – Renter Requirement Sets	241
42	Data Dictionary – Renter Requirement Documents	242
43	Data Dictionary – Renter Requirement Reviews	243
44	Data Dictionary – Booking Payment Quotes	246
45	Data Dictionary – Payment Methods	248
46	Data Dictionary – Payments	249
47	Data Dictionary – Payment Proofs	251
48	Data Dictionary – Rate Cards	252
49	Data Dictionary – Booking Rate Quotes	253
50	Data Dictionary – Rental Transactions	255
51	Data Dictionary – Maintenance Records	257
52	Data Dictionary – Forecast Runs	259
53	Data Dictionary – Demand Forecasts	260
54	Data Dictionary – Forecast Inputs	262
55	Data Dictionary – Forecast Demand Coverage	263
56	Data Dictionary – Supply Evaluations	263
57	Data Dictionary – Supply Evaluation Vehicles	265
58	Data Dictionary – Allocation Recommendation Batches	266
59	Data Dictionary – Allocation Recommendations	267
60	Data Dictionary – Allocation Recommendation Candidates	269
61	Data Dictionary – Notifications	270
62	Data Dictionary – Notification Preferences	272
63	Data Dictionary – Operational Notification Conditions	272
64	Data Dictionary – Email Deliveries	274
65	Data Dictionary – Audit Events	275
66	Data Dictionary – Backup Runs	277
67	Data Dictionary – Backup Artifacts	278
68	Data Dictionary – Recovery Drills	279
69	Data Dictionary – Public Contact Settings	281
70	Data Dictionary – Public Contact Locations	282
71	Data Dictionary – Contact Inquiries	283
72	Development Tools	318

73	Desktop and Tablet System Requirements for Staff and Admin	334
74	Mobile System Requirements for Renter	336
75	Profile of Respondents	341
76	Evaluation Respondent Matrix	343
77	Likert Scale Interpretation Table	364
78	Statistical Techniques Used for Data Analysis	365

LIST OF FIGURES

Figure No.	Description	Page
1	Context Diagram - Level 0	12
2	Data Flow Diagram - Level 1	14
3	Fishbone Diagram	20
4	Use Case Diagram - Register / Log in Customer Account	95
5	Use Case Diagram - Authenticate User / Login	96
6	Use Case Diagram - Manage Access Permissions	97
7	Use Case Diagram - Manage Customer Records	98
8	Use Case Diagram - Manage Vehicle and Branch Records	99
9	Use Case Diagram - Manage Maintenance Records	100
10	Use Case Diagram - View Vehicle Catalog	101
11	Use Case Diagram - View Rental Policy and Agreement	102
12	Use Case Diagram - Create and Manage Reservation	103
13	Use Case Diagram - View Booking Calendar	104
14	Use Case Diagram - Check Vehicle Availability	105
15	Use Case Diagram - Verify Requirements	106
16	Use Case Diagram - Validate Payment Record	107
17	Use Case Diagram - Forecast Demand using Weighted Moving Average	108
18	Use Case Diagram - Evaluate Context-Aware Factors	109
19	Use Case Diagram - Recommend Fleet Allocation	110
20	Use Case Diagram - Assign Vehicle	111
21	Use Case Diagram - Manage Rental Transaction	112
22	Use Case Diagram - Track Booking Status	113
23	Use Case Diagram - Receive Booking Notifications	114
24	Use Case Diagram - Monitor Fleet Status	115
25	Use Case Diagram - Process Vehicle Return and Settlement	116
26	Use Case Diagram - Record Penalties and Damages	117
27	Use Case Diagram - Manage Notifications and Alerts	118
28	Use Case Diagram - Generate Reports	119
29	Use Case Diagram - View Dashboard / Operational Summary	120
30	Owner Sign In and Dashboard Access	155
31	Owner Dashboard Review and Report Export	156
32	Booking Approval and Rejection	157

33	Customer Verification Management	158
34	Fleet Search, Filter, and Service Action	159
35	Maintenance Scheduling and Service Recording	160
36	Payment Verification	161
37	Notification Triage	162
38	Reports and Analytics Review	163
39	Decision Support Actions	164
40	Branch Management	165
41	Users and Roles Management	166
42	Settings Management	167
43	Staff Sign In and Restricted Access	168
44	Staff Booking Queue Management	169
45	Staff Calendar Review	170
46	Customer Registration	171
47	Customer Sign In	172
48	Customer Vehicle Search	173
49	Browse Vehicles and Reserve	174
50	Booking Request	175
51	Requirement Submission	176
52	Payment Proof Submission	177
53	Dashboard Tracking	178
54	Contact Inquiry	179
55	Session and Role Authorization	180
56	Oauth Sign In or Sign Up	181
57	Logout	182
58	Error and Not Found Recovery	183
59	Database Schema Overview	219
60	Database Schema Detail 1: Demand Coverage, Public Contact Locations, and Notification Conditions	220
61	Database Schema Detail 2: Authentication, Profiles, Branches, Vehicle Categories, Vehicles, and Payment Methods	221
62	Database Schema Detail 3: Bookings, Requirement Sets, Payments, Rentals, and Maintenance	222
63	Database Schema Detail 4: Operational State Events, Forecast Runs, Forecasts, Finder Context, and Booking Idempotency (continued)	223
64	Database Schema Detail 5: Notification Preferences, Audit Events, Email Deliveries, Backup Runs, and Rate Cards (continued)	224
65	Database Schema Detail 6: Rate Cards, Vehicle Images, Backup Artifacts, Recovery Drills, and Public Contact Settings (continued)	225

66	Database Schema Detail 7: Public Contact Settings, Contact Inquiries, and Quotation Records (continued)	226
67	Database Schema Detail 8: Requirement Documents, Payment Proofs, Forecast Inputs, and Supply Evaluations	227
68	Database Schema Detail 9: Requirement Reviews, Supply Evaluation Vehicles, and Allocation Recommendations	228
69	Iterative Waterfall Model	317

Chapter 1

INTRODUCTION

Project Context

The car rental industry provides flexible transportation services for individuals, businesses, travelers, and customers who need temporary access to vehicles. As customer demand varies by location, vehicle type, rental schedule, travel purpose, and seasonal condition, car rental businesses must manage vehicle availability efficiently to maintain service quality and operational sustainability. This becomes more challenging for small-scale car rental businesses operating across multiple branches because vehicles are not always evenly distributed. One branch may experience high demand or vehicle shortages, while another branch may have idle or underutilized units. Recent studies on vehicle rental and fleet management confirm that demand fluctuations, supply-demand mismatch, and branch-level fleet imbalance remain persistent issues in transportation-related operations, affecting both service quality and resource utilization (Brar et al., 2022; Schroer et al., 2021; Suganya et al., 2026).

In small-scale car rental operations, these challenges are often worsened by manual or semi-manual management practices. Existing studies show that manual booking, paperwork-based transactions, fragmented records, and limited vehicle monitoring may result in delayed processing, inaccurate records, overlapping reservations, and poor visibility of vehicle availability (Linghuraj et al., 2024; Marafo et al., 2025). Although web-based car rental management systems have been developed to support reservations, customer registration, vehicle booking, payment processing,

and inventory tracking, many of these systems remain focused on transaction management rather than decision support. As a result, operational data such as rental history, vehicle usage, and branch-level demand may be recorded but not fully analyzed to support forecasting, vehicle allocation, and operational planning.

These conditions are evident in the current operations of the participating car rental business, the target client and operational context of this study. The business operates through its Manila and Antipolo operations and manages customer inquiries, booking coordination, rental schedules, payment validation, renter verification, vehicle monitoring, and branch coordination using several separate tools. Based on the client interview, customer inquiries and booking discussions are primarily handled through Facebook Messenger and other communication channels, while rental schedules are monitored through Google Calendar. Some operational records may also be handled through spreadsheets, mobile devices, and communication histories. Vehicle monitoring is supported through external tools such as Apple Find My, AirTags, and GPS-related applications; however, these tools remain separate from reservation records, payment validation, customer records, maintenance information, and reports. This setup shows that the business still depends heavily on manual coordination and owner judgment when verifying renters, confirming payments, checking schedules, assigning vehicles, and monitoring rental status.

The current workflow also shows the need for proper access control. Owners require access to payment information, rental approvals, vehicle allocation decisions, reports, maintenance monitoring, and operational records, while staff or administrators may only need limited access to booking schedules, calendar-related information, customer coordination, and operational updates. Customers or renters, on the other

hand, should only access their own booking requests, submitted requirements, payment proof, booking status, and rental-related notifications. This need for separated access levels supports the inclusion of secure authentication and role-based access control in the proposed system.

Beyond record centralization, multi-branch car rental operations also require decision-support features for vehicle allocation and operational planning. Without analytical support, vehicle assignment and branch allocation decisions are commonly based on manual judgment, which may lead to reactive planning. This means that businesses respond to shortages or idle units only after they occur instead of anticipating demand in advance. Recent studies emphasize that demand forecasting can help transportation and rental-related systems estimate future demand using historical usage or rental data. In addition, rule-based and threshold-based decision strategies can support allocation decisions by translating operational priorities into structured and interpretable decision rules. These findings support the proposed system's use of Weighted Moving Average forecasting and rule-based fleet allocation recommendations.

Selected operational and external context inputs may also support administrative vehicle-assignment and branch-allocation decisions. These inputs may include weather condition, road condition, route feasibility, route accessibility, travel distance, reference fuel efficiency, and estimated fuel consumption. In the proposed system, these factors will serve as supporting information for the Owner/Admin and will not independently determine vehicle assignment or branch allocation. The Customer-Side Vehicle Recommendation remains primarily based on

customer-provided rental requirements, vehicle availability, maintenance readiness, and encoded vehicle information.

Given these gaps, this study proposes A Web-Based Car Rental Management System with Demand Forecasting and Context-Aware Fleet Allocation for Small-Scale Multi-Branch Businesses. The proposed system aims to centralize customer records, vehicle records, reservations, rental transactions, payment validation records, fleet status, maintenance records, notifications, and report generation for the participating car rental business. It also extends beyond basic transaction processing by incorporating demand forecasting using Weighted Moving Average, rule-based fleet allocation recommendations, vehicle utilization analysis, idle vehicle detection, and selected operational context inputs such as fuel efficiency, estimated fuel consumption, travel distance, road accessibility, route feasibility, and weather condition, depending on data availability and development feasibility. Through these features, the system seeks to transform routine operational data into actionable insights that can help managers anticipate demand, reduce idle units, improve vehicle allocation, support branch coordination, and make more cost-aware and context-informed decisions while keeping final operational decisions under the owner or authorized administrator.

Technical Background

This section discusses the existing operational environment, hardware resources, software tools, network access, data storage practices, security procedures, and business workflow of the target client. It focuses on the current tools, devices, and procedures used in daily car rental operations, particularly in handling customer

inquiries, reservation coordination, payment checking, vehicle monitoring, recordkeeping, and operational communication.

Equipment/Hardware

Table 1

Existing Hardware of the Business

Equipment	Quantity
Laptop	2
iPad	2
iPhones	5
GPS Trackers / AirTags	20

Table 1 presents the existing hardware used by the target client in its daily operations. Based on the client interview, the business primarily uses iPhones, iPads, and laptops to handle customer inquiries, booking coordination, schedule monitoring, payment checking, and branch-related communication. These devices support the business's home-based and mobile workflow, especially when coordinating vehicle release, return, and customer updates.

The business also uses GPS-related tracking devices and Apple AirTags to monitor selected rental vehicles during active rentals. These tools help the owners maintain visibility over the vehicles; however, they operate separately from booking records, payment validation, customer information, vehicle records, maintenance

records, and reports. As a result, vehicle monitoring information is not automatically connected to the other operational records of the business.

Software

Table 2

Existing Software and Digital Tools Used by the Business

Software / Application	Purpose / Use
iOS	Operating system for iPhones and iPads
Windows OS	Operating system for laptop devices
Facebook / Facebook Page	Customer inquiries and business promotion
Facebook Messenger	Customer communication and reservation coordination
Viber	Internal communication and operational coordination
Safari / Google Chrome	Access to web-based platforms and online services
Apple Find My	Monitoring of AirTags installed in vehicles
AKSH GPS	Vehicle monitoring and tracking
Google Calendar	Booking schedule monitoring and coordination
Google Sheets	Storage and monitoring of selected customer, booking, sales, or operational records

Table 2 presents the existing software and digital tools used by the target client. The business currently uses iOS and Windows devices, Facebook Messenger for customer communication, Google Calendar for booking schedules, Google Sheets for selected operational records, and vehicle monitoring applications such as Apple Find My and AKSH GPS.

Apple Find My is used to monitor AirTags installed in selected vehicles, while AKSH GPS is an existing GPS-related monitoring tool used by the business to help track vehicle location or movement during active rentals. These applications are external monitoring tools and are not directly connected to the business's booking, payment, customer, maintenance, and reporting records.

Although these software tools support day-to-day operations, they are used separately. Customer inquiries, rental schedules, payment references, renter information, vehicle monitoring details, and selected operational records are handled through different applications. This setup may result in fragmented information, repeated manual coordination, and difficulty in retrieving complete and updated records when needed.

Peopleware/Manpower

The daily operations of the target client involve business owners, administrators or staff, and customers or renters. The business owners oversee major operational decisions, including booking approval, payment verification, vehicle assignment, maintenance monitoring, vehicle monitoring, and report review. Administrators or staff

assist with reservation coordination, booking schedule monitoring, customer communication, calendar management, and selected operational updates.

Customers or renters participate in the rental process by sending inquiries, submitting booking requests, providing rental requirements, submitting proof of payment, and receiving booking confirmations or rental-related updates. Since these users have different roles in the current business workflow, their access to operational information also differs. Owners handle sensitive payment, approval, and vehicle-related decisions, while staff members assist in daily coordination tasks. Customers are only involved in their own booking requests, submitted requirements, payment proof, and rental status updates.

Table 3

System Users and Operational Roles of the Business

Operational Role	Responsibilities
Admin / Business Owners	Has full access to operational records, payment information, rental approvals, vehicle allocation decisions, maintenance monitoring, and report viewing. Responsible for overseeing branch operations and making final operational decisions.
Operational Staff	Responsible for reservation coordination, booking schedule monitoring, customer communication, calendar management, and operational updates. Access to sensitive financial records and payment-related information is limited.
Renters	Individuals who inquire about vehicle availability, submit reservation requests, provide rental requirements, and receive booking confirmations and operational updates.

Network Infrastructure/Architecture

The target client does not operate through a formal enterprise network infrastructure. Its current operations rely on home internet, mobile data, phones, tablets, laptops, messaging applications, and browser-accessible tools. This setup supports flexible rental coordination across the Manila and Antipolo operations, but it may also cause scattered records and communication delays because information is distributed across multiple platforms.

Since the business relies on different devices and internet connections, operational updates are usually coordinated manually through messaging applications, calendar tools, mobile devices, and spreadsheets. This current setup allows the owners and staff to communicate and monitor operations remotely, but it does not provide a unified network or centralized access point for all rental records, vehicle information, payment details, schedules, and monitoring updates.

Storage, Backup, and Recovery Procedure

The target client currently stores operational information through a combination of mobile devices, messaging applications, Google Calendar, Google Sheets, and manual records. Booking details, customer information, payment references, vehicle schedules, and rental-related notes may be stored in separate locations. Because of this, retrieving complete, accurate, and updated records may require checking multiple applications or communication threads.

The current storage practice does not appear to follow a formal centralized backup and recovery procedure. Records may depend on the availability of individual devices, messaging histories, spreadsheets, and calendar entries. This setup may increase the risk of missing information, duplicated records, accidental deletion, or difficulty in recovering complete records when a device, file, or application becomes unavailable.

Security Procedures

The target client currently applies manual security procedures before confirming rental transactions. Customers are required to submit valid identification, driver's license information, and other rental requirements for verification. These submitted requirements are manually checked by the owners or authorized personnel before a booking is approved.

Payment confirmation is also manually verified by checking payment references, screenshots, and the business's bank or e-wallet account. If the submitted payment proof or reference number does not match the actual transaction record, the payment may be marked as invalid, rejected, or requiring resubmission. These practices help reduce rental and payment risks, but they also require careful handling of sensitive customer information, uploaded documents, and payment details.

Because customer documents, payment proof, and operational records contain sensitive information, access to these records is currently handled through the owners and authorized personnel. However, since many records are managed through

separate applications and manual checking, maintaining privacy, accuracy, and controlled access remains an important concern in the current workflow.

Policies and Procedures

The target client follows a controlled rental process to ensure that customers are properly verified, payments are confirmed, and vehicles are prepared before release. The usual workflow begins when a customer sends a rental inquiry through Facebook Messenger or other communication channels. The owner or staff then gathers the necessary rental details, such as the rental date, rental duration, preferred vehicle, number of passengers or seats needed, and branch-related vehicle availability.

After the initial inquiry, the customer is required to submit the necessary rental documents for verification. These may include valid identification, driver's license information, and other requirements needed by the business before approving the rental request. Once the submitted requirements are reviewed and approved, the customer may proceed with the required down payment.

Payment confirmation is performed manually by the owner or authorized personnel. The customer provides proof of payment, such as a screenshot and reference number, and the payment is checked against the business's bank or e-wallet account. If the payment details are valid, the reservation may proceed. If the proof of payment or reference number does not match the actual transaction record, the payment may be rejected or marked for resubmission.

Before vehicle turnover, the selected vehicle is prepared and checked by the owner or staff. This includes checking the vehicle's availability, physical condition, existing damages, fuel status, and other rental-related conditions. The rental agreement, possible penalties, existing damages, return schedule, and important rental reminders are also explained to the customer before the vehicle is released.

During the rental period, the business monitors selected vehicles using existing GPS-related tools and tracking devices. These monitoring tools are used separately from the rental records and are not directly connected to customer records, payment validation, maintenance records, or operational reports. Upon return, the vehicle is inspected for damages, fuel level, late return issues, remaining balance, penalties, and other settlement concerns before the rental transaction is closed.

These procedures show that the current rental workflow involves several manual and semi-manual activities, including customer inquiry handling, requirement verification, payment checking, vehicle preparation, vehicle monitoring, return inspection, and settlement processing. Since these activities are handled through separate tools and manual coordination, the business needs accurate recordkeeping, controlled approval procedures, and proper handling of customer, payment, and vehicle-related information.

Context Diagram

Figure 1. Context Diagram - Level 0

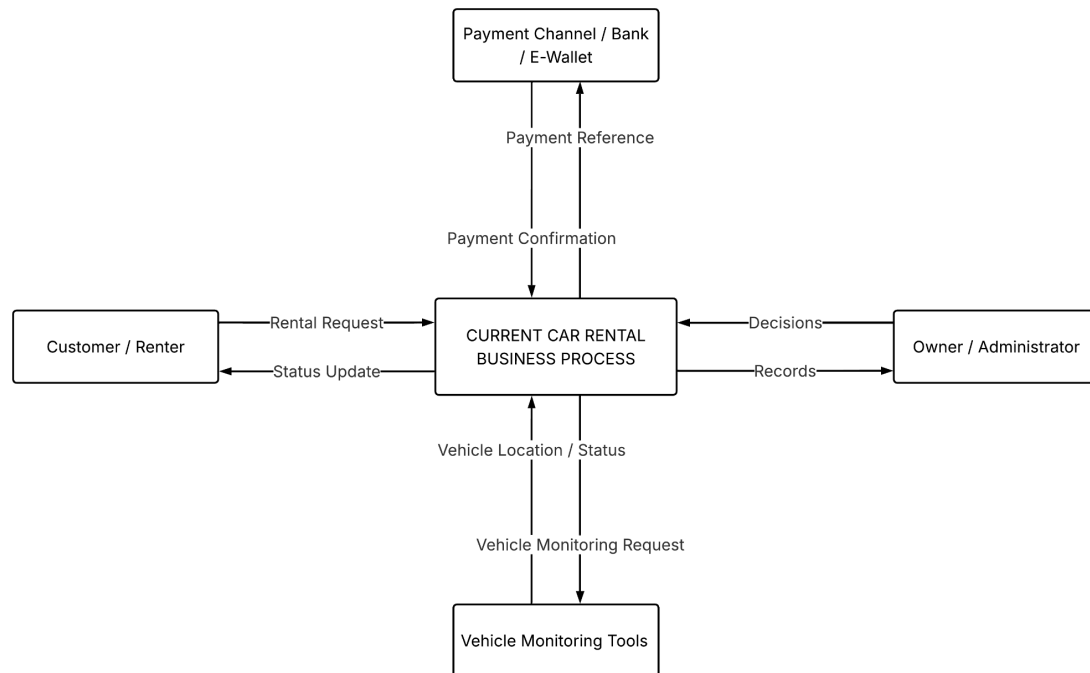


Figure 1 presents the Level 0 Context Diagram of the current car rental business process of the target rental business. The diagram illustrates the interaction between the current rental operations and its external entities, namely the Customer or Renter, Owner or Administrator, Payment Channel or Bank or E-Wallet, and Vehicle Monitoring Tools. It shows how rental information, payment-related information, vehicle monitoring information, operational decisions, and operational records flow between the business process and the involved external entities.

The Customer or Renter provides rental requests and related rental information to the business process, while the business provides status updates regarding reservations and rental transactions. These interactions support the coordination of

rental inquiries, booking requests, and customer-related updates throughout the rental process.

Payment Channel or Bank or E-Wallet services support payment verification activities by receiving payment references and providing payment confirmation information. These services assist the business in validating customer payments before reservation approval and vehicle release.

Vehicle Monitoring Tools provide vehicle location and status information during active rental periods and receive monitoring-related requests from the business process. These tools support the monitoring of rented vehicles using existing tracking technologies utilized by the business.

The Owner or Administrator provides operational decisions related to reservation approval, payment validation, vehicle assignment, and other rental management activities. In return, the business process provides operational records and information needed for monitoring and decision-making.

Overall, the context diagram presents the current manual and semi-manual workflow of the target rental business and shows how customers, management personnel, payment services, and vehicle monitoring tools interact with the existing rental operations.

Data Flow Diagram

Figure 2. Data Flow Diagram - Level 1

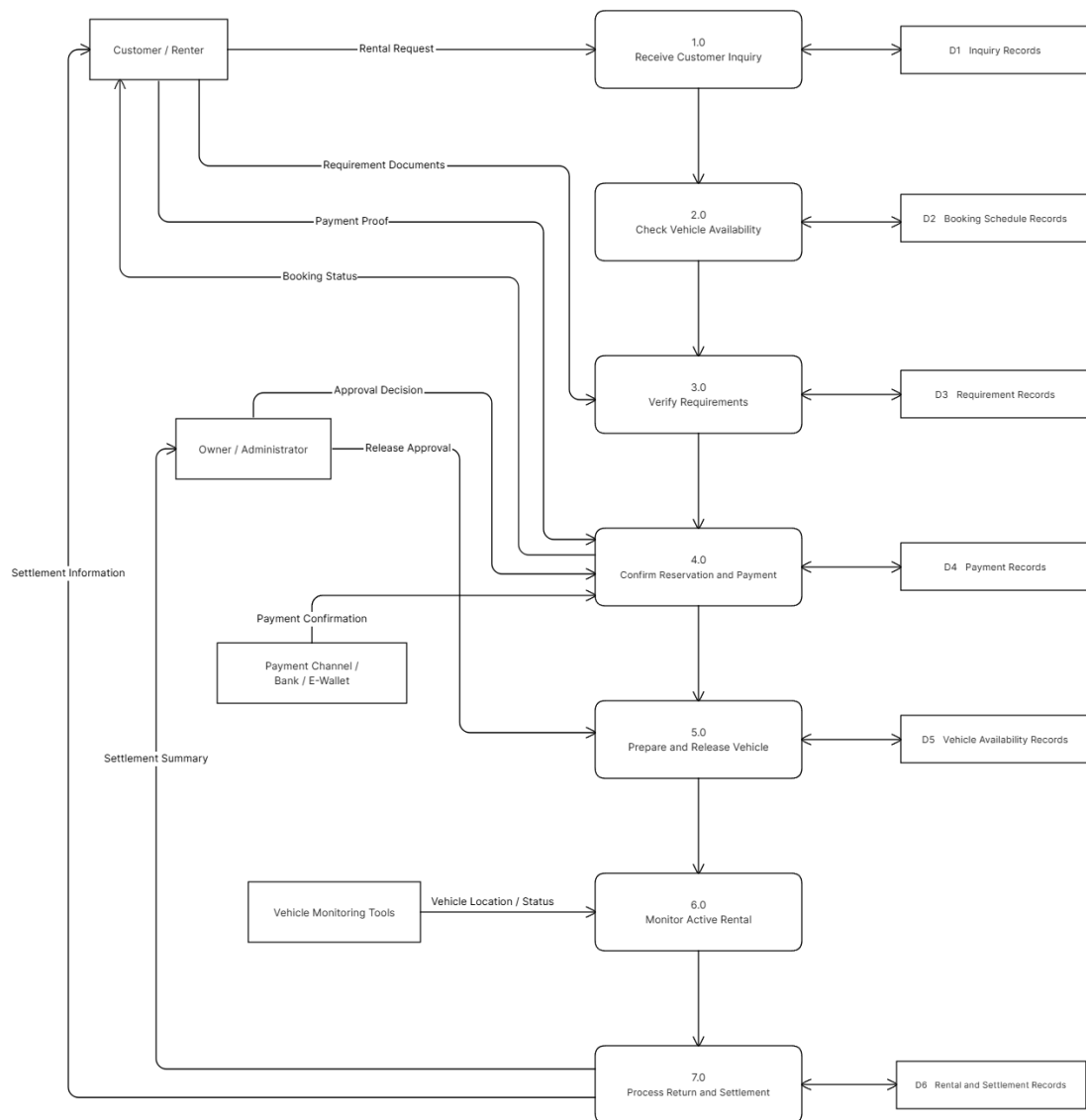


Figure 2 presents the Level 1 Data Flow Diagram of the current car rental business process of the target rental business. The diagram illustrates the major operational processes, external entities, and records involved in the current manual and semi-manual rental operations. It shows how rental-related information flows

between the Customer or Renter, Owner or Administrator, Payment Channel or Bank or E-Wallet, Vehicle Monitoring Tools, and the operational records maintained by the business.

Process 1.0: Receive Customer Inquiry handles the initial receipt and recording of customer rental requests and inquiry information. Customer-provided rental details are collected and stored to support succeeding rental activities.

Process 2.0: Check Vehicle Availability manages the checking of vehicle availability and reservation schedules based on the customer's requested rental details. Availability information is reviewed to determine whether a suitable vehicle can be assigned for the requested rental period.

Process 3.0: Verify Requirements handles the collection and verification of customer-submitted requirements and supporting documents. Verification activities are performed before reservation approval and payment processing can proceed.

Process 4.0: Confirm Reservation and Payment manages reservation confirmation and payment validation activities after customer requirements have been verified. Payment proof and payment confirmation information are reviewed before the reservation is finalized.

Process 5.0: Prepare and Release Vehicle handles the preparation of the assigned rental vehicle prior to turnover. This process includes vehicle readiness checking, availability confirmation, and release authorization before vehicle pickup or delivery.

Process 6.0: Monitor Active Rental manages rental monitoring activities during the active rental period. Vehicle location and status information obtained from existing monitoring tools are used to support operational monitoring and rental tracking activities.

Process 7.0: Process Return and Settlement handles vehicle return inspection and settlement activities after the rental period. This process includes the review of vehicle condition, fuel status, penalties, damages, remaining balances, and other settlement-related concerns before operational records are updated.

Overall, the Level 1 Data Flow Diagram illustrates the current operational workflow of the target rental business by showing how customer inquiries, availability checking, requirement verification, payment confirmation, vehicle preparation, rental monitoring, and settlement activities are manually and semi-manually managed within the business.

Problem Analysis

This section presents the analysis of the major operational problems encountered in small-scale multi-branch car rental operations, particularly in the context of the target client. It identifies the root causes of existing inefficiencies, explains their operational effects, and aligns the identified problems with the proposed system requirements. The analysis focuses on issues related to fragmented record management, manual operational processes, limited fleet visibility, weak utilization of historical operational data, maintenance monitoring concerns, and the absence of

structured decision-support mechanisms for vehicle recommendation, branch allocation, and operational planning.

Based on the client interview conducted, the target client currently relies on multiple separate tools and manual coordination practices to manage reservations, scheduling, vehicle monitoring, renter verification, payment validation, maintenance updates, and operational communication across its Manila and Antipolo operations. Customer inquiries and booking coordination are primarily handled through Facebook Messenger and other communication platforms, while reservation schedules are monitored through Google Calendar. Vehicle monitoring is conducted using existing external tools such as AKSH GPS, Apple Find My, installed tracking devices, and AirTags. AKSH GPS refers to a GPS-related vehicle monitoring tool currently used by the client to help track selected rental vehicles during active rentals. Apple Find My and AirTags are also used as external monitoring tools for selected vehicles. However, these tools operate separately from the client's booking records, customer information, payment validation, maintenance records, and operational reports. Therefore, the concern addressed in this study is not the absence of GPS tracking, but the lack of a centralized platform where rental records, vehicle status, maintenance information, and monitoring-related updates can be organized and accessed together.

The current setup shows that the business still depends heavily on manual and semi-manual coordination when handling inquiries, checking vehicle availability, verifying renter requirements, validating payments, updating schedules, and monitoring rental status. Although these tools support daily operations, the use of separate applications may lead to fragmented records, repeated manual checking, inconsistent updates, and difficulty in retrieving complete and updated information. Studies on

web-based car rental management systems show that manual or fragmented processes may cause delays in reservation handling, inaccurate recordkeeping, limited visibility of vehicle availability, and inefficient transaction monitoring (Harissa, 2025; Khan, 2020; K. S., 2025; Pawestri & Putro, 2024). These findings support the need for a centralized system that can organize customer records, booking details, payment validation records, vehicle information, maintenance data, and operational reports in one platform.

In multi-branch rental operations, vehicle demand may vary depending on branch location, rental schedule, vehicle type, and customer needs. One branch may experience high demand or vehicle shortage, while another branch may have idle or underutilized vehicles. If vehicle assignment and branch allocation are based mainly on manual judgment, the business may respond to shortages only after they occur instead of preparing for possible demand changes. Studies on rental vehicle and carsharing operations show that fleet imbalance, idle vehicles, relocation concerns, and uneven demand distribution can affect vehicle availability, utilization, and service quality (Favier et al., 2024; Fernández et al., 2020; Moein & Awasthi, 2020). These concerns support the need for decision-support features that can help the owner or administrator assess branch-level demand and vehicle availability before making allocation decisions.

Another concern involves the limited use of historical rental and operational data for planning. The demand forecasting component of the proposed system will use qualifying historical booking records as the basis for estimating short-term vehicle demand. Qualifying bookings will be aggregated according to calendar week, branch, and vehicle category to determine the recorded demand for each completed weekly period. These weekly demand observations will serve as inputs to the Weighted

Moving Average method, which will generate short-term demand forecasts for up to three succeeding weekly periods. Other rental and vehicle records, such as rental duration and vehicle usage, may support related operational analyses such as vehicle utilization monitoring but will not directly define the demand count used in the forecasting computation. Demand forecasting studies in carsharing, transportation, and rental-related operations show that historical demand and usage data can be used to estimate short-term demand and support planning decisions (Adhyapak, 2020; Alencar et al., 2021; Maghfur & Tranggono, 2025; Moein & Awasthi, 2020). In addition, studies using the Weighted Moving Average method show that it can be applied to short-term forecasting by assigning greater importance to more recent observations (Ariantini & Supartha, 2022; Yuliani et al., 2022). If sufficient historical observations are unavailable, the system will indicate insufficient historical data rather than generate a forecast from incomplete inputs. The resulting forecasts will be treated as short-term decision-support estimates rather than as artificial intelligence-based or guaranteed predictions of future demand.

Vehicle assignment and fleet allocation may also be affected by selected operational context inputs. These include fuel efficiency, estimated fuel consumption, travel distance, weather condition, road condition, route feasibility, and route accessibility. Fuel efficiency refers to how far a vehicle can travel per unit of fuel, while estimated fuel consumption refers to the projected fuel use for a trip based on travel distance and the vehicle's fuel efficiency. Route feasibility and route accessibility refer to whether a route is practical, reachable, and suitable based on available operational information. These factors will be presented as supporting information during administrative review of eligible vehicles and proposed branch allocations. They will not alter the Weighted Moving Average forecast or automatically determine candidate

ranking, vehicle assignment, or transfer approval; the final decision will remain with the owner or authorized administrator. Studies on context-aware vehicle services and fleet optimization show that operational factors such as travel conditions, fuel-related factors, route-related information, maintenance, and cost-related variables can support transportation and fleet decision-making when used as decision-support inputs (Agarwal & Shinde, 2022; Chavhan et al., 2022; Labib et al., 2026; Naim et al., 2025).

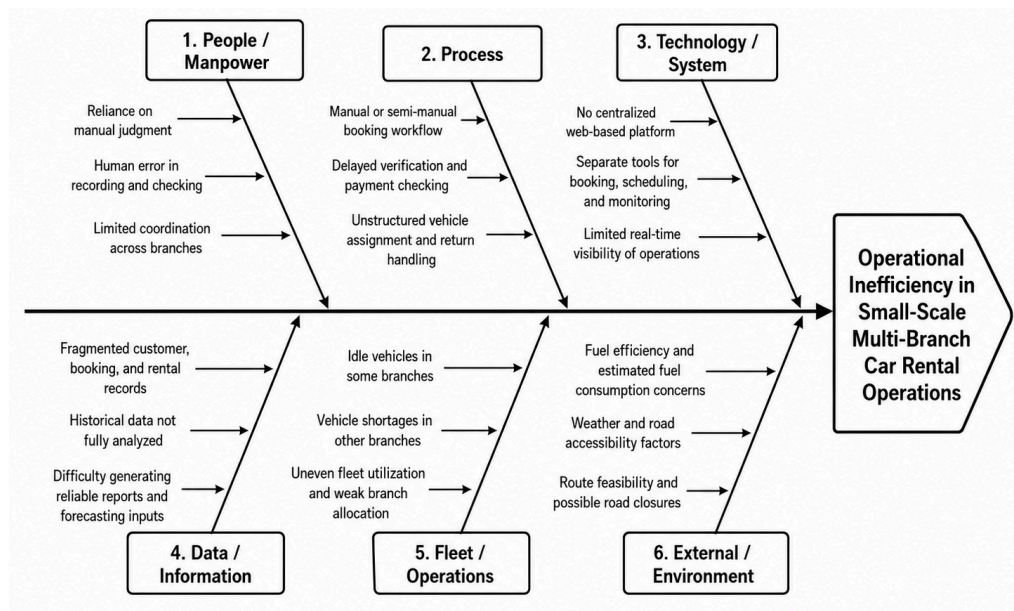
Vehicle maintenance is also an important part of fleet visibility and vehicle recommendation. In the current setup, maintenance updates may depend on manual checking and separate records, which can make it difficult to review a vehicle's maintenance status, mileage, preventive maintenance schedule, and service history. This is important because vehicles under maintenance, vehicles with high mileage, or vehicles with poor condition should not be recommended for rental assignment without proper review. Studies on fleet monitoring and maintenance information systems emphasize that maintenance records, vehicle condition, diagnostics, and service history are important for operational safety, efficiency, and fleet decision-making (Fauzi et al., 2025; Gurusinghe, 2021; Lopez et al., 2025). Therefore, maintenance-related data should be considered in the problem analysis because it affects vehicle availability, rental readiness, and recommendation suitability.

Overall, the identified operational issues demonstrate the need for a centralized web-based car rental management and decision-support system capable of organizing reservation management, customer records, payment validation, vehicle status, maintenance records, reports, demand forecasting, and rule-based fleet allocation in one platform. By centralizing operational records and providing forecasting and recommendation support, the system aims to assist the target client in improving

operational visibility, reducing fragmented records, supporting vehicle utilization monitoring, and making more informed vehicle assignment and branch allocation decisions. However, the system will not replace existing GPS-related tools or perform live GPS tracking. Existing monitoring tools such as AKSH GPS, Apple Find My, AirTags, and installed tracking devices will remain external to the system, while the proposed system will focus on centralizing rental, vehicle, maintenance, monitoring, and decision-support records.

Fishbone Diagram

Figure 3. Fishbone Diagram



The Fishbone Diagram shows that the main problem is operational inefficiency in small-scale multi-branch car rental operations. The root causes are grouped into six major categories: People/Manpower, Process, Technology/System, Data/Information, Fleet/Operations, and External/Environment. These categories show that the problem

is not caused by a single issue only, but by the combined effects of manual workflows, fragmented information, separate monitoring tools, limited forecasting, maintenance visibility concerns, and the absence of structured decision-support mechanisms.

Under People/Manpower, one of the main causes is the reliance of staff and owners on manual judgment when checking reservations, assigning vehicles, verifying payments, and coordinating branch operations. Since decisions are often based on immediate availability rather than organized historical data or structured analysis, vehicle assignment becomes reactive. Manual handling also increases the possibility of human error, especially in recording customer details, rental schedules, payment information, vehicle status, and maintenance updates.

Under Process, the problem is caused by manual or semi-manual booking, delayed reservation processing, and unstructured rental workflows. When reservations and vehicle assignments are handled through messaging applications, calls, walk-in inquiries, or informal communication, staff may experience difficulty in checking vehicle availability accurately. This may lead to delayed approvals, missed updates, repeated checking, and scheduling conflicts. These issues show the need for a more structured rental process that can organize booking, verification, payment checking, vehicle release, return, and settlement activities.

Under Technology/System, the absence of a centralized web-based system limits the ability of staff and managers to access updated information across branches. Although the business uses digital tools such as messaging applications, Google Calendar, spreadsheets, Apple Find My, AKSH GPS, and AirTags, these tools operate separately. The issue is not the lack of GPS-related monitoring tools, but the lack of integration between vehicle monitoring information and other operational records such

as booking records, payment validation, customer information, maintenance records, and reports.

Under Data/Information, fragmented and scattered records prevent the business from using operational data effectively. Rental transactions, vehicle usage, customer records, payment references, vehicle status, maintenance updates, and branch-level demand may be recorded but are not organized in one centralized platform. This limits the ability of managers to retrieve complete records, forecast demand, identify peak periods, monitor vehicle utilization, detect idle vehicles, and generate reliable reports.

Under Fleet/Operations, the main concerns are idle vehicles, vehicle shortages, uneven utilization, limited maintenance visibility, and branch-level allocation difficulties. One branch may experience a shortage of available vehicles while another may have idle or underutilized units. In addition, if maintenance records, mileage, preventive maintenance schedules, and vehicle condition are not properly monitored, vehicles that are not suitable for rental may still be considered during manual assignment. These issues show the need for fleet monitoring, maintenance record management, and rule-based recommendation support.

Under External/Environment, vehicle assignment may also be affected by selected operational context inputs such as fuel efficiency, estimated fuel consumption, travel distance, weather condition, road condition, route feasibility, and route accessibility. These factors may help the owner or administrator assess whether a vehicle is suitable for a rental request. However, these inputs should be treated as supporting criteria only and should depend on available and reliable data sources.

Among the identified causes, the most critical issues are fragmented records, manual booking coordination, separate vehicle monitoring tools, limited vehicle availability visibility, limited use of historical rental data, and weak maintenance record monitoring. These were prioritized because they directly affect booking accuracy, requirement verification, payment validation, vehicle assignment, branch coordination, vehicle readiness, and fleet visibility. Addressing these issues through a centralized system can help reduce scattered records, missed updates, inconsistent information, idle vehicles, and reactive operational decisions.

Problem and Solution Statement

Small-scale multi-branch car rental businesses experience operational inefficiencies due to manual or semi-manual booking processes, fragmented records, delayed rental transactions, limited centralized vehicle availability monitoring, and weak branch-level coordination. These issues increase the risk of human error, double-booking, inaccurate reports, missed updates, and slow transaction processing. In addition, vehicle allocation decisions are often based on manual judgment rather than historical data or structured analysis, making it difficult for managers to anticipate branch-level demand and distribute vehicles efficiently. As a result, some branches may experience vehicle shortages while others may have idle or underutilized units.

The target client already uses existing vehicle monitoring tools such as AKSH GPS, Apple Find My, AirTags, and other tracking devices during active rentals. However, these tools operate separately from reservation records, customer records, payment validation records, maintenance records, and operational reports. Therefore, the operational problem is not the absence of live GPS tracking, but the lack of a

centralized platform where booking records, vehicle status, maintenance information, monitoring-related updates, and reports can be organized and accessed together.

Furthermore, existing car rental management systems often focus mainly on reservations, customer records, rental transactions, and inventory tracking. While these features improve basic operations, they provide limited support for demand forecasting, fleet allocation, maintenance-based vehicle recommendation, and rule-based decision support using selected operational context inputs. This limitation becomes a concern because vehicle assignment may also be affected by passenger capacity, vehicle availability, vehicle condition, maintenance status, fuel efficiency, estimated fuel consumption, travel distance, weather condition, road condition, route feasibility, and route accessibility. Without a centralized and decision-support-driven system, small-scale multi-branch car rental businesses may continue to experience fragmented records, delayed operations, limited fleet visibility, and reactive decision-making.

To address these problems, this study proposes a web-based car rental management and decision-support system for small-scale multi-branch car rental businesses. The system will centralize customer records, vehicle records, booking requests, rental transactions, payment validation records, fleet monitoring records, maintenance records, and report generation in one browser-accessible platform. Through centralized data management, the system will help organize scattered records, support transaction processing, provide updated vehicle availability monitoring, and assist branch coordination.

The proposed system will also include short-term demand forecasting using the Weighted Moving Average method and two distinct rule-based decision-support functions. The Customer-Side Vehicle Recommendation will assist customers in

identifying suitable available vehicles according to their stated rental requirements before booking submission. The administrative Branch Allocation Recommendation will use forecasted vehicle requirements, projected available supply, maintenance readiness, vehicle availability, idle-vehicle information, and applicable operational context to identify possible redistribution of vehicles between branches. Applicable contextual information such as weather condition, road condition, route feasibility, route accessibility, travel distance, reference fuel efficiency, and estimated fuel consumption will serve as supporting information for administrative review rather than as automatic decision criteria.

The system will not replace the client's existing GPS-related tools or perform live GPS tracking. Existing tools such as AKSH GPS, Apple Find My, AirTags, and installed tracking devices will remain external to the system. Instead, the proposed system will focus on centralizing rental, vehicle, maintenance, monitoring, and decision-support records. Final vehicle assignment, branch allocation, payment verification, and operational decisions will remain under the owner or authorized administrator.

Table 4

Problem – Requirements Matrix

Problem	ID	Solution / Requirement
1. Reservation, customer, payment, and rental records are handled through separate tools and manual coordination, causing fragmented and inconsistent operational information.	R1	The system shall provide a centralized reservation and rental management module for recording booking requests, customer details, rental schedules, payment status, and transaction history.

	R2	The system shall provide online booking request management that allows administrators to review, approve, reject, and monitor reservations while checking vehicle availability.
2. Renter verification, requirement submission, and rental policy handling are manually performed, making the approval process time-consuming and difficult to track.	R3	The system shall provide a requirement submission and verification module where customers can upload required documents and administrators can review verification status.
	R4	The system shall provide access to rental policies, agreements, penalties, and booking conditions for customer reference and administrative validation.
3. Vehicle records, availability, branch assignment, condition, and maintenance information are difficult to monitor when managed through separate tools.	R5	The system shall provide a vehicle and fleet management module for managing vehicle records, availability status, branch assignment, condition details, and maintenance information.
	R6	The system shall provide operational fleet-status monitoring for tracking active rentals, idle vehicles, vehicle availability, branch assignment, vehicle condition, maintenance status, and other recorded fleet-status information. This functionality shall not include live GPS tracking or direct integration with external vehicle-monitoring hardware.

Continuation of **Table 4. Problem – Requirements Matrix**

Problem	ID	Solution / Requirement
4. Vehicle allocation decisions are mostly based on manual judgment,	R7	The system shall implement demand forecasting using a

which may result in idle vehicles in one location and shortages in another.		three-period Weighted Moving Average method with fixed weights of 0.50, 0.30, and 0.20 to estimate vehicle demand for up to three succeeding weekly periods per branch and vehicle category based on qualifying historical booking records. The first forecasted week shall use the three most recent completed actual-demand observations, while succeeding forecast periods shall be generated recursively using the most recent available actual and forecasted values.
	R8	The system shall provide rule-based branch allocation recommendations for the Owner/Admin by comparing forecasted vehicle requirements with projected available vehicle supply per branch and vehicle category and by considering vehicle availability, maintenance readiness, idle-vehicle information, and applicable operational context. The generated recommendations shall be advisory and shall not automatically transfer or reassign vehicles.
5. The business lacks analytical tools for evaluating vehicle utilization, idle units, branch demand, and operational performance.	R9	The system shall provide analytical reports for vehicle utilization, idle vehicle detection, branch demand, forecasted vehicle shortage and surplus, booking records, payment records, maintenance information, and fleet performance to support operational monitoring and administrative decision-making.
6. Customers have limited structured support for identifying available vehicles that match their rental needs, while administrators lack a systematic mechanism for considering applicable operational and contextual information when making vehicle assignment and	R10	The system shall provide a Customer-Side Find Your Ride feature that uses rule-based eligibility filtering and deterministic ranking to identify suitable available vehicles based on rental dates and times, passenger capacity,

branch allocation decisions.		large-bag capacity, total rental budget, vehicle availability, and maintenance readiness. The Finder shall use vehicle capacities maintained by the Owner/Administrator and present explainable matches, including why a vehicle fits the customer's group and luggage requirements. The generated matches shall assist customers in vehicle selection before submitting a booking request and shall not constitute booking approval or guaranteed vehicle assignment.
	R11	The system shall provide context-aware decision support for the Owner/Admin using applicable operational and external context information, including weather condition, road condition, route feasibility, route accessibility, travel distance, reference fuel efficiency, and estimated fuel consumption. These contextual inputs shall serve as supporting information for administrative vehicle-assignment and branch-allocation decisions and shall not independently determine the final operational action.

Continuation of **Table 4. Problem – Requirements Matrix**

Problem	ID	Solution / Requirement
7. Operational updates such as booking status, payment reminders, verification results, return schedules, and maintenance alerts may be missed when communication is handled manually.	R12	The system shall provide notification and alert features for booking updates, payment reminders, verification results, vehicle return schedules, maintenance reminders, and low vehicle availability.

8. Sensitive customer, payment, and operational records require secure handling and reliable storage.	R13	The system shall implement secure authentication, role-based access control, and restricted access to customer records, payment information, uploaded requirements, and operational reports.
	R14	The system shall provide centralized database storage with backup and recovery procedures to support data integrity, record availability, and operational continuity.

Purpose and Description

This study aims to develop a web-based car rental management and decision-support system for the target client, a small-scale car rental business operating through its Manila and Antipolo operations. The proposed system is intended to centralize the management of customer records, vehicle records, booking requests, rental transactions, payment validation records, maintenance records, fleet status, notifications, and operational reports in one browser-accessible platform.

The system will support both customer-side and administrative-side functions. Customers or renters will be able to view available vehicles, submit booking requests, upload rental requirements, provide proof of payment, view rental policies, track booking status, and receive notifications. Owners and authorized administrators will be able to manage reservations, verify customer requirements, manually validate payments, assign vehicles, monitor fleet status, record maintenance information, record physical vehicle returns and operational inspection outcomes (financial

settlement remains outside the current workflow), generate reports, and manage user access based on assigned roles.

Beyond basic rental transaction management, the system will include a demand forecasting feature using the Weighted Moving Average method to estimate short-term vehicle demand per branch and vehicle category for up to three succeeding weekly periods. Qualifying historical booking records will be aggregated by calendar week, and the three most recently completed weekly observations will be assigned fixed descending weights of 0.50, 0.30, and 0.20. The first forecasted week will be calculated using the three most recent actual weekly demand observations, while the second and third forecasted weeks will be generated recursively using the most recent available actual and previously forecasted values. The three-period window defines the inputs required for each calculation; it does not represent the total evidence needed to establish predictive accuracy or seasonal behavior. The resulting forecasts are short-term planning estimates. They do not explain the causes of demand changes and do not constitute an artificial intelligence or autonomous prediction model.

The system will provide two separate rule-based recommendation functions. The Customer-Side Vehicle Recommendation will assist customers before booking by filtering and ranking eligible vehicles according to passenger capacity, total rental budget, rental duration, vehicle preference, availability, and maintenance readiness. Separately, the administrative Branch Allocation Recommendation will compare forecasted vehicle requirements with projected available supply to identify branch shortages and surpluses and determine whether an eligible vehicle transfer may be recommended. Applicable contextual information, including weather condition, road condition, route feasibility, route accessibility, travel distance, reference fuel efficiency,

and estimated fuel consumption, will serve as supporting information for administrative review. All recommendation outputs remain advisory, and final operational decisions remain under the Owner/Admin.

The maintenance-related functions of the system will include the recording of vehicle maintenance status, mileage, preventive maintenance schedule, service history, and related maintenance updates. These details may help the owner or authorized administrator determine whether a vehicle is available, suitable for assignment, or should be excluded from recommendation due to maintenance concerns.

The system will not replace the client's existing GPS-related monitoring tools or provide live GPS tracking. Existing tools such as AKSH GPS, Apple Find My, AirTags, and other tracking devices will remain external to the system. Instead, the proposed system will focus on centralizing rental records, vehicle status, maintenance records, monitoring-related updates, and decision-support records.

Overall, the system is intended to assist the owner or authorized administrator in managing rental operations, organizing records, monitoring vehicle status, reviewing maintenance information, generating reports, and making more informed vehicle assignment and branch allocation decisions. Final approval, payment validation, vehicle assignment, branch allocation, and other business decisions will remain under the control of the owner or authorized administrator.

Specific Objectives

To address the identified operational problems of the participating car rental business, particularly fragmented rental records, manual booking coordination, limited fleet visibility, separated vehicle monitoring tools, and the need for structured decision-support features, this study establishes the following specific objectives. These objectives serve as the basis for the development and evaluation of the proposed web-based car rental management and decision-support system.

The specific objectives of this study are:

1. To assess the existing car rental management process of the target client in terms of reservation handling, rental transactions, vehicle availability monitoring, fleet utilization, payment verification, and vehicle maintenance.
2. To develop a centralized web-based car rental management system for managing customer records, vehicle records, booking requests, rental transactions, payment validation records, maintenance records, fleet status, notifications, and operational reports.
3. To implement a Customer-Side Vehicle Recommendation feature that filters and ranks vehicles using passenger and large-luggage capacity, a stated total rental budget compared with a provisional daily-rate reference estimate, the requested rental period, vehicle preference, availability, and maintenance readiness. An optional destination is retained for administrative review and does not affect customer recommendation ranking.
4. To implement a demand forecasting feature using the Weighted Moving Average method to estimate short-term weekly booking demand per branch and vehicle category from confirmed booking starts. These booking-count forecasts are planning inputs and do not directly measure concurrent vehicle occupancy.

5. To implement an administrative rule-based branch allocation feature that compares forecast-based planning requirements with projected eligible supply, identifies branch shortages and transferable surpluses, and recommends eligible candidate vehicles for review without automatically executing transfers.
6. To incorporate weather, incident-based road information, route availability, travel distance, reference fuel efficiency, and estimated fuel consumption into administrative review of vehicle assignments and proposed branch allocations, with source, freshness, coverage, and missing-data limitations clearly identified.
7. To support vehicle maintenance monitoring by recording maintenance status, preventive maintenance schedule, mileage or odometer reading, vehicle condition, maintenance history, repair history, maintenance cost, remarks, and related updates that may affect vehicle availability and recommendation.
8. To provide administrative reports and dashboard summaries for booking records, payment records, vehicle utilization, idle vehicle detection, maintenance records, branch demand, and branch allocation recommendations.
9. To evaluate the developed system based on selected ISO/IEC 25010 software quality characteristics, specifically functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety.

Scope and Limitations

This study focuses on the design and development of a web-based car rental management and decision-support system for a small-scale, multi-branch car rental

business. The system is developed for the target car rental business, specifically covering its Manila and Antipolo operations, and is intended to support the centralized management of customer records, vehicle records, booking requests, rental transactions, payment verification, vehicle availability, maintenance records, branch assignments, and operational reporting.

The proposed system will provide a centralized web-based platform accessible through standard desktop and mobile browsers. It will support customer-side functions such as account registration, vehicle catalog viewing, Customer-Side Vehicle Recommendation, booking request submission, requirement upload, proof-of-payment submission, booking status tracking, rental policy viewing, and customer notifications. These features are intended to provide customers with a more organized way of viewing vehicles, submitting rental requests, and monitoring the status of their bookings.

The Customer-Side Vehicle Recommendation is a customer-side vehicle selection support feature. It allows customers to provide passenger and large-bag counts, a total rental budget for comparison with the daily-rate reference estimate, rental dates and times, and an optional vehicle-category preference. The optional destination is retained for administrative review and does not change customer-side eligibility or ranking. The system applies predefined rule-based recommendation logic to match these customer inputs with suitable available vehicles based on passenger capacity, vehicle availability, rental constraints, vehicle preference, and other defined customer-matching rules. The recommended vehicles are then displayed to the customer for consideration and selection before submitting a booking request.

The Customer-Side Vehicle Recommendation does not automatically approve reservations, guarantee vehicle assignment, or replace administrative review. It is intended only to assist customers in identifying vehicles that may be suitable for their stated rental needs. Final booking approval, vehicle assignment, payment validation, and other rental decisions will remain under the owner or authorized administrator.

For the administrative side, the system will include modules for booking management, customer management, vehicle and fleet management, payment verification, maintenance monitoring, calendar and scheduling, report generation, notification management, user role management, demand forecasting, operational fleet-status monitoring, and branch allocation recommendation. These modules are intended to assist owners and authorized staff in managing daily operations, monitoring vehicle availability and status, verifying customer submissions, maintaining vehicle records, and organizing branch-related information.

The administrative decision-support features will include demand forecasting using the Weighted Moving Average method, vehicle utilization analysis, idle vehicle detection, branch demand and supply analysis, maintenance-based vehicle readiness review, and rule-based branch allocation recommendation. For forecasting, qualifying historical booking records will be aggregated by calendar week, branch, and vehicle category. The three most recently completed weekly demand observations will be assigned fixed descending weights of 0.50, 0.30, and 0.20 to generate forecasts for up to three succeeding weekly periods. The first forecasted week will be calculated using actual completed weekly demand, while the second and third forecasted weeks will be generated recursively using the most recent available actual and previously forecasted values. At least three complete historical weekly observations will be required before

an initial forecast is generated. This minimum permits the WMA calculation only; it is not treated as sufficient evidence of stable forecast accuracy or seasonal behavior. In the current study, multiple eligible target weeks from the labeled synthetic time series will be used only to verify forecast generation and error computation. Real-world forecast performance will require a separate future evaluation using sufficient verified client or post-implementation observations. For each forecasted week, the Branch Allocation Recommendation shall compare the rounded-up booking-count forecast, used as a planning proxy, with projected eligible supply. Proposed transfer quantities shall not exceed the destination shortage, the source surplus after reserving its own requirement, or the number of eligible candidates. Actual booking and maintenance schedules shall be checked before an operational decision because weekly booking counts do not measure concurrent occupancy. All resulting outputs remain advisory.

The decision-support component is strictly limited to rule-based logic and statistical forecasting. The system does not employ artificial intelligence, machine learning algorithms, autonomous optimization models, dynamic pricing algorithms, automated fraud detection, computer vision, automated license plate recognition, or automated decision execution. System-generated recommendations and analytical outputs are intended only to support human decision-making. Final decisions regarding booking approval, payment validation, vehicle assignment, vehicle transfer, branch allocation, and other operational actions will remain under the control of the owner or authorized administrator.

The context-aware decision-support component is limited to selected operational and external context inputs that may support administrative vehicle-assignment and branch-allocation decisions. These inputs may include weather

condition, road condition, route feasibility, route accessibility, travel distance, estimated or reference fuel efficiency expressed in kilometers per liter (km/L), and estimated fuel consumption. For weather information, the system will primarily use the Open-Meteo Forecast API, with the OpenWeather One Call API 3.0 serving as the fallback provider. Destination geocoding uses Geoapify as the primary provider and LocationIQ as the fallback, subject to Philippine location-quality checks. Route distance and travel-time estimates use TomTom Orbis Routing with HERE Routing as fallback. Incident information uses TomTom Traffic with HERE Traffic as fallback. These outputs support administrative review and do not establish vehicle-specific travel permission.

If a designated primary provider is unavailable or fails to return sufficiently usable information, the system may attempt to obtain comparable information from the designated fallback provider. If neither provider supplies reliable information, authorized manual contextual information may be recorded where available; otherwise, the affected contextual factor will be classified as Unavailable or Unknown rather than being assigned an assumed favorable value. External contextual information remains separate from the primary customer-need criteria used by the Customer-Side Vehicle Recommendation and serves primarily as supporting information for Owner/Admin review.

External weather and incident information will only be evaluated for periods where sufficiently current provider data are available. Because the system's three-week demand-forecast horizon may extend beyond the supported forecast period of an external weather provider, weather information is not guaranteed to be available for every forecasted weekly period. The demand forecast, projected-supply analysis,

shortage and surplus calculations, and other internal operational analyses shall continue independently of external contextual-data availability.

Fuel-efficiency information used by the system will be treated as estimated or reference information rather than continuously measured actual fuel performance. Reference fuel-efficiency values may be based on manufacturer specifications or owner-provided vehicle information and will be expressed in kilometers per liter (km/L). Estimated fuel consumption may be derived from the available travel-distance and reference fuel-efficiency information when applicable. Actual fuel consumption may vary depending on driving behavior, passenger or cargo load, traffic, terrain, road and weather conditions, and vehicle maintenance condition. Therefore, fuel-related information will serve only as supporting information for operational decision-making and will not guarantee actual trip fuel consumption or automatically determine vehicle assignment.

The system does not include direct GPS tracking integration or live vehicle location monitoring. External tools currently used by the target car rental business, such as AKSH GPS, Apple Find My, AirTags, and other tracking devices, will remain independent systems and will not be connected directly to the proposed platform. The proposed system will not retrieve live coordinates, perform geofencing, display real-time map tracking, connect directly to GPS hardware, or replace the client's existing vehicle-monitoring tools. Instead, operational fleet-status monitoring within the proposed system will focus on vehicle records, availability, branch assignment, rental status, vehicle condition, return status, maintenance records, and monitoring-related notes or updates recorded by authorized users.

Payment-related features are limited to payment-status tracking, proof-of-payment upload, transaction-reference recording, and administrative verification. The system does not include built-in automated payment gateway integration within the current scope. All payment validation will be performed manually by an authorized owner or administrator using the uploaded proof of payment and transaction reference, which will be checked against the business's external bank or e-wallet records. Automated payment gateway integration is outside the scope of the present study and may be considered as a future system enhancement.

The system is designed specifically for small-scale, multi-branch car rental operations and is not intended for enterprise-level fleet management, logistics optimization systems, or real-time dispatch platforms. It does not include advanced route optimization engines, automatic dispatching, full telematics integration, or large-scale fleet relocation optimization.

Forecasting is limited to the Weighted Moving Average method because of its simplicity, interpretability, and suitability for limited historical operational data. The three-week input window and rolling three-week output horizon define the calculation procedure; they do not by themselves validate predictive accuracy or establish seasonal patterns. Forecast uncertainty is expected to increase for succeeding periods because the second- and third-week projections partially depend on previously forecasted values. In the current study, Mean Absolute Percentage Error (MAPE) will be computed from original one-week-ahead forecasts and corresponding realized values in the labeled synthetic time series solely to verify forecast-error computation. Periods with zero realized synthetic demand will be reported separately and excluded from MAPE because percentage error is undefined when the denominator is zero. The

evaluation will report the synthetic coverage period and number of valid observations. Lower MAPE values indicate smaller errors within the test series, but they will not be interpreted as empirical predictive accuracy or assigned qualitative accuracy classifications. The second- and third-week forecasts will serve primarily as longer-horizon planning projections.

Recommendation outputs are rule-based and depend on the accuracy and completeness of the information used by the corresponding recommendation process. Customer-side recommendations depend primarily on customer-provided rental requirements, vehicle availability, and encoded vehicle information. Administrative allocation recommendations depend on forecasted demand, branch and vehicle records, vehicle availability, condition and maintenance information, and applicable contextual inputs. Incomplete, inaccurate, outdated, or unavailable data may affect the resulting recommendations.

The system requires stable internet connectivity because it is a web-based application. Performance may vary depending on device capability, browser compatibility, internet speed, server availability, and the responsiveness of external data sources used for applicable contextual information. Although the system supports both desktop and mobile browsers, the user experience may vary across devices and screen sizes.

At the current stage of the study, no real historical client records are available. All booking, rental, vehicle, maintenance, utilization, forecasting, and allocation records used for development, testing, demonstration, and evaluation are synthetic dummy data created to represent plausible operating scenarios. The synthetic dataset may be used to verify WMA calculations, forecast finalization, utilization analysis, maintenance

conflicts, shortage and surplus detection, Customer-Side Vehicle Recommendation, and Branch Allocation Recommendation, including controlled edge cases. Within the forecasting workflow, the term actual demand refers to the realized qualifying booking count for a completed week in the synthetic time series; it does not mean actual client demand. Synthetic MAPE results will therefore be labeled as functional-test results and will not be presented as evidence of real-world predictive accuracy, seasonal behavior, or long-term operational effectiveness. Each reported result will identify the dataset as synthetic, state its coverage period and valid observation count, and disclose relevant limitations. The study's present evaluation will primarily assess system functionality, rule conformance, software quality, usability, and feasibility.

Overall, the study aims to develop a centralized, practical, and decision-support-enabled car rental management system intended to support operational organization, booking coordination, fleet visibility, maintenance monitoring, and administrative decision-making for the target car rental business while ensuring that critical operational decisions remain under human control.

Definition of Terms

The following terms are operationally defined to provide a clear understanding of the concepts used in this study.

AKSH GPS. This refers to an existing vehicle tracking system and its companion mobile application used by the target car rental business for vehicle monitoring and security purposes. The tool supports vehicle location and other monitoring-related

functions. In this study, AKSH GPS serves only as an external monitoring tool and is not directly integrated with the proposed system for real-time GPS tracking or automated vehicle location updates.

Application Programming Interface (API). This refers to a software interface that allows the proposed system to communicate with external services or data sources. In this study, APIs may be used to obtain selected contextual information for administrative decision support. Open-Meteo serves as the primary weather-data provider, with OpenWeather One Call API 3.0 as the fallback weather provider. Geoapify provides destination geocoding with LocationIQ as fallback. TomTom routing and traffic services provide route distance, travel-time estimates, and incident information, with corresponding HERE services as fallback. API-derived information remains supporting information and does not automatically determine vehicle assignment or branch allocation.

Booking Request. This refers to a rental request submitted by a customer through the proposed system. It includes essential rental details such as the preferred vehicle, rental date, rental duration, branch or service location, customer information, and other booking-related requirements needed for administrative review.

Branch Allocation Recommendation. This refers to the system-generated advisory output that identifies a possible redistribution of eligible vehicles between branches based on the Rule-Based Branch Allocation process. In this study, the recommendation may identify the source branch, destination branch, vehicle category, forecasted vehicle requirement, projected available supply, shortage, surplus, recommended transfer quantity, eligible candidate vehicles, and applicable operational or contextual

reasons. The recommendation does not automatically modify vehicle branch assignments, and final transfer decisions remain under the Owner/Admin.

Car Rental Management System. This refers to a web-based information system designed to support major car rental operations, including customer records, vehicle records, booking requests, rental transactions, payment verification, maintenance monitoring, operational fleet-status monitoring, report generation, demand forecasting, customer-side vehicle recommendation, and branch allocation recommendation.

Centralized Database. This refers to a structured data storage component that stores and organizes system records in a unified location. In this study, it is used to manage customer records, vehicle records, booking information, rental transactions, payment records, maintenance data, forecasting records, recommendation records, and other operational information.

Context-Aware Decision Support. This refers to the use of selected operational and external context information to support administrative decision-making. In this study, applicable contextual information may include weather condition, road condition, route feasibility, route accessibility, travel distance, estimated or reference fuel efficiency, and estimated fuel consumption. These inputs serve only as supporting information for administrative vehicle assignment and branch allocation decisions and do not automatically determine the final operational action.

Context-Aware Fleet Allocation. This refers to the use of applicable operational and contextual information within the administrative branch allocation recommendation process. In this study, context-aware fleet allocation considers forecasted demand, vehicle availability, branch-level fleet status, vehicle condition, maintenance readiness,

and applicable contextual information such as travel distance, road condition, route feasibility, route accessibility, weather condition, estimated or reference fuel efficiency, and estimated fuel consumption. The resulting output is advisory and does not automatically transfer or assign vehicles.

Customer Requirement Submission. This refers to the process by which customers upload or provide required rental documents, such as valid identification cards and driver's licenses, for administrative verification before a booking request can proceed.

Customer-Side Vehicle Recommendation. This refers to the customer-facing rule-based feature that assists customers in identifying suitable available vehicles based on their stated rental requirements. In this study, the feature applies eligibility filtering and deterministic ranking using passenger capacity, large-luggage capacity, a stated total rental budget, rental duration, vehicle preference, vehicle availability, maintenance readiness, and applicable vehicle information. Destination or travel-area information is retained for administrative context review and does not affect customer eligibility or ranking. Vehicles that do not satisfy required availability, maintenance-readiness, passenger-capacity, or budget conditions are excluded from the recommendation results. The resulting recommendations provide vehicle-selection guidance before booking submission and do not constitute booking approval or guaranteed vehicle assignment.

Decision-Support System. This refers to a system component that assists users in making decisions by organizing data, analyzing relevant information, and presenting useful analytical outputs or recommendations. In this study, the decision-support component assists owners and authorized administrators through demand forecasting, vehicle utilization analysis, idle vehicle detection, operational fleet-status information,

maintenance-based vehicle readiness review, context-aware decision support, and branch allocation recommendation. Final operational decisions remain under authorized personnel.

Demand Forecasting. This refers to the process of estimating upcoming vehicle demand using historical qualifying booking records. In this study, qualifying bookings are aggregated by calendar week, branch, and vehicle category. A three-period Weighted Moving Average using fixed weights of 0.50, 0.30, and 0.20 is used to generate demand forecasts for up to three succeeding weekly periods. The first forecasted week is computed using the three most recently completed actual weekly demand observations, while the second and third forecasted weeks are generated recursively using the most recent available actual and previously forecasted demand values. The resulting forecasts serve as short-term decision-support information for operational and branch-allocation planning.

Estimated Fuel Consumption. This refers to the projected amount of fuel that may be required for a trip based on available travel-distance information and the vehicle's estimated or reference fuel-efficiency value. In this study, estimated fuel consumption is calculated by dividing the estimated travel distance in kilometers by the vehicle's reference fuel efficiency in kilometers per liter (km/L). The resulting value serves only as supporting operational information and does not represent guaranteed actual fuel consumption.

External Context Data Services. This refers to external services used to obtain selected contextual information required by the administrative decision-support component. In this study, Open-Meteo and OpenWeather One Call API 3.0 serve as the primary and fallback weather-data providers, respectively, while Geoapify and

LocationIQ provide primary and fallback geocoding, respectively. TomTom routing and traffic services provide route and incident information, with corresponding HERE services as fallback. If a primary service is unavailable or fails to provide usable information, the system may request the corresponding information from the designated fallback provider. If neither source provides reliable information, authorized manual information may be recorded where available; otherwise, the affected contextual factor is recorded as Unavailable or Unknown rather than being assumed.

Fleet Allocation. This refers to the administrative process of assigning, retaining, or redistributing vehicles across branches according to operational requirements. In this study, fleet allocation may consider forecasted demand, vehicle availability, branch-level fleet status, vehicle condition, maintenance readiness, and applicable contextual information. Final allocation decisions remain under the owner or authorized administrator.

Fleet Management. This refers to the process of managing vehicle records, branch assignments, vehicle availability, maintenance status, rental activity, vehicle condition, and other fleet-related information to support organized car rental operations.

Fleet Monitoring. This refers to operational fleet-status monitoring within the proposed system, which allows authorized users to view vehicle availability, active rental status, idle vehicles, branch assignments, vehicle condition, maintenance status, return status, and other recorded fleet-related information. In this study, fleet monitoring does not refer to live GPS tracking, geofencing, real-time location monitoring, or direct integration with vehicle-tracking hardware.

Forecasted Demand. This refers to the estimated qualifying vehicle demand generated using the Weighted Moving Average method for a specific branch, vehicle category, and future weekly period. In this study, the system generates forecasted demand for up to three succeeding weekly periods. The first forecasted period is derived from completed actual-demand observations, while later forecast periods may incorporate previously generated forecast values through recursive computation. Forecasted demand may contain decimal values for analytical reporting and visualization. When a forecast is used to determine the number of vehicles potentially required for branch-allocation planning, the corresponding weekly forecast is rounded upward to the nearest whole vehicle.

Fuel Efficiency. This refers to an estimated or reference value representing the distance a vehicle can travel per unit of fuel, expressed in kilometers per liter (km/L). In this study, the recorded value may be based on manufacturer specifications or owner-provided vehicle information and is treated as reference information rather than continuously measured actual fuel performance. The value may be used together with estimated travel distance to calculate estimated fuel consumption and shall serve only as supporting operational information.

Historical Rental Data. This refers to past booking or rental records that a deployed system may use to analyze demand patterns, vehicle usage, branch activity, and rental frequency. At the current stage of this study, the participating business has not provided real historical client records; therefore, the implemented system is populated with a labeled synthetic dataset for development and functional evaluation. The three most recent completed weeks are used as inputs to each WMA calculation. Within the synthetic time series,

completed-week observations are called actual demand only to distinguish realized test values from forecast values; they are not empirical client observations. Consequently, MAPE and other outputs derived from the current dataset demonstrate computational behavior and rule conformance rather than real-world predictive accuracy.

Idle Vehicle. This refers to an active and rental-ready vehicle that remains eligible for rental but has recorded no rental activity for at least fourteen consecutive days. In this study, a vehicle is not classified as idle when it is currently rented, unavailable because of maintenance, inactive, or otherwise prevented from being offered for rental. Idle-vehicle identification serves as an operational indicator for utilization monitoring and branch-allocation review and does not automatically result in vehicle transfer.

ISO/IEC 25010. This refers to a software product quality model used as a basis for evaluating relevant quality characteristics of the developed system. In this study, applicable ISO/IEC 25010 quality characteristics are used to structure the evaluation of system quality and acceptability (Monteverde et al., 2023; Bondoc, 2023; Mendoz et al., 2025; Adyaputra et al., 2025).

Maintenance Monitoring. This refers to the system feature used to record, view, and monitor vehicle maintenance information, including mileage or odometer readings, preventive maintenance schedules, maintenance history, vehicle condition, service records, and maintenance-related reminders.

Maintenance Readiness. This refers to the operational determination of whether a vehicle is suitable for rental, recommendation, assignment, or branch-allocation consideration based on its recorded maintenance information. In this study, a vehicle is

treated as not maintenance-ready when an active maintenance activity is in progress, required preventive maintenance is due or overdue, its recorded condition indicates that it is unsafe or unsuitable for rental, or an unresolved maintenance or repair concern prevents operational use. Vehicles that are not maintenance-ready are excluded from Customer-Side Vehicle Recommendation results and branch-transfer candidate selection.

Mean Absolute Percentage Error (MAPE). This refers to a forecasting error measure used to evaluate the difference between forecasted demand and corresponding actual demand. In this study, MAPE is calculated only for valid forecast observations where actual demand is greater than zero because percentage error is undefined when actual demand is zero. The resulting value represents the average absolute forecasting error as a percentage of actual demand, with lower MAPE values indicating smaller forecasting errors. The study does not assign qualitative accuracy classifications unless a verified interpretation basis is formally adopted.

Multi-Branch Car Rental Operations. This refers to car rental business operations conducted across more than one branch or operating location. In this study, it refers to the Manila and Antipolo operations of the target car rental business, where vehicle availability, customer demand, maintenance information, and branch-level coordination are managed through the proposed centralized system.

Operational Efficiency. This refers to the organization and performance of rental-related processes in terms of transaction coordination, record management, fleet visibility, and administrative decision support. In this study, operational efficiency represents an intended area of support of the proposed system and is not treated as a

proven long-term operational outcome unless supported by appropriate real-world effectiveness measurements.

Payment Verification. This refers to the manual administrative process of checking customer-submitted proof of payment and transaction references against the business's external bank or e-wallet records. In this study, payment verification is performed by an authorized owner or administrator, and the proposed system does not automatically process, approve, or verify payments through an integrated payment gateway.

Requirement Verification. This refers to the administrative process of reviewing customer-submitted documents and renter information to determine whether the customer satisfies the business's requirements before proceeding with the corresponding stage of the booking process.

Rental Transaction. This refers to the rental process recorded within the proposed system, including booking approval, requirement verification, payment verification, vehicle assignment, vehicle release, active rental-status monitoring, return processing, applicable charges or penalties, and final settlement.

Role-Based Access Control (RBAC). This refers to a security mechanism that restricts system access according to assigned user roles and responsibilities. In this study, RBAC is used to separate the access privileges of owners or administrators, operational staff, and customers, particularly for sensitive records such as customer documents, payment information, operational reports, and administrative functions (Dedhia, 2026; Lu et al., 2024).

Rule-Based Branch Allocation. This refers to the use of predefined operational rules to generate advisory vehicle-transfer recommendations between branches. In this study, the system compares the rounded-up forecasted vehicle requirement with projected available vehicle supply for each branch and vehicle category to identify shortages and surpluses. A potential transfer may be recommended when one branch has a forecasted shortage and another branch has a corresponding surplus, subject to vehicle availability, maintenance readiness, booking or rental commitments, idle-vehicle information, and applicable operational context. The recommended transfer quantity is limited by the destination shortage and source surplus. Final vehicle-transfer and branch-allocation decisions remain under the Owner/Admin.

System Acceptability. This refers to the degree to which intended users and evaluators consider the developed system suitable for its intended purpose based on the software quality characteristics selected for the study. In this study, system acceptability is assessed through the adopted ISO/IEC 25010-based evaluation instrument.

Vehicle Availability. This refers to the recorded operational status indicating whether a vehicle is available for rental or is otherwise reserved, rented, under maintenance, unavailable, or assigned to an existing rental transaction or branch-related activity.

Vehicle Utilization. This refers to the proportion of eligible operational days within a selected reporting period during which a vehicle was actually used for rental operations. In this study, vehicle utilization is calculated by dividing recorded rental days by eligible operational days and multiplying the result by 100. Days when the vehicle was inactive or unavailable because of maintenance are excluded from eligible

operational days. Vehicle utilization serves as an analytical indicator for fleet monitoring and does not independently determine vehicle-transfer or branch-allocation decisions.

Web-Based System. This refers to an information system accessed through a web browser using compatible devices such as laptops, tablets, and mobile phones. In this study, the proposed system is web-based to support access across the target client's existing devices without requiring separate application installation.

Weighted Moving Average (WMA). This refers to a forecasting method that estimates future values by assigning different weights to recent observations, with greater importance generally assigned to more recent data. In this study, a three-period WMA with fixed weights of 0.50, 0.30, and 0.20 is used to generate short-term vehicle-demand forecasts per branch and vehicle category. The method is applied recursively when producing the second- and third-week projections within the system's three-week forecast horizon because of its simplicity, interpretability, and suitability for operations with limited historical data (Rahayu et al., 2022; Sariati et al., 2026; Paramitha & Saifuddin, 2023; Khalid, 2024).

Chapter 2

REVIEW OF LITERATURE, STUDIES, AND SYSTEMS

This chapter presents the review of related literature, studies, and systems that served as the foundation for the proposed web-based car rental management system with demand forecasting and context-aware fleet allocation for small-scale multi-branch businesses. It discusses relevant local and foreign literature on car rental operations, fleet management, web-based management systems, demand forecasting, rule-based allocation, and decision-support systems.

The review also examines existing systems and operational challenges related to reservation management, vehicle utilization, fleet monitoring, and branch coordination. These studies helped identify the research gaps addressed by the proposed system and served as the basis for the development of the study.

A. Car Rental Industry and Operational Context

- **Car Rental Operations and Existing Management Challenges**

The car rental industry continues to serve an important role in modern transportation by providing temporary vehicle access for individuals, tourists, business travelers, and customers who need flexible mobility without long-term vehicle ownership. In the digital economy, car rental services are increasingly expected to provide faster booking, wider vehicle access, transparent availability, secure customer processing, and reliable transaction handling. Kumar and Singh (2024) explained that traditional car rental businesses previously depended on physical rental offices, paperwork, limited vehicle choices, and restricted availability, but online platforms have

allowed customers to rent vehicles more conveniently from different locations and at different times. This shows that the current situation of car rental businesses is moving from purely manual service delivery toward digital reservation and management platforms.

Despite this shift, many small-scale and medium-sized rental businesses still rely on manual or semi-manual processes. Dhimi et al. (2024) noted that many vehicle rental companies continue to depend on manual procedures for inventory, rental transactions, and customer records. These manual practices create difficulties because staff must repeatedly check vehicle availability, verify customer information, update schedules, and record transactions by hand. When these tasks are not supported by a centralized system, rental operations become slower and more prone to human error.

Manual booking is one of the most common problems in traditional car rental operations. Customers may need to inquire through phone calls, messaging applications, walk-in visits, or social media pages before confirming a reservation. While these channels may work for low transaction volumes, they become inefficient when several customers inquire about the same vehicle or rental date. Mhatre et al. (2026) stated that traditional vehicle rental systems that depend on manual booking may cause delays, booking conflicts, and inefficient vehicle availability management. This directly relates to small-scale car rental businesses because delays in confirming availability can result in missed reservations, customer frustration, or incorrect booking information.

Overlapping reservations are another operational issue caused by manual or disconnected booking records. When availability is recorded in notebooks, calendars, spreadsheets, or chat threads, staff may fail to update records immediately after

confirming a reservation. This creates a risk that the same vehicle may be promised to more than one customer. Saritha et al. (2026) emphasized that distributed car rental systems are designed to manage vehicle availability, customer reservations, and billing workflows while reducing booking conflicts through a responsive web interface. This supports the need for reservation validation and vehicle availability locking in the proposed system.

Fragmented records also weaken car rental operations. In manual or semi-manual settings, customer details, vehicle records, payment references, rental agreements, maintenance schedules, and booking histories may be stored in separate devices, notebooks, spreadsheets, and messaging applications. Jyothi and Sheethal (2025) identified vehicle booking, maintenance automation, billing management, customer authentication, fleet optimization, reporting tools, and operational efficiency as key concerns in car rental systems. Similarly, Rahman and Mohd Rusli (2023) explained that manual car rental booking makes it difficult for owners to track customer data, booking history, vehicle information, and rental transactions efficiently. Their web-based car rental business management system was designed to help car owners manage rental records more systematically while providing customers with an easier booking process.

Delayed rental transactions are also common when payment validation, requirement checking, vehicle assignment, and return settlement are handled manually. Ganeshkar et al. (2021) presented an online car rental management system that allows users to view vehicle listings, register, book cars online, and process payment through the system. Ogbiti and Aaron (2024) also developed a web-based car

rental management system with customer registration, online car reservation, reservation management, and administrative functions to streamline the rental process.

Poor monitoring of vehicle availability is another major challenge in car rental businesses. In a manual setup, staff may not immediately know whether a vehicle is available, reserved, rented out, under maintenance, assigned to another branch, or awaiting return. This weak visibility can cause wrong booking decisions, underutilized units, or customer dissatisfaction. Web-based rental systems address this problem by maintaining updated vehicle records and availability status. For example, Mhatre et al. (2026) study explained that web-based platforms allow customers to browse vehicles, check availability, and make reservations, while administrators can manage vehicle records and booking information through a centralized dashboard.

- **Manual and Fragmented Recordkeeping in Car Rental Operations**

Manual and fragmented recordkeeping remains one of the common operational concerns in car rental businesses, especially in organizations that still rely on paperwork, spreadsheets, messaging applications, and separate digital tools for daily transactions. In traditional car rental operations, customers often need to visit rental offices, fill out forms, and wait for manual approval, making the process time-consuming and inconvenient. Ogbiti and Aaron (2024) emphasized that many car rental companies still depend on manual processes or basic non-web-based systems, which can limit operational efficiency, slow down rental processing, and create difficulties in managing customer data, rental history, payments, and customer preferences. This shows that manual recordkeeping does not only affect customer convenience but also weakens the internal ability of the business to maintain complete and updated operational records.

In car rental management, fragmented information occurs when important records are stored in different locations or systems instead of one centralized platform. Van der Aa et al. (2017) explained that process information becomes fragmented when information about the same business process is spread across several documents, systems, or artifacts. This fragmentation increases the effort needed to access the right information and may affect process execution and decision-making. In the context of car rental operations, this may happen when booking details are stored in chat histories, schedules are monitored through calendars, payments are checked through screenshots or banking applications, and vehicle status or maintenance records are kept in separate files. As a result, staff or owners may need to check multiple sources before confirming a reservation, validating payment, assigning a vehicle, or generating a report.

Manual recordkeeping also increases the possibility of errors and delays in rental operations. Keerthi and Kumar (2024) stated that traditional car rental processes involve several manual tasks, including booking management, vehicle allocation, tracking vehicle availability, and maintaining service schedules. These manual processes are prone to errors and inefficiencies and may also cause delays in billing and invoicing. Similarly, Abayon et al. (2023) identified manual or Excel-based booking management, maintenance reminders, payment tracking, and vehicle tracking as operational challenges in a car rental business. These findings are relevant to small-scale car rental businesses because manual handling of records can lead to inaccurate booking information, missed updates, delayed payment confirmation, and difficulty in monitoring vehicle availability.

Fragmented records can also affect the quality and reliability of business information. Van der Aa et al. (2017) found that fragmented process information may result in maintenance issues, inefficient process execution, and poor process outcomes. When information is outdated, incomplete, or difficult to locate, users may perform tasks based on incorrect information. In car rental operations, this may result in double-booking, incorrect vehicle assignment, missed maintenance schedules, inaccurate payment status, or unreliable reports. Since car rental transactions involve several connected activities, such as customer verification, booking approval, payment validation, vehicle release, monitoring, and return settlement, fragmented records can disrupt the continuity of the rental process.

The need for centralized recordkeeping is therefore important in improving car rental operations. Ogbiti and Aaron (2024) explained that a web-based car rental management system can address the inefficiencies of traditional manual systems by automating the rental process, improving data management, and allowing companies to manage fleets, track reservations and payments, and maintain comprehensive customer records. Keerthi and Kumar (2024) also emphasized that a car rental management system integrates key functions such as reservation management, vehicle tracking, maintenance scheduling, billing, invoicing, customer management, and reporting. These features support the need for a centralized platform where customer records, vehicle information, reservations, payments, maintenance records, and reports can be accessed and managed more efficiently.

In relation to the proposed system, the reviewed literature shows that manual and fragmented recordkeeping can negatively affect operational efficiency, booking

accuracy, customer record management, payment monitoring, maintenance tracking, and decision-making. Although some car rental businesses already use digital tools, the use of separate and disconnected platforms may still cause fragmented information if records are not integrated into one system. Therefore, this theme supports the development of a centralized web-based car rental management system that can organize rental transactions, customer records, vehicle availability, payment validation, maintenance information, and operational reports in a single platform. By reducing fragmented records and manual coordination, the proposed system can help improve record accuracy, process visibility, and administrative decision-making in small-scale multi-branch car rental operations.

- **Multi-Branch Car Rental Operations and Fleet Coordination**

The operational problem becomes more complex when a car rental business operates across more than one branch. In multi-branch operations, vehicles are not only rented and returned; they must also be assigned, monitored, relocated, and balanced across different locations. A branch may experience high demand while another branch may have idle vehicles. This creates a fleet coordination problem where the business must decide which vehicles should remain in a branch, which vehicles should be transferred, and how availability should be monitored across locations.

Research on vehicle-sharing and carsharing operations supports the importance of balancing vehicle supply and demand across service areas. Boysen et al. (2024) explained that regional imbalance occurs when actual vehicle supply does not match projected demand levels, making it necessary to align vehicle distribution

with expected demand. Although their study focuses on carsharing, the concept is relevant to multi-branch car rental businesses because both settings deal with vehicle availability across different locations. If one branch has high demand but limited units, while another branch has idle vehicles, the business may lose potential bookings and reduce fleet utilization.

Eliyan and Kerbach (2024) also emphasized that fleet size is sensitive to demand in vehicle relocation problems. Their review shows that vehicle relocation and resource allocation are important because service quality depends on the ability to provide vehicles where customers need them. Brandizzi et al. (2022) similarly noted that vehicle-sharing services face management challenges involving relocation strategies and user clustering. These studies strengthen the argument that multi-branch car rental operations require more than simple booking records; they need a system that can support fleet visibility, branch-level monitoring, and informed allocation decisions.

For small-scale multi-branch businesses, coordination is often done through private messages, calls, shared calendars, or informal updates between owners and staff. This setup may work temporarily, but it becomes risky when reservations increase or when multiple users update records separately. Saritha et al. (2026) proposed a distributed car rental management suite that integrates reservation management, fleet allocation, and customer billing using a scalable and modular architecture. The study found that such systems can improve vehicle availability management, customer reservations, billing workflows, transaction management, and booking conflict reduction. This directly supports the proposed system's multi-branch reservation and fleet allocation features.

- **Web-Based Car Rental Management Systems**

Web-based car rental management systems are developed to address the limitations of manual and semi-manual rental operations. These systems provide browser-based access to core rental functions such as customer registration, vehicle browsing, reservation processing, availability checking, payment monitoring, vehicle records, and administrative reporting. Since they are accessible through internet-connected devices, they are suitable for small businesses that use phones, tablets, and laptops across different locations.

Ogbiti and Aaron (2024) showed that a web-based car rental management system can allow customers to register, view available cars, select a vehicle, make reservations online, and manage reservations, while administrators handle system functions. This supports the proposed system's reservation and customer management modules. Similarly, Rahman and Mohd Rusli (2023) developed a web-based car rental business management system that allows customers to search for available cars, make reservations, check booking status, and view rental history. At the same time, the system helps car owners manage vehicle details, booking requests, customer records, feedback, and reports in a more organized manner.

Web-based systems also improve record organization. Bakale et al. (2023) described a MERN stack car rental platform that stores and manages customer information, vehicle information, and rental data while supporting payment processing and fast customer request handling. Karche et al. (2025) likewise noted that traditional booking systems often rely on manual processes, resulting in inefficiency, data redundancy, and customer dissatisfaction, while a digital platform supports real-time

vehicle rental management. These studies support the need for centralized customer records, vehicle records, and rental transaction histories in the proposed system.

In terms of transaction monitoring, web-based systems make it easier for administrators to verify booking status, payment status, customer details, and vehicle assignment. Rahman and Mohd Rusli (2023) emphasized that a web-based car rental business management system can help owners monitor customer information, booking status, rental history, and reports in one platform. This is relevant to the proposed system because rental operations involve several connected activities: customer inquiry, requirement verification, payment confirmation, vehicle assignment, rental monitoring, return checking, and final settlement.

- **Vehicle Monitoring Tools and Limitations of Non-Integrated GPS System**

Vehicle monitoring tools are widely used in fleet-based operations because they help owners and managers observe vehicle location, movement, and operational status during active use. In car rental businesses, vehicle monitoring is important because rented vehicles are temporarily controlled by customers; therefore, owners need tools that help them verify vehicle location, monitor active rentals, reduce risk, and support operational decision-making. However, the value of vehicle monitoring becomes stronger when tracking data is connected to other business records such as reservations, customer information, payment validation, vehicle condition, maintenance history, and reports.

Farahpoor, Esparza, and Soriano (2024) discussed fleet management as an area affected by operational concerns such as equipment breakdowns, rising

maintenance costs, inefficient resource utilization, and outdated telematics systems. Their study proposed an IoT-driven fleet management system that supports real-time communication, data gathering, monitoring, and analysis. This shows that vehicle monitoring should not be treated only as a location-checking tool. Instead, monitoring data becomes more useful when it is part of a system that supports operational decisions.

Naim et al. (2025) also emphasized the importance of integrating GPS, artificial intelligence, and predictive maintenance in fleet management. Their study described how real-time GPS tracking, route optimization, and predictive maintenance may work together to improve fleet operations. This supports the idea that vehicle monitoring is more effective when it is connected to other decision-support processes. For a car rental business, GPS information becomes more useful when it can support vehicle assignment, route awareness, maintenance planning, and operational reports.

Sutabri et al. (2024) presented a GPS-based vehicle monitoring system integrated with operational vehicle maintenance. Their study showed that monitoring vehicle position and condition in real time can help optimize travel routes, reduce travel time, reduce operational costs, and support maintenance planning. The system did not focus only on vehicle location; it also considered vehicle-related indicators such as fuel percentage and tire pressure. This is relevant to the proposed system because car rental operations require not only vehicle tracking but also vehicle condition monitoring, maintenance history, and readiness status.

In the case of the participating car rental business, the client already uses external vehicle monitoring tools such as AKSH GPS, Apple Find My, and AirTags. These tools help the client monitor selected vehicles, but they are not directly

connected to the proposed car rental management system. Therefore, these tools should be treated as existing external monitoring tools rather than system-integrated modules. The proposed system will not perform live GPS tracking or automatically retrieve location data from these external applications. Instead, it will centralize vehicle records, branch assignment, vehicle availability, rental status, maintenance records, vehicle condition, return status, and monitoring notes based on updates entered by authorized users.

The limitation of non-integrated GPS systems is that vehicle monitoring information remains separated from the main operational records of the business. When vehicle location is checked in one application while bookings, payment proof, customer requirements, rental schedules, and maintenance records are stored in other platforms, the owner or staff must manually compare data across several tools. This may cause delayed coordination, repeated checking, fragmented records, and limited fleet visibility. Although external GPS tools remain useful for location monitoring, they do not automatically update booking status, vehicle availability, maintenance status, or branch-level fleet reports.

This limitation supports the proposed system's focus on centralized car rental management. The system does not replace AKSH GPS, Apple Find My, or AirTags. Instead, it organizes the operational records that are currently fragmented across messaging applications, calendar tools, spreadsheets, payment screenshots, and external monitoring applications. This approach is aligned with the current scope of the study because it improves record centralization, fleet visibility, and operational coordination without claiming direct GPS integration. Full GPS integration may be

considered as a future enhancement when API access, device compatibility, cost, and implementation feasibility are already established.

- **Vehicle Monitoring Tools and Limitations of Non-Integrated GPS Systems**

Vehicle monitoring and tracking systems commonly require the integration of positioning hardware, communication modules, and software applications to provide automatic or real-time vehicle location monitoring. Yu, Hlaing, and Aung (2026) developed a vehicle location and status tracking system that integrated GPS, GSM/GPRS, OBD technologies, and a web-based application to provide real-time location tracking, trip analysis, and telemetry monitoring. Their study shows that live vehicle tracking is not only a record-keeping function, but a technology-dependent process that requires GPS-enabled hardware, communication infrastructure, automated data updates, and supporting software.

Related literature also explains that vehicle tracking systems may be active or passive depending on how tracking data are collected, transmitted, and used. Alhwety and Elfadil (2024) described active vehicle tracking systems as systems that provide real-time tracking, geofencing, cellular tracking, or satellite-based tracking, while passive tracking systems store vehicle information for later retrieval and reporting. They also noted that vehicle tracking system implementation may involve concerns such as compatibility, scalability, cost-effectiveness, privacy, data transmission reliability, and technical issues. This supports the need to clearly define whether a proposed system performs direct GPS tracking or only uses vehicle monitoring records as supporting operational data.

In the case of Web-based Car Rental Management System, direct GPS integration is not treated as a core requirement because the business already uses AKSH GPS as an external vehicle monitoring tool. The proposed system will not replace AKSH GPS or directly retrieve live GPS coordinates from it. Instead, the system will support the business process by storing vehicle records, plate numbers, renter details, booking records, rental transactions, and manual monitoring notes. Through these records, authorized users can identify which renter or rental transaction is connected to a vehicle being monitored through AKSH GPS or other external tools. For example, when the owner checks a monitored vehicle through AKSH GPS using its plate number, the proposed system can be used to find the corresponding vehicle record, current rental transaction, renter information, booking details, and relevant monitoring remarks.

This limitation is appropriate because the main purpose of the proposed monitoring function is administrative tracking and record linkage, not direct live GPS tracking. The system will store monitoring sources, location notes, monitoring status, remarks, and recorded staff updates based on external tools such as AKSH GPS, Apple Find My, AirTags, or manual observations. However, it will not claim automatic geofencing, live map tracking, real-time GPS coordinate retrieval, or direct GPS hardware/API integration unless such integration is formally implemented in the future. This ensures that the system scope remains realistic and aligned with the actual operational practice of the client.

- **Security, Data Privacy, and Role-Based Access in Web-Based Systems**

Security, data privacy, and role-based access are important considerations in the development of web-based information systems because these systems commonly

handle sensitive user records, transaction details, uploaded documents, and administrative data. In online management systems, unauthorized access may lead to privacy violations, data misuse, inaccurate records, and loss of user trust. For this reason, modern web-based systems require secure authentication, access restrictions, and proper handling of user information to ensure that only authorized users can access specific functions and records.

Role-Based Access Control, or RBAC, is one of the most commonly used approaches for managing user permissions in web-based systems. RBAC restricts system access based on the role assigned to each user, such as administrator, staff, customer, or owner. Romadhan and Mansur (2026) developed a web-based educational payment information system using RBAC to protect financial transaction data. Their system assigned different access privileges to users such as Super Admin, Treasurer, Student Guardian, and Principal. The study found that RBAC effectively prevented unauthorized access to sensitive financial information and improved data accuracy and accountability. This is relevant to the proposed car rental management system because the system will also handle payment proof, booking records, customer information, and administrative data that should not be equally accessible to all users.

Similarly, Cardos et al. (2025) evaluated the impact of RBAC and data privacy measures on user satisfaction and security compliance in a digital evaluation and feedback system. Their findings showed that RBAC received favorable ratings in terms of usability and security compliance, although they also emphasized the need for clear role definitions and privacy communication. This indicates that RBAC is not only a technical security mechanism but also a factor that affects user confidence and satisfaction. In the proposed car rental system, this supports the need to clearly define

access levels for owners, administrators, and customers so that users understand what information they can access and what records are restricted.

The use of RBAC has also been applied in inventory and management systems that handle sensitive operational data. Purba and Hidayasari (2026) implemented RBAC in a web-based drug stock management information system using Laravel and MySQL. Their system used RBAC to restrict access based on assigned roles and achieved a 100 percent pass rate in black-box testing. The study also reported a System Usability Scale score of 76, which was interpreted as good. This shows that RBAC can improve data security without preventing users from effectively using the system. The findings are relevant to the proposed car rental system because both systems involve the management of important records, such as inventory or fleet data, transaction data, and role-specific administrative functions.

Aside from access restriction, secure authentication and monitoring are also necessary in web-based systems. Kanagamalliga et al. (2026) proposed a web-based system with JSON Web Token authentication, RBAC, encrypted credential storage, audit logging, and real-time alerts for suspicious activities. Their study showed high accuracy in access enforcement, reliable threat detection, complete audit coverage, and low response latency. These findings suggest that sensitive web-based systems should not rely only on login credentials, but should also include mechanisms for monitoring access and detecting possible unauthorized activities. For the proposed car rental system, this supports the use of secure login, role-based access, and possible logging of important activities such as document viewing, payment verification, booking approval, and record updates.

Multi-layer security is also important when web systems transmit and store sensitive data. Manoj et al. (2025) designed a scalable RBAC-based system that

integrated security safeguards against common web application vulnerabilities such as SQL injection, cross-site request forgery, and cross-site scripting. Their system also used encryption and one-time password authentication to improve data protection. The study found that the combination of RBAC, encryption, and authentication mechanisms improved protection of data in transit and at rest. This is relevant to the proposed car rental system because customer documents, payment proof, and rental records may be transmitted between users and the web platform. Therefore, the system should include appropriate security safeguards to reduce unauthorized access and data exposure.

Authentication security is also important in browser-based systems that support multiple users and access levels. Dedhia (2026) proposed a secure web authentication framework using JSON Web Tokens stored in HTTP-only cookies, role-based access control, and secure password hashing. The study found that the combination of JWT and RBAC improved authentication security, strengthened session management, and reduced risks related to common web vulnerabilities. For the proposed car rental system, this supports the need for secure session handling, especially because the system will be used by customers, administrators, and owners who access different modules such as booking, payment verification, vehicle records, and reports.

Data privacy is also essential in systems that process customer and transaction information. Lu et al. (2024) discussed the use of differential privacy for protecting sensitive transaction data while preserving the usefulness of aggregated information for analysis. Their study emphasized that privacy-preserving techniques can reduce the risk of unauthorized disclosure while still allowing systems to generate useful business insights. This is relevant to the proposed system because rental transaction data may be used for reports, demand forecasting, and branch-level analysis. The system should

therefore protect individual customer records while still allowing authorized users to generate operational reports and forecasting outputs.

In relation to the proposed web-based car rental management system, security and privacy are necessary because the system will manage customer accounts, renter requirements, uploaded identification documents, proof of payment, booking records, rental history, vehicle records, and operational reports. The reviewed studies show that RBAC is an effective approach for restricting access based on user responsibilities, while secure authentication, encryption, audit logging, and privacy-preserving practices strengthen the protection of sensitive information. These concepts support the proposed system's need for role-based access control, where owners may have access to financial and operational records, administrators may manage reservations and vehicle status, and customers may only access their own booking details and submitted requirements.

- **Software Quality Evaluation Using ISO/IEC 25010**

Software quality evaluation is an important part of system development because it determines whether a developed system meets its intended requirements and satisfies the needs of its users. For web-based information systems, evaluation is necessary to assess whether the system functions correctly, performs efficiently, is easy to use, protects data, and can be maintained or improved in the future. ISO/IEC 25010 is commonly used as a software quality model because it provides structured quality characteristics for evaluating software products, including functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

Monteverde et al. (2023) developed a web-based rental house smart finder system using Rapid Application Development and evaluated the system using ISO/IEC 25010. The study assessed the system based on quality characteristics such as functional suitability, performance efficiency, usability, reliability, security, maintainability, and portability. The evaluation result obtained an overall mean of 3.67 out of 4.00, which was interpreted as very effective. This study is relevant to the proposed car rental management system because both systems are web-based rental-related platforms. It shows that ISO/IEC 25010 can be used as an appropriate framework for evaluating whether a rental-domain system is functional, usable, reliable, secure, and ready for implementation.

Bondoc (2023) also used ISO/IEC 25010 to evaluate a web-based smart attendance monitoring system with QR code and SMS notification. The study used two respondent groups: IT experts and school personnel. IT experts evaluated the system using all eight ISO/IEC 25010 characteristics, while school personnel evaluated selected characteristics such as functional suitability, performance efficiency, and usability. The study found that the system conformed to the assessed ISO/IEC 25010 standards. This dual-respondent evaluation design is relevant to the proposed car rental system because the system may also be evaluated by both technical evaluators and actual or potential users, including owners, administrators, and customers.

Mendoz et al. (2025) evaluated a web-based system for maternal support and infant health tracking using ISO/IEC 25010. Their study involved 60 respondents, including 40 users and 20 technical evaluators, and used weighted mean and Likert scale analysis. The results showed that both user and technical respondent groups gave favorable ratings, with usability rated highest by users and functionality rated highest by technical respondents. This study supports the use of ISO/IEC 25010 in

evaluating web-based systems with different user groups. For the proposed car rental system, this approach is useful because customer users may focus more on ease of use and booking experience, while technical evaluators may focus more on system functionality, security, reliability, and maintainability.

Adyaputra et al. (2025) combined ISO/IEC 25010 with the Technology Acceptance Model in assessing user satisfaction with local government websites. The study found that functional suitability, usability, and reliability significantly influenced perceived usefulness, while performance efficiency and security were identified as priority areas for improvement. These findings show that software quality is closely related to user satisfaction and acceptance. In the context of the proposed car rental management system, this means that the system should not only include the required features but should also be easy to use, reliable, responsive, and secure so that customers, administrators, and owners can confidently use it in daily operations.

Islam et al. (2021) evaluated an enterprise resource planning system using the ISO/IEC 25010 quality model. Their study involved users with different roles, including department heads, IT professionals, and daily users, and used all eight ISO/IEC 25010 quality characteristics. The study showed that ISO/IEC 25010 is suitable for evaluating multi-role and multi-module management systems. This is directly relevant to the proposed car rental system because it will include multiple user roles and modules, such as customer management, booking management, vehicle management, payment verification, maintenance monitoring, forecasting, allocation recommendation, and report generation.

Ismail et al. (2025) developed a web-based information system for food availability and production and evaluated it using ISO/IEC 25010 across all eight quality aspects. The study used an Agile development approach and involved expert and user

evaluation. The findings showed that the system was feasible for implementation and conformed to the ISO/IEC 25010 quality characteristics. This study supports the idea that ISO/IEC 25010 can be applied to management information systems that support data processing, monitoring, and reporting. The proposed car rental system shares similar characteristics because it will process operational records, monitor vehicle and booking status, and generate reports for decision-making.

Nuzula and Rochimah (2023) evaluated a human resource information system using ISO/IEC 25010, focusing on functional suitability and usability sub-characteristics such as functional completeness, correctness, appropriateness, learnability, operability, and user interface aesthetics. Their study showed that ISO/IEC 25010 can be used not only for overall system evaluation but also for identifying specific strengths and weaknesses in system functions and interface design. This is useful for the proposed car rental system because the proponents may use ISO/IEC 25010 results to identify which modules need improvement, such as booking, payment verification, vehicle assignment, forecasting output, or dashboard usability.

Laudza and Sofyan (2024) evaluated an academic information system using all eight ISO/IEC 25010 standards and found that seven of the eight characteristics were fully met, while security only reached a medium level. This finding highlights that security may become a weak point in web-based system evaluation if it is not deliberately considered during design and development. For the proposed car rental management system, this reinforces the need to include security measures such as authentication, RBAC, restricted document access, and protection of payment-related records before conducting the final ISO/IEC 25010-based evaluation.

- **Research Instrument Development Using ISO/IEC 25010**

Aside from being used as a software quality evaluation framework, ISO/IEC 25010 can also serve as the basis for developing the research instrument of the study. The research instrument for evaluating the proposed system will be developed using the ISO/IEC 25010 software product quality model because it provides a recognized framework for assessing the quality of ICT and software products. ISO/IEC 25010 defines a product quality model composed of quality characteristics and sub-characteristics that serve as a reference for specifying, measuring, and evaluating software product quality (International Organization for Standardization & International Electrotechnical Commission [ISO/IEC], 2023). These quality characteristics may be translated into questionnaire indicators that can be used to evaluate whether the developed system satisfies the required software quality standards.

Recent studies support the use of ISO/IEC 25010 as a basis for system evaluation instruments. Ariningsih and Muhammad (2024) evaluated a Ticket Management System using ISO/IEC 25010, questionnaires, black-box testing, and Likert-scale scoring. Their study shows that ISO/IEC 25010 characteristics and sub-characteristics can be translated into questionnaire items for measuring the quality of an implemented information system. Similarly, Nugraha and Rachman (2024) evaluated the Ixitask web-based application using ISO/IEC 25010 through a descriptive quantitative method, focusing on functional suitability, reliability, interaction capability, and security, with validity and reliability testing. These studies show that ISO/IEC 25010 can be used as a practical basis for developing system evaluation instruments.

In relation to Web-based Car Rental Management System, ISO/IEC 25010 is appropriate as the basis for the research instrument because the system must be evaluated according to measurable software quality characteristics. The questionnaire

items may be organized according to selected quality characteristics such as functional suitability, performance efficiency, compatibility, interaction capability or usability, reliability, security, maintainability, and flexibility or portability, depending on the finalized evaluation scope. This ensures that the evaluation instrument is aligned with an internationally recognized software quality model and with recent system evaluation practices.

Overall, the reviewed studies show that ISO/IEC 25010 is a suitable and widely used framework for evaluating web-based systems and management information systems. It provides a structured basis for measuring the quality of the proposed system in terms of functional suitability, performance efficiency, compatibility, usability or interaction capability, reliability, security, maintainability, and portability or flexibility, depending on the finalized evaluation scope. The reviewed studies also show that ISO/IEC 25010 can guide the development of research instruments by translating software quality characteristics into measurable questionnaire indicators. For the proposed car rental management system, ISO/IEC 25010 is appropriate because the system is multi-role, multi-module, web-based, and intended for actual operational use. Using ISO/IEC 25010 will allow the proponents to evaluate whether the system satisfies user needs, supports business operations, protects sensitive data, and remains usable and maintainable for future improvement.

- **Use of 4-Point Likert Scale in System Evaluation Studies**

Likert-type scales are widely used psychometric instruments for measuring attitudes, opinions, perceptions, and evaluation judgments in research contexts. Koo

and Yang (2025) explain that Likert-type scales allow respondents to indicate levels of agreement or disagreement and that Likert-type scale variations may use different numbers of points, including four to seven response options. They also note that even-numbered scales remove the neutral midpoint and compel respondents to take a more definite stance.

Kankaraš and Capecchi (2025) explain that the use of a neutral response option in Likert-type scales remains debated. While a neutral midpoint may be valid for respondents who are genuinely neutral, some respondents may also use the neutral option as an escape response, especially when they are uncertain or unwilling to provide a clear answer. Mariano et al. (2024) also examined whether Likert scales should include neutral response categories and emphasized that the inclusion or exclusion of a neutral option is a methodological decision in survey design. Their study made the researchers decide on the use of a neutral category based on the purpose of the instrument and the type of responses needed.

In relation to Web-based Car Rental Management System, the 4-point Likert scale is appropriate because the evaluation requires respondents to determine whether each system quality indicator is acceptable or not acceptable. Since the questionnaire will be based on ISO/IEC 25010 quality indicators, the respondents are expected to evaluate the system's functional suitability, performance, usability or interaction capability, reliability, security, and maintainability through clear agreement or disagreement responses. The absence of a neutral midpoint helps reduce undecided answers and encourages respondents to provide a definite evaluation.

The following scale will be used for the evaluation instrument:

4 - Strongly Agree

3 - Agree

2 - Disagree

1 - Strongly Disagree

This scale is suitable for questionnaire statements such as “The system provides the required booking functions correctly” or “The system protects sensitive user and transaction information.” The results may then be interpreted using frequency, percentage, and weighted mean to summarize the evaluators’ assessment of the system.

B. Fleet Management and Operational Decision-Making

- **Fleet Management and Vehicle Utilization**

Fleet management plays a major role in transportation and car rental operations because companies must ensure that vehicles are available in the right quantity, at the right locations, and at the right time. Effective fleet management helps businesses improve operational efficiency, reduce costs, and maintain customer satisfaction. However, transportation and rental companies often experience several operational problems that affect profitability and service quality.

One major problem in fleet management is the presence of idle vehicles. Idle vehicles are fleet units that remain unused for long periods in certain locations. These vehicles continue to generate maintenance, insurance, parking, and depreciation expenses even though they are not producing revenue. According to Brandizzi et al. (2022), vehicle-sharing systems must continuously monitor vehicle locations and usage patterns to avoid inefficiencies and improve utilization. When too many vehicles are

allocated to low-demand areas, businesses experience financial losses because valuable resources remain inactive.

Another common challenge is vehicle shortages. Vehicle shortages occur when customer demand exceeds the number of available vehicles at a specific location. This problem is especially common at airports, tourist destinations, and busy urban areas during peak seasons. Customers may experience long waiting times, booking cancellations, or service dissatisfaction due to insufficient vehicle availability. Favier et al. (2024) explain that one-way rentals often create shortages in high-demand regions while causing an oversupply of vehicles in other areas. This imbalance creates operational pressure for fleet managers who must relocate vehicles efficiently to maintain service quality.

Closely connected to shortages is the issue of underutilized fleet units. In many transportation systems, some vehicles are used frequently while others remain inactive or are only partially utilized. This uneven utilization leads to inefficient use of company resources because businesses continue paying operational costs for vehicles that generate little or no revenue. Favier et al. (2024) note that successful fleet management requires balancing service levels, relocation costs, and utilization rates to maximize operational performance.

Transportation and car rental companies also face uneven demand across locations. Demand is not distributed equally because customer activity is heavily influenced by geography, tourism, population density, and economic activity. High-demand locations such as city centers, airports, shopping districts, and tourist attractions often experience heavy vehicle usage, while suburban or remote areas experience lower demand. Brandizzi et al. (2022) found that demand in Barcelona's vehicle-sharing system was concentrated within a very small portion of the city,

showing how customer demand tends to cluster geographically. Uneven demand creates difficulties for fleet managers because vehicles may accumulate in low-demand areas while becoming unavailable in locations where customers need them most.

As a result of these demand imbalances, companies frequently experience poor vehicle distribution. Vehicles may become concentrated in one location because of one-way trips, tourism patterns, or seasonal travel behavior. For example, tourists may rent vehicles in major cities and return them in rural or leisure destinations, leaving shortages in urban pickup locations. Favier et al. (2024) demonstrated this issue in New Zealand, where one-way rentals caused vehicles to accumulate in the South Island while shortages developed in other regions. Poor distribution increases relocation costs and reduces the efficiency of fleet operations.

To address these challenges, businesses increasingly rely on demand forecasting. Demand forecasting is the process of predicting future customer demand using historical data, statistical methods, and analytical models. Forecasting helps companies estimate how many vehicles will be required, where they will be needed, and when demand levels are expected to rise or fall. Accurate forecasting allows businesses to allocate vehicles proactively instead of reacting after shortages or oversupply problems occur.

Demand forecasting provides several operational advantages. It helps reduce idle vehicles by preventing oversupply in low-demand locations while also minimizing shortages in busy areas. Forecasting also improves relocation planning because fleet managers can move vehicles before imbalances become severe. In addition, forecasting improves customer satisfaction because customers are more likely to find available vehicles when and where they need them. Sangaraju et al. (2024) explain that forecasting models such as ARIMA, regression analysis, and machine learning

techniques are increasingly used in rental vehicle systems to improve revenue prediction and resource allocation. Similarly, Akay (2024) identifies demand forecasting as one of the major themes in car rental and transportation management research.

Historical rental and vehicle usage data are essential for effective demand forecasting. Transportation companies collect data related to rental frequency, pickup and drop-off locations, booking patterns, trip durations, seasonal demand, and peak operating periods. By analyzing historical information, businesses can identify recurring trends and customer behavior patterns. For example, companies may discover that airport rentals increase during holiday seasons, that urban demand peaks during evenings, or that tourist destinations require more vehicles during weekends.

Brandizzi et al. (2022) demonstrated how historical usage data could support relocation planning and customer clustering strategies in vehicle-sharing systems. Similarly, Favier et al. (2024) used historical movement patterns to evaluate different fleet allocation and relocation models in rental vehicle operations. These studies show that historical data allows businesses to make more informed operational decisions and improve overall fleet performance.

Historical data analysis also supports dynamic pricing and revenue optimization. When companies can predict increases in demand, they can adjust prices and reposition vehicles strategically to maximize profitability. Sangaraju et al. (2024) argue that combining forecasting techniques with dynamic pricing and intelligent resource allocation can significantly improve both revenue generation and customer satisfaction in rental vehicle systems.

Modern fleet management systems also integrate forecasting technologies with intelligent transportation systems and traffic control systems. Kotsialos and

Vassilakopoulou (2023) explain that real-time traffic information, GPS tracking, and collaborative information-sharing systems help transportation companies improve routing decisions, reduce delays, and increase operational efficiency. These technologies allow fleet operators to respond more effectively to changing traffic conditions and customer demand.

In conclusion, fleet management in transportation and car rental operations involves managing vehicle allocation, utilization, and customer demand across multiple locations. Problems such as idle vehicles, vehicle shortages, underutilized fleet units, uneven demand, and poor vehicle distribution can reduce operational efficiency and customer satisfaction. Demand forecasting and historical data analysis provide important solutions by helping businesses predict future demand, improve relocation strategies, optimize vehicle allocation, and maximize profitability. As transportation systems become increasingly data-driven, forecasting and intelligent fleet management technologies will continue to play a critical role in improving operational performance and service quality.

- **Customer-Side Vehicle Recommendation**

Recommendation systems are commonly used to help users select suitable options based on their preferences, requirements, or contextual needs. In the transportation and tourism field, Sabet et al. (2022) developed THOR, a hybrid recommender system for personalized travel experiences. Their study supports the idea that recommendation systems can assist users by matching available options with user preferences and contextual requirements. Although THOR focuses on travel experiences rather than car rental operations, it provides conceptual support for customer-facing recommendation as a decision-support feature.

Vehicle recommendation has also been studied in relation to vehicle selection. Boteju and Munasinghe (2020) proposed a vehicle recommendation system using a hybrid recommender algorithm and natural language processing approach, while Prabowol, Nasrun, and Nugrahaeni (2019) developed a car selection recommendation system using item-based collaborative filtering. These studies show that vehicle selection can be supported through system-generated suggestions. However, the present study does not claim to use collaborative filtering, natural language processing, or machine learning. Instead, the proposed Customer-Side Vehicle Recommendation uses rule-based logic because the recommendation criteria must remain simple, transparent, and aligned with the operational requirements of a small-scale car rental business.

In this study, the Customer-Side Vehicle Recommendation is placed in the customer-side module. It allows customers to input passenger capacity, destination or travel area, budget, rental duration, and vehicle preference. The system then applies predefined rule-based logic to display suitable available vehicles before the customer submits a booking request. This separates customer-side vehicle selection support from admin-side branch allocation recommendation, which remains focused on demand forecasting, fleet monitoring, maintenance records, and operational planning.

- **Idle Vehicle Detection and Vehicle Utilization Monitoring**

Idle vehicle detection and vehicle utilization monitoring are important aspects of fleet management because they help determine whether vehicles are being used efficiently or left unused for a certain period. In rental and shared mobility operations,

vehicles that remain idle may still require parking, cleaning, monitoring, maintenance, and administrative attention even when they are not producing revenue. Vehicle utilization monitoring therefore supports operational visibility by showing which vehicles are frequently used, which vehicles are rarely used, and which locations may have excess or insufficient vehicle supply.

Abouelela et al. (2023) developed a fleet utilization prediction framework for shared mobility using open-source big data and machine learning. Their study measured fleet utilization through the daily number of trips per vehicle and examined how historical demand, temporal patterns, and contextual variables can be used to predict vehicle usage. The study shows that utilization monitoring can support operational policies related to fleet deployment, planning, and resource use. It also demonstrates that historical mobility records can be transformed into useful indicators for understanding whether fleet assets are being used efficiently.

Eliyan and Kerbache (2024) reviewed vehicle relocation in one-way carsharing systems and explained that flexible pickup and return arrangements may create vehicle imbalance. In one-way systems, some stations may accumulate excess vehicles while other stations experience vehicle shortages. Their review emphasized that vehicle rebalancing, relocation, resource allocation, and station planning are important operational concerns in fleet-based services. This supports the idea that vehicle availability should not be viewed only at the individual vehicle level, but also at the location or branch level.

Zhu et al. (2024) proposed an idle vehicle rebalancing approach for autonomous mobility-on-demand systems. Their study explained that imbalance between mobility demand and vehicle supply can reduce operational efficiency and service quality. Vehicle rebalancing was presented as a way of dispatching idle vehicles to areas with higher demand in order to improve service rates and reduce passenger waiting time. Although the study focused on autonomous mobility, it shows that idle vehicles can become useful operational resources when their location, availability, and demand context are monitored.

Zhong et al. (2025) studied carsharing operation optimization by considering economic and social benefits. Their study developed an optimization model involving site location, fleet size, site size, and pricing decisions. The study shows that vehicle distribution and utilization are important in transport services that depend on shared or rented vehicles. It also highlights that operational planning must consider how fleet resources are positioned and used in relation to travel demand.

Skrzynski et al. (2026) examined vehicle relocation decisions in free-floating carsharing schemes. Their study combined demand prediction, vehicle availability prediction, and decision-making methods to support relocation planning. The study shows that relocation decisions become more effective when demand and availability are considered together. This suggests that idle vehicle detection should not be based only on whether a vehicle is currently unused, but should also consider demand patterns, availability, and the location where the vehicle is assigned.

This indicates that idle vehicle detection and vehicle utilization monitoring support fleet visibility, resource planning, and allocation decisions. These studies suggest that underutilized vehicles, location-based imbalance, and branch-level vehicle shortages can be identified more effectively when booking history, rental activity, availability status, and location assignment are monitored together. In the context of Web-based Car Rental Management System, these findings support the inclusion of vehicle utilization reports, idle vehicle reports, fleet status dashboards, and branch-level monitoring to help the Owner/Admin review vehicle use and make informed allocation decisions.

- **Vehicle Maintenance Monitoring and Preventive Maintenance Scheduling**

Vehicle maintenance monitoring and preventive maintenance scheduling are essential in fleet operations because they help maintain vehicle safety, reliability, and availability. Preventive maintenance is performed before serious failure occurs and may be based on scheduled service dates, mileage intervals, inspection findings, vehicle condition, or service history. In fleet-based operations, maintenance monitoring helps reduce unexpected downtime, avoid delayed service, and ensure that only roadworthy vehicles are used for operation.

Barreiro-Zambrano et al. (2026) developed an integrated preventive maintenance management and alert system for light vehicles. Their system monitored the distance traveled by a vehicle and generated alerts for services such as oil changes, brake inspections, and timing-belt replacements. The study shows that preventive maintenance reminders can help users become aware of upcoming service requirements before vehicle problems become more serious. It also demonstrates that

distance-based monitoring can be used as a practical basis for maintenance scheduling.

Crespo del Castillo and Parlikad (2024) discussed dynamic fleet management by integrating predictive maintenance, preventive maintenance, and operational workload balancing. Their study explained that traditional maintenance strategies may become inefficient when they rely only on fixed schedules or reactive repair. The authors emphasized that maintenance planning affects fleet availability, cost, and workload distribution because vehicles must be assigned either to operation, preventive maintenance, predictive maintenance, or idle status. This shows that maintenance planning should be connected to operational decision-making rather than treated as a separate activity.

Arena et al. (2022) reviewed predictive maintenance in the automotive sector and discussed statistical, stochastic, and artificial intelligence-based approaches for maintenance decision-making. Their review explained that data-driven maintenance can help prevent possible failures and support timely maintenance actions. The study also shows that vehicle maintenance decisions increasingly depend on organized and reliable data, including vehicle condition, usage patterns, and failure-related information. This supports the importance of structured maintenance records in fleet operations.

Giannoulidis et al. (2024) presented a predictive maintenance framework in the Navarchos Fleet Management System. Their study focused on applying predictive maintenance in a fleet environment where information may be dynamic, incomplete, or

continuously changing. The Navarchos case shows that fleet management systems can support maintenance decisions by organizing vehicle-related data, monitoring status changes, and applying maintenance logic within a broader system architecture. This highlights the importance of updating maintenance records as vehicles are used, inspected, repaired, and returned to service.

Mahale et al. (2025) reviewed artificial intelligence-driven predictive maintenance in vehicles and emphasized that predictive maintenance can improve vehicle reliability, reduce operating costs, and support better maintenance scheduling. Their review discussed the role of vehicle health monitoring, diagnostics, data quality, and maintenance records in improving maintenance strategies. The study also identified challenges such as data quality, scalability, and integration, which means that maintenance systems must be designed carefully to ensure that recorded data can support reliable decisions.

It shows that maintenance monitoring and preventive maintenance scheduling help improve fleet readiness, vehicle safety, reliability, and operational control. These studies indicate that maintenance decisions should consider mileage, service dates, maintenance alerts, vehicle condition, and service history. In the context of Web-based Car Rental Management System, these findings support the inclusion of maintenance records, PMS scheduling, vehicle status logs, mileage tracking, and maintenance readiness dashboards. These features can help authorized users monitor service schedules, identify vehicles due for PMS, and prevent vehicles with unresolved maintenance issues from being assigned to renters.

- **Mileage, Vehicle Condition, and Maintenance History as Fleet Decision Factors**

Mileage, vehicle condition, and maintenance history are important decision factors in fleet management because they help determine whether a vehicle is suitable for operation, rental assignment, preventive maintenance, or further inspection. A vehicle may appear available based on schedule, but it may still be unsuitable for use if it has high mileage, poor condition, unresolved repair issues, or incomplete maintenance records. In fleet-based operations, these factors help managers evaluate vehicle readiness and reduce the risk of assigning vehicles that may be unsafe, unreliable, or likely to require service.

Chen et al. (2024) proposed a mileage-aware method for vehicle maintenance demand prediction. Their study emphasized that mileage should be considered when predicting vehicle maintenance needs because vehicle use contributes to component wear, aging, and possible maintenance demand. The study also noted that some maintenance prediction methods focus only on partial maintenance needs, while overall vehicle maintenance demand requires a broader view of the vehicle's usage and service requirements. This shows that mileage is not only a numerical record of vehicle use but also a useful indicator for anticipating future maintenance needs.

Arena et al. (2022) reviewed predictive maintenance in the automotive sector and discussed how statistical, stochastic, and artificial intelligence-based methods can support maintenance decisions. Their review explained that the availability of condition-monitoring data allows vehicle-related problems to be detected earlier and helps maintenance actions become more timely and appropriate. The study also

highlighted that predictive maintenance depends on organized data and proper interpretation of vehicle condition. This supports the idea that vehicle condition records and maintenance-related data are necessary for improving fleet reliability and reducing unexpected failures.

Chaudhuri and Ghosh (2024) developed a predictive maintenance approach for vehicle fleets using hybrid deep learning-based ensemble methods. Their study used both sensor data and historical maintenance data to support time-between-failures modeling and vehicle fleet maintenance prediction. Although their approach uses advanced computational methods, the study demonstrates that historical maintenance records are valuable for understanding vehicle reliability and supporting preventive management. This reinforces the importance of maintaining complete maintenance history in fleet operations, especially when vehicles are repeatedly used and exposed to different operating conditions.

Crespo del Castillo and Parlikad (2024) discussed dynamic fleet management by integrating predictive maintenance, preventive maintenance, and operational workload balancing. Their study explained that maintenance planning affects fleet availability because vehicles must be allocated either to operation, preventive maintenance, predictive maintenance, or idle status. This means that maintenance is not only a technical activity but also an operational decision factor. If maintenance planning is not connected to vehicle assignment and workload distribution, fleet operations may experience higher costs, reduced availability, or service delays.

Chen et al. (2025) reviewed vehicle maintenance demand prediction and explained that many existing studies focus on specific vehicle components, such as

suspension, brakes, or batteries. Their review emphasized the need to move from isolated component-level prediction toward holistic vehicle health management. This is important in fleet decision-making because a vehicle's readiness cannot be judged from one factor alone. Mileage, condition status, maintenance records, unresolved issues, and service history must be considered together to form a more complete understanding of the vehicle's operational condition.

Overall, the reviewed studies show that mileage, vehicle condition, and maintenance history are significant factors in fleet decision-making. Mileage data can indicate usage level and possible wear, condition records can show readiness or risk, and maintenance history can reveal completed service activities, recurring problems, or unresolved issues. In the context of Web-based Car Rental Management System, these findings support the use of odometer records, vehicle condition status, maintenance history, PMS information, and audit records as internal decision-support data. These factors can help the Owner/Admin determine whether a vehicle is ready for rental, due for inspection, under maintenance, or unsuitable for assignment until cleared.

- **Demand Forecasting in Transportation, Car Rental, and Vehicle-Sharing Operations**

Demand forecasting is an important decision-support process in transportation, car rental, and vehicle-sharing operations because it helps businesses estimate future customer demand based on historical data, usage patterns, and other related factors. In rental-based transportation services, demand is not always consistent across time and location. Some branches, stations, or service areas may experience high demand

during certain periods, while others may have lower usage or idle vehicles. Without demand forecasting, businesses may rely only on manual judgment or current vehicle availability, which can lead to reactive decision-making, vehicle shortages, poor vehicle distribution, and underutilized fleet units.

In car rental and vehicle-sharing operations, forecasting helps managers anticipate where and when vehicles will be needed. This is particularly important in multi-branch or multi-location operations because each location may experience different demand levels. Historical rental transactions, reservation records, return data, and vehicle usage patterns can be analyzed to estimate future demand and support better planning. When demand forecasts are connected to fleet monitoring and allocation decisions, businesses can prepare vehicles in advance, reduce shortages, and improve overall vehicle utilization.

Suganya et al. (2026) proposed an integrated AI-driven forecasting and optimization framework called RentOptima for intelligent vehicle rental management. Their study used branch-wise historical rental, usage, and maintenance data to forecast demand and support operational decisions. The framework applied time-series forecasting models such as ARIMA, Prophet, and LSTM to estimate future rental demand across multiple locations. The study showed that forecasting can help align fleet distribution with predicted demand and improve the efficiency of vehicle allocation and maintenance scheduling. This supports the proposed capstone system because it shows that historical rental and operational data can be used to produce branch-level demand forecasts that guide fleet allocation decisions.

Alencar et al. (2021) examined carsharing demand forecasting using univariable and multivariable models. Their study emphasized that daily demand variation can contribute to fleet imbalance in carsharing systems, especially in large

urban areas. They also found that adding meteorological data, such as weather-related variables, can improve forecasting performance depending on the forecasting model and time horizon. This finding supports the idea that demand forecasting may be strengthened by considering contextual factors, especially weather conditions, which may influence customer travel behavior and rental demand. For the proposed car rental management system, this supports the inclusion of demand forecasting and selected context-aware factors to assist managers in anticipating demand more accurately.

Zhao et al. (2021) focused on station-level short-term demand forecasting in carsharing systems. Their study proposed a station-embedding-based hybrid neural network that combines spatial station features and time-series patterns to predict rental and return demand. The study emphasized that accurate short-term demand forecasting is necessary for vehicle relocation because operators need to know which stations are likely to experience higher rental or return activity. Although their method uses a more advanced forecasting model than the proposed capstone system, the operational principle is relevant. It shows that location-specific forecasting can support vehicle distribution decisions, which is similar to the proposed system's goal of forecasting demand per branch.

Martin et al. (2021) also demonstrated the relationship between demand prediction and fleet relocation in free-floating car-sharing services. Their study explained that predicting demand across geographic regions and time periods is necessary for determining where vehicles should be positioned. They used predicted utilization as an input for relocation optimization, showing that forecasting is not only useful for reporting but also for operational planning. This is relevant to the proposed system because the demand forecasting feature is intended to support rule-based fleet allocation recommendations. In other words, the system will not only predict future

demand but also use those predictions to guide managers in assigning or redistributing vehicles across branches.

Huang et al. (2021) addressed demand-supply imbalance in carsharing systems under demand uncertainty. Their study proposed a two-stage stochastic programming approach that considers tactical planning and real-time vehicle relocation decisions. The findings showed that active relocation and planning under uncertain demand can improve service outcomes. This supports the importance of forecasting in rental operations because demand uncertainty can lead to shortages, lost rental opportunities, and poor service availability. For small-scale multi-branch car rental businesses, forecasting can help managers prepare for expected demand instead of responding only after vehicle shortages occur.

Overall, the reviewed studies show that demand forecasting is a useful tool for improving planning and decision-making in transportation, car rental, and vehicle-sharing operations. Historical rental data, vehicle usage records, reservation patterns, and location-based demand information can help estimate future vehicle needs. These forecasts can then support fleet monitoring, branch coordination, and vehicle allocation. In the proposed web-based car rental management system, qualifying historical booking records will be aggregated by calendar week, branch, and vehicle category, and a three-period Weighted Moving Average will be used to generate short-term forecasts for up to three succeeding weekly periods. The resulting forecast horizon can help managers view expected demand trends in advance, identify possible future shortages or surpluses, and support more proactive fleet-planning decisions.

Therefore, demand forecasting is directly connected to the proposed system's decision-support function. By transforming historical rental records into future demand estimates, the system can help small-scale multi-branch car rental businesses move from reactive vehicle assignment to more proactive and data-informed planning. This supports the development of a car rental management system that does not only record transactions but also assists managers in making better operational decisions.

- **Weighted Moving Average and Forecasting Methods**

Accurate demand forecasting is a foundational requirement for any fleet management or resource planning system. Among the classical time series techniques, the Weighted Moving Average (WMA) method has been consistently recognized in recent literature as a practical and reliable approach for short-term demand prediction, particularly in small-scale operational environments where computational simplicity and interpretability are prioritized.

The WMA method estimates future demand by computing a weighted mean of a fixed number of past observations, where more recent data points are assigned higher weights than older ones. This design reflects the intuitive assumption that recent patterns are more indicative of near-future behavior than distant historical records. Formally, the forecast for a future period is expressed as the sum of the products of each historical observation and its corresponding weight, divided by the total sum of weights. This differentiates WMA from the Simple Moving Average (SMA), which treats all historical periods equally. As Rahayu et al. (2022) demonstrated in forecasting product inventory for a small reseller, the WMA method produced a MAPE of just

0.18% using a 3-month weighted window, confirming its precision in short-horizon, low-volatility demand environments.

The suitability of WMA for short-term forecasting has been validated across multiple operational contexts. A comparative study by Sariat et al. (2026) evaluating SMA and WMA for MSME production planning found that the WMA model with a 4-period window achieved the lowest MAPE of 8.36%, significantly outperforming SMA (MAPE of 14.40%) and longer-period variants of both methods. The study attributes this advantage to WMA's responsiveness to recent demand shifts, noting that shorter historical periods combined with appropriate weighting more effectively capture evolving demand patterns in dynamic operational environments. Similarly, Paramitha and Saifuddin (2023), comparing WMA, SMA, and Single Exponential Smoothing for industrial demand forecasting, found WMA to produce the smallest error across all key accuracy metrics, recommending it as the most appropriate method for operational production planning.

Beyond individual case studies, a broader review of time series forecasting models by Khalid (2024) situates the moving average family—including WMA—as particularly well-suited for short-term operational forecasting. The review notes that while advanced machine learning models offer improvements for long-term strategic prediction, traditional time series methods remain effective and efficient for immediate operational needs, particularly when data volumes are limited and computational resources are constrained. This finding aligns with the work of Lezama Lope et al. (2023), who developed an Analogue Moving Average (AnMA) method for very short-term load forecasting and demonstrated that moving average-based techniques deliver high accuracy, speed, robustness, and low maintenance overhead relative to

more complex alternatives—attributes that are especially valuable in small-scale operational settings.

The advantages of the WMA method make it particularly appropriate for the forecasting module of the proposed fleet management system. First, its computational simplicity requires no specialized infrastructure and can be implemented efficiently in a web-based or lightweight application environment. Second, WMA is highly interpretable: administrators and operators can understand and verify forecasts without statistical expertise, supporting confidence in system outputs. Third, the method is lightweight in terms of data requirements—it operates on a short rolling window of historical data, making it viable even when historical records are limited, as is common in small-to-medium scale operations. Fourth, by assigning greater weight to recent observations, WMA is naturally attuned to gradual shifts in demand trends, making it more responsive than equal-weight methods in environments where usage patterns evolve over time.

However, the WMA method carries recognized limitations that must be acknowledged in system design. Because it derives forecasts from recent historical observations, WMA may not fully capture sudden or irregular changes in demand caused by unexpected events or abrupt operational shifts. In addition, WMA does not inherently model complex seasonal patterns or external contextual factors, which may limit its ability to represent demand changes that are not reflected in recent historical records. These limitations reinforce the need to treat WMA output as short-term decision-support information rather than as a guaranteed prediction of future demand. In the proposed system, WMA will therefore be implemented using a fixed three-period window and predefined descending weights of 0.50, 0.30, and 0.20. The system will generate a rolling forecast for up to three succeeding weekly periods. The first

forecasted week will use the three most recently completed actual weekly observations, while the second and third forecasted weeks will be generated recursively using the newest available actual and previously forecasted values. Because recursive forecasting progressively incorporates predicted values, the farther forecast periods will be treated primarily as planning projections. The same procedure will be applied consistently across branches and vehicle categories to preserve reproducibility. The Owner/Admin may review and use the generated forecasts for operational planning, but the computed WMA values themselves will not be manually altered.

- **Rule-Based Decision Support and Fleet Allocation**

Beyond demand forecasting, operational fleet management requires a structured mechanism for translating forecast outputs into concrete vehicle allocation decisions. Rule-based decision support systems provide this mechanism by encoding expert operational knowledge into explicit IF-THEN conditional logic, enabling the system to generate consistent, auditable, and actionable recommendations without requiring complex optimization algorithms or machine learning infrastructure.

A rule-based system operates on a knowledge base composed of decision rules derived from domain expertise. Each rule specifies a condition—such as the number of available vehicles, current deployment status, forecasted demand, or route priority—and a corresponding action or recommendation. When the system evaluates a request or operational scenario, the inference engine matches current data against the rule set to produce a recommendation. This structure makes rule-based systems particularly transparent: every recommendation can be traced back to the rule or

combination of rules that triggered it, which is important for operational accountability and for allowing administrators to audit or modify system behavior over time.

A particularly instructive recent example is provided by Haber et al. (2026), who developed a rule-based expert system for dispatch planning in liquid transportation logistics. Their system incorporated 28 decision rules formulated from the collective expertise of experienced planners, engineers, and senior managers, enabling automated evaluation of vehicle, trailer, and driver combinations for each operational order. Tested against real operational data from a Turkish logistics firm, the rule-based system consistently produced more cost-effective, balanced, and efficient dispatch plans than manual planning by experienced human personnel. Critically, the system reduced planning errors and reduced the variability introduced by individual human judgment—demonstrating that codifying expert logic into a rule structure can yield measurable operational improvements even in complex, multi-variable resource allocation problems.

The application of rule-based systems to fleet allocation is further supported by Mavrin and Makarova (2026), who developed a Decision Support System (DSS) for waste transportation fleet management. Their system integrated simulation with a structured vehicle competitiveness assessment module, and the optimization experiment yielded a 40.64% reduction in total logistics costs compared to the baseline—a statistically significant improvement attributed to the structured, rule-driven allocation of fleet assets based on operational requirements. This confirms that even in the absence of highly sophisticated optimization engines, systematically applied decision rules can produce substantial efficiency gains over ad hoc or manual allocation practices.

Meira et al. (2022) further contextualize the role of decision support systems in fleet operations through a literature review and DSS prototype for beer delivery route management. Their work illustrates how rule-based routing and allocation logic—built around Vehicle Routing Problem frameworks—can be practically implemented in small-to-medium fleet management applications, offering a structured template that balances operational complexity with system feasibility.

For the purposes of the proposed fleet management system, rule-based allocation provides the logical layer that connects forecast outputs to deployment decisions. Based on the WMA-generated demand estimate for a given period, the system applies a set of predefined allocation rules—such as prioritizing vehicles with the highest availability, matching vehicle type to route category, or distributing deployments to avoid fleet concentration—to generate a recommended allocation plan. This approach mirrors the logic described by Mohamad et al. (2026), who found that integrating predictive analytics with rule-based decision frameworks significantly improved fleet planning and resource allocation in a resource-constrained institutional setting, reducing underutilization and improving service coverage.

The use of rule-based systems in operational decision support is also well-established in adjacent expert system literature. The work of an expert system for freight coordination in logistics (2021) highlights how encoding domain knowledge into structured rule sets enables organizations to automate operational decisions consistently, reduce dependence on individual expertise, and improve overall logistics coordination—outcomes directly relevant to fleet allocation in the proposed system.

In summary, the combination of WMA-based forecasting and rule-based allocation logic constitutes a technically sound and operationally appropriate foundation for the system's decision-support modules. WMA provides reliable short-term demand

estimates with minimal computational overhead, while the rule-based allocation layer translates those estimates into concrete vehicle deployment recommendations in a manner that is transparent, auditable, and adaptable to the specific operational context of the fleet being managed.

- **Rule-Based Vehicle Recommendation Systems**

Rule-based vehicle recommendation systems represent one of the most established and interpretable approaches to operational decision-making in fleet management, car rental, and transportation contexts. Unlike opaque machine-learning recommenders, rule-based systems encode expert knowledge as IF–THEN rules or fuzzy-logic inference structures, producing transparent, auditable, and easily maintained recommendations — qualities especially valuable for applications where users (drivers, dispatchers, customers) must understand and trust the system's output. The recent literature shows that rule-based approaches remain highly relevant in 2021–2026, particularly when integrated with modern web technologies, fuzzy reasoning, and multi-criteria decision frameworks.

Rule-based expert systems for vehicle and resource selection. The most directly relevant recent study is Haber, Haklı, Uğuz, and Gerz (2026), who developed a rule-based expert system for resource planning in liquid transportation, published in *Sustainability*. Their system encoded 28 decision rules derived from expert knowledge — covering trailers, ISO tanks, vehicles, and driver allocation — and evaluated multiple resource combinations to recommend the most suitable allocation for each order. When tested with real operational data from a Turkish logistics company, the rule-based system produced more cost-effective, efficient, and balanced dispatch plans

than manual planning, while reducing planning errors and variations arising from human judgment. The authors explicitly emphasized that rule-based expert systems serve as "reliable and scalable decision-support tools for complex dispatch planning problems," demonstrating consistent performance across different operational scenarios. This work directly validates the rule-based approach as an alternative or complement to machine learning in vehicle allocation contexts.

Earlier work by Topal, Yürüyen, Ulutaş, Karabašević, and Karaköy (2021) framed vehicle selection in freight transportation as a multi-criteria decision problem and used fuzzy BWM and fuzzy SWARA methods to determine which criteria freight companies value most when selecting transportation vehicles. Their finding that vehicle selection inherently involves multiple, sometimes conflicting criteria reinforces the case for rule-based and fuzzy approaches over single-objective optimization. Badulescu, Vanegas, and Cheikhrouhou (2025) extended this thinking with a Fuzzy Multi-Criteria Decision-Making (FMCDM) framework — integrating F-DEMATEL, F-AHP, and F-TOPSIS — for sustainable truck selection in a Swiss transportation company. Their results identified efficiency, truck capacity, and fuel consumption as key decision drivers, providing a structured rule-and-criteria approach for fleet-level vehicle recommendations.

Fuzzy-logic rule systems for transport route and vehicle recommendation. Several recent studies focus specifically on fuzzy-logic rule systems for transport decisions. Akhmedova, Zhazhneva, and Pianova (2023) developed a mobile expert system for selecting optimal cargo transport routes using fuzzy logic combined with geoinformation technologies and the OpenWeatherMap API for weather-aware

decisions. Their system used Dijkstra's and Jena's algorithms for shortest-path calculation and a simplified fuzzy inference algorithm for route recommendation under multiple influencing factors. This work is particularly relevant because it integrates rule-based reasoning with weather data — exactly the integration the present study undertakes. Zulkarnaen and Gustian (2022) implemented a Mamdani-style fuzzy decision support system specifically for car rental, achieving 90% accuracy in vehicle selection by considering rental distance, time, and customer demand — a clear precedent for rule-based recommendation in the car rental domain.

Gouveia, de la Iglesia, Abrantes, Rivero, Gouveia, and Váz (2025) proposed a hybrid fuzzy-logic decision support model combining Fuzzy TOPSIS with a Fleet Renewal Priority Index (FRPI) to prioritize vehicle alternatives for SMEs and public entities seeking sustainable mobility. The model evaluated economic, environmental, and operational criteria (total cost of operation, CO₂ emissions, maintenance needs, autonomy, infrastructure compatibility), using linguistic judgments converted into triangular fuzzy numbers to handle uncertainty and subjectivity. A companion 2025 study by the same team applied fuzzy logic to evaluate electric vehicle attractiveness, demonstrating that fuzzy approaches are well-suited to scenario-based, multi-criteria fleet decisions. Nemade and Pujeri (2023) used hybrid fuzzy-logic rules in the Internet of Vehicle Things context (ECFL-IoVT) to select reliable next vehicles for emergency communication — another illustration of rule-based vehicle selection under operational constraints.

Knowledge-based and IF–THEN rule systems in adjacent recommendation domains. Beyond strictly vehicle recommendation, the literature shows rule-based

systems being applied successfully across many related decision problems, demonstrating the broad maturity of the technique. Górski, Heesch, Dziendzikowski, and Dworakowski (2022) developed a fuzzy-logic-based recommendation system for processing algorithms in machine condition monitoring, showing 5–14.5% higher accuracy than arbitrary selection — useful as a precedent for predictive maintenance rule sets in fleet management. Chakrabarti, Ahmad, and Quix (2021) built a knowledge-based rule engine for visualization recommendation, demonstrating that rule-based recommendation systems extract data characteristics, match them to recommendation outputs, and operate effectively with minimal user input. Syed, Bhargava, and Sai Srinadh (2023) used simple IF–THEN rules in a knowledge-based expert system for material selection in racing bicycle forks — a methodological template that maps directly to vehicle selection logic.

Several Indonesian and Asian studies between 2024 and 2026 illustrate the continued relevance of rule-based systems for selection and recommendation problems implemented as practical web applications. Pratami, Tursina, and Septiriana (2024) built a rule-based recommendation system for bicycle model selection using the Laravel framework with Lemond-method rules, achieving an 82.9% user-satisfaction score in UAT testing — a useful template for the present study's recommendation engine implementation. Fajar, Atmaja, Hafidhoh, Ismar, and Ivansyah (2026) implemented a rule-based expert system in a Central Java vocational-school admission system using Agile Scrum and verified IF–THEN rules. Dumadi, Munawir, and Amaldi (2025) developed a rule-based expert system using Laravel with forward chaining inference for student major selection, reaching 85.25% UAT satisfaction. Sholihin and Hidayati (2024) used forward chaining in an expert system for programming language

selection. Although these works are not in vehicle recommendation per se, they document the design patterns — IF–THEN rule representation, forward chaining inference, Laravel-based web implementation, UAT validation — that current rule-based recommendation systems use.

Patil (2021) applied classifier rules and fuzzy logic in a voter recommendation system, comparing decision-tree rules with PART; the result that PART outperformed pure decision-tree rules is informative for designing rule-set structures. Karimian (2022) developed a rule-based weighted expert system for instructor selection using decision-tree rules with majority voting, achieving 85.55% accuracy with low computational complexity — an important property for transparent, deployable rule-based systems. Muraina, Agoi, Omorojor, Adedokun, and Ajetunmobi (2022) used VisiRule, an artificial-intelligence rule-based expert system, to support algorithm selection — demonstrating the broader pattern of rule-based decision support for selection problems.

Rule-based recommendation embedded in fleet and rental management systems. Some recent fleet and rental management studies have explicitly embedded recommendation logic into operational systems. Sahu, Katlana, Sharma, Baniya, Sureliya, and Kanungo (2024) developed OnDemand Vehicle Ride, an online vehicle rental management system with a recommendation module that suggests vehicles to customers based on their requirements; while the authors used machine learning rather than pure rule-based logic, the architecture mirrors what a rule-based recommender would occupy in such a system. Gurusinghe (2021) built an automated fleet management system for Cash-In-Transit (CIT) operations that allocated vehicles

and drivers based on rule-driven factors including productivity, cost reduction, secure transport, vehicle condition, distance, and number of jobs — essentially a rule-based allocation engine. Liu, Wu, and Wu (2023) built an app-based recommender system using association rules from pairwise comparison data on automobiles, showing that rule-mining techniques (with adjustable support and confidence thresholds) can deliver effective car recommendations.

C. Context-Aware Transportation and Operational Intelligence

- **Fuel Cost, Fuel Efficiency, and Vehicle Selection**

Fuel cost and fuel efficiency are important considerations in transportation operations because they directly influence travel expenses, vehicle utilization, and operational decision-making. In car rental businesses, customers do not only choose vehicles based on seating capacity, rental price, or vehicle type. They may also consider how much fuel a vehicle is expected to consume, especially when the rental involves long-distance travel. For rental operators, fuel efficiency is also relevant because it can support better vehicle recommendation, reduce unnecessary operating costs, and improve the suitability of vehicles for specific rental purposes.

Recent studies show that fuel efficiency can be used as a decision-support factor in transportation systems. Vyas et al. (2022) developed DriveBFR, a driver behavior and fuel-efficiency-based recommendation system that uses machine learning to provide fuel-efficient driving suggestions. Their study emphasized that fuel efficiency can be transformed into practical recommendations by analyzing driving behavior and fuel-related performance. This supports the idea that fuel efficiency should not only be treated as a basic vehicle specification but also as a factor that can guide users in selecting a more economical vehicle.

Fuel efficiency can also be connected to route and vehicle assignment decisions. Moawad et al. (2024) proposed a real-time energy and cost-efficient vehicle route assignment recommender system that predicts vehicle energy consumption and cost based on route characteristics and fleet constraints. Although the study focused on route assignment, its concept is relevant to car rental operations because it shows that vehicles can be recommended based on expected travel distance, energy or fuel consumption, and estimated trip cost. In a car rental system, this can support the recommendation of vehicles that are more suitable for a customer's intended destination or travel purpose.

Fuel price variability further affects transportation planning and vehicle selection. Peng et al. (2025) discussed how dynamic fuel prices can be considered in optimization systems to support cost-sensitive decisions. Their study showed that fuel-related planning can be adjusted depending on changing fuel price conditions. This is relevant to car rental operations because estimated fuel cost may vary depending on current fuel prices, travel distance, and vehicle fuel efficiency. Therefore, a car rental management system may use fuel price and fuel consumption data to provide more realistic and practical vehicle recommendations.

Vehicle selection is also considered a multi-criteria decision-making problem. Şimşek et al. (2025) presented a decision-support framework for vehicle selection using multi-criteria decision-making and machine learning. Their study demonstrated that vehicle alternatives can be ranked based on several benefit and cost-related criteria. Similarly, Alrashdi et al. (2025) proposed an intelligent decision-support framework for assessing alternative vehicle technologies and emphasized the importance of structured ranking in transportation decisions. These studies support the

use of systematic criteria in recommending vehicles, such as fuel efficiency, vehicle type, route distance, expected cost, and suitability for the customer's travel needs.

In the context of a web-based car rental management system, fuel efficiency can strengthen vehicle recommendation by allowing the system to suggest vehicles based on estimated fuel consumption and trip requirements. For example, a fuel-efficient vehicle may be recommended for long-distance travel, while another vehicle may be more suitable for short city trips or heavier passenger needs. This makes the recommendation more practical because the system considers not only vehicle availability but also the expected cost and operational suitability of the vehicle.

- **Travel Distance and Estimated Fuel Consumption in Vehicle Assignment**

The accurate estimation of travel distance and fuel consumption has become a central concern in modern vehicle assignment and recommendation systems, particularly as transportation operators face rising fuel costs, sustainability mandates, and pressure to reduce greenhouse gas emissions. The literature reveals a steady evolution from simple distance-based assignment models toward sophisticated approaches that combine geographical attributes, vehicle-specific characteristics, and real-time conditions to produce fuel-aware vehicle recommendations.

Conrad (2022) presented statistical techniques for estimating vehicle fuel consumption in urban road networks based on vehicle and geographical factors. The work uses OpenStreetMap mapping data and Shuttle Radar Topography Mission elevation data to generate paths that minimize fuel consumption, introducing the concept of a "fuel consumption estimating function" as an extension of conventional distance-estimating functions widely used in logistics planning. The models incorporate

vehicle weight, elevation, and regional travel speed characteristics as independent variables, demonstrating that fuel consumption can be reliably estimated from geographical data alone for long-range planning purposes. This work provides a foundational methodology for embedding fuel estimation directly into route and vehicle assignment logic.

Stokic, Momčilović, and Dimitrijević (2023) developed a bilinear interpolation model for estimating commercial vehicles' fuel consumption and exhaust emissions under different operating conditions. By incorporating average speed, road slope, and vehicle load/capacity utilization, the model achieved estimation deviations of only about 5% from real-world consumption values. This level of precision is significant for fleet operators because it demonstrates that fuel consumption can be predicted accurately enough to drive vehicle assignment decisions — for example, recommending lighter vehicles for hilly routes or restricting heavily loaded vehicles to flatter paths.

The relationship between distance and fuel consumption is rarely linear, and the literature increasingly recognizes that detailed environmental and behavioral factors must be considered. Fahmin et al. (2022) presented an eco-routing algorithm that computes the most fuel-economical route by considering detailed mobility profiles, driving behavior (aggressive vs. moderate), and vehicle type (truck vs. car). Their experiments on a real road network showed that this nuanced approach can reduce fuel consumption by up to 35% compared to traditional shortest-path or one-size-fits-all eco-routing methods. Similarly, Mahecha Núñez et al. (2023) demonstrated that incorporating real road network slopes into urban freight routing can increase fuel consumption estimates by 28.67% compared to flat-network assumptions —

highlighting that route feasibility for fuel-aware vehicle recommendation must account for terrain.

For real-time and at-scale applications, Moawad et al. (2021) developed a neural network recommender system for assigning vehicles to routes based on energy and cost criteria. Using machine learning to estimate energy consumption of candidate vehicles over given routes with limited internal vehicle dynamics, their system implements a complete recommendation logic supporting (1) single-trip recommendations with a top-k vehicles star ranking and (2) general n-vehicles-to-m-trips assignment problems. The system has been deployed in the POLARIS Transportation System Simulation Tool, demonstrating that fuel-aware vehicle assignment is a practical, deployable capability rather than a purely theoretical exercise.

Dhaou (2022) addressed the implementation side of fuel estimation by presenting a pipelined VLSI architecture for real-time fuel estimation algorithms. The work compared three architectures — binary search, direct address table, and approximation-based — and demonstrated that real-time, low-power fuel estimation suitable for in-vehicle and eco-routing applications is achievable. This implementation-oriented work complements the algorithmic literature by showing that fuel estimation is technologically feasible at the device level.

Recent work extends fuel-aware assignment to data-driven and spatio-temporal modeling. Patil et al. (2025) presented a Graph Neural Network–based approach for fuel consumption estimation using spatio-temporal modeling and traffic flow

predictions, combining traffic prediction uncertainty (via Adaptive Conformal Prediction) with Monte Carlo simulations of expected traffic flows. For passenger vehicles, predicted fuel consumption showed a maximum error of only 5.6% compared to observed values, and 8.4% for heavy-duty trucks. This study demonstrates that fuel-aware vehicle recommendation systems can now incorporate predicted traffic conditions, not just static route characteristics, into the assignment decision.

Reddy, Vurubindi, and Uyyala (2026) applied a feedforward artificial neural network to estimate fuel efficiency in heavy-duty vehicles using distance-based modeling, achieving 93.5% classification accuracy by aggregating data from similar vehicle profiles. The system uses telemetry features such as speed, throttle, gear, and fuel rate, and emphasizes scalability for integration with real-time vehicle telemetry — providing a practical basis for fuel-aware vehicle selection in fleet operations.

Together, these studies establish that travel distance and estimated fuel consumption are no longer treated as independent inputs to vehicle assignment; they are integrated, condition-aware, and increasingly predictive components of intelligent vehicle recommendation systems. Yet, as Patil et al. (2025) and Fahmin et al. (2022) note, this integration remains absent from most consumer-facing transportation and rental platforms, which continue to recommend vehicles based on inventory and price rather than projected fuel consumption over the specific route, terrain, and weather conditions the trip will encounter.

- **Road Conditions, Route Feasibility, and Road Accessibility**

Road conditions, route feasibility, and road accessibility are important factors in transportation and rental planning because they affect whether a vehicle is suitable for a specific trip. A route may be available in terms of distance, but it may not always be practical due to road closures, poor road surface, traffic congestion, flooding, construction, or limited accessibility. In car rental operations, these factors can affect travel time, customer experience, vehicle safety, and the possibility of vehicle damage. Because of this, road-related information can support better decision-making in vehicle recommendation and rental planning.

Context-aware transportation studies show that route recommendation should consider external conditions rather than relying only on distance or shortest-path calculations. Liu et al. (2022) proposed Hydra, a personalized and context-aware transportation recommendation system that integrates multi-source urban data such as road networks, points of interest, station data, and weather-related context. Their study emphasized that some travel queries may have no feasible route under certain conditions. This supports the need for transportation systems to determine whether a planned route is feasible before recommending travel options.

Real-time road condition data can also improve transportation planning. Dubey et al. (2025) developed RoadAware-DIRL, a system for on-the-fly path replanning using real-time road condition data. Their study used contextual cost mapping to assign higher penalties to hazardous routes and support route adjustments when road conditions change. This is relevant to car rental operations because road hazards, closures, or poor road conditions may affect a customer's planned trip. A route-aware system can help identify possible route concerns and support safer and more reliable rental planning.

Road inspection and safety assessment systems also demonstrate the importance of road condition monitoring. Gabbar et al. (2023) developed HAIS, a highway automated-inspection system that collects road condition data and supports road safety assessment. Their study showed that road condition information can be gathered, assessed, and used to support transportation-related decisions. In relation to car rental operations, this supports the idea that road accessibility and safety conditions should be considered when recommending vehicles, especially for routes that may involve poor road quality or possible hazards.

Route feasibility also affects vehicle suitability. A compact and fuel-efficient vehicle may be appropriate for city travel, but it may not be the best option for long-distance routes, rough roads, or areas with limited accessibility. In contrast, a larger or more durable vehicle may be more appropriate for difficult routes, even if it consumes more fuel. This means that vehicle recommendation should not be based on fuel efficiency alone. It should also consider the customer's destination, road conditions, route distance, and accessibility concerns.

In car rental operations, integrating route conditions and road accessibility into the system can help reduce poor vehicle matching and improve decision-making. If a route has possible road concerns, the system can recommend a more suitable vehicle or notify the operator that the selected vehicle may not be ideal for the intended trip. This supports a more context-aware approach to rental planning, where vehicle availability, fuel efficiency, and route feasibility are considered together before finalizing a recommendation.

Overall, the reviewed studies show that fuel efficiency and route awareness are valuable factors in transportation decision-support systems. Fuel-related studies support the use of fuel consumption, fuel cost estimation, and multi-criteria vehicle

ranking, while route-related studies support the consideration of road conditions, route feasibility, and accessibility. These concepts support the proposed web-based car rental management system by strengthening its ability to recommend vehicles that are not only available, but also fuel-efficient, cost-aware, and suitable for the customer's planned route.

- **Road Condition Data Sources and Route Accessibility Assessment**

Effective context-aware transportation systems require reliable data on road condition and accessibility — information that historically was expensive to collect and slow to update. Recent literature documents a clear paradigm shift: from dedicated, costly surveys conducted by transportation agencies to crowdsourced and connected-vehicle approaches that produce continuous, low-cost, large-scale road condition data.

Dong and Li (2021), publishing in *Sensors* (MDPI), demonstrated the viability of smartphone-based road surface monitoring by collecting accelerometer data at low sampling rates from Android-based applications during normal driving. Using power spectral density analysis and k-means unsupervised machine learning, they achieved an average defect detection accuracy of 84%. Their finding that smartphones in ordinary drivers' vehicles can supply quasi-real-time road condition information at hourly or daily granularity established crowdsourced road sensing as a credible alternative to traditional surveys.

Sattar (2021), in a master's thesis from Ryerson University, developed a probabilistic-based crowdsourcing technique using smartphone linear accelerometers,

gyroscopes, and GPS sensors to detect road surface anomalies. Single-survey detection achieved approximately 80% accuracy, but when multiple drivers traversed the same road segments, the probabilistic integration of detections improved accuracy by an additional 5–20%. The accompanying web-based GIS platform demonstrated how such data can be visualized for real-time monitoring — directly relevant to how route accessibility information should be presented to system users.

Building on these foundations, Lyu et al. (2024) proposed a Detecting and Clustering Framework that uses smartphone GPS, orientation, and accelerometer data with a wavelet scattering transformation and LSTM neural network to identify Bump Features of Road Surfaces (BFRSs). The framework's two-stage clustering method considers both distances and directions, producing road surface condition maps applicable to urban road maintenance and route planning. Gamage, Thotawatththa, and Wijayasiri (2022) similarly used crowdsourced smartphone data combined with fuzzy logic and machine learning to identify and classify road anomalies by severity — an important advancement because it allows transportation systems to distinguish between minor irregularities and severe hazards that would render a route inaccessible for certain vehicle types.

Connected vehicle data has emerged as a complementary source. Chen et al. (2021), publishing in *Transportation Research Record*, demonstrated that production connected vehicles can supply road surface readings closely correlated with Class 1 pavement profilers: a regional experiment in Detroit covering 64 miles of roadway found that 83% of traveled miles had a 0.8 or higher correlation with reference equipment, and 85% of measurements had absolute errors under 50 in/mi. This finding

establishes that connected vehicle fleets can supply broad, continuous road condition data without dedicated survey equipment.

Jeong and Jo (2024), publishing in the IEEE Internet of Things Journal (Q1), addressed the practical real-world challenges of smartphone-based pavement monitoring through IRI-Net, a fully convolutional neural network designed to estimate the International Roughness Index from anonymous vehicles' vibrations at variable driving speeds. By incorporating a data management strategy tailored to low-resolution crowdsourced GPS data and validating against conventional inertial profiler measurements across 29 vehicles, the work represents a significant step toward operationally deployable crowdsourced pavement assessment.

Sirocchi, Klopfenstein, and Bogliolo (2022), publishing in IEEE Access (Q1), analyzed the SmartRoadSense (SRS) platform — a large-scale operational mobile crowdsensing system that computes a crowdsensed road roughness index (PPE) from smartphone sensor data. Their joint statistical analysis with open datasets demonstrated that road category, speed limit, and other internal/external features affect PPE in predictable ways, validating the SRS platform as a tool for assessing road quality conditions at city- and regional-scale.

Road accessibility — particularly for users with mobility needs — represents an emerging extension of road condition sensing. Aly, Youssef, and Agrawala (2021) presented the AccessMap system, which passively crowdsources smartphone sensor measurements to mark indoor and outdoor spaces as visually impaired- and/or wheelchair-accessible. Evaluation in multiple countries achieved average precision and

recall around 89.8% and 86.3%, increasing to 98.7% and 99% with as few as seven encounters per semantic. Building on this, Nenadic, Palazzi, and Zanella (2025) developed a road accessibility mapping system using smartphone-based sensing with threshold-based methods and dynamic rolling windows, designed for integration with applications like Google Maps so users with specific mobility needs can choose routes based on real-time road condition information.

The most recent contribution, Khandakar et al. (2025), introduced RoadSens-4M, a multimodal smartphone-and-camera dataset that integrates GPS, accelerometer, gyroscope, magnetometer, gravity, and orientation sensor data with GIS, weather, and video data — explicitly designed to support holistic road-way analysis for traffic management, infrastructure development, and smart transportation systems. The integration of weather information directly with road condition data within a single dataset reflects the convergence of context-aware data sources discussed in this study.

Collectively, this literature confirms that road condition and accessibility data — once a bottleneck for context-aware transportation systems — are now available at scale through crowdsourced smartphone sensing, connected vehicle telemetry, and integrated multimodal datasets. The implication for route accessibility assessment is significant: transportation systems can now query real-time, granular road condition data alongside weather and traffic data when deciding whether a particular vehicle on a particular route at a particular time represents a feasible and safe assignment.

- **Weather-Aware Transportation Decision Support**

Weather conditions are among the most consequential yet underappreciated variables in transportation decision-making. Rain, fog, snow, and low visibility do not merely slow vehicles; they fundamentally restructure which routes are feasible, which vehicle types are appropriate, and whether travel should proceed at all. Tobin et al. (2024) formalized this relationship by developing an impact-based decision support forecasting tool for surface transportation hazards, systematically linking precipitation type, local hour of day, and visibility levels to crash probability and roadway restriction thresholds. Their framework proposes severity classifications that can scale weather forecast information directly into actionable transport guidance — a model that demonstrates how meteorological data can be embedded into operational decision logic rather than treated as background information.

The behavioral dimension of weather-aware transportation is equally significant. Hussein et al. (2021) used speed data from Weigh-in-Motion (WIM) sensors across multiple regions in Ohio, combined with weather observations, to develop a probabilistic trip-planning model for snowstorm conditions. Their findings reveal that under heavy snow with low visibility, drivers do not simply select the shortest path — they shift their decision criteria toward alternate routes perceived as safer or more likely to be cleared. Some drivers reduced speeds to as low as 20 mph and showed measurable hesitation behaviors during storm events. This study confirms that weather-aware decision support must model driver behavior probabilistically, not deterministically, and must account for route attractiveness factors such as road clearance status and terrain, not just travel time.

At the systems level, Čelar et al. (2024) reviewed how Intelligent Transportation Systems (ITS) mitigate adverse weather impacts on traffic safety and efficiency. Their review documents that ITS applications — including variable message signs,

automated speed advisories, and weather-responsive signal controls — demonstrably reduce accident rates and congestion during inclement weather. Critically, they identify a persistent gap: most deployed ITS solutions remain reactive, responding to weather events after they occur, and few integrate multi-source meteorological and traffic data into truly predictive, proactive frameworks.

Pradeeppraja et al. (2025) addressed this gap with WeatherSync-ITS, a heterogeneous data fusion system combining real-time meteorological department feeds, traffic sensor infrastructure, and incident reporting using a Space-Temporal Graph Attention Network (ST-GAT). The system achieved 92% prediction accuracy for traffic and incident events and provided weather-conditioned route optimization that delivered 15–25% travel time improvements over classical shortest-path algorithms. Its frontend, built with React for visualization and supported by a Node.js/TypeScript API backend, also represents an example of how weather-responsive systems can present actionable recommendations through dashboards and real-time alerts.

Data fusion between weather and traffic streams is a recurring theme in this literature. Nasser and Simon (2025), publishing in the *Journal of Advanced Transportation*, presented the MI-SMHA (multi-input sequential multihead attention) model, which integrates weather variables — temperature, wind speed, precipitation, visibility, and humidity — directly into traffic prediction tasks using attention mechanisms drawn from sequential modeling. The model outperformed several state-of-the-art baselines and demonstrated that weather conditions are not merely noise in traffic data but structured, predictable contributors to traffic pattern shifts.

Similarly, the SmartFlow AI system (2026) integrates live OpenWeatherMap data with OSRM routing and Folium map visualization in a Streamlit-based web interface, automatically recommending alternative routes when high congestion is

predicted from combined weather and traffic inputs. The system's design illustrates how weather-aware logic can be delivered to end users through interactive web-based dashboards — a UI/UX consideration relevant to how recommendation results and alerts should be presented in context-sensitive transportation platforms.

Collectively, these studies establish that effective weather-aware transportation decision support requires:

- (1) probabilistic modeling of weather impacts on driver behavior and road conditions,
- (2) real-time fusion of meteorological and traffic data streams,
- (3) proactive rather than reactive recommendation logic, and
- (4) accessible visual interfaces — dashboards, route maps, and alerts — that translate complex data into actionable guidance.

- **Weather Data Sources and Weather API Use in Transportation Systems**

Weather data has emerged as one of the most heavily used external data sources in modern transportation systems, with a remarkably standardized pattern of API consumption across the recent literature. The dominant source by a clear margin is OpenWeatherMap (OWM), with Open-Meteo and several other providers serving complementary roles. The literature documents both the technical architectures used to integrate these APIs and the operational benefits gained from real-time weather awareness.

Kolambe et al. (2025), publishing in the *International Journal of Environmental Science*, presented a representative reference implementation of OpenWeatherMap integration: a Django-based MVC architecture providing real-time access to temperature, humidity, wind speed, and precipitation data with multilingual support,

GPS-based location detection, and severe weather alert notifications. While the application targets general users rather than transportation operators, its architecture — a backend that consumes the OWM API, processes and stores the data, and presents both routine forecasts and severe-weather alerts through an interactive frontend — is the template that recurs across nearly every transportation-oriented application in the recent literature.

Sandhip et al. (2025) deployed this template in an intelligent traffic monitoring and autonomous navigation system, integrating MapBox for dynamic mapping and OpenWeatherMap for weather analysis to predict traffic congestion, monitor weather conditions, and provide alternative routes using real-time data. Importantly, the system also includes an automated incident detection mechanism (a Raspberry Pi camera module) and pushes instant alerts to users — illustrating how weather APIs are combined with other real-time data streams to produce comprehensive context-aware navigation.

Priyadarshini et al. (2025) presented the Integrated Weather-Aware Route Optimisation Dashboard (IWARD), explicitly addressing the problem that Google Maps and AccuWeather typically work as separate systems, forcing users to manually cross-reference weather forecasts against travel plans. IWARD merges real-time route planning with up-to-date weather forecasts using API-driven live data, historical patterns, and machine learning techniques including Random Forest, Gradient Boosting, and Reinforcement Learning to predict localized weather and adjust travel routes dynamically. This work directly motivates the integrated approach pursued in this study.

The dominance of OpenWeatherMap is reflected in further examples across the recent literature: Bajad (2024) used the OpenWeather API with various machine learning algorithms for rainfall prediction with a user-friendly interface, demonstrating its accessibility for student and developer projects. Khakim and Astutik (2026) integrated OpenWeatherMap with IoT-based irrigation systems, showing that the API's range of use cases extends from transportation to agriculture and that its JSON deserialization workflow has become a de facto standard.

Two additional studies illustrate the broader use of weather APIs in transportation. Pradeepraja et al. (2025) integrated the Indian Meteorological Department's weather feeds with traffic sensor infrastructure in the WeatherSync-ITS framework, achieving 92% traffic prediction accuracy and 15–25% travel time improvement over classical shortest-path algorithms — demonstrating that national meteorological department feeds, where available, serve as a higher-fidelity alternative to commercial APIs for region-specific deployments. Bhuvanraj et al. (2026) presented the Weather-Resilient Road Safety Vision System, which uses YOLOv8 deep learning for pothole, speed-bump, lane, and road-sign detection while incorporating diverse weather conditions into model training; the system surfaces detection results through a Streamlit-and-Plotly interactive dashboard with grade heatmaps and is designed for smart city ITS integration.

A clear practical pattern emerges from this literature: transportation systems consume weather APIs through standardized JSON-over-HTTPS interfaces, retrieve a common set of parameters (temperature, humidity, wind speed, precipitation, visibility),

apply analytical or machine learning logic to relate these to routing and safety decisions, and present results to users through dashboards, map overlays, and alerts. The data sources are mature, the integration patterns are well established, and the question for any new transportation decision-support system is not whether to use weather data, but which provider best fits the application's geographic scope, data resolution, refresh rate, and licensing constraints.

- **API Integration and Context-Aware Decision-Support Systems**

The practical delivery of weather-aware, context-sensitive transportation intelligence depends heavily on Application Programming Interface (API) integration — the technical mechanism through which disparate real-time data sources are accessed, combined, and acted upon within a unified system. APIs allow systems to query live weather feeds, retrieve current traffic conditions, access map and routing services, and incorporate road condition data without owning the underlying data infrastructure. The literature confirms that multi-source API integration is now foundational to modern intelligent transportation and travel-planning systems.

Penumala and Naganaboina (2025) demonstrated this paradigm by building a context-aware navigation system that integrates OpenStreetMap (OSM) APIs with real-time environmental APIs — covering air quality index, temperature, wind speed, rainfall forecasts, and cloudiness — alongside points-of-interest APIs for hotels, restaurants, fuel stations, and transit stops. The unified platform adjusts route recommendations and travel insights based on both user preferences and live environmental conditions, providing a clear example of how multi-source API integration produces richer, more contextually relevant navigation experiences than systems relying on a single data stream. Their work underscores that the value of

context-aware decision support lies not in any single data source but in the seamless orchestration of multiple APIs.

Sandhip et al. (2025) presented a traffic monitoring and navigation system that integrates MapBox (for dynamic mapping) and OpenWeatherMap APIs (for weather analysis) to predict congestion, monitor weather conditions, and provide alternative route recommendations in real time. The system adds an IoT layer — a camera-enabled Raspberry Pi module for traffic anomaly detection — and sends automated incident alerts to users via cloud-based integration. The resulting architecture illustrates the convergence of weather APIs, mapping APIs, and IoT sensors within a single decision-support pipeline, and its alert notification interface provides a reference model for how transportation systems should surface critical information to users.

Niroshan et al. (2025), publishing in *MethodsX*, designed a unified data-sharing API that integrates real-time air quality, noise, and traffic monitoring data from diverse sensor networks to support urban governance across four planning domains: user demand, transport, freight and logistics, and city infrastructure. Unlike frameworks that address environmental or traffic monitoring separately, their API unifies multi-source data with real-time processing, enabling data-driven decisions across overlapping urban planning needs. This architecture demonstrates a scalable approach to multi-domain context awareness where a single API layer can serve multiple decision-support applications simultaneously.

Joshna et al. (2026) extended API integration to full travel planning by combining location-based search, distance calculation, transportation availability, and weather forecasting APIs into a real-time intelligent travel planning and cost management system. The system generates personalized itineraries, adjusts plans in

response to weather-triggered route changes, and supports group travel cost sharing — all through RESTful API architecture. Their evaluation found significantly better planning efficiency and cost transparency compared to traditional multi-platform approaches, and the system's modular design illustrates how API-driven context awareness can scale from individual trip planning to fleet-level operational decision support.

Vdovic et al. (2021), published in the International Conference on Telecommunications, described an API for advanced analytics of contextually enriched automotive data — combining vehicle-generated telematics with location, weather, and traffic context stored in big data platforms. Their two demonstrated use cases — contextually enriched data visualization and eco-efficient driving pattern evaluation — show how weather and traffic APIs can augment raw vehicle data to support both operational recommendations and sustainability analysis.

Across these studies, a clear architectural pattern emerges: effective context-aware decision-support systems commonly rely on API integration, where weather data, traffic data, road condition data, and mapping or routing data are accessed through external services and processed within a unified system. These systems commonly present contextual information through dashboards, maps, alerts, and recommendation interfaces to support operational decision-making.

In relation to the proposed system, context-aware decision support will not be implemented as autonomous environmental analysis, advanced artificial intelligence, or complex route optimization. Instead, selected contextual information such as weather condition, road condition, route feasibility, route accessibility, travel distance, reference fuel efficiency, and estimated fuel consumption will serve as supporting information for administrative vehicle-assignment and branch-allocation decisions. These contextual

inputs will supplement the system's primary internal operational information, including forecasted demand, projected vehicle supply, vehicle availability, and maintenance readiness, and will not independently create a branch shortage or surplus or automatically determine the final operational action. The Customer-Side Vehicle Recommendation will remain a separate customer-facing function based primarily on customer rental requirements, vehicle eligibility, availability, maintenance readiness, and deterministic ranking rules. Contextual information may be obtained from available external data sources, predefined operational references, or authorized manual input according to the finalized implementation and fallback strategy.

- **Comparison of API and Data Sources for Context-Aware Inputs**

A defining characteristic of context-aware transportation systems is their reliance on multiple external APIs for different categories of data: weather, traffic, mapping and routing, road conditions, and location services. The reviewed literature does not present formal head-to-head benchmarks of commercial APIs, but it does reveal a clear pattern of API selection and combination that allows a comparative analysis of the practical strengths and trade-offs of the available data sources.

For weather data, the comparison is between commercial and open-source providers. OpenWeatherMap dominates the literature (Sandhip et al., 2025; Priyadarshini et al., 2025; Kolambe et al., 2025; Bajad, 2024; Khakim & Astutik, 2026), valued for its global coverage, well-documented JSON API, multilingual support, and free tier that is generous enough for student and prototype projects. Open-Meteo is the principal alternative (Jeyabharathi et al., 2026), preferred when applications require fully open, no-API-key access, high-resolution forecasts, and transparent data

provenance. National meteorological department feeds (Pradeeppraja et al., 2025) provide the highest fidelity for region-specific deployments, particularly where local terrain or monsoon patterns are critical, but they typically require institutional access agreements and are less portable across geographies. The trade-off across these options is convenience versus fidelity: OpenWeatherMap maximizes ease of integration; national feeds maximize local accuracy; Open-Meteo balances both for open-data use cases.

For mapping and routing, the literature documents three primary choices. OpenStreetMap (OSM) and its derivatives — particularly OSRM (Open Source Routing Machine) and OSM-based geocoding via Nominatim — are widely adopted for routing-heavy applications (SmartFlow AI, 2026; Penumala & Naganaboina, 2025; Jeyabharathi et al., 2026). OSM's strength is openness — no API key restrictions, no per-query costs, and full data transparency — but its routing quality depends on local map completeness, which varies by region. MapBox is selected for applications requiring polished, interactive mapping with strong visualization capabilities (Sandhip et al., 2025), balancing commercial reliability with developer-friendly APIs. Google Maps API (via the Distance Matrix and Google Maps APIs) appears in transit and location-tracking applications (Suresh et al., 2023), valued for its real-time traffic information, comprehensive POI coverage, and global geocoding accuracy — but constrained by usage quotas and per-query pricing for production deployments.

For road condition data, the comparison is between centrally curated and crowdsourced sources. Government and agency datasets — such as those derived from inertial profilers and dedicated survey vehicles — remain the gold standard for

accuracy but suffer from limited coverage and infrequent updates (typically annual or biennial). Crowdsourced smartphone sensing platforms (Dong & Li, 2021; Sattar, 2021; Lyu et al., 2024; Gamage et al., 2022) trade some per-measurement accuracy for vastly greater coverage and refresh rate, achieving 80–84% defect detection accuracy from single-driver data and 95%+ accuracy with multi-driver probabilistic integration.

For traffic and contextual data, the literature reveals an emerging pattern of unified, multi-source platforms rather than single-API solutions. Niroshan et al. (2025) demonstrated a single API layer integrating real-time air quality, noise, and traffic monitoring from diverse sensor networks. Vdovic et al. (2021) showed how vehicle-generated telematics data can be contextually enriched with weather, location, and traffic data through a unified API. These approaches reflect a key insight: for context-aware decision support, the comparative question is no longer "which single API is best?" but rather "what combination of APIs provides the most complete and reliable context, and how should they be orchestrated?"

The reviewed literature demonstrates that context-aware transportation systems may rely on combinations of weather, mapping, routing, traffic, incident, and other external data sources rather than on a single service. Weather APIs can provide meteorological information that supports transportation assessment, while mapping and routing services can provide location, travel-distance, travel-time, and route-related information. Traffic- and incident-related services may further provide information concerning road closures, restrictions, road works, flooding, and other disruptions that affect route movement. The reviewed studies also show that the suitability of an external data source depends on factors such as geographic coverage, available data,

service accessibility, implementation requirements, and reliability. These findings provide the basis for adopting primary and fallback external data providers for the proposed system. The specific API providers, fallback mechanisms, normalization rules, and implementation procedures selected for the study are presented in Chapter 3.

D. Related Systems and Research Gaps

- **Related Car Rental, Fleet Management, and Transportation Systems**

Existing car rental and fleet management systems have evolved significantly in their operational capabilities, yet a consistent pattern emerges across the reviewed literature: these systems excel at administrative and logistical functions while largely neglecting the contextual intelligence — weather awareness, road condition monitoring, and real-time environmental data — that would make them genuinely decision-supportive for vehicle selection and deployment.

The Smart Car Rental Management System reviewed by M.A. et al. (2023) represents a comprehensive operational platform covering customer registration and authentication, vehicle inventory management, reservation handling, rental duration tracking, payment processing, and reporting analytics. It incorporates data mining for demand forecasting and fleet utilization optimization. However, the system's vehicle recommendation logic is driven by inventory availability and pricing — not by vehicle suitability for current road or weather conditions. A customer renting a vehicle during a typhoon or flood event receives the same recommendation workflow as one renting on a clear day.

The Online Car Rental Management System described by Surabhi (2025) adds GPS tracking, dynamic pricing, real-time vehicle availability, and automated

notifications, and presents these through a user-friendly web interface. While the GPS component enables location tracking, it is used for fleet management rather than route feasibility assessment or weather-adapted dispatching. The system's analytics cover rental patterns and performance metrics but do not integrate environmental data that would allow operators to assess whether a given vehicle is appropriate for prevailing road conditions.

Dhami et al. (2024), reviewing online vehicle rental management systems, document the industry-wide shift from manual processes to digital platforms, emphasizing improvements in operational efficiency, customer access, and transaction accuracy. Their analysis, however, notes that the literature on rental system design gives little attention to contextual intelligence — the capacity to match vehicle type, route, and departure timing to real-world environmental conditions. This gap is characterized as a structural limitation of platforms designed primarily around booking workflow optimization rather than transportation quality.

Adishesha et al. (2026), in a comprehensive review of online car rental systems published in the *International Research Journal of Modernization in Engineering Technology and Science*, similarly identify that existing platforms prioritize user convenience, e-commerce functionality, and customer satisfaction metrics, with few addressing vehicle-to-condition matching or safety advisory features. Their review notes that emerging trends in the field — including AI-driven demand forecasting and dynamic pricing — have not yet extended to weather- or road-condition-aware recommendations.

Routaray and Dileep Kumar (2024) presented Auto Mobilize Pro, a car rental analytics solution using machine learning algorithms for demand forecasting, fleet utilization optimization, and maintenance prediction. While the system provides

financial analysis tools and real-time maintenance insights that reduce downtime, it remains focused on internal operational performance. Environmental context — weather conditions, road accessibility, or route feasibility — is absent from its recommendation engine.

Beyond car rental, the transportation and fleet management literature presents systems of increasing sophistication. Meira et al. (2022) reviewed decision support systems for fleet management and routing, covering the Vehicle Routing Problem and the Travelling Salesman Problem, and prototyped a DSS for beer delivery fleet management. Their work reflects the state of fleet optimization systems: strong in computational route efficiency, but disconnected from real-time weather and road condition inputs.

Heidari et al. (2024), publishing in *Kybernetes* (a Q1 journal), proposed a resilient vehicle routing DSS using a multi-layer perceptron neural network for disruption prediction, k-means clustering for customer grouping, and mixed-integer programming for route planning. The system explicitly addresses resilience under uncertainty, including dynamic replanning — a significant capability — but its disruption model focuses on logistical uncertainty rather than environmental conditions such as weather events or road closures.

In terms of UI/UX presentation, the reviewed systems commonly feature web-based dashboards displaying fleet status, booking queues, and performance analytics; tabular reports on revenue and utilization; and GPS map views for vehicle location tracking. Alert systems, where present, are primarily operational (maintenance due, booking confirmation) rather than safety- or condition-driven. None of the reviewed car rental systems present weather overlays on maps, condition-based vehicle suitability indicators, or route feasibility alerts — design elements that would

meaningfully differentiate a context-aware transportation decision-support system from a standard rental management platform.

- **Comparison of Existing Car Rental and Fleet Management Systems**

Existing car rental systems usually focus on digitizing basic rental operations such as customer registration, vehicle browsing, reservation scheduling, booking management, payment processing, and customer communication. These features help reduce manual work and allow customers to access rental services more conveniently. However, many car rental systems are still mainly transaction-oriented and provide limited support for demand forecasting, branch-level allocation, vehicle utilization analysis, and context-aware recommendation.

Wan Abdul Rahman and Mohd Rusli (2023) developed a web-based car rental business management system, showing that web-based car rental management is already an established area of system development research. However, the available description does not confirm whether the system includes GPS integration, demand forecasting, maintenance monitoring, or decision-support features. This indicates that some existing car rental systems address digital transaction management but do not fully explain how the system supports analytical planning or multi-branch fleet decisions.

Atanasio, Shibghatullah, and Islam (2022) developed a mobile application for fleet vehicle tracking. This shows that fleet tracking is also studied as a separate system category. However, tracking-focused applications usually center on vehicle location and monitoring rather than complete rental workflows. Such systems may not

include customer verification, payment validation, rental agreements, booking approval, return settlement, vehicle recommendation, or branch allocation. This supports the distinction between vehicle tracking systems and car rental management systems. A tracking application may help locate vehicles, but it does not automatically solve fragmented booking records, payment proof checking, maintenance tracking, and branch-level reporting unless it is connected to the core management platform.

Akbiyik et al. (2024) developed a decision support system for internal logistics operations management. Their system combined vehicle tracking, demand forecasting, and predictive demand-vehicle assignment. Although the study was conducted in an internal logistics context rather than a car rental business, it provides strong support for integrating tracking, forecasting, and assignment logic in fleet-related operations. This is relevant to the proposed car rental system because vehicle assignment and branch allocation also require the use of demand, availability, vehicle suitability, and operational constraints.

Mavrin and Makarova (2026) developed a decision support system for waste transportation using simulation and vehicle competitiveness assessment. Their study showed that fleet-related decision support can improve operational planning and reduce logistics costs. Although the domain is different from car rental, the study supports the value of structured decision-support systems in fleet management. It suggests that vehicle selection, assignment, and allocation can be improved when supported by data, criteria, and systematic recommendation logic.

The comparison of existing systems shows that car rental systems, vehicle tracking applications, and fleet decision-support systems often exist as separate categories. Car rental systems commonly manage reservations and transactions. GPS tracking systems focus on vehicle location. Fleet decision-support systems focus on forecasting, assignment, simulation, or optimization. The research gap is the need for a practical small-scale car rental system that centralizes rental operations while also providing selected decision-support features.

The proposed system addresses this gap by combining centralized customer records, vehicle records, booking requests, payment validation, maintenance monitoring, operational reports, customer-side vehicle recommendation, demand forecasting, and branch allocation recommendation. Unlike standalone GPS tools, the proposed system does not directly integrate live tracking from AKSH GPS, Apple Find My, or AirTags. Instead, it treats these as external monitoring tools while centralizing the operational records needed for decision-making. Unlike basic booking systems, it includes forecasting and rule-based recommendation logic. Unlike complex logistics optimization systems, it remains practical for a small-scale multi-branch car rental business by using interpretable and feasible methods such as Weighted Moving Average and rule-based decision criteria.

Overall, the reviewed studies support the development of a web-based car rental management and decision-support system that is more than a reservation platform. The system is positioned as a practical tool for improving record centralization, customer booking access, vehicle availability monitoring, maintenance tracking, customer-side vehicle recommendation, forecasting, branch allocation

support, and operational decision-making for small-scale multi-branch car rental businesses.

- **Limitations and Research Gaps in Existing Systems**

Lack of weather integration in vehicle recommendation logic. Existing car rental and fleet management systems do not incorporate weather data into vehicle selection or deployment recommendations. Customers are matched to vehicles based on availability and pricing, not on whether those vehicles are appropriate for prevailing or forecasted weather conditions. This represents a safety gap with direct consequences for service reliability during adverse weather events.

Absence of road condition awareness. While road condition monitoring technologies exist in the literature — including IoT-based sensor systems, GPS/accelerometer-based roughness detection, and V2X communication frameworks — none of the reviewed car rental or fleet management systems incorporate road condition data into their operational decision logic. Route feasibility is not assessed before vehicle dispatch or rental confirmation.

Reactive rather than proactive decision support. Most existing systems respond to conditions after they have developed: congestion is rerouted after it forms, incidents are logged after they occur, bookings are processed without environmental pre-screening. The literature on intelligent transportation systems consistently identifies this reactivity as a key limitation, and proactive, predictive frameworks represent an active research frontier.

Siloed data architectures. Weather data, traffic data, road condition data, and rental/fleet management data are maintained in separate systems with limited integration. The API integration literature demonstrates that unified multi-source

architectures are technically feasible, but existing rental and fleet platforms have not adopted this approach. Decision-makers must consult multiple platforms — weather apps, navigation apps, rental dashboards — to form a complete operational picture.

Limited contextual intelligence in recommendation engines. Vehicle recommendation in existing systems is inventory-driven. The literature on context-aware navigation and route recommendation systems (Penumala & Naganaboina, 2025; Nasution et al., 2021) demonstrates that recommendation engines can incorporate vehicle type, road characteristics, weather conditions, and traffic patterns into a unified scoring model — but this capability has not been applied to the car rental domain.

Insufficient UI integration of contextual data. Related systems present operational data through dashboards and reports, but weather and road condition information — when available at all — is displayed in separate interfaces rather than integrated into the recommendation workflow. Effective decision support requires that contextual information be surfaced at the point of decision, not retrieved separately.

- **Gaps in Existing Systems Regarding Forecasting, Maintenance, and Context-Aware Allocation**

While modern car rental, fleet management, and transportation systems have advanced significantly in functional scope, the recent literature points to three persistent and interrelated gaps that limit their effectiveness as truly intelligent decision-support tools: weaknesses in demand forecasting, fragmented predictive maintenance integration, and the near-absence of context-aware vehicle allocation. These gaps are particularly relevant to the present study because they describe

precisely the capability shortfall that a weather-, road-, and fuel-aware vehicle recommendation system is designed to address.

Demand forecasting gaps. Demand forecasting in the car rental and shared-mobility sectors remains far less mature than in adjacent industries such as airlines and hospitality. Ukandu and Kalesanwo (2025) reported that approximately 50% of car rental companies experience demand forecasting inaccuracies severe enough to depress fleet utilization to 70–75%, well below the 85–90% optimal level. Their integrated customer segmentation and forecasting framework — combining clustering algorithms (DBSCAN, Agglomerative, Fuzzy-C-Means) with ARIMA, regression, and Holt–Winters models — achieved a Mean Absolute Error of only 29.36, but the authors note that most operational car rental systems still rely on fixed pricing charts, staff intuition, and broad seasonal assumptions rather than data-driven prediction. Yang et al. (2021) similarly observed that multitype car rental demand forecasting is hampered by the high probability of vehicle upgrading/downgrading, customer subjective preferences, and the need to repair constrained historical demand before forecasting — challenges that single-method forecasting models cannot adequately address. ShivajiRao et al. (2024), in a multi-orthogonal review of urban mobility demand forecasting, concluded that while machine learning, deep learning, and even quantum learning methods have been applied to demand prediction, prevailing approaches suffer from computational limitations and a persistent gap between forecasting research and operational deployment. The most recent applied work — Suganya et al.'s (2026) RentOptima framework and Uday Kumar et al.'s (2026) dynamic pricing system — explicitly identifies the same gap: existing car rental systems

set prices and allocate fleets manually, without evaluating historical trends, regional variations, or multiple influencing factors simultaneously.

The dynamic and non-linear nature of mobility demand compounds these problems. G. S. et al. (2024) noted that demand for rides can be influenced by weather, special events, and unforeseen catastrophes — dynamic factors that conventional forecasting models, focused on seasonal and trend components, capture poorly. Alzaidi, Shakya, and Khargharia (2022) demonstrated that incorporating external inputs such as temperature and holidays into vehicle demand forecasting substantially improves accuracy, yet such external integration is rare in deployed rental systems. The gap is therefore not one of theoretical impossibility but of practical adoption: the methods exist in the research literature, but commercial rental and fleet systems have not yet absorbed them into their core recommendation logic.

Predictive maintenance gaps. A second persistent gap concerns predictive maintenance. Chen, Jia, and Zhou (2025), in a systematic review of vehicle maintenance demand prediction published in *Applied Sciences*, identified a principal challenge: while machine learning and deep learning models can accurately predict failures in specific components (suspensions, batteries), these approaches "predominantly operate in isolation," and the field has not successfully transitioned from single-component failure prediction to holistic Vehicle Health Management (VHM). Crespo del Castillo and Parlikad (2024), publishing in *Reliability Engineering & System Safety*, identified a more operational gap: predictive maintenance is rarely integrated with calendar-based preventive maintenance and operational workload allocation, despite the fact that these three functions interact strongly to determine fleet availability

and cost. Their optimization model demonstrated that integrating all three dimensions can substantially reduce fleet operating costs, but they explicitly note that this integration remains absent from most commercial fleet management systems.

Several recent studies confirm that the gap is being addressed in research but lags in practice. Chaudhuri and Ghosh (2024) developed a hybrid deep learning ensemble for vehicle fleet predictive maintenance using IoT data, and Sharma et al. (2025) built the DriveSure platform reporting 30–50% reductions in unplanned downtime through machine learning–based maintenance prediction. Nagy and Lakatos (2024), however, emphasized that while premium-segment vehicles increasingly support predictive maintenance through rich sensor and data structures, mass-market and rental-fleet vehicles still lack the sensor infrastructure to make predictive maintenance broadly practical. Kansal and Ediga (2024) and Mittal et al. (2024) demonstrated that IoT-enabled predictive maintenance is technically feasible and can produce customized maintenance schedules per vehicle, yet acknowledge that integration with broader fleet management decision systems remains limited. Priyanka et al. (2025) and Zayats et al. (2026) reinforced that scalable, end-to-end pipelines connecting predictive maintenance models to fleet operational decisions are still emerging rather than mature.

Context-aware vehicle allocation gaps. The third — and perhaps most directly relevant — gap is the limited use of contextual data (weather, road conditions, real-time traffic, route characteristics) in vehicle allocation and dispatch decisions. Xu et al. (2021), publishing in *IEEE Transactions on Intelligent Transportation Systems* (Q1), demonstrated that network-flow-based vehicle dispatch in ride-hailing systems can

significantly improve service rates, but their model and most successor models optimize based on supply and demand alone, without incorporating environmental conditions that affect feasibility and safety. Han et al. (2024), in their GARLIC framework presented at AAAI, explicitly identified the limitation that current vehicle dispatch systems "struggle to navigate the complexities of urban traffic dynamics, including unpredictable traffic conditions" — and proposed reinforcement learning with multi-view graphs to address this, but again without integrating weather or road-surface data. Daoud (2021, 2022) similarly explored multi-agent and decentralized vehicle allocation but framed the problem in terms of communication and coordination rather than environmental context.

A small but growing body of work explicitly recognizes the context-aware allocation gap. Hu et al. (2023) proposed the Road-aware Grid-based Multi-Agent Reinforcement Learning (RGMARL) approach, which incorporates road information into vehicle dispatching decisions and demonstrates a 5.75% improvement in order response rate over road-agnostic baselines — an explicit acknowledgement that contextual data improves allocation quality. Li, Parker, and Hao (2021) addressed stochastic travel times in on-demand ride-sharing dispatch, noting that "existing ride-sharing research seldom considers travel time uncertainty, which leads to inaccurate dispatch allocations." Wang et al. (2025) introduced "sink proximity" as a way to consider future network states in online dispatch but without environmental inputs. Potluri and Kumar (2021) advocated context-aware multi-data fusion for ride-share scheduling using big data techniques. Hssini (2025), publishing in IEEE Access (Q1), is among the few to combine context-aware allocation with robust optimization in a hybrid framework for disaster blood logistics — demonstrating that the

methodology can be made operational when the application domain demands it, but underscoring that mainstream rental and fleet systems have not adopted such approaches.

The IoT-driven fleet management literature similarly highlights what is missing. Farahpoor, Esparza, and Soriano (2024), publishing in IEEE Access, noted that traditional telematics systems exhibit "closed compatibility with specific brands, unique network protocols, and insufficient data analysis and decision-making support." Raut et al. (2025) reviewed real-time fleet monitoring systems and concluded that current systems face persistent challenges in latency, scalability, security, and — crucially — data integration across disparate sources. Rathnayaka et al. (2021), Babiola et al. (2023), Singaraju (2025), Anderson (2022), Lee et al. (2026), and Zahid et al. (2025) collectively demonstrate that real-time fleet management systems are now widespread, but they focus primarily on vehicle tracking, emissions monitoring, route optimization, and predictive maintenance — not on context-aware vehicle-to-trip recommendation that integrates weather, road conditions, terrain, and fuel projections. Abediasl et al. (2024) integrated emissions estimation into an intelligent fleet management system, illustrating that context-aware extensions are technically achievable when the development effort is invested. Brar, Kasture, and Su (2022) showed that EV rental fleets in particular suffer from supply-demand mismatch due to limited control over fleet distribution — a gap that contextual, predictive allocation could close. Sangaraju et al. (2024) and Alanazi and Almutairi (2026) reinforced that effective resource allocation and revenue generation for rental vehicles depends on integrating multiple data dimensions that current systems handle in isolation.

E. Synthesis of the Study

The literature reviewed in this chapter collectively establishes both the operational context and the theoretical foundations of the proposed web-based car rental management system with demand forecasting and context-aware fleet allocation for small-scale multi-branch businesses.

Studies on car rental operations and existing management challenges confirm that traditional and semi-manual rental practices create persistent inefficiencies — including manual booking delays, overlapping reservations, fragmented customer and vehicle records, and delayed transaction processing — that reduce operational reliability and limit the ability of owners to make timely decisions (Kumar & Singh, 2024; Dhami et al., 2024; Mhatre et al., 2026; Rahman & Mohd Rusli, 2023). These findings directly reflect the operational conditions of the target client, the participating car rental business, where reservations, payment verification, customer requirements, and vehicle availability are currently managed across separate devices, messaging platforms, and scheduling tools. The reviewed studies on web-based car rental management systems demonstrate that centralizing these functions into a single browser-accessible platform can improve record organization, transaction processing, vehicle monitoring, and overall operational efficiency (Ogbiti & Aaron, 2024; Bakale et al., 2023; Karche et al., 2025), which supports the proposed system's core reservation and rental management modules.

The challenge of managing vehicles across more than one location is addressed by studies on multi-branch operations and fleet coordination, which show that branch-level fleet imbalance — where one location experiences shortages while another has idle units — is a structural problem in rental and vehicle-sharing operations requiring more than basic booking records (Boysen et al., 2024; Eliyan & Kerbach,

2024; Brandizzi et al., 2022). For small-scale operators like the participating car rental business, this imbalance is typically managed through informal coordination, which introduces risk as reservation volumes grow. The literature on distributed car rental systems and fleet monitoring demonstrates that reservation validation, availability locking, and branch-level fleet visibility can reduce booking conflicts and improve vehicle distribution across locations (Saritha et al., 2026), which directly informs the proposed system's multi-branch fleet monitoring and allocation features.

Security, data privacy, and role-based access control are consistently identified as essential requirements in web-based systems that handle sensitive user records, payment information, uploaded documents, and administrative data. The reviewed studies show that Role-Based Access Control effectively restricts system functions based on assigned user roles, reduces the risk of unauthorized access, and improves data accountability across multi-user platforms (Romadhan & Mansur, 2026; Cardos et al., 2025; Purba & Hidayasari, 2026). Secure authentication frameworks using JSON Web Tokens, encrypted credential storage, and audit logging further strengthen access management and session security (Kanagamalliga et al., 2026; Dedhia, 2026; Manoj et al., 2025). These findings are directly relevant to the proposed system because the participating car rental business will handle customer-submitted documents, payment records, rental transactions, and operational reports. Therefore, secure authentication and restricted access are necessary to ensure that only authorized internal users can access sensitive records and management functions.

Fleet management literature confirms that idle vehicles, vehicle shortages, uneven demand distribution, and poor vehicle allocation are recurring problems in transportation and rental operations, and that demand forecasting using historical rental and usage data can help businesses anticipate where and when vehicles will be

needed rather than reacting after imbalances occur (Brandizzi et al., 2022; Favier et al., 2024; Sangaraju et al., 2024). Studies on demand forecasting in car rental and vehicle-sharing operations demonstrate that branch-level historical rental records, reservation patterns, and usage data can be analyzed to produce short-term demand estimates that support fleet planning and allocation decisions (Suganya et al., 2026; Zhao et al., 2021; Martin et al., 2021; Huang et al., 2021). The Weighted Moving Average method is validated across multiple small-scale and operational contexts as a practical, interpretable, and computationally lightweight forecasting approach that captures recent demand trends without requiring complex infrastructure (Rahayu et al., 2022; Sariati et al., 2026; Paramitha & Saifuddin, 2023; Khalid, 2024). These studies collectively support the proposed system's WMA-based demand forecasting module, which will estimate short-term vehicle demand per branch from historical booking records to help managers anticipate demand and prepare vehicle availability in advance.

Beyond forecasting, rule-based decision support systems provide the logical mechanism for translating demand estimates into concrete vehicle allocation recommendations. Studies on rule-based systems in transportation and fleet management show that encoding expert operational knowledge into explicit IF-THEN conditional logic produces consistent, auditable, and actionable recommendations without requiring advanced optimization algorithms (Haber et al., 2026; Mavrin & Makarova, 2026; Meira et al., 2022; Mohamad et al., 2026). These findings support the proposed system's rule-based fleet allocation feature, which will apply predefined allocation rules — based on forecasted demand, vehicle availability, branch assignment, and operational conditions — to generate vehicle distribution recommendations that guide rather than replace managerial judgment.

Context-aware factors further strengthen the administrative decision-support capability of the proposed system. Studies on fuel efficiency and transportation planning demonstrate that travel distance and fuel-consumption estimates can provide useful supporting information when evaluating vehicle and fleet movements, while studies on road conditions, route feasibility, and accessibility show that road incidents, closures, and other operational restrictions may affect whether a proposed movement remains practical (Vyas et al., 2022; Moawad et al., 2024; Liu et al., 2022; Dubey et al., 2025; Gabbar et al., 2023). Weather-aware transportation studies likewise show that precipitation, visibility, and other adverse weather conditions may support operational review when incorporated as advisory information (Tobin et al., 2024; Hussein et al., 2021; Celar et al., 2024; Pradeeppraja et al., 2025). API integration makes these contextual inputs technically feasible by allowing external weather, routing, and incident-related information to be obtained through web-based services (Penumala & Naganaboina, 2025; Sandhip et al., 2025; Joshna et al., 2026; Vdovic et al., 2021). In the proposed system, these contextual factors will support Owner/Admin review of vehicle-assignment and branch-allocation decisions and will remain separate from the primary Customer-Side Vehicle Recommendation logic. Missing contextual information will not prevent the system from performing its available internal demand, supply, vehicle-availability, and maintenance-readiness analyses and will not be silently interpreted as a favorable condition.

The reviewed systems also show the operational value of notifications and alert mechanisms in improving user awareness and reducing missed updates. In the proposed system, alerts for booking status, payment reminders, verification results, return schedules, maintenance reminders, and low vehicle availability can support more timely coordination among customers and authorized internal users.

The reviewed car rental systems provide a basis for transaction management, customer registration, booking processing, and reporting (M.A. et al., 2023; Surabhi, 2025; Dhami et al., 2024; Adishesha et al., 2026; Routaray & Dileep Kumar, 2024). Related decision-support work provides a basis for examining explicit allocation rules and contextual operational information (Meira et al., 2022; Heidari et al., 2024). The contribution proposed in this study is their integration for the participating small-scale, multi-branch business: weekly booking-demand forecasts, transparent shortage and surplus calculations, eligible transfer candidates, and external context presented for administrative review. The study does not claim that these individual techniques are new or absent from all commercial rental systems. Its contribution will be assessed through implementation, traceable outputs, functional tests, and user evaluation within the stated scope.

The reviewed studies show that existing car rental systems generally address booking management, customer records, vehicle records, payment processing, and reporting. For this study, the implementation gap is the participating business's need for connected weekly demand forecasting, rule-based branch allocation, and context-informed administrative review. This gap is significant because small-scale multi-branch businesses require not only centralized records but also practical decision-support tools for anticipating demand, reducing idle vehicles, improving vehicle distribution, and considering operational factors such as fuel efficiency, weather conditions, road accessibility, possible road closures, and route feasibility. Therefore, the proposed system addresses this gap by integrating centralized rental management, Weighted Moving Average demand forecasting, rule-based fleet allocation recommendations, and selected context-aware factors within one web-based platform.

Finally, the literature on software quality evaluation using ISO/IEC 25010 establishes the framework as an appropriate basis for assessing the proposed system based on selected software quality characteristics. In this study, the relevant characteristics include functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety (Monteverde et al., 2023; Bondoc, 2023; Mendoz et al., 2025; Adyaputra et al., 2025; Islam et al., 2021; Ismail et al., 2025). Studies using multiple evaluation perspectives, involving both technical evaluators and actual users, further support the use of separate evaluation groups such as owners, administrators, staff, customers or potential renters, and IT evaluators, since each group interacts with different modules and functions of the proposed system. Findings from prior evaluations also highlight that security requires deliberate attention during development, especially because systems that handle customer information, uploaded requirements, payment records, and role-based access may become vulnerable when security is not considered from the early stages of design and implementation (Laudza & Sofyan, 2024).

Taken together, the reviewed literature provides a coherent and converging foundation for the proposed system. The operational challenges of small-scale multi-branch car rental businesses are well-documented, while the selected technical approaches, including web-based architecture, Weighted Moving Average forecasting, rule-based allocation, role-based access control, API-supported or predefined contextual data inputs, and ISO/IEC 25010-based evaluation, are supported by related literature. The identified research gap confirms the need for a unified platform that integrates car rental management, demand forecasting, and context-aware fleet allocation in a small-scale multi-branch setting, which the participating car rental business represents.

Chapter 3

METHODOLOGY

This chapter presents the methodologies, procedures, development tools, testing procedures, and evaluation methods to be used in the development of the proposed Web-based Car Rental Management System with demand forecasting and context-aware fleet allocation. It discusses the requirements analysis, system development methodology, process model, development tools, testing procedures, system requirements, and evaluation procedures that will guide the design, development, implementation, and assessment of the proposed system.

The methodologies and procedures presented in this chapter are intended to ensure that the proposed system is developed using structured software engineering practices and aligned with the operational requirements of the participating car rental business.

Requirements Analysis

This section presents the identified functional and non-functional requirements of the proposed Web-based Car Rental Management System. The requirements were derived from client interviews, workflow analysis, observation of current business processes, related literature, and system planning activities. These requirements served as the basis for the design, development, testing, and evaluation of the proposed system.

The following system requirements were identified:

R1 - The system shall provide a centralized reservation and rental management module for recording booking requests, customer details, rental schedules, payment status, and transaction history.

R2 - The system shall provide online booking request management that allows administrators to review, approve, reject, and monitor reservations while checking vehicle availability.

R3 - The system shall provide a requirement submission and verification module where customers can upload required documents and administrators can review verification status.

R4 - The system shall provide access to rental policies, agreements, penalties, and booking conditions for customer reference and administrative validation.

R5 - The system shall provide a vehicle and fleet management module for managing vehicle records, availability status, branch assignment, condition details, and maintenance information.

R6 - The system shall provide fleet monitoring features for tracking active rentals, idle vehicles, vehicle availability, and branch-level fleet status.

R7 - The system shall implement demand forecasting using a three-period Weighted Moving Average method with fixed weights of 0.50, 0.30, and 0.20 to estimate vehicle demand for up to three succeeding weekly periods per branch and vehicle category

based on qualifying historical booking records. The first forecasted week shall use the three most recent completed actual-demand observations, while succeeding forecast periods shall be generated recursively using the most recent available actual and forecasted values.

R8 - The system shall provide rule-based branch allocation recommendations for the Owner/Admin by comparing forecasted vehicle requirements with projected available vehicle supply per branch and vehicle category and by considering vehicle availability, maintenance readiness, idle-vehicle information, and applicable operational context. The generated recommendations shall be advisory and shall not automatically transfer or reassign vehicles.

R9 - The system shall provide analytical reports for vehicle utilization, idle vehicle detection, branch demand, forecasted vehicle shortage and surplus, booking records, payment records, maintenance information, and fleet performance to support operational monitoring and administrative decision-making.

R10 - **The system shall provide a Customer-Side Vehicle Recommendation feature that uses rule-based eligibility filtering and deterministic ranking based on passenger and large-luggage capacity, a stated total rental budget compared with a daily-rate reference estimate, rental duration, vehicle preference, availability, and maintenance readiness. An optional destination shall be retained for administrative review and shall not affect customer ranking.** The generated recommendations shall assist customers in vehicle selection before submitting a booking request and shall not constitute booking approval or guaranteed vehicle assignment.

Reservation Management	✓	✓							✓	✓		✓		✓
Booking Approval Management	✓	✓	✓	✓								✓	✓	
Requirement Submission & Verification			✓									✓	✓	✓
Rental Policy & Agreement Management				✓									✓	✓
Payment Record Management	✓								✓			✓	✓	✓
Rental Transaction Management	✓			✓		✓			✓			✓		✓
Vehicle Record Management		✓			✓	✓		✓		✓	✓			✓
Maintenance Monitoring					✓	✓		✓	✓	✓	✓	✓		✓
Fleet Monitoring Dashboard		✓			✓	✓		✓	✓		✓	✓		✓

Continuation of **Table 5. Requirement – Features Matrix**

Requirement Features	R 1	R 2	R 3	R 4	R 5	R 6	R 7	R 8	R 9	R 10	R 11	R 12	R 13	R 14
Demand Forecasting Module							✓	✓	✓					✓
Fleet Allocation Recommendation Module					✓	✓	✓	✓	✓		✓			✓

Customer-Side Vehicle Recommendation Module		✓			✓					✓				✓
Context-Aware Decision Support					✓	✓		✓			✓			✓
Reports & Analytics Dashboard	✓					✓	✓	✓	✓				✓	✓
Notification & Alert Management		✓	✓		✓	✓						✓		
Centralized Database, Backup & Recovery	✓		✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓

Table 5 shows the relationship between the identified system requirements and the proposed system features. Each feature was mapped to one or more requirements to ensure that the system directly addresses the operational problems identified in the study. The matrix also confirms that the proposed system is not limited to basic transaction processing, but includes fleet monitoring, analytical reporting, forecasting, rule-based recommendations, context-aware decision support, security, and centralized data management.

The requirements analysis establishes the foundation for the succeeding design specifications, including the use case diagrams, use case reports, activity diagrams, database schema, and data dictionary. It ensures that all system designs and development activities remain aligned with the actual needs of the target client and the objectives of the study.

Use Case Diagrams

Figure 4. General Use Case Diagram



Figure 4 illustrates the general use case diagram, showing the identified system use cases and their associations with the Owner/Admin, Operations Staff, and Customer/Renter actors.

Figure 5. Use Case Diagram

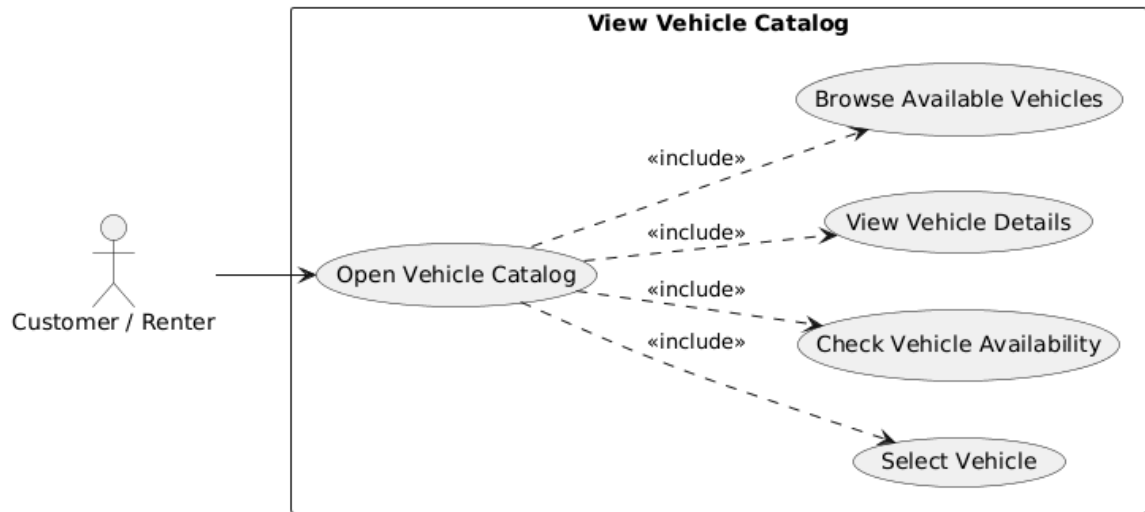


Figure 5 illustrates the view vehicle catalog use case, showing how the Customer/Renter opens the vehicle catalog, browses available vehicles, views vehicle details, checks vehicle availability, and selects a vehicle for consideration before submitting a booking request.

Figure 6. Use Case Diagram - Use Smart Vehicle Finder

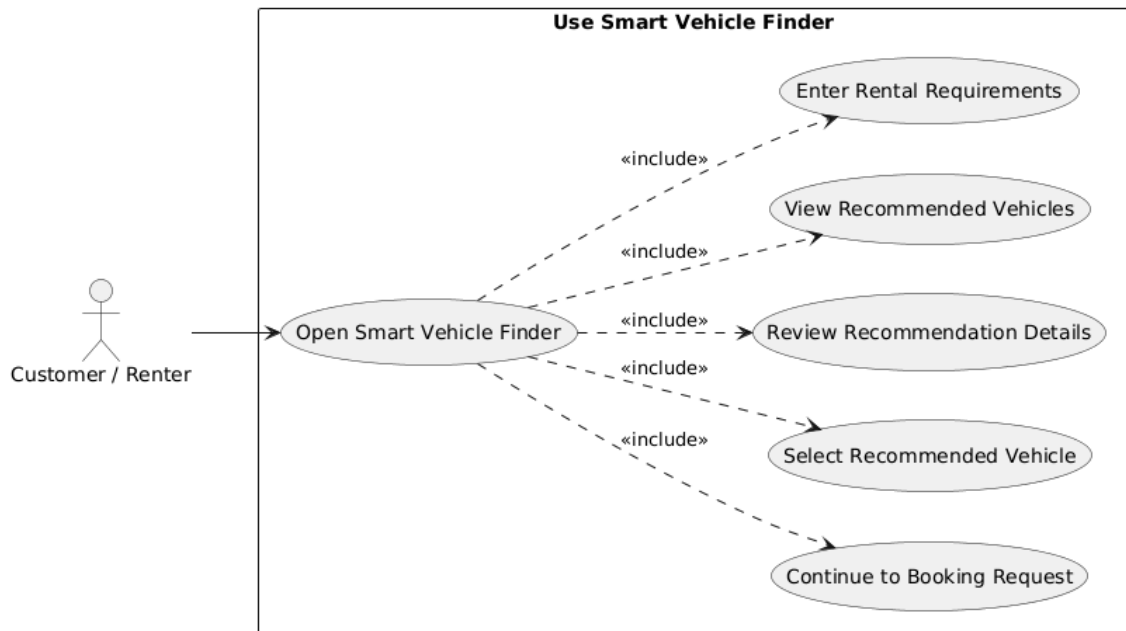


Figure 6 illustrates the use Smart Vehicle Finder use case, showing how the Customer/Renter enters rental requirements, reviews the recommended vehicles and their recommendation details, selects a suitable recommended vehicle, and continues to the booking request. The recommendation assists vehicle selection and does not constitute booking approval or guaranteed vehicle assignment.

Figure 7. Use Case Diagram - Register Account

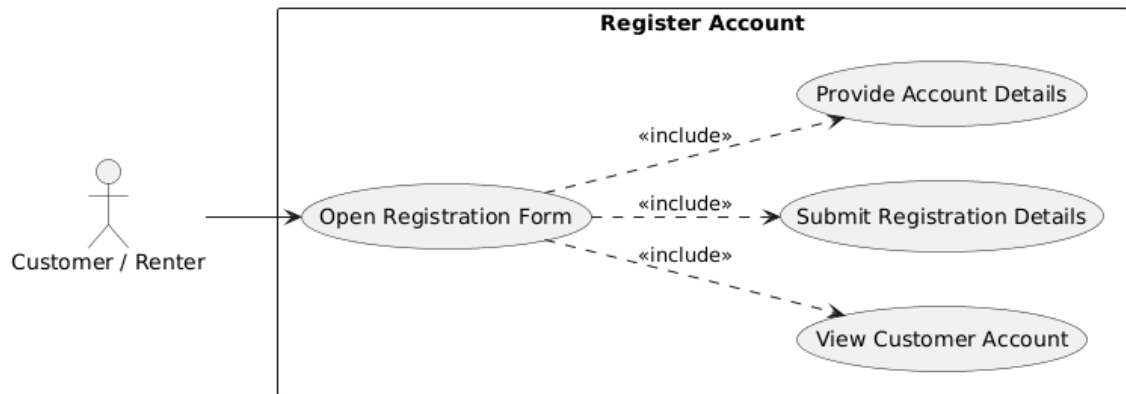


Figure 7 illustrates the register account use case, showing how the Customer/Renter opens the registration form, provides account details, submits the registration details, and accesses the customer account.

Figure 8. Use Case Diagram - Login

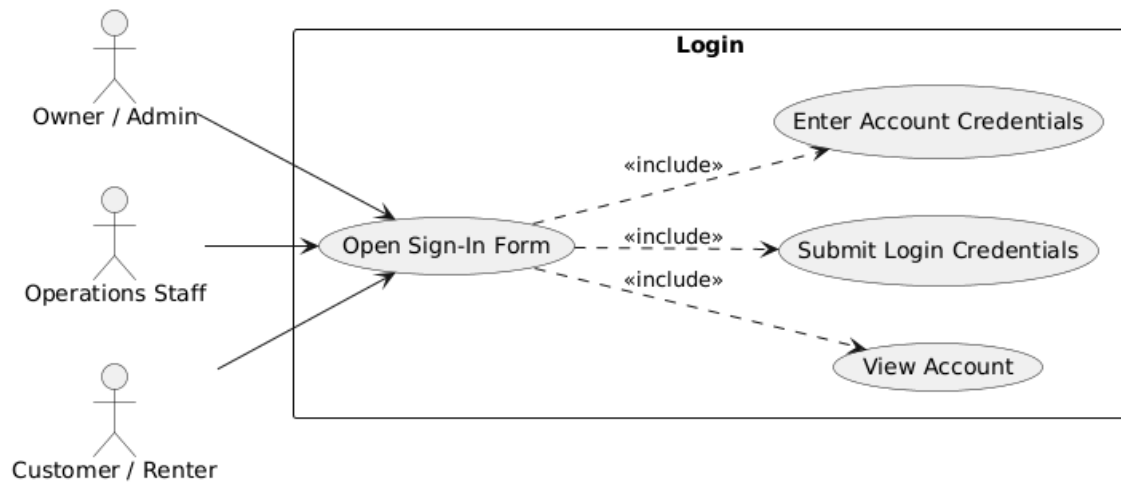


Figure 8 illustrates the login use case, showing how the Owner/Admin, Operations Staff, and Customer/Renter open the sign-in form, enter their account credentials, submit the login credentials, and access their respective accounts.

Figure 9. Use Case Diagram - Manage Reservations

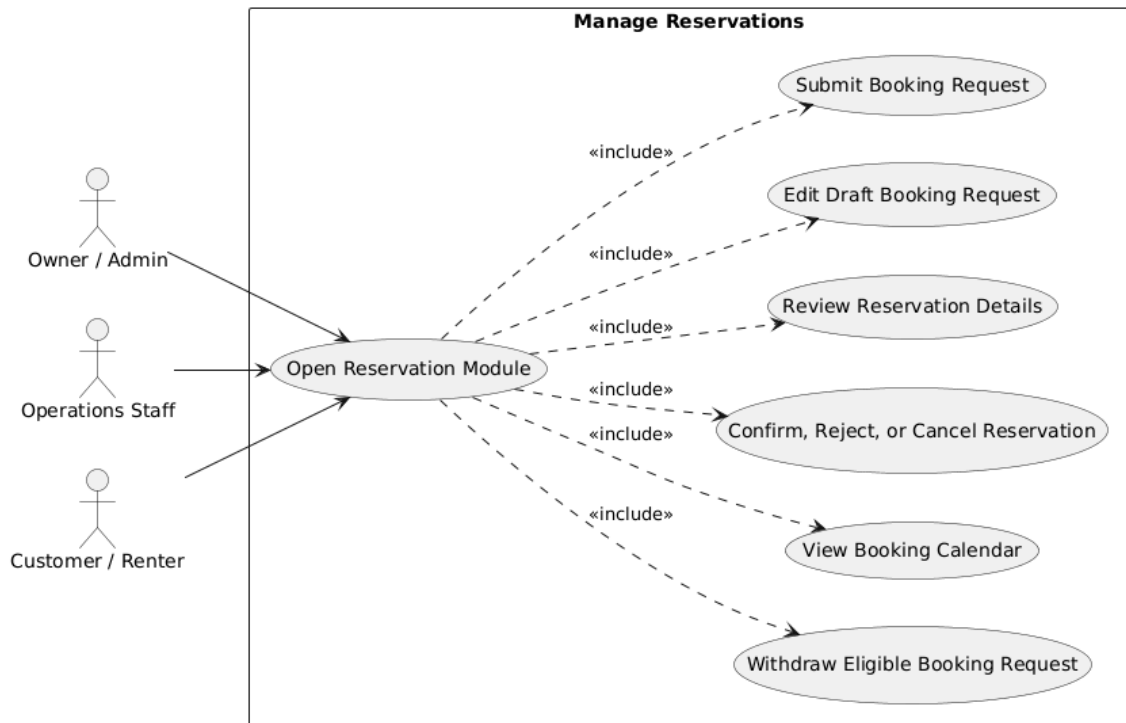


Figure 9 illustrates the manage reservations use case, showing how the Customer/Renter submits and manages eligible booking requests, while the Owner/Admin and Operations Staff review reservation information and booking schedules according to their permitted access. The Owner/Admin performs reservation confirmation, rejection, or cancellation decisions.

Figure 10. Use Case Diagram - Verify Customer Requirements

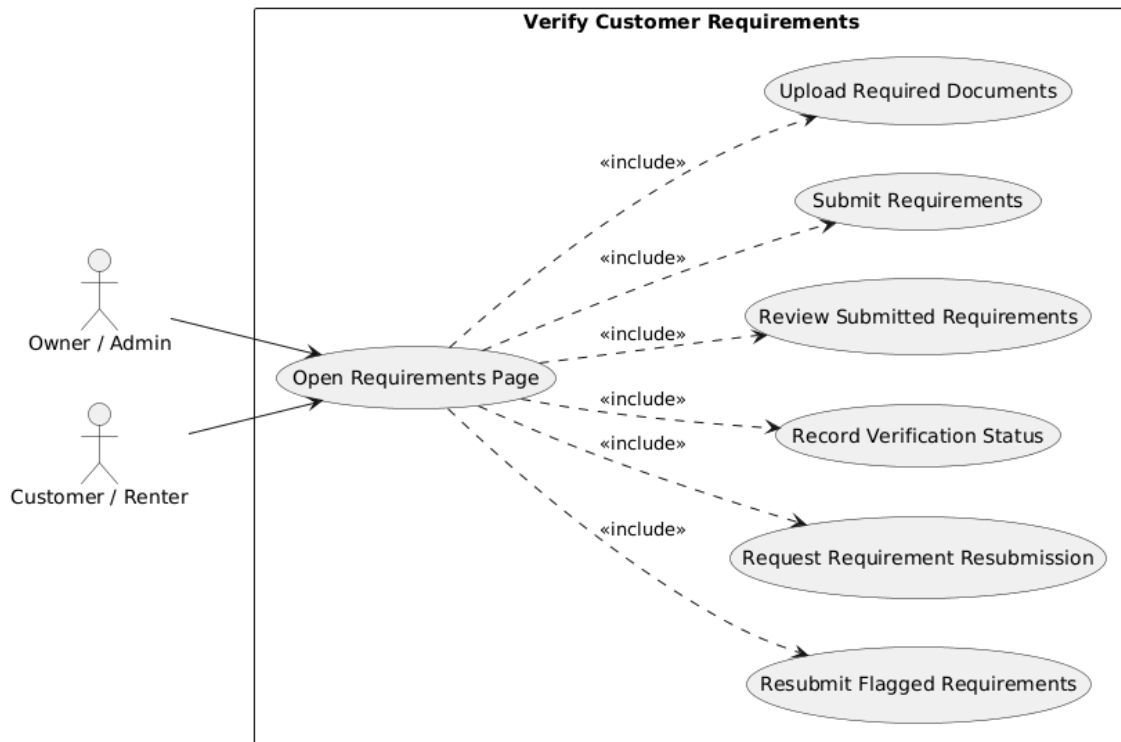


Figure 10 illustrates the verify customer requirements use case, showing how the Customer/Renter uploads and submits required documents, while the Owner/Admin reviews the submitted requirements and records the verification result. When corrections are required, the Owner/Admin requests resubmission and the Customer/Renter resubmits the flagged requirements.

Figure 11. Use Case Diagram - Verify Payment

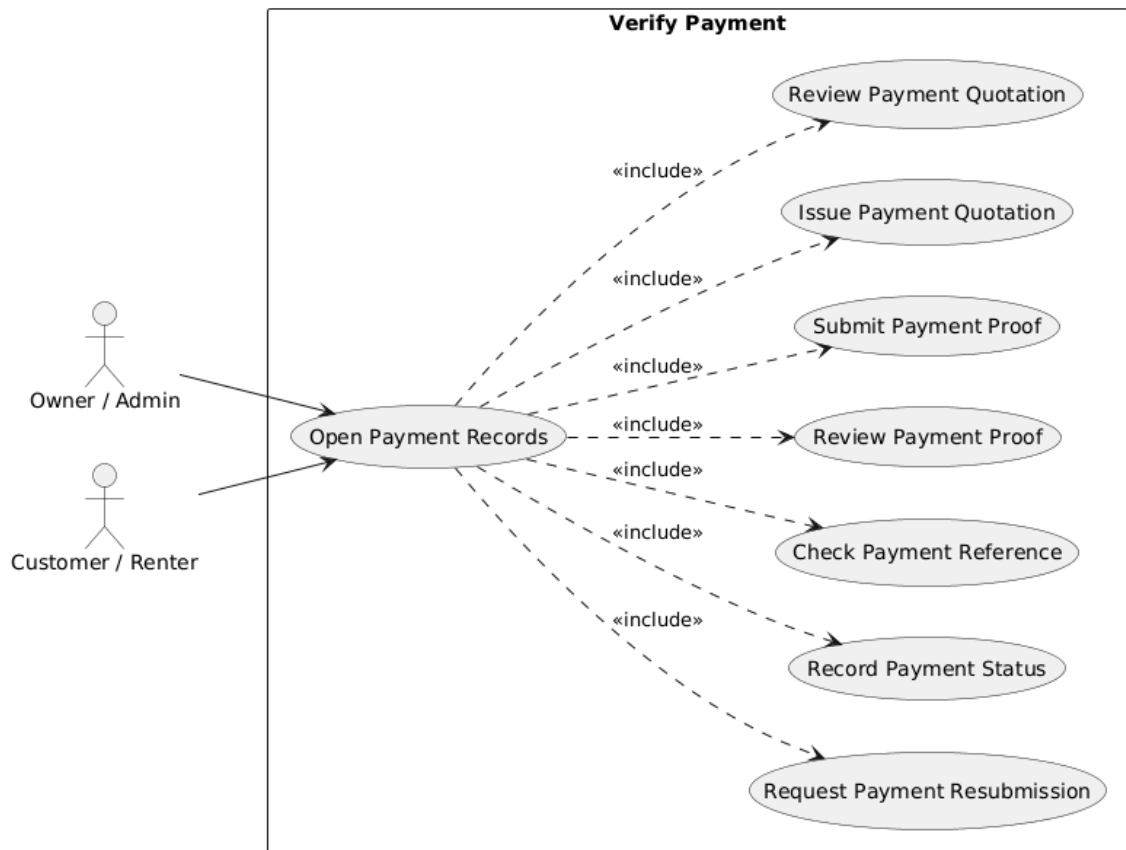


Figure 11 illustrates the verify payment use case, showing how the Owner/Admin establishes the required payment amount and reviews the submitted payment information, while the Customer/Renter submits proof of payment. The Owner/Admin checks the payment reference, records the payment status, and may request payment resubmission when necessary.

Figure 12. Use Case Diagram - Manage Rental Transactions

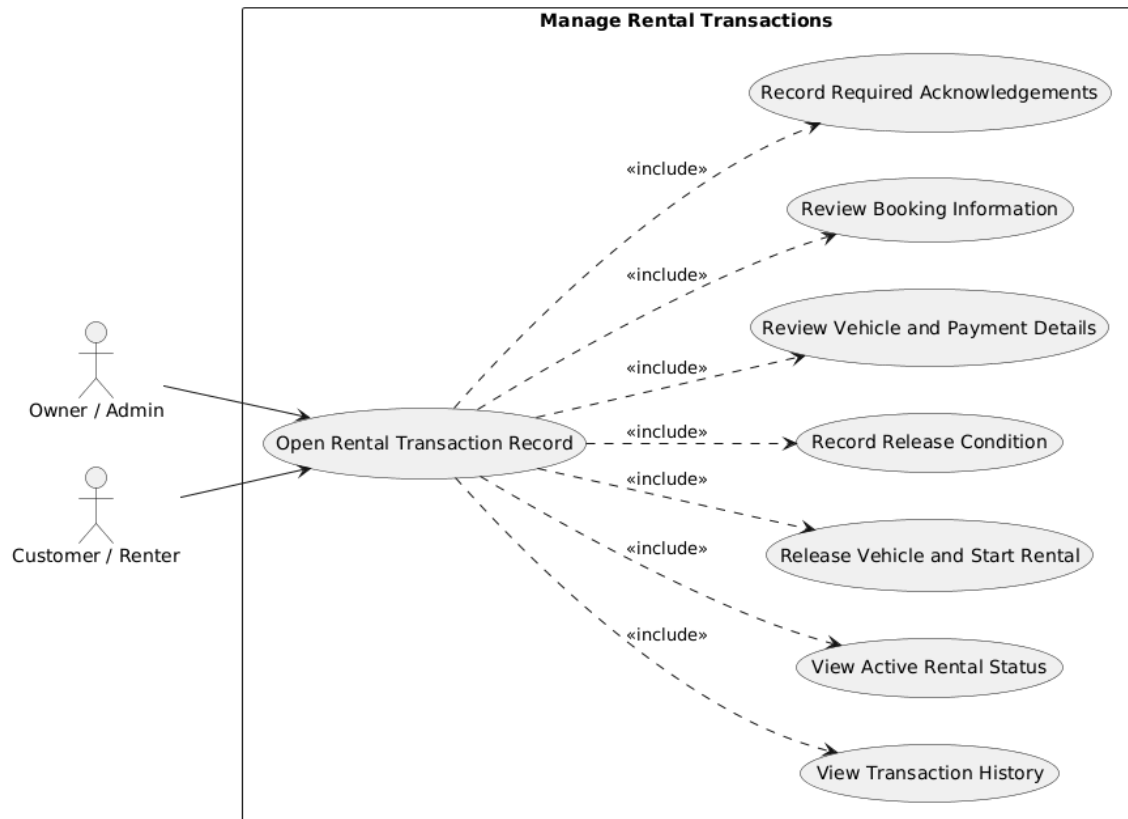


Figure 12 illustrates the manage rental transactions use case, showing how the Owner/Admin reviews booking, vehicle, and payment information before releasing the vehicle and starting the rental. The Customer/Renter can view the active rental status and transaction history related to the booking.

Figure 13. Use Case Diagram - Process Vehicle Return

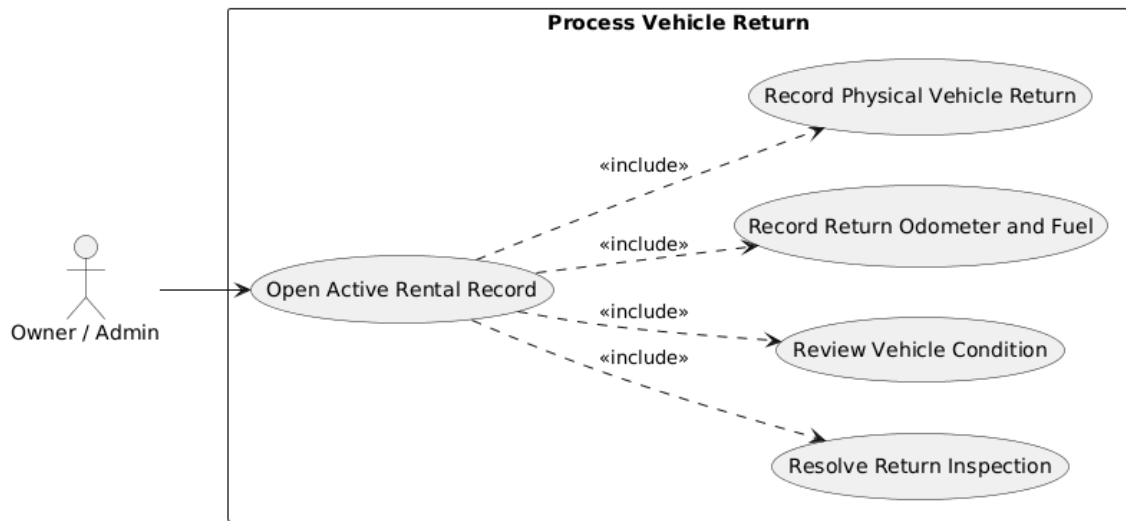


Figure 13 illustrates the process vehicle return use case, showing how the Owner/Admin opens the active rental record, records the return date and time, odometer and fuel information, reviews the vehicle condition, records return details, and resolves the return inspection.

Figure 14. Use Case Diagram - View Dashboard

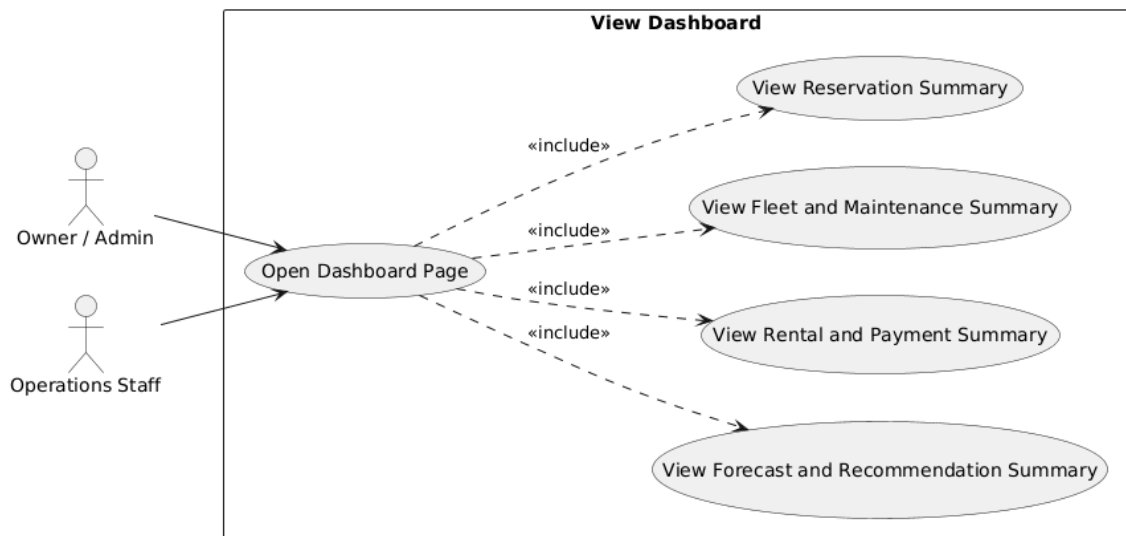


Figure 14 illustrates the view dashboard use case, showing how authorized users access the dashboard and review reservation, fleet and maintenance, rental and payment, and forecast and recommendation summaries according to their permitted access.

Figure 15. Use Case Diagram - Manage Customer Records

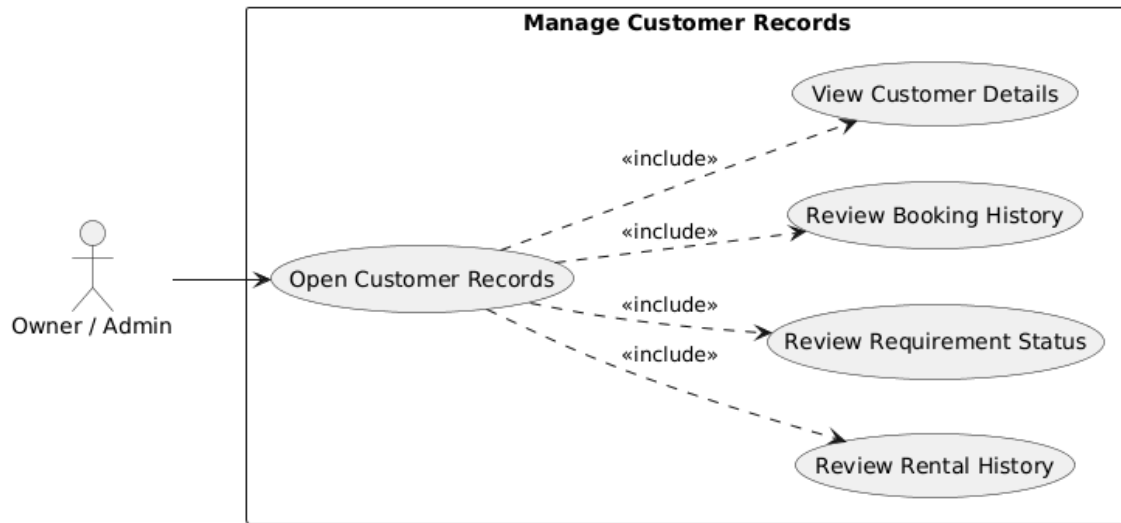


Figure 15 illustrates the manage customer records use case, showing how the Owner/Admin opens customer records, views customer details, and reviews booking, requirement, and rental history information.

Figure 16. Use Case Diagram - Manage Vehicle Records

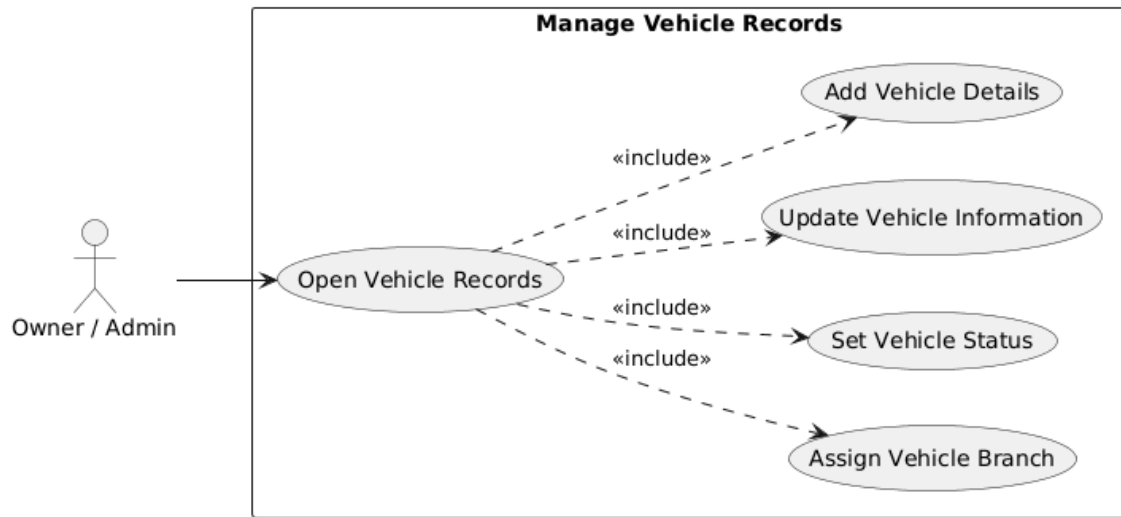


Figure 16 illustrates the manage vehicle records use case, showing how the Owner/Admin opens vehicle records, adds vehicle details, updates vehicle information, sets vehicle status, and assigns vehicles to their corresponding branches.

Figure 17. Use Case Diagram - Manage Maintenance Records

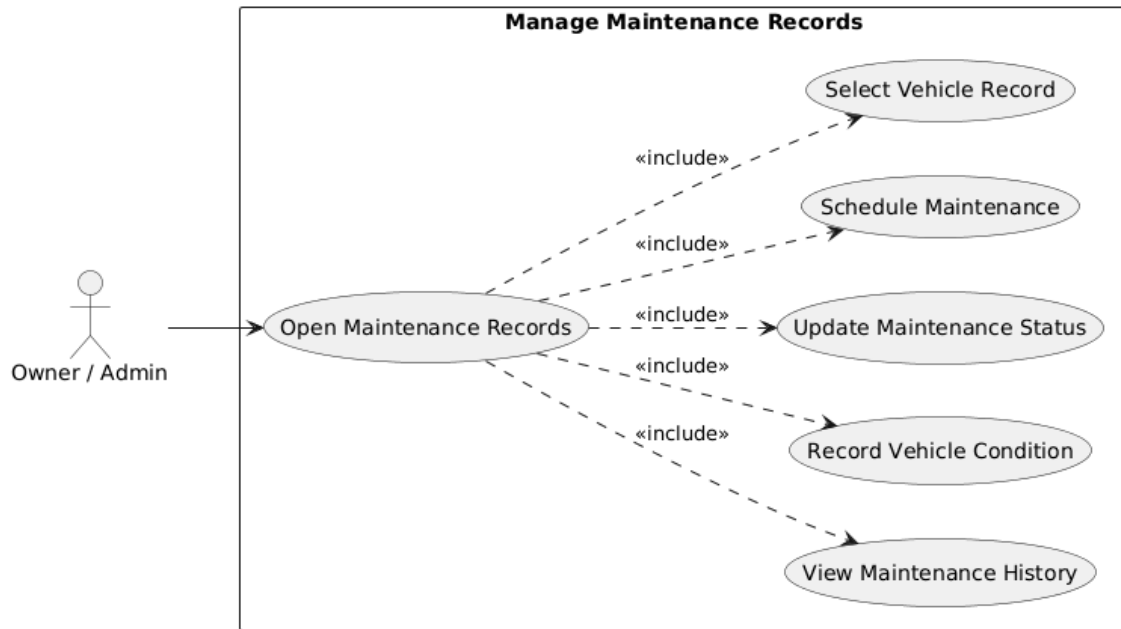


Figure 17 illustrates the manage maintenance records use case, showing how the Owner/Admin selects a vehicle record, schedules maintenance, updates maintenance status, records vehicle condition, and reviews maintenance history.

Figure 18. Use Case Diagram - View Forecasted Demand

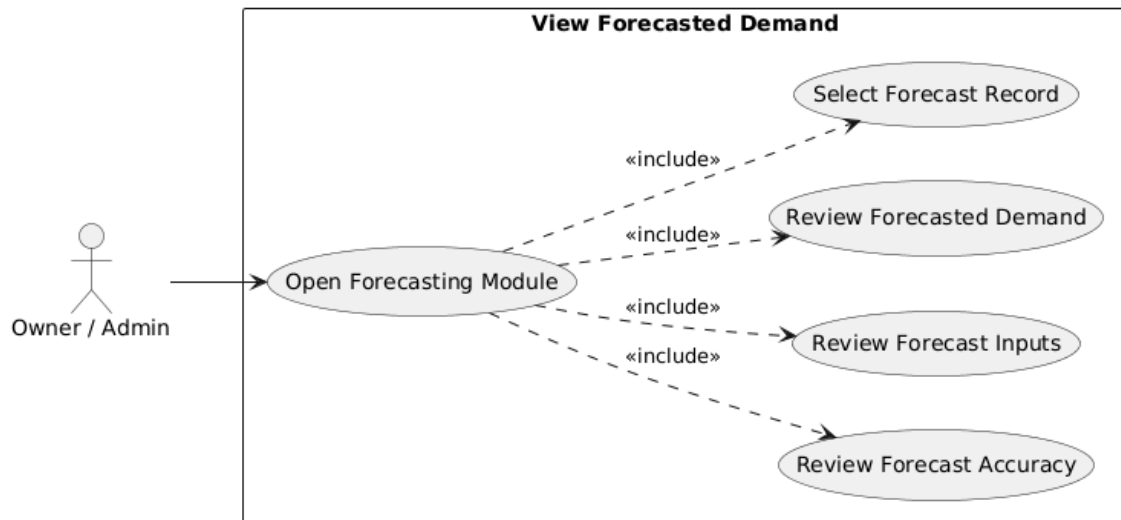


Figure 18 illustrates the view forecasted demand use case, showing how the Owner/Admin opens the forecasting module, selects the relevant forecast record, and reviews the forecasted demand, forecast inputs, and available forecast-accuracy information.

Figure 19. Use Case Diagram - View Fleet Allocation Recommendation

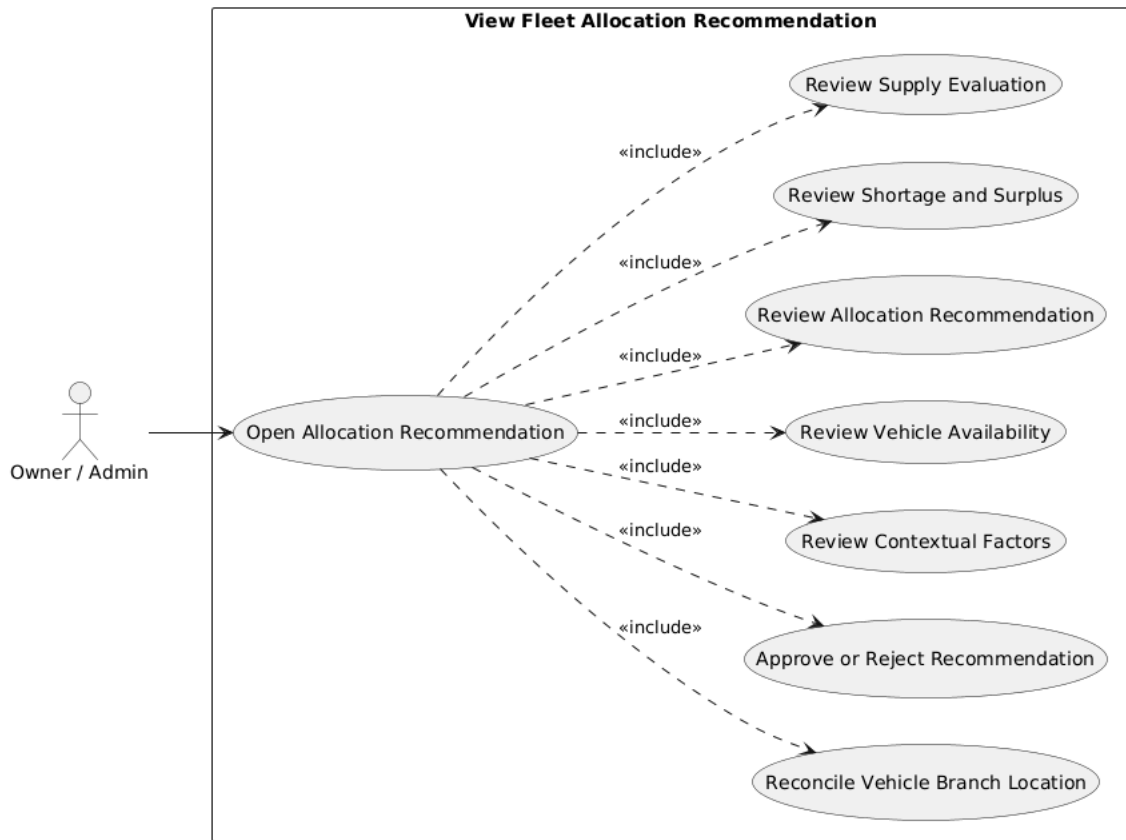


Figure 19 illustrates the view fleet allocation recommendation use case, showing how the Owner/Admin reviews supply evaluation information, identified shortages and surpluses, vehicle availability, contextual factors, and the resulting allocation recommendation. The Owner/Admin may approve or reject the recommendation and separately reconcile a vehicle's branch location when an operational transfer is carried out.

Figure 20. Use Case Diagram - View Generated Reports

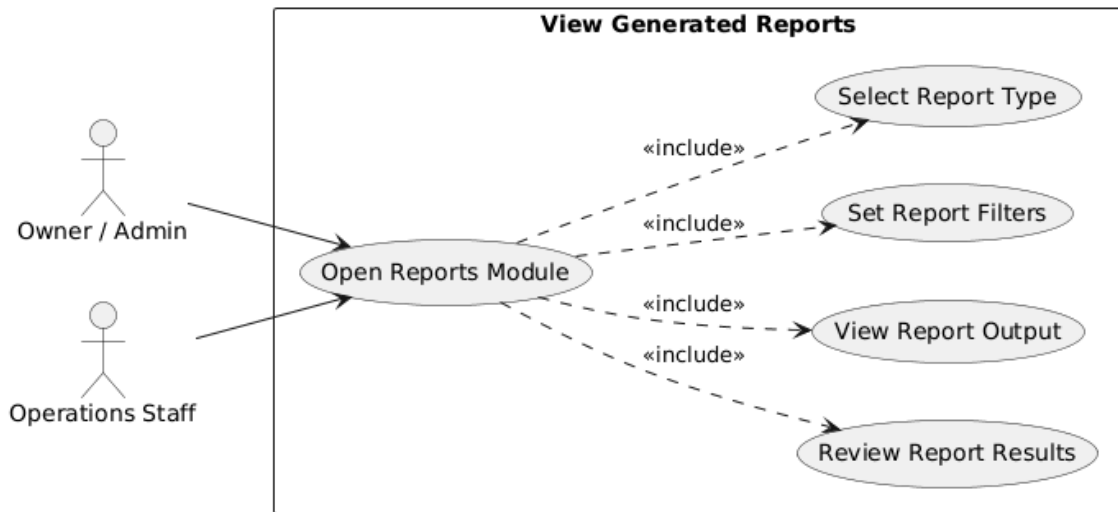


Figure 20 illustrates the view generated reports use case, showing how the Owner/Admin and Operations Staff open the reports module, select a report type, apply report filters, view the report output, and review the resulting operational information according to their permitted access.

Figure 21. Use Case Diagram - Configure Notifications

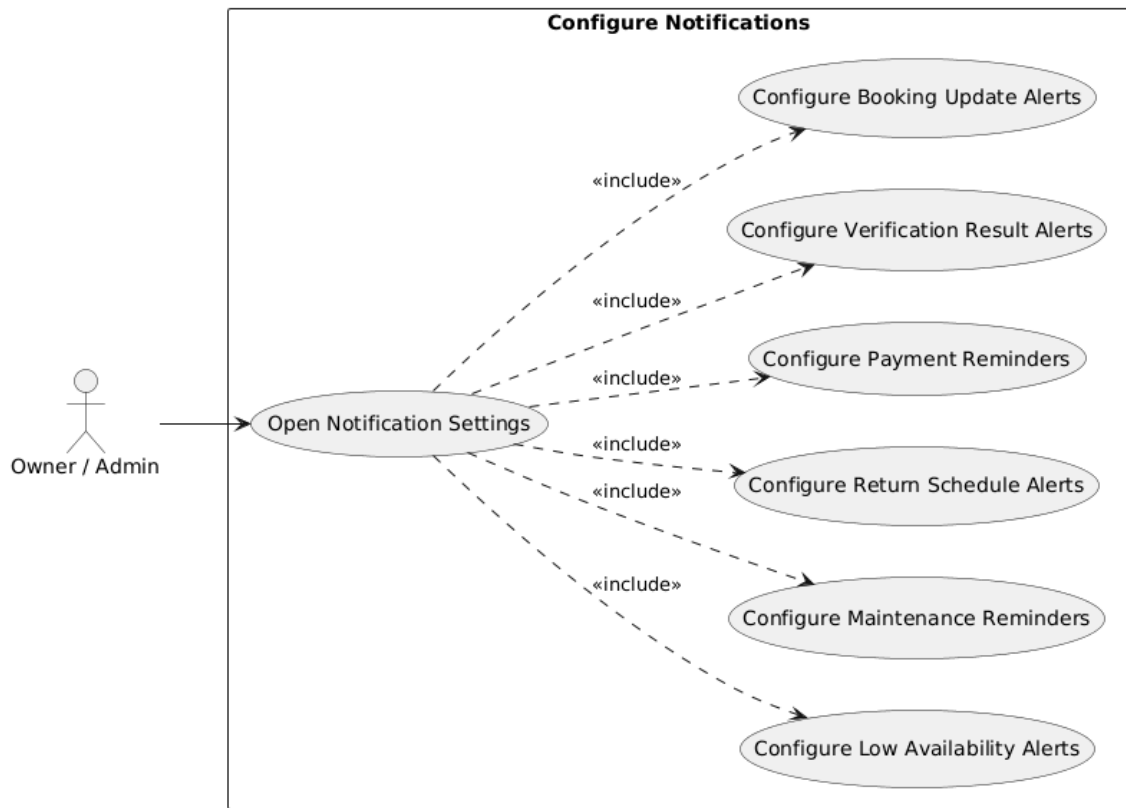


Figure 21 illustrates the configure notifications use case, showing how the Owner/Admin configures booking updates, verification alerts, payment reminders, return schedule alerts, maintenance reminders, and low availability alerts.

Use Case Reports

Table 6

Use Case Report – View Vehicle Catalog

Use Case Name	View Vehicle Catalog
Actors	Customer / Renter
Description	This use case allows the customer/renter to browse available vehicles, view vehicle details, check vehicle availability, and select a vehicle for consideration.
Precondition	The customer/renter has access to the web-based system and the vehicle catalog is available.
Flow of Events	Customer / Renter 1. The customer/renter opens the vehicle catalog. 2. The customer/renter browses available vehicles. 3. The customer/renter views vehicle details. 4. The customer/renter checks vehicle availability. 5. The customer/renter selects a vehicle.
Post Conditions	The customer/renter has reviewed available vehicle information and may proceed with the selected vehicle toward a booking request.
Exceptions	1. If no vehicles match the selected availability or criteria, the customer/renter may revise the selection. 2. If a selected vehicle is unavailable, the customer/renter must select another available vehicle.

Table 7

Use Case Report – Use Smart Vehicle Finder

Use Case Name	Use Smart Vehicle Finder
Actors	Customer / Renter
Description	This use case allows the customer/renter to provide rental requirements and review vehicle recommendations that assist in selecting a suitable vehicle.
Precondition	The customer/renter has access to the Smart Vehicle Finder and vehicle information is available for recommendation.
Flow of Events	Customer / Renter 1. The customer/renter opens the Smart Vehicle Finder. 2. The customer/renter enters the rental requirements. 3. The customer/renter views recommended vehicles. 4. The customer/renter reviews recommendation details. 5. The customer/renter selects a recommended vehicle. 6. The customer/renter continues to the booking request.
Post Conditions	The customer/renter has selected a recommended vehicle and may be carried into the booking process, but the recommendation does not constitute booking approval, vehicle reservation, or guaranteed vehicle assignment.
Exceptions	1. If required rental information is incomplete, the customer/renter completes the missing information. 2. If no suitable vehicle recommendation is available, the customer/renter may revise the rental requirements. 3. If the selected recommendation becomes unavailable or no longer satisfies the booking conditions, the customer/renter must refresh the recommendations or select another eligible vehicle.

Table 8

Use Case Report – Register Account

Use Case Name	Register Account
Actors	Customer / Renter
Description	This use case allows a customer/renter to create an account for accessing permitted customer-facing rental functions.
Precondition	The customer/renter has access to the web-based system and is on the registration page.
Flow of Events	Customer / Renter 1. The customer/renter opens the registration form. 2. The customer/renter provides the required account details. 3. The customer/renter submits the registration details. 4. The customer/renter proceeds according to the account confirmation status.
Post Conditions	The customer/renter account is created. Account access becomes available after any required email confirmation and successful sign-in.
Exceptions	1. If required registration details are incomplete, the system prompts the customer/renter to complete the missing information. 2. If a duplicate account is detected, the system informs the customer/renter and prevents duplicate registration. 3. If email confirmation is required, the customer/renter must confirm the registered email address before signing in and accessing the customer account.

Table 9

Use Case Report – Login

Use Case Name	Login		
Actors	Customer / Renter; Owner / Admin; Operations Staff		
Description	This use case allows authorized users to log into the system and access functions permitted for their respective roles.		
Precondition	The user has a valid account and is on the sign-in page.		
Flow of Events	Customer / Renter	Owner / Admin	Operations Staff
	1. Opens the sign-in form.	1. Opens the sign-in form.	1. Opens the sign-in form.
	2. Enters account credentials.	2. Enters account credentials.	2. Enters account credentials.
	3. Submits the login credentials.	3. Submits the login credentials.	3. Submits the login credentials.
	4. Views the customer account.	4. Views the authorized account with full system access.	4. Views the authorized account with permitted operational access.
Post Conditions	The user is logged into the system and can access features permitted for the assigned role.		
Exceptions	1. If invalid credentials are entered, the system prompts the user to provide the correct information. 2. If the account is inactive or unauthorized, access is not granted.		

Table 10

Use Case Report – Manage Reservations

Use Case Name	Manage Reservations		
Actors	Customer / Renter; Owner / Admin; Operations Staff		
Description	This use case supports customer booking requests and authorized reservation review and management. Successful payment verification normally triggers booking confirmation automatically when confirmation requirements are satisfied.		
Precondition	Required vehicle and booking information is available, and the user has the appropriate access for the intended reservation action.		
Flow of Events	Customer / Renter	Owner / Admin	Operations Staff
	1. Opens the reservation module.	1. Opens the reservation module.	1. Opens the reservation module.
	2. Submits a booking request.	2. Reviews reservation details.	2. Reviews permitted reservation details.
	3. Edits a draft booking request when needed.	3. Confirms, rejects, or cancels the reservation when permitted.	3. Views the booking calendar.
	4. Withdraws an eligible booking request when needed.	4. Views the booking calendar.	
		5. Reviews any confirmation exception that requires administrative action.	
Post Conditions	The booking request or reservation reflects the authorized action performed and is available for subsequent applicable rental processes.		
Exceptions	1. If required booking information is incomplete, the customer/renter must complete the missing information. 2. If the requested vehicle or schedule is unavailable, the booking request must be revised. 3. Actions outside an actor's permitted role are not available to that actor. 4. If automatic confirmation cannot proceed because assignment, vehicle readiness, schedule, or other confirmation requirements are not satisfied, the booking remains for Owner/Admin review.		

Table 11

Use Case Report – Verify Customer Requirements

Use Case Name	Verify Customer Requirements	
Actors	Customer / Renter; Owner / Admin	
Description	This use case allows the customer/renter to submit required documents and allows the owner/admin to review the submitted requirements and record their verification status.	
Precondition	A relevant customer or booking record exists and requirements are available for submission or review.	
Flow of Events	Customer / Renter	Owner / Admin
	1. Opens the requirements page.	1. Opens the requirements page.
	2. Uploads the required documents.	2. Reviews the submitted requirements.
	3. Submits the requirements.	3. Records the verification status.
	4. Resubmits flagged requirements when requested.	4. Requests requirement resubmission when corrections are required.
Post Conditions	The submitted requirements have a recorded verification status or are returned to the customer/renter for required correction.	
Exceptions	1. If required documents are missing or unreadable, corrected requirements are requested. 2. If submitted information requires correction, the owner/admin requests resubmission. 3. If one or more submitted requirements require correction, the Owner/Admin requests resubmission and the Customer/Renter replaces the flagged requirement before resubmitting.	

Table 12

Use Case Report – Verify Payment

Use Case Name	Verify Payment	
Actors	Customer / Renter; Owner / Admin	
Description	This use case allows the Customer/Renter to review the issued quotation and submit payment proof, while the Owner/Admin reviews the system-calculated quotation, enters the applicable delivery fee, issues the quotation, verifies payment proof, and records the payment status.	
Precondition	The Customer/Renter requirements are verified, the applicable booking exists, and the booking is eligible for quotation and payment processing.	
Flow of Events	Customer / Renter	Owner / Admin
	1. Opens the payment records.	1. Opens the payment records.
		2. Reviews the calculated payment quotation.
	2. Reviews the issued quotation.	3. Enters the applicable delivery fee.
		4. Issues the payment quotation.
	3. Submits payment proof for the exact required payment amount.	5. Reviews submitted payment proof.
		6. Checks the payment reference and submitted amount.
	4. Resubmits payment proof when requested.	7. Records the payment status.
		8. Requests payment resubmission when necessary.
Post Conditions	The payment information has a recorded status or has been returned for correction or resubmission.	
Exceptions	1. If payment proof is unclear or incomplete, corrected payment information is requested. 2. If the payment reference cannot be confirmed, the payment is not treated as successfully verified. 3. If the submitted amount does not match the exact required payment amount, payment verification cannot proceed. 4. If successful payment verification cannot automatically confirm the booking because confirmation requirements	

	are not satisfied, the booking requires Owner/Admin review.
--	---

Table 13

Use Case Report – Manage Rental Transactions

Use Case Name	Manage Rental Transactions	
Actors	Customer / Renter; Owner / Admin	
Description	This use case supports authorized processing and viewing of rental transactions associated with eligible bookings. Vehicle release includes recording the release condition and the required acknowledgements for the rental agreement, vehicle condition, and return schedule.	
Precondition	The booking is confirmed, an eligible vehicle is assigned, requirements and payment are verified, and all release prerequisites are satisfied.	
Flow of Events	Customer / Renter	Owner / Admin
	1. Opens the applicable rental transaction information.	1. Opens the rental transaction record.
		2. Reviews the booking information.
		3. Reviews vehicle and payment details.
	2. Views the active rental status.	4. Records the release condition, including applicable odometer, fuel, and condition information.
		5. Records the required acknowledgements for the rental agreement, vehicle condition, and return schedule.
	3. Views the transaction history.	6. Releases the vehicle and starts the rental.
		7. Views the active rental status and transaction history.

Post Conditions	The applicable rental transaction information is recorded and available for subsequent rental monitoring or return processing.
Exceptions	1. If required transaction information is incomplete, the transaction cannot proceed until the required information is resolved. 2. If required booking, requirement, payment, or vehicle conditions do not permit release, the rental is not started.

Table 14

Use Case Report – Process Vehicle Return

Use Case Name	Process Vehicle Return
Actors	Owner / Admin
Description	This use case allows the Owner/Admin to record the physical return of a rented vehicle, record return condition details, and separately resolve the post-return inspection.
Precondition	An active rental record exists and the vehicle is presented or eligible for return processing.
Flow of Events	1. The owner/admin opens the active rental record. 2. The owner/admin records the return date and time. 3. The owner/admin records the return odometer and fuel information. 4. The owner/admin reviews the vehicle condition. 5. The owner/admin records the return details. 6. The owner/admin resolves the return inspection.
Post Conditions	The vehicle return and inspection information are recorded for the applicable rental transaction.
Exceptions	1. If required return information is incomplete, the return cannot be finalized until the required information is provided. 2. If the return inspection identifies an issue requiring resolution, the return remains subject to the applicable inspection resolution.

Table 15

Use Case Report – View Dashboard

Use Case Name	View Dashboard	
Actors	Owner / Admin; Operations Staff	
Description	This use case allows authorized internal users to view operational summaries available to their respective roles.	
Precondition	The user is logged in with authorized internal access and applicable dashboard information is available.	
Flow of Events	Owner / Admin	Operations Staff
	1. Opens the dashboard page.	1. Opens the dashboard page.
	2. Views the reservation summary.	
	3. Views the fleet and maintenance summary.	2. Views dashboard information permitted for Operations Staff.
	4. Views the rental and payment summary.	
	5. Views the forecast and recommendation summary.	
Post Conditions	The owner/admin obtains a centralized operational overview for monitoring and decision-making.	
Exceptions	1. If dashboard data are incomplete, the system displays only the available summary information. 2. If summary values require review, the owner/admin may open the related module for verification.	

Table 16

Use Case Report – Manage Customer Records

Use Case Name	Manage Customer Records
Actors	Owner / Admin
Description	This use case allows the owner/admin to review customer information and relevant booking, requirement, and rental history.
Precondition	The owner/admin is logged in and applicable customer records exist.
Flow of Events	1. The owner/admin opens the customer records. 2. The owner/admin views customer details. 3. The owner/admin reviews booking history. 4. The owner/admin reviews requirement status. 5. The owner/admin reviews rental history.
Post Conditions	The owner/admin has reviewed the applicable customer record and related history.
Exceptions	1. If the requested customer record cannot be found, no customer details are available for review. 2. If related history is unavailable, only existing customer information is displayed.

Table 17

Use Case Report – Manage Vehicle Records

Use Case Name	Manage Vehicle Records
Actors	Owner / Admin
Description	This use case allows the owner/admin to manage vehicle information, status, and branch assignment.
Precondition	The owner/admin is logged in and applicable vehicle or branch information is available.
Flow of Events	1. The owner/admin opens the vehicle records. 2. The owner/admin adds vehicle details. 3. The owner/admin updates vehicle information. 4. The owner/admin sets the vehicle status. 5. The owner/admin assigns the vehicle branch.
Post Conditions	The authorized vehicle record changes are stored and become available to applicable fleet and rental functions.
Exceptions	1. If required vehicle information is incomplete, the record cannot be completed until the required information is provided. 2. If a branch assignment is invalid, a valid branch must be selected.

Table 18

Use Case Report – Manage Maintenance Records

Use Case Name	Manage Maintenance Records
Actors	Owner / Admin
Description	This use case allows the owner/admin to manage maintenance information and vehicle condition for fleet readiness monitoring.
Precondition	The owner/admin is logged in and the applicable vehicle record exists.
Flow of Events	1. The owner/admin opens the maintenance records. 2. The owner/admin selects the vehicle record. 3. The owner/admin schedules maintenance. 4. The owner/admin updates the maintenance status. 5. The owner/admin records the vehicle condition. 6. The owner/admin views the maintenance history.
Post Conditions	The applicable maintenance information and vehicle condition are recorded and available for subsequent review.
Exceptions	1. If required maintenance information is incomplete, the record cannot be completed until the required information is provided. 2. If the selected vehicle cannot be found, another valid vehicle record must be selected.

Table 19

Use Case Report – View Forecasted Demand

Use Case Name	View Forecasted Demand
Actors	Owner / Admin
Description	This use case allows the owner/admin to select and review available forecast information for rental-demand analysis.
Precondition	The owner/admin is logged in and applicable forecasting information is available.
Flow of Events	1. The owner/admin opens the forecasting module. 2. The owner/admin selects a forecast record. 3. The owner/admin reviews the forecasted demand. 4. The owner/admin reviews the forecast inputs. 5. The owner/admin reviews the available forecast accuracy information.
Post Conditions	The owner/admin has reviewed the applicable demand forecast and its supporting information.
Exceptions	1. If forecast information is unavailable or insufficient, only available forecast information can be reviewed. 2. If the selected forecast record is unavailable, another available record must be selected.

Table 20

Use Case Report – View Fleet Allocation Recommendation

Use Case Name	View Fleet Allocation Recommendation
Actors	Owner / Admin
Description	This use case allows the owner/admin to review fleet allocation decision-support information and take the applicable authorized action.
Precondition	The owner/admin is logged in and applicable supply, availability, contextual, and allocation information is available.
Flow of Events	1. The owner/admin opens the allocation recommendation. 2. The owner/admin reviews the supply evaluation. 3. The owner/admin reviews identified shortage and surplus information. 4. The owner/admin reviews the allocation recommendation. 5. The owner/admin reviews vehicle availability. 6. The owner/admin reviews contextual factors. 7. The owner/admin approves or rejects the recommendation. 8. The owner/admin separately reconciles the vehicle branch location when an operational transfer is carried out.
Post Conditions	The recommendation has been reviewed and the owner/admin's applicable decision is recorded. Any actual branch-location reconciliation remains a separate operational action.
Exceptions	1. If required supporting information is unavailable, only available recommendation information can be reviewed. 2. If the recommendation is not accepted, no corresponding branch-location reconciliation is performed on that basis.

Table 21

Use Case Report – View Generated Reports

Use Case Name	View Generated Reports	
Actors	Owner / Admin; Operations Staff	
Description	This use case allows authorized internal users to select and review operational reports according to their permitted access.	
Precondition	The user is logged in with authorized internal access and applicable report information is available.	
Flow of Events	Owner / Admin	Operations Staff
	1. Opens the reports module.	1. Opens the reports module.
	2. Selects the report type.	2. Selects an available report type.
	3. Sets report filters.	3. Sets permitted report filters.
	4. Views the report output.	4. Views permitted report output.
	5. Reviews the report results.	5. Reviews permitted report results.
Post Conditions	The authorized user has reviewed the selected report information according to the user's permitted access.	
Exceptions	1. If no records match the selected filters, no matching report information is available. 2. If selected criteria are invalid, the criteria must be revised. 3. Report information outside the Operations Staff role's permitted access is not available.	

Table 22

Use Case Report – Configure Notifications

Use Case Name	Configure Notifications
Actors	Owner / Admin
Description	This use case allows the owner/admin to configure supported notification and reminder settings.
Precondition	The owner/admin is logged in and applicable notification settings are available for configuration.
Flow of Events	1. The owner/admin opens the notification settings. 2. The owner/admin configures booking update alerts. 3. The owner/admin configures verification result alerts. 4. The owner/admin configures payment reminders. 5. The owner/admin configures return schedule alerts. 6. The owner/admin configures maintenance reminders. 7. The owner/admin configures low availability alerts.
Post Conditions	The configured notification and reminder settings are stored for applicable system notifications.
Exceptions	1. If required notification settings are incomplete, the settings must be completed before they can be saved. 2. If configured alert criteria are invalid, the owner/admin must revise the criteria.

Design Specifications**Activity Diagram.**

The Activity Diagram visually represents the workflow or sequence of activities that take place in the system, showing how users or components interact with it to accomplish a specific process. This diagram is especially helpful in illustrating the logic and flow of operations, from beginning to end, including decisions, parallel processes, and system responses.

Figure 30. Register/ Log In Customer Account

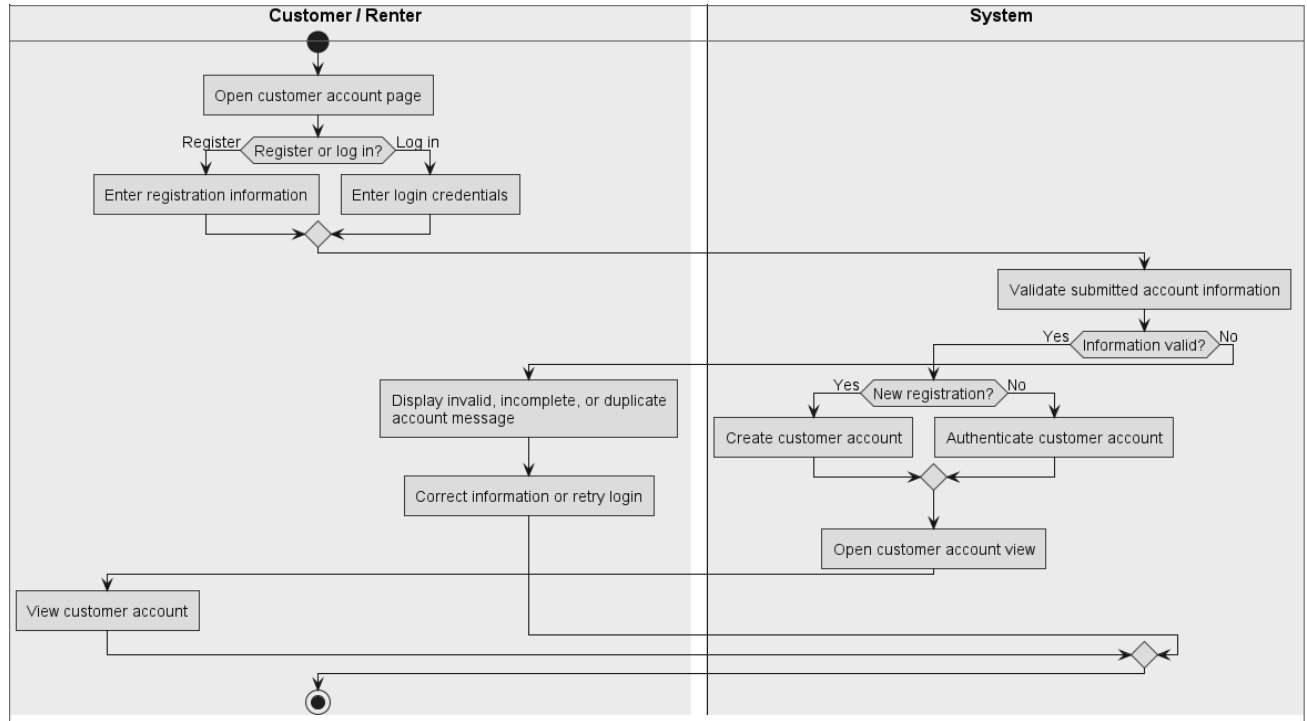


Figure 31. Authenticate User Login

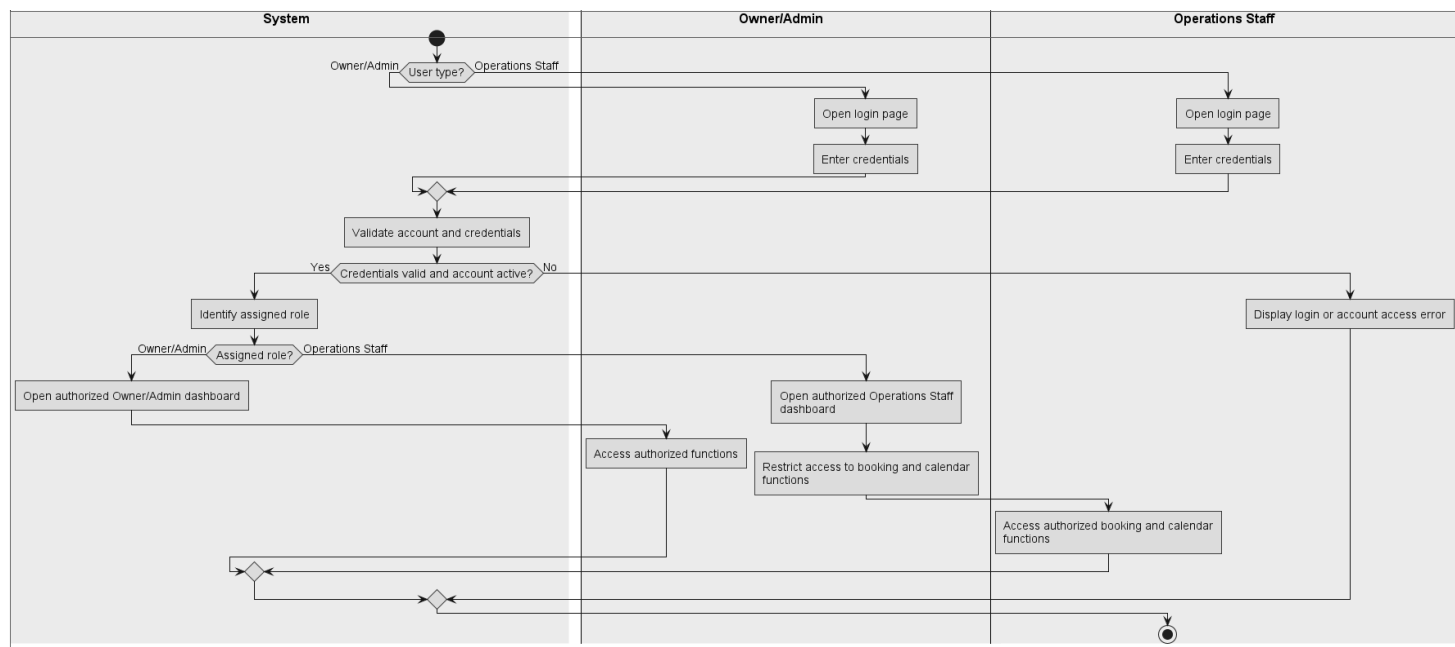


Figure 32. Manage Access Permissions

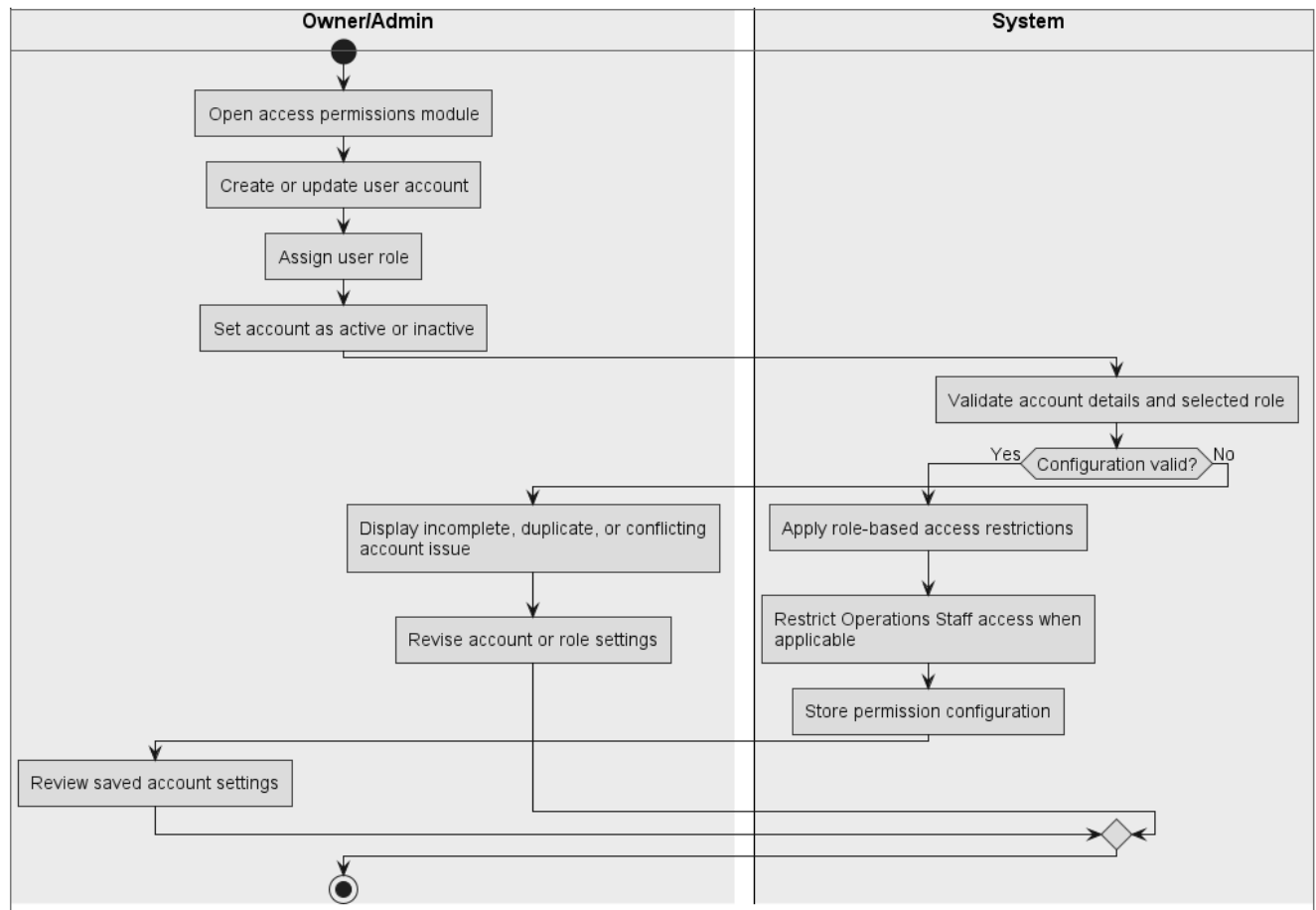


Figure 34. Manage Vehicle and Branch Records

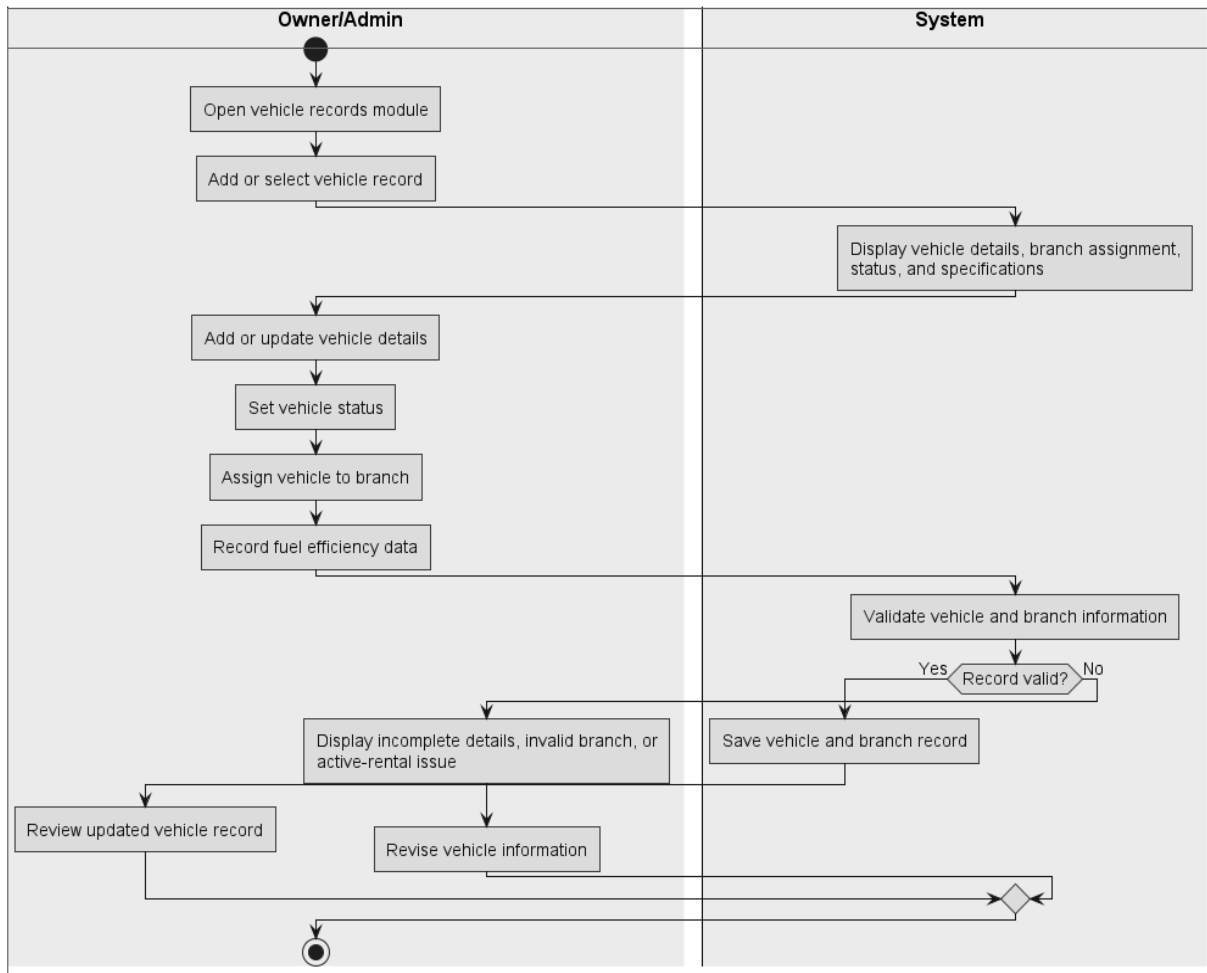


Figure 35. Manage Maintenance Records

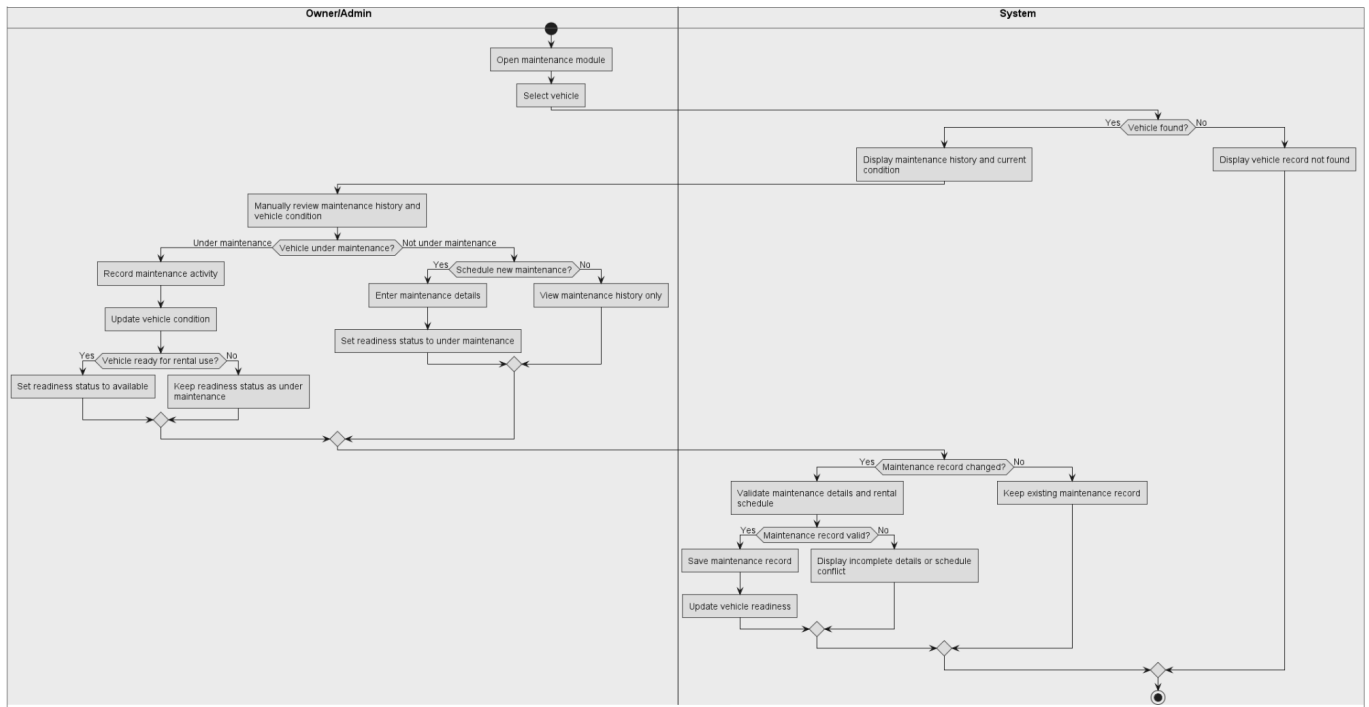


Figure 36. View Vehicle Catalog

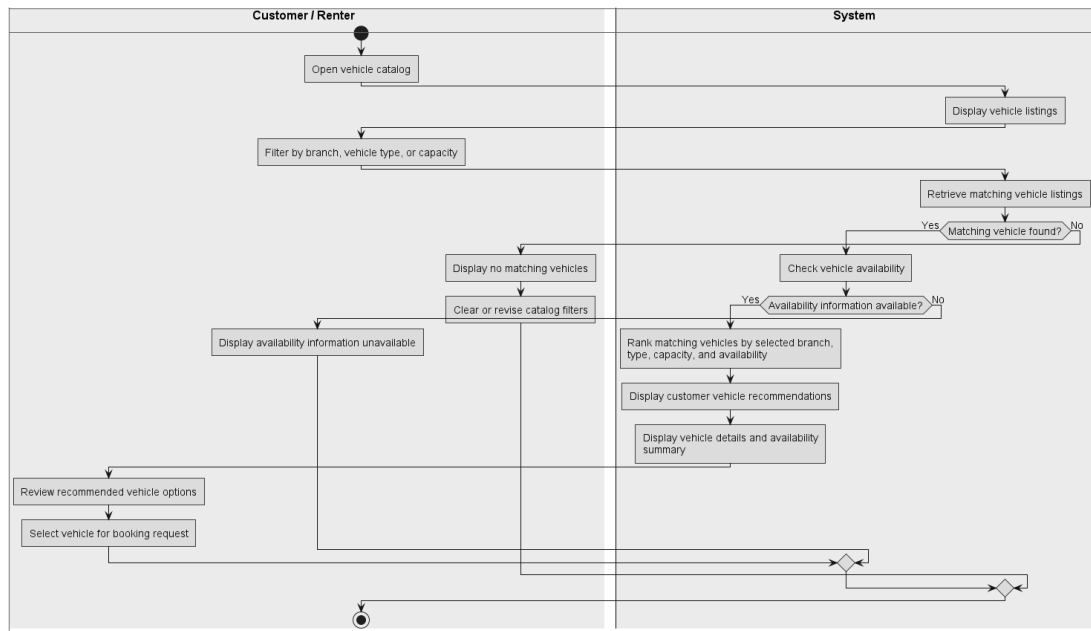


Figure 37. View Rental Policy Agreement

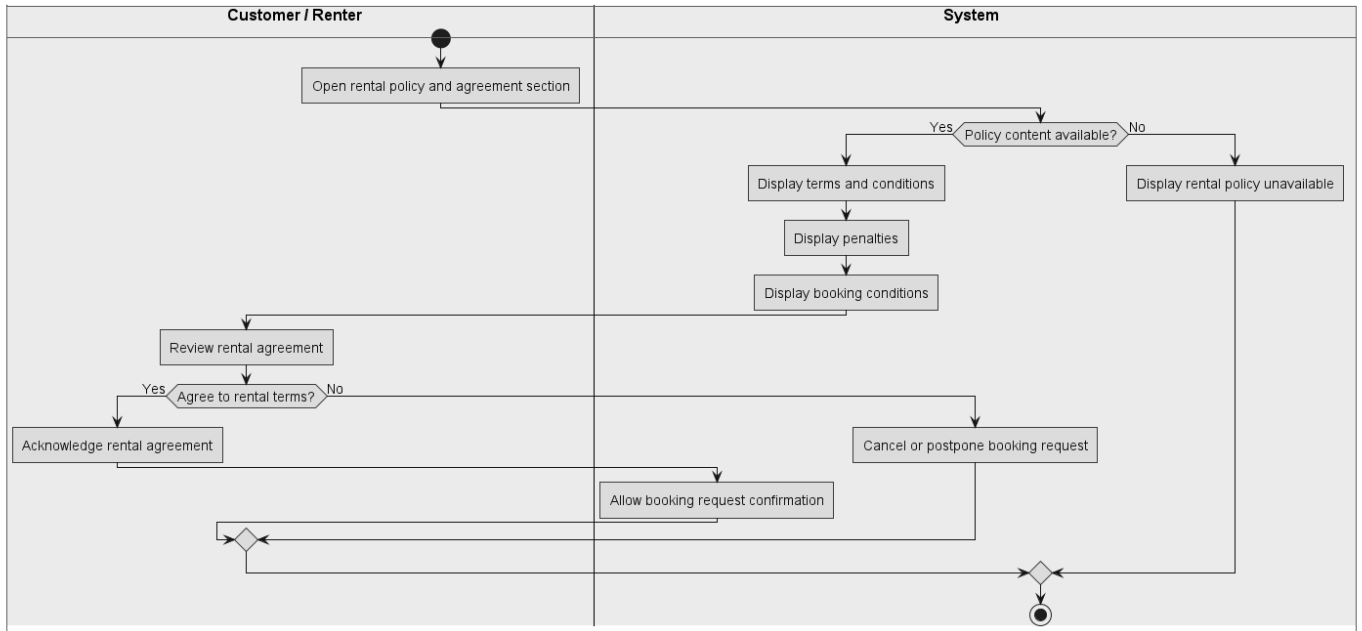


Figure 38. Create and Manage Reservation

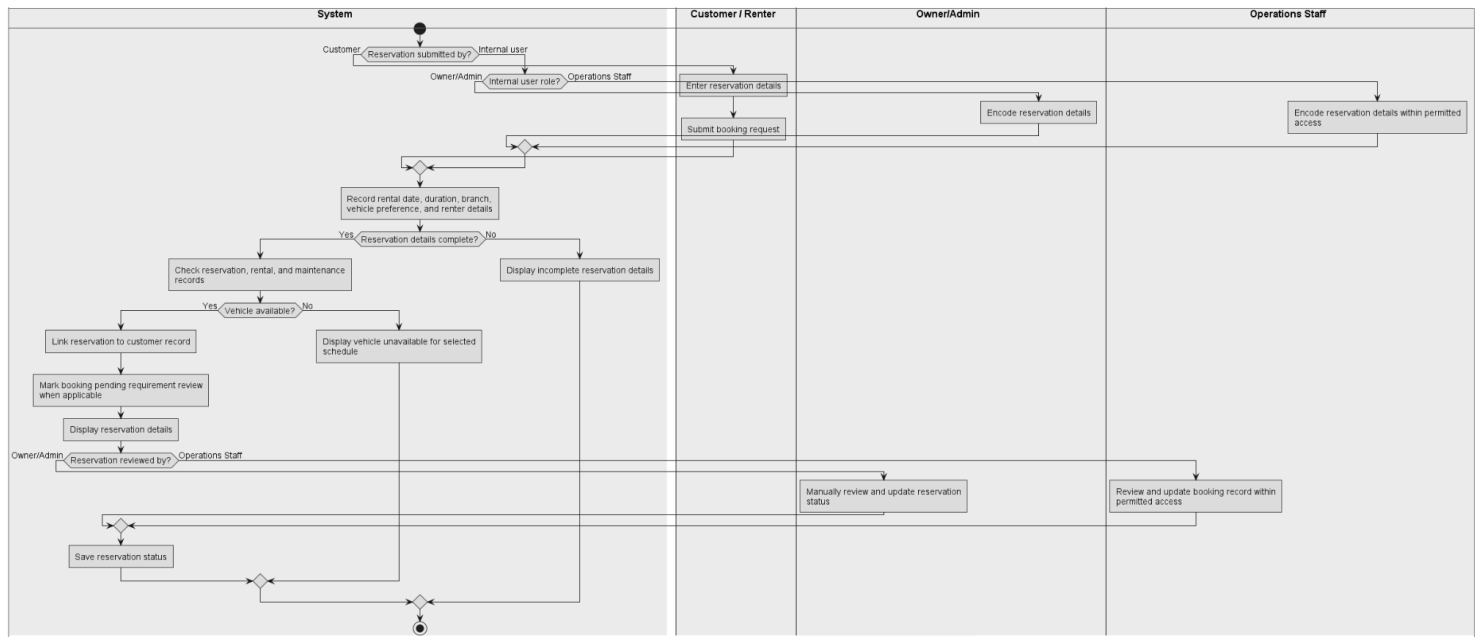


Figure 39. View Booking Calendar

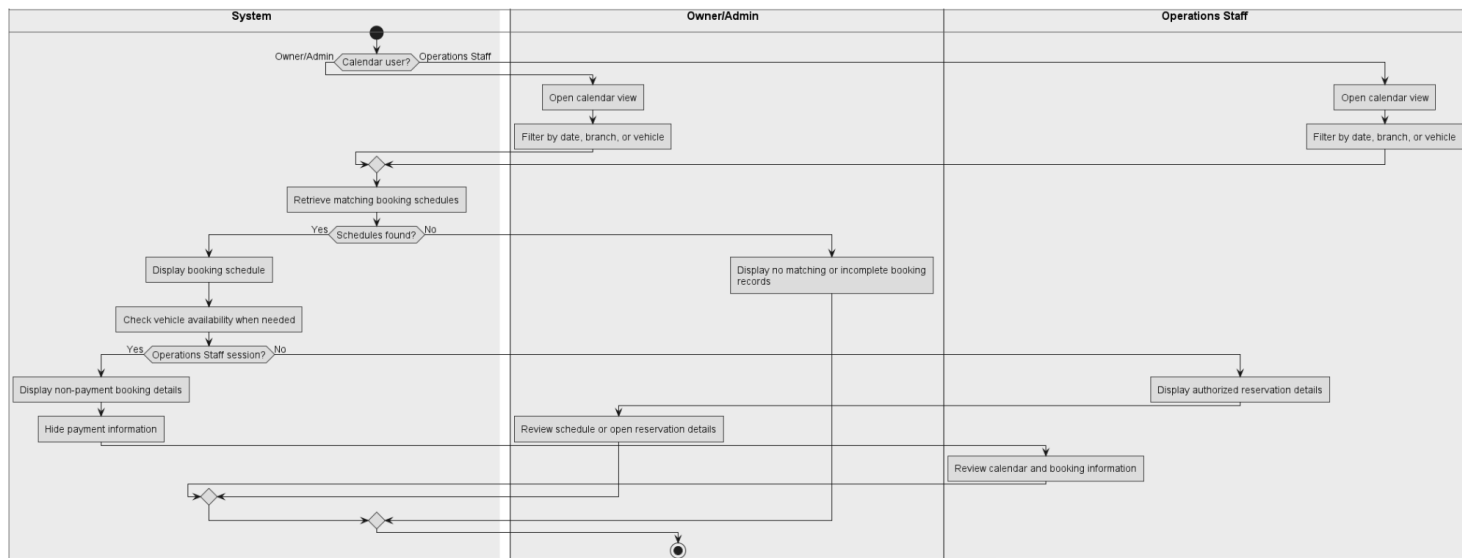


Figure 40. Check Vehicle Availability

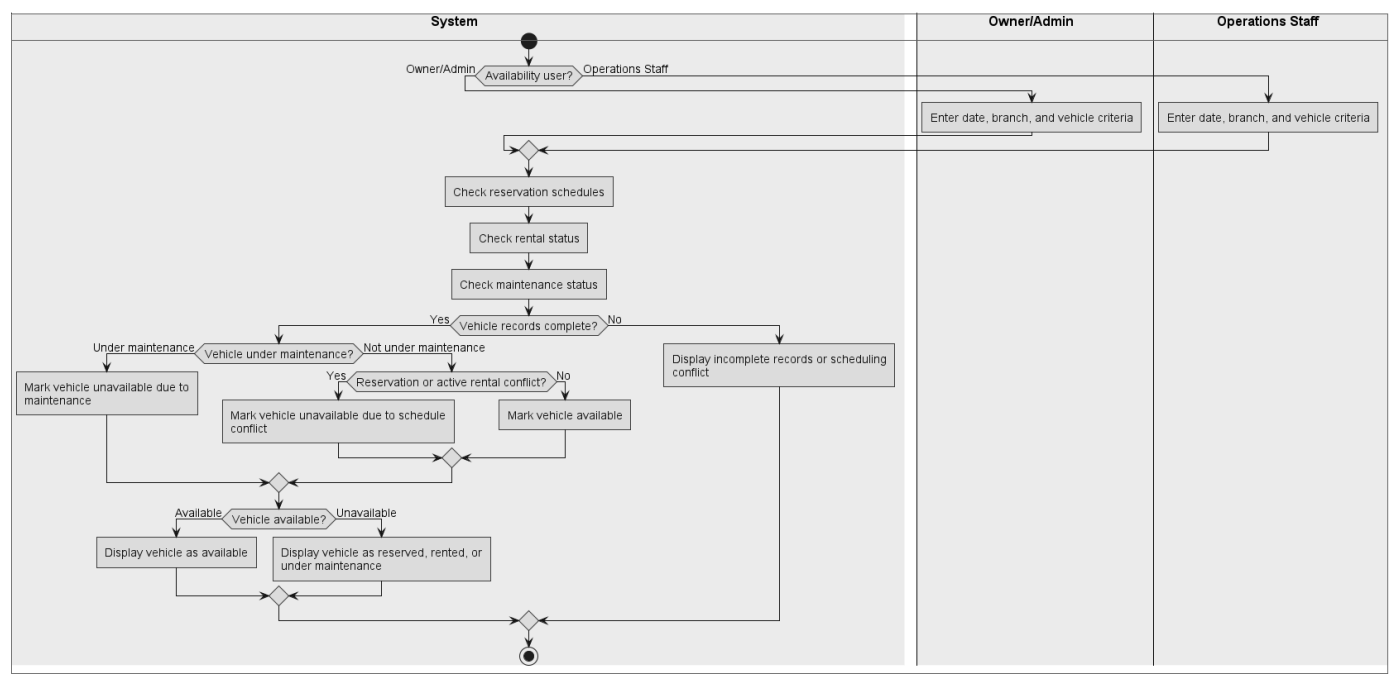


Figure 41. Verify Requirements

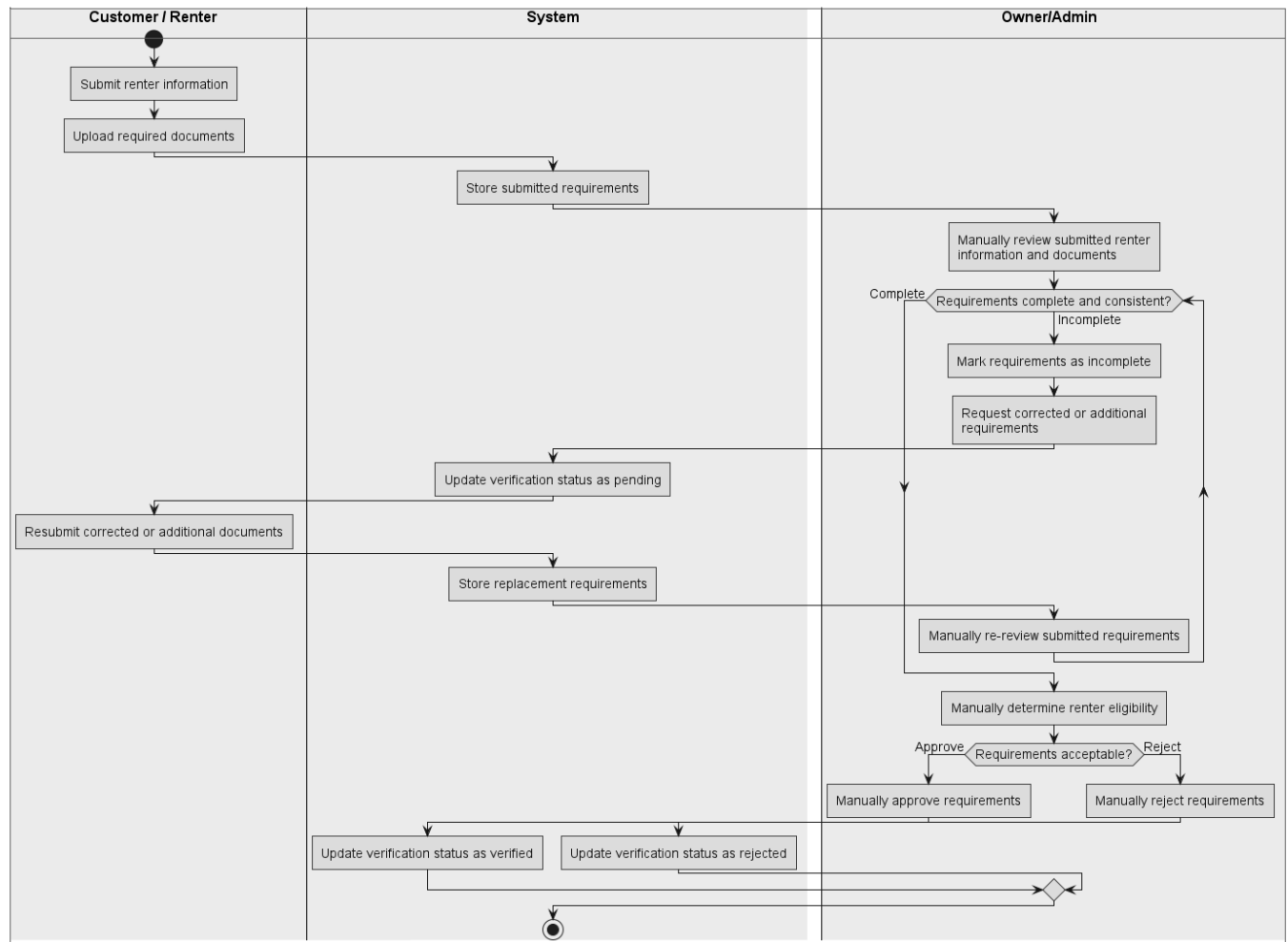


Figure 42. Validate Payment Records

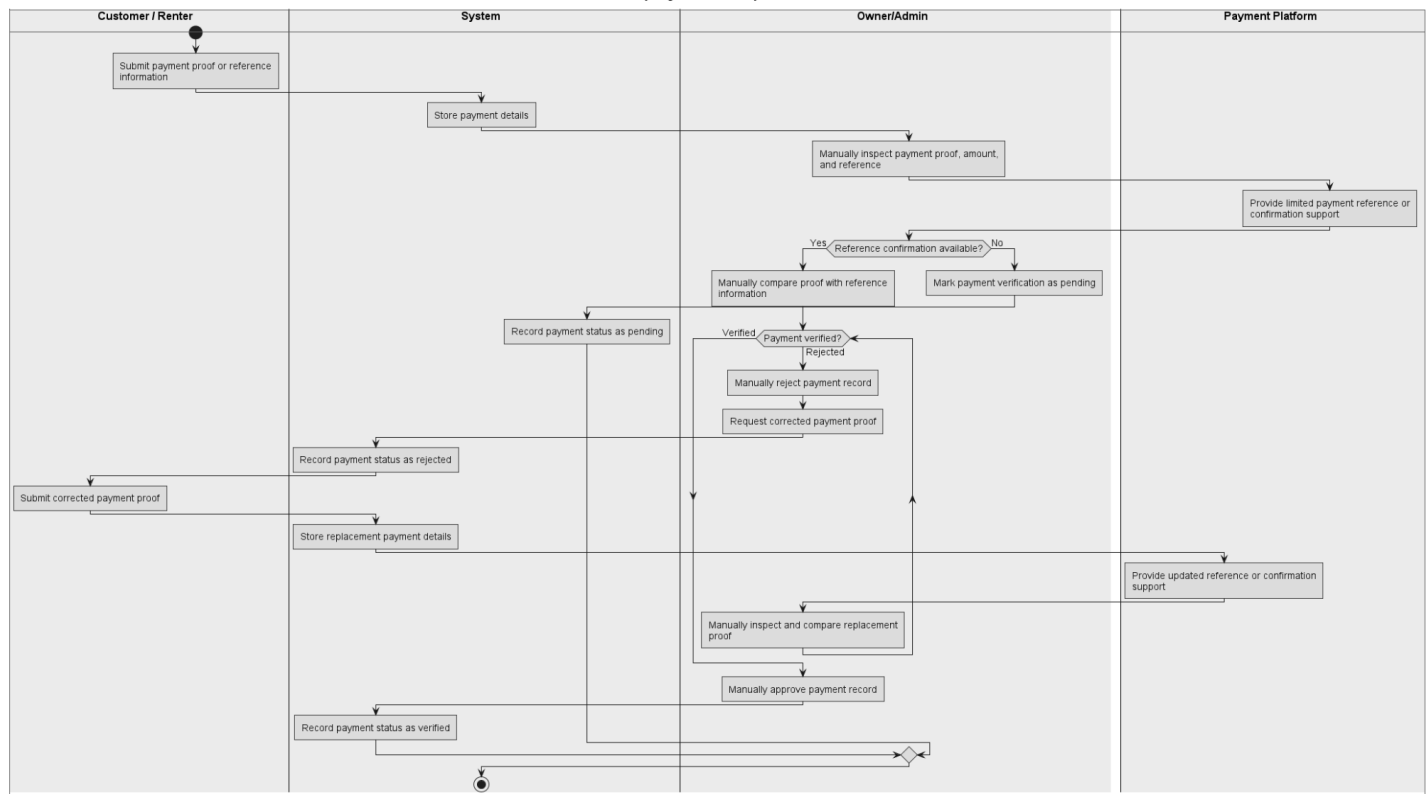


Figure 43. Forecast Demand Using WMA

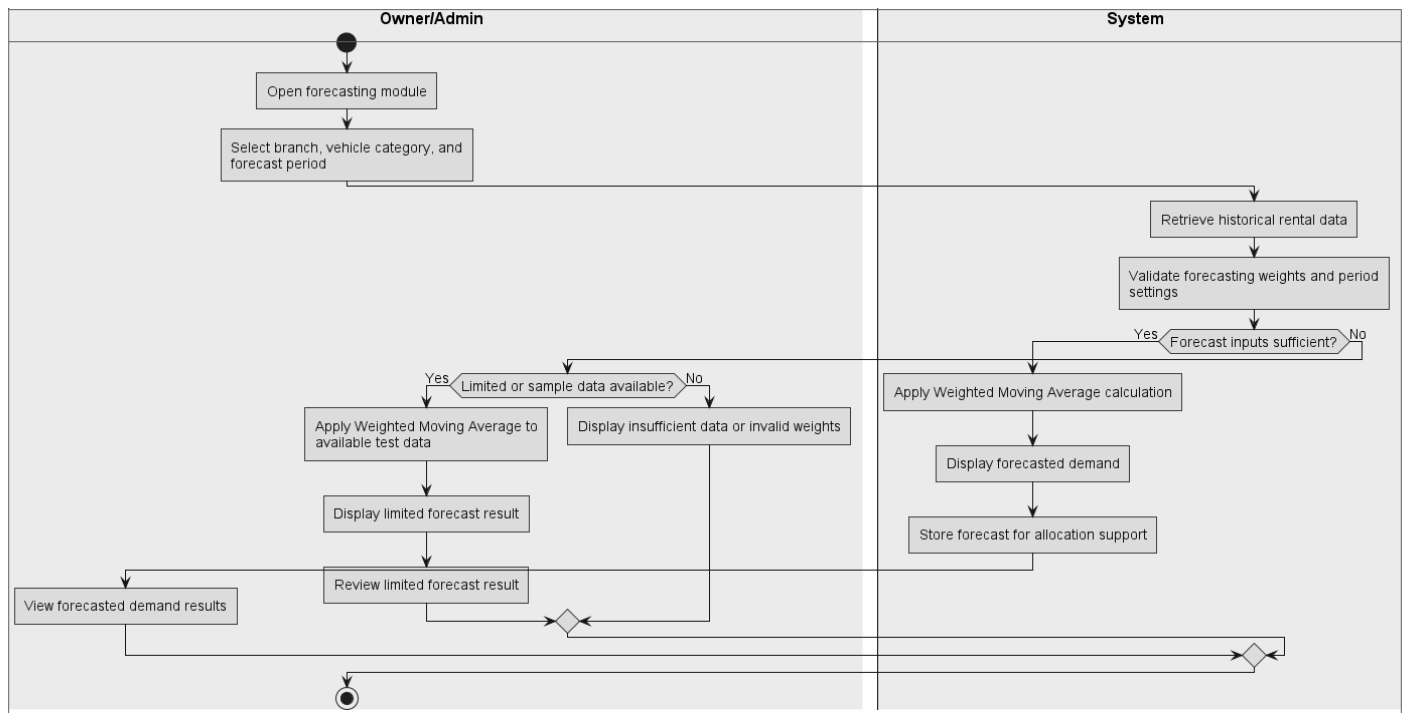


Figure 44. Evaluate Context Aware Factors

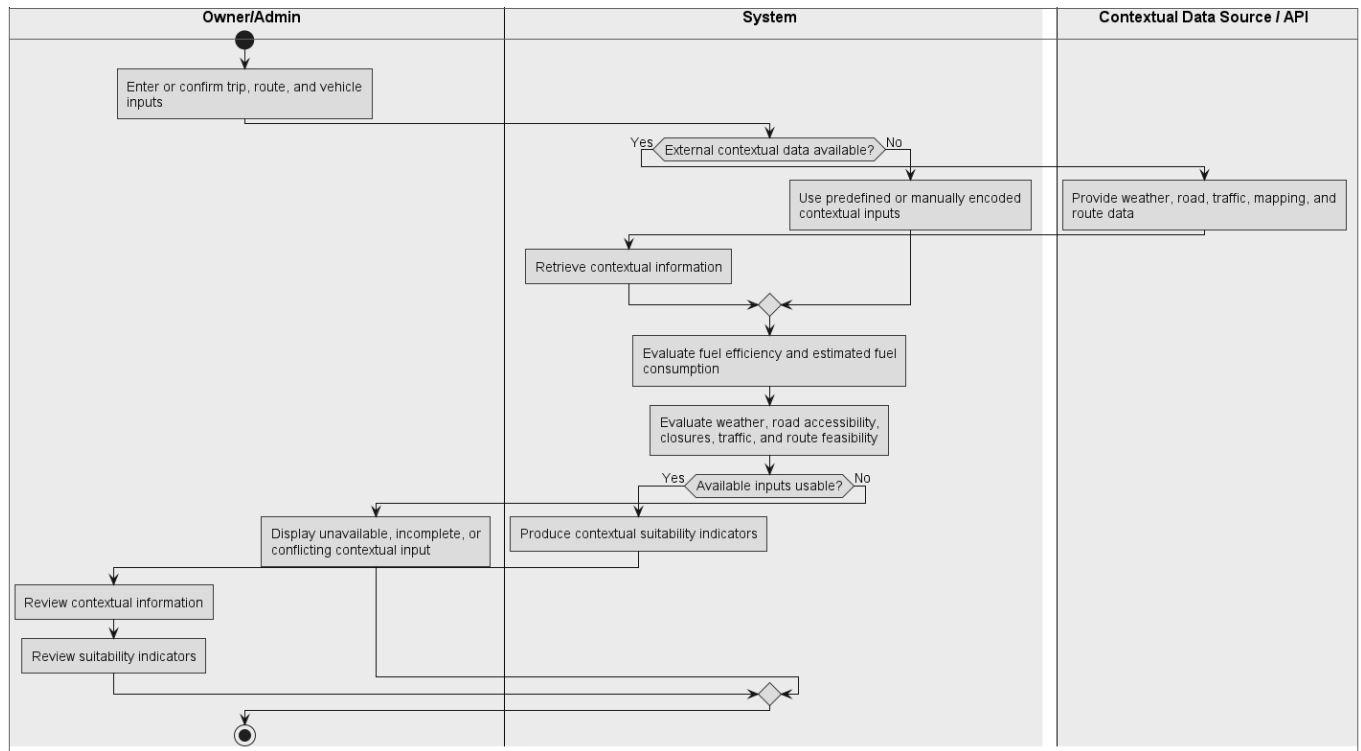


Figure 45. Recommend Fleet Allocation

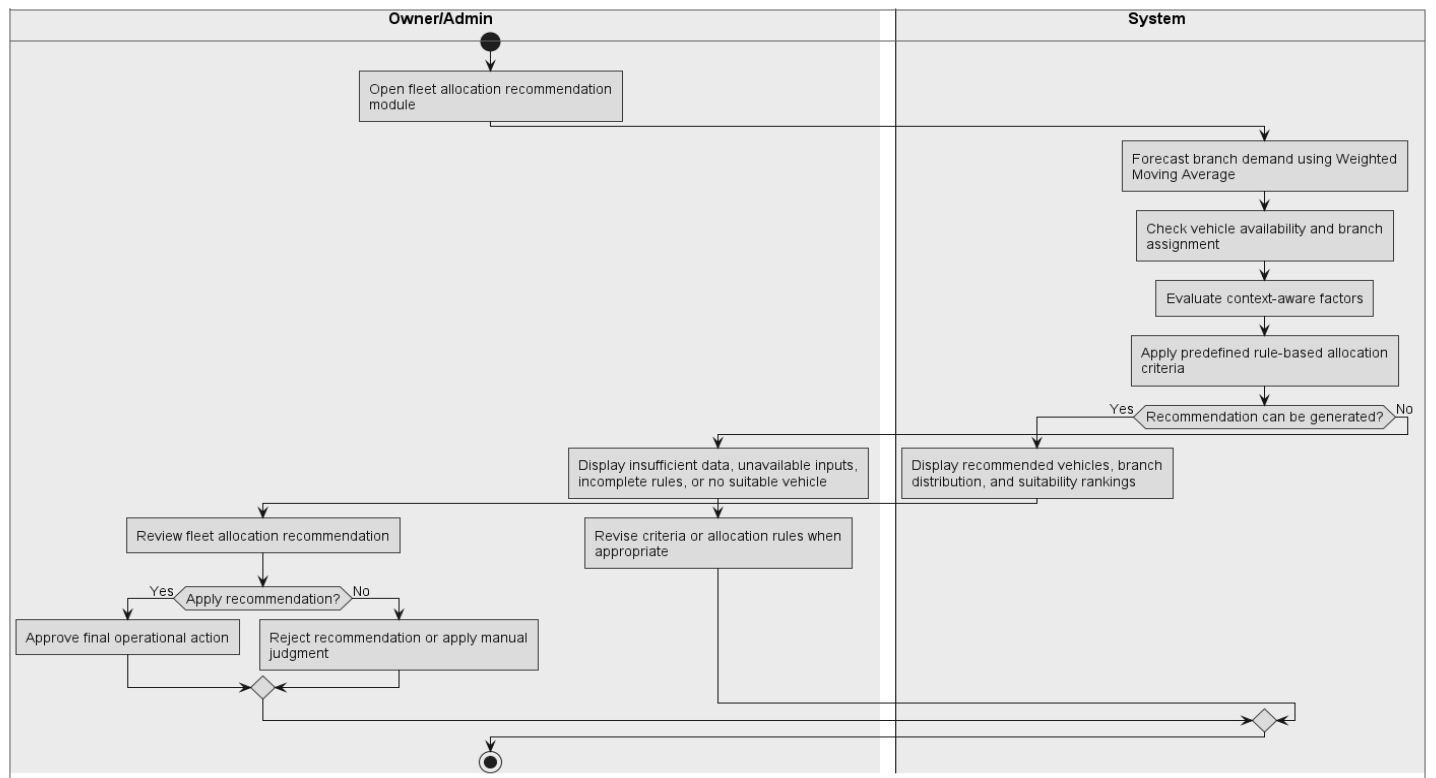


Figure 47. Manage Rental Transaction

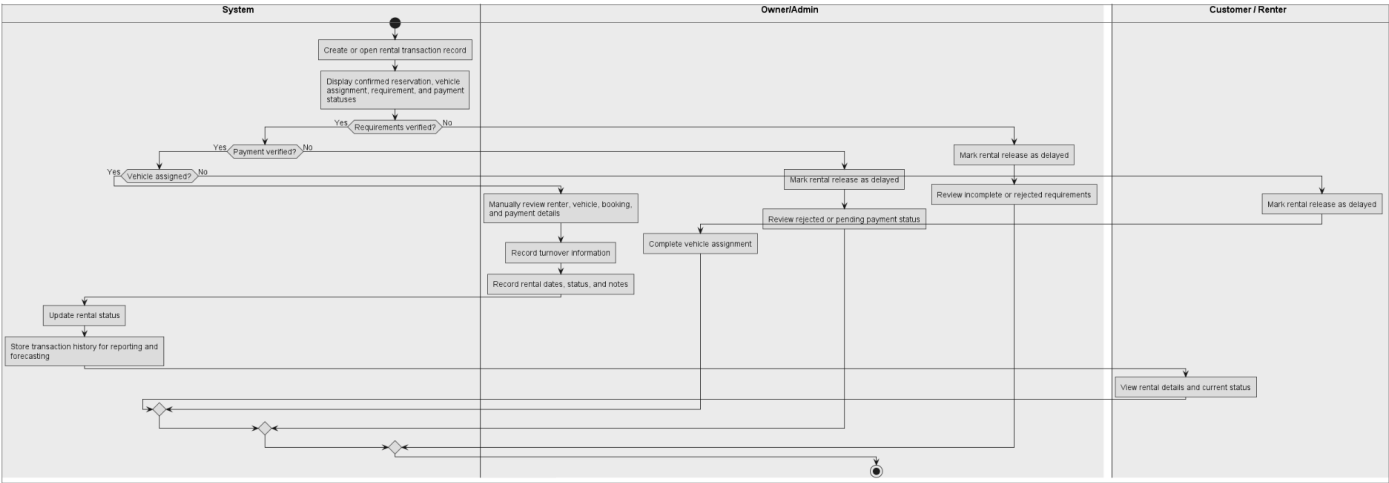


Figure 48. Track Booking Status

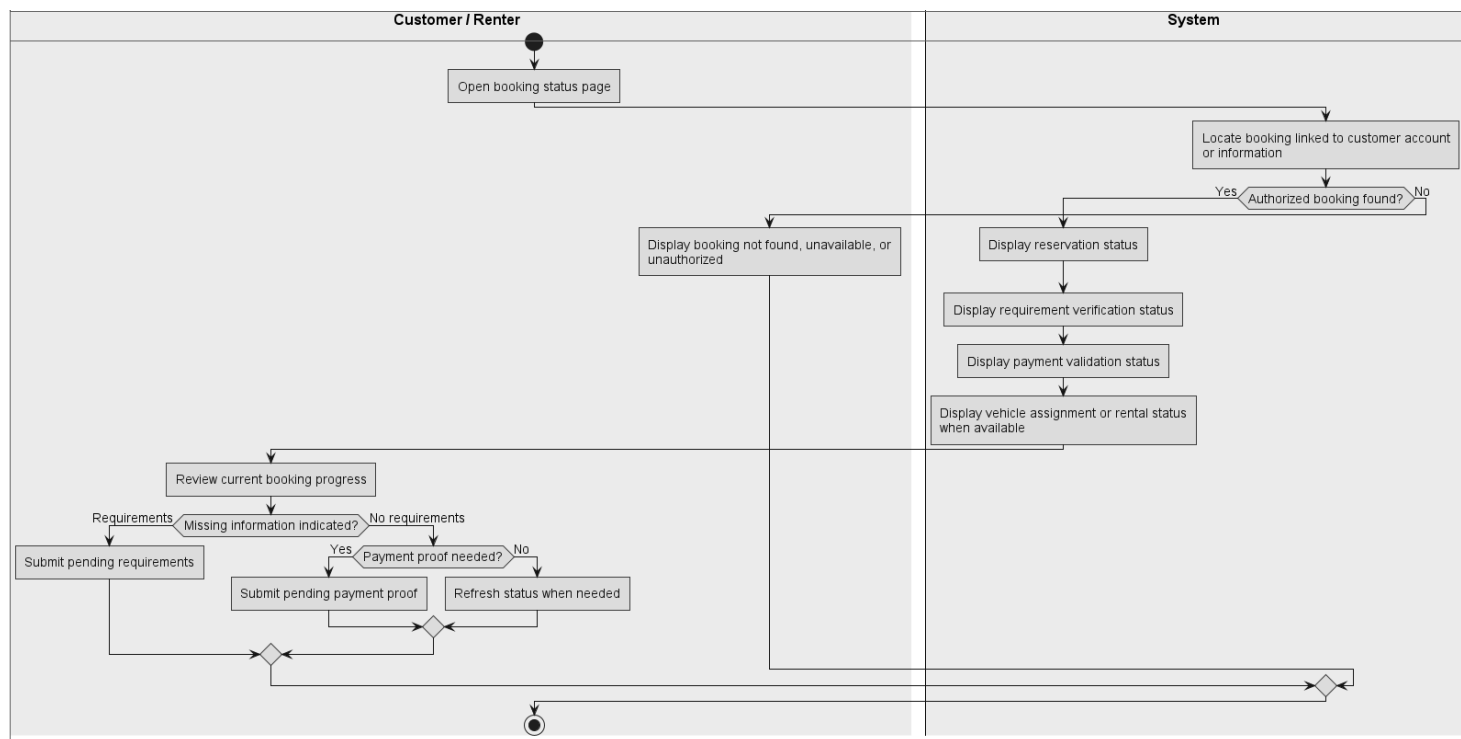


Figure 49. Browse Vehicles and Reserve

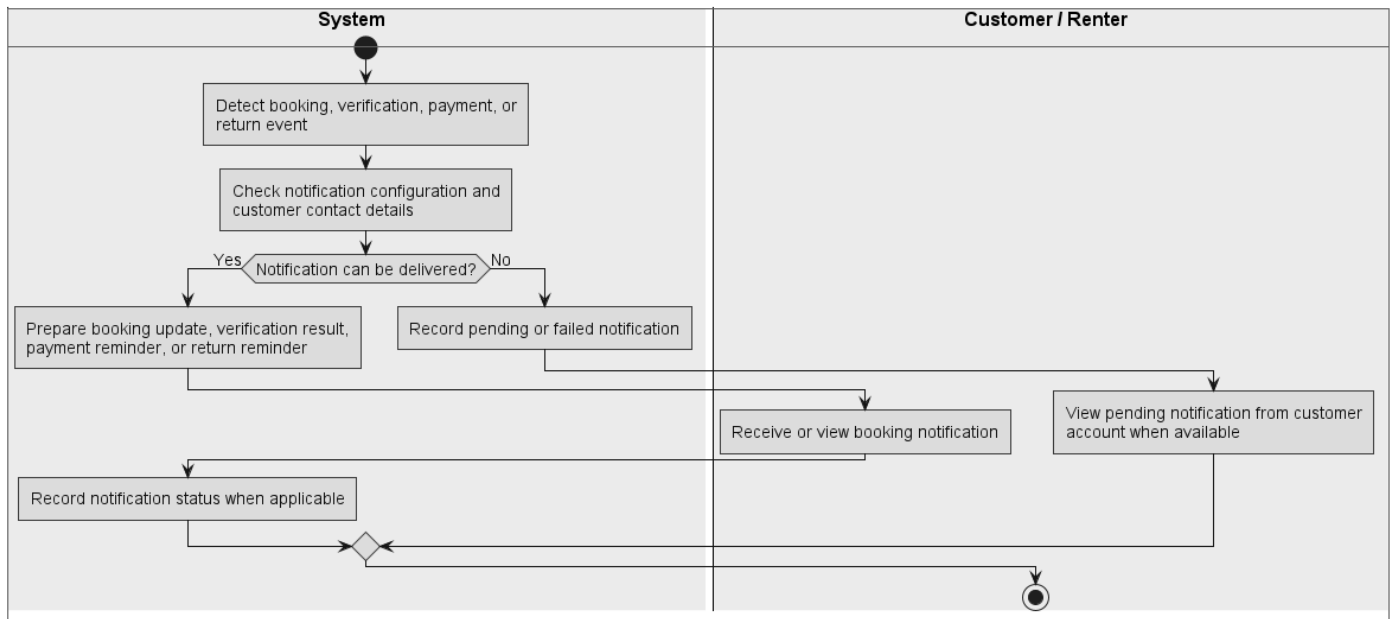


Figure 50. Monitor Fleet Status

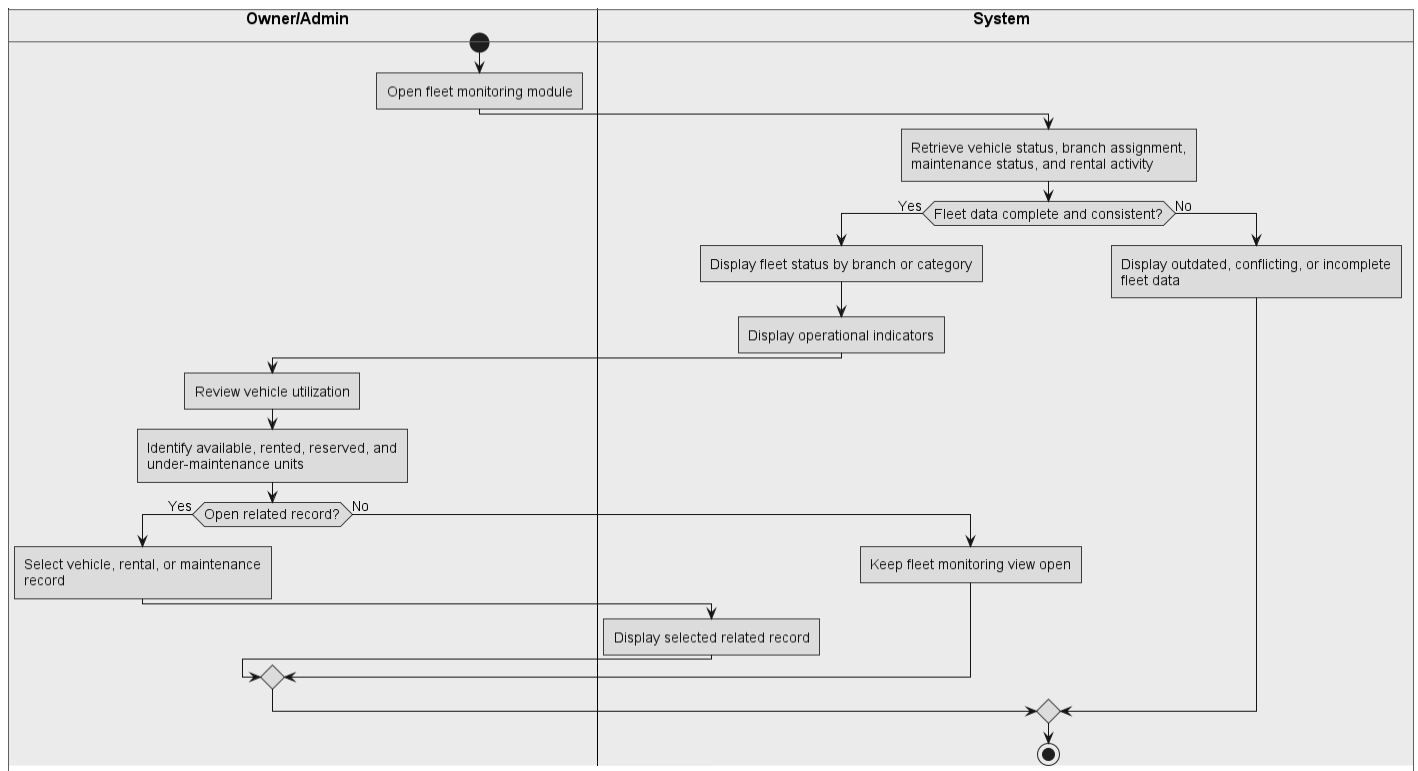


Figure 51. Process Vehicle Return Settlement

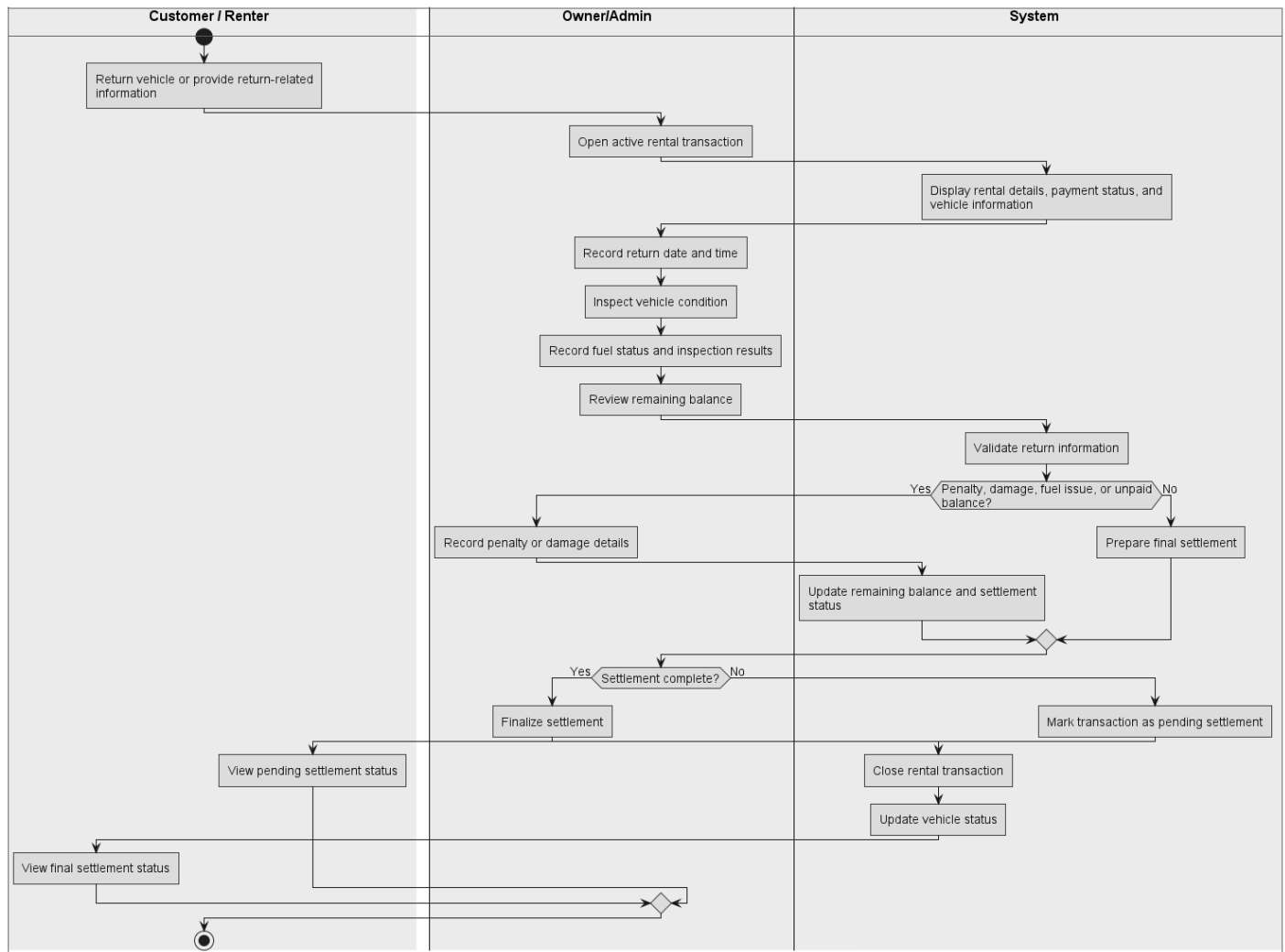


Figure 52. Record Penalties and Damages

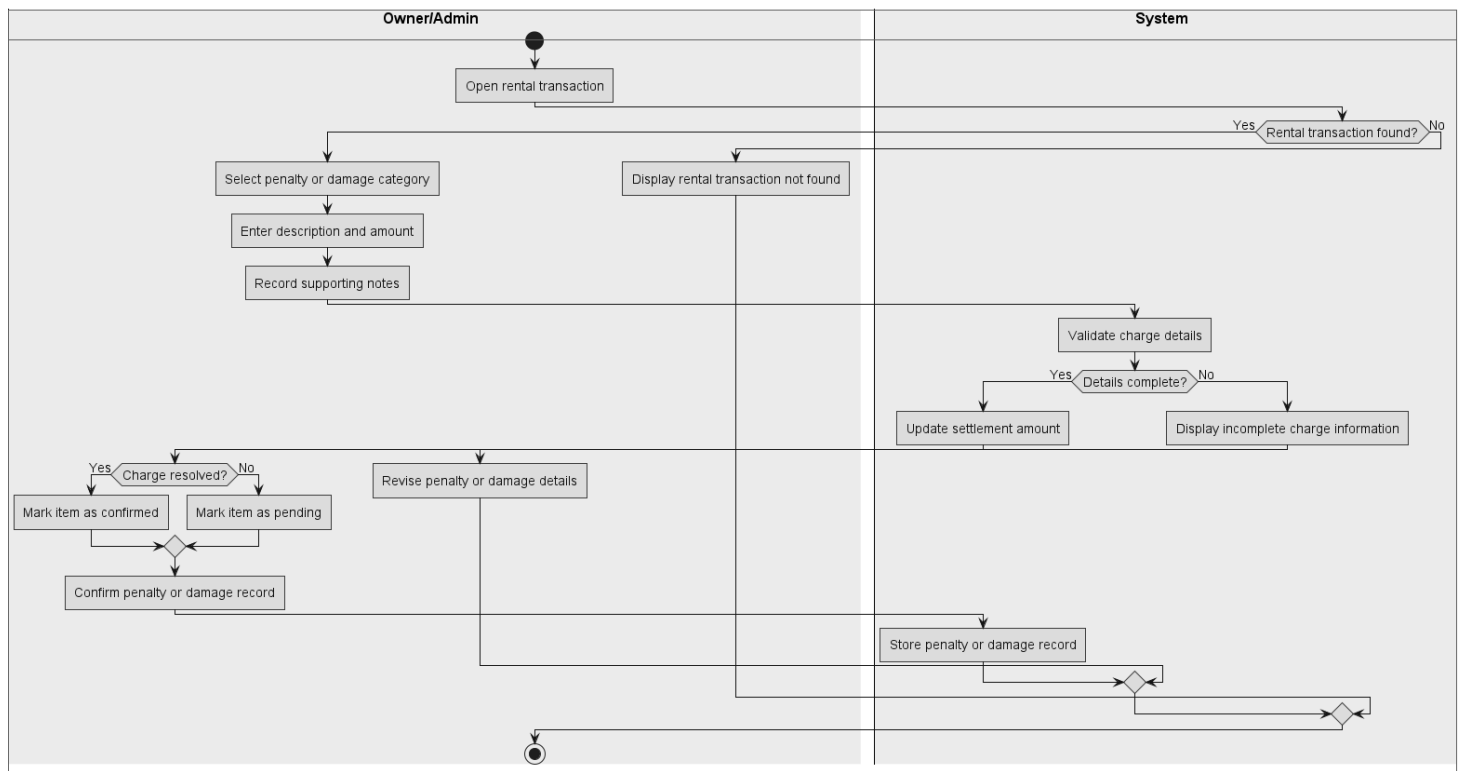
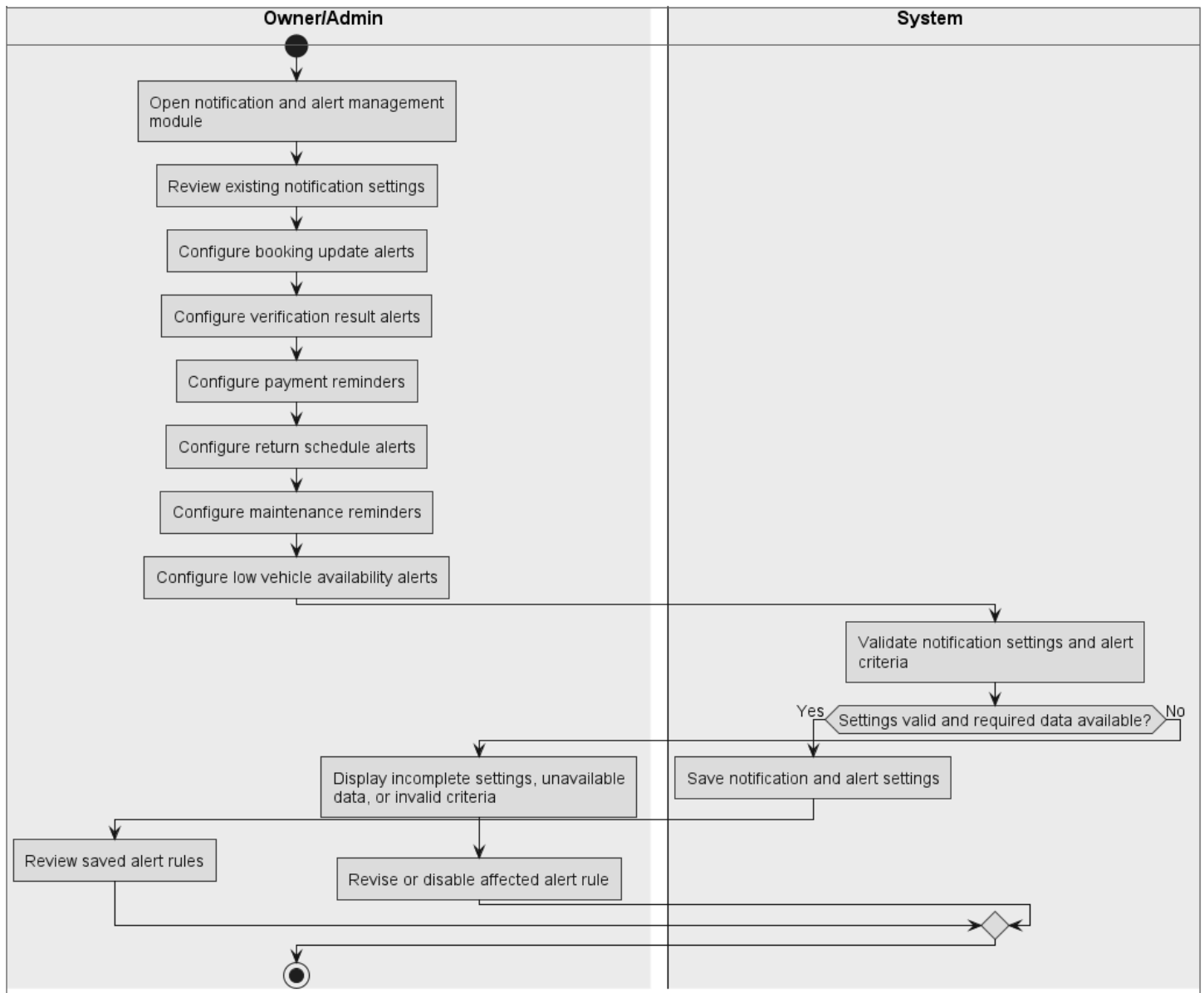


Figure 53. Manage Notifications and Alerts



```

graph TD
    subgraph OwnerAdmin [Owner/Admin]
        Start(( )) --> Open[Open reports module]
        Open --> Select[Select report type]
        Select --> SetFilters[Set date, branch, vehicle category, or other filters]
        SetFilters --> End(( ))
    end

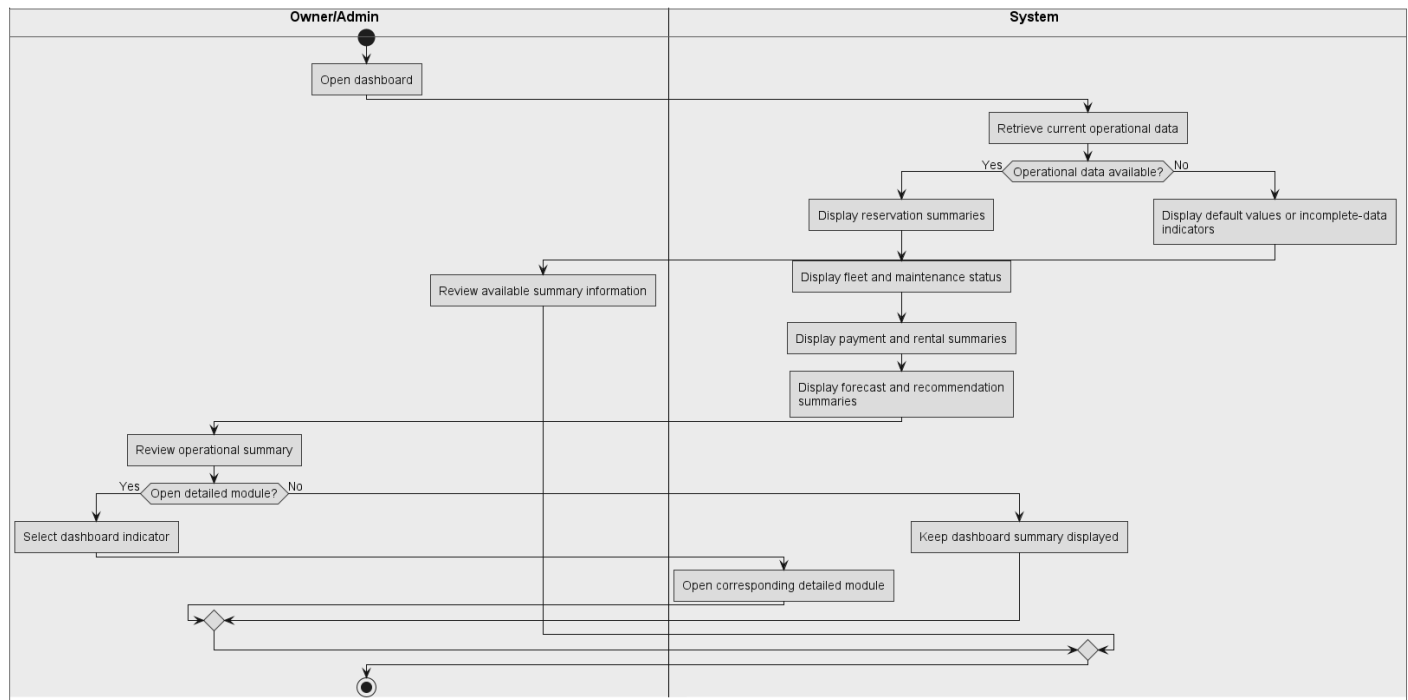
    subgraph System [System]
        SetFilters --> Validate[Validate report criteria]
        Validate --> Criteria{Criteria valid?}
        Criteria -- No --> DisplayInvalid[Display invalid report criteria]
        DisplayInvalid --> DisplayNone[Display no matching records]
        DisplayNone --> AdjustFilters[Adjust filters or view available data only]
        AdjustFilters --> ViewReport[View generated report]
        ViewReport --> Revise{Revise report?}
        Revise -- Yes --> AdjustFilters
        Revise -- No --> KeepReport[Keep generated report]
        AdjustFilters --> End
        KeepReport --> End
    end

    Criteria -- Yes --> Retrieve[Retrieve matching operational records]
    Retrieve --> Found{Matching records found?}
    Found -- No --> DisplayNone
    Found -- Yes --> Generate[Generate report output]
    Generate --> End
  
```

The flowchart is divided into two swimlanes: **Owner/Admin** and **System**.

- Owner/Admin Lane:**
 - Starts with a black circle (Start).
 - Process: **Open reports module** → **Select report type** → **Set date, branch, vehicle category, or other filters**.
 - Ends with a bullseye (End).
- System Lane:**
 - Receives input from the Owner/Admin lane at the **Set date, branch, vehicle category, or other filters** step.
 - Process: **Validate report criteria** → **Criteria valid?** (Decision).
 - If **Criteria valid?** is **No**, it goes to **Display invalid report criteria**.
 - If **Criteria valid?** is **Yes**, it goes to **Retrieve matching operational records**.
 - From **Retrieve matching operational records**, it goes to **Matching records found?** (Decision).
 - If **Matching records found?** is **No**, it goes to **Display no matching records**.
 - If **Matching records found?** is **Yes**, it goes to **Generate report output**.
 - From **Display no matching records**, it goes to **Adjust filters or view available data only**.
 - From **Adjust filters or view available data only**, it goes to **View generated report**.
 - From **View generated report**, it goes to **Revise report?** (Decision).
 - If **Revise report?** is **Yes**, it loops back to **Adjust filters or view available data only**.
 - If **Revise report?** is **No**, it goes to **Keep generated report**.
 - Both **Generate report output** and **Keep generated report** lead to the final end point (bullseye).

Figure 55. View Dashboard Operational Summary



Database Schema

The database schema represents 40 public tables supporting application profiles, branches, vehicles, bookings, requirement and payment verification, rental release and return inspection, maintenance, forecasting, supply evaluation, allocation recommendations, notifications, audit events, backup metadata, and public contact records. Supabase Auth is shown through its external auth.users reference. The overview and continued detail views show table attributes, keys, and relationships; database checks, functions, triggers, and access policies are documented in the implementation rather than reproduced in the diagram.

Figure 60. Database Schema Detail 1: Demand Coverage, Public Contact Locations, and Notification Conditions

public.forecast_demand_coverage <table><tr><td>id</td><td>smallint</td><td>NN</td></tr><tr><td>tracking_started_at</td><td>timestampz</td><td>NN</td></tr><tr><td>created_at</td><td>timestampz</td><td>NN</td></tr></table>	id	smallint	NN	tracking_started_at	timestampz	NN	created_at	timestampz	NN	public.public_contact_locations <table><tr><td>id</td><td>uuid</td><td>NN</td></tr><tr><td>name</td><td>text</td><td>NN</td></tr><tr><td>address</td><td>text</td><td>NN</td></tr><tr><td>note</td><td>text</td><td></td></tr><tr><td>sort_order</td><td>smallint</td><td>NN</td></tr><tr><td>is_active</td><td>boolean</td><td>NN</td></tr><tr><td>created_at</td><td>timestampz</td><td>NN</td></tr><tr><td>updated_at</td><td>timestampz</td><td>NN</td></tr></table>	id	uuid	NN	name	text	NN	address	text	NN	note	text		sort_order	smallint	NN	is_active	boolean	NN	created_at	timestampz	NN	updated_at	timestampz	NN	public.operational_notification_conditions <table><tr><td>condition_type</td><td>text</td><td>NN</td></tr><tr><td>related_entity_type</td><td>text</td><td>NN</td></tr><tr><td>related_entity_id</td><td>uuid</td><td>NN</td></tr><tr><td>is_active</td><td>boolean</td><td>NN</td></tr><tr><td>occurrence_count</td><td>bigint</td><td>NN</td></tr><tr><td>title</td><td>text</td><td>NN</td></tr><tr><td>message</td><td>text</td><td>NN</td></tr><tr><td>activated_at</td><td>timestampz</td><td></td></tr><tr><td>resolved_at</td><td>timestampz</td><td></td></tr><tr><td>last_evaluated_at</td><td>timestampz</td><td>NN</td></tr></table>	condition_type	text	NN	related_entity_type	text	NN	related_entity_id	uuid	NN	is_active	boolean	NN	occurrence_count	bigint	NN	title	text	NN	message	text	NN	activated_at	timestampz		resolved_at	timestampz		last_evaluated_at	timestampz	NN
id	smallint	NN																																																															
tracking_started_at	timestampz	NN																																																															
created_at	timestampz	NN																																																															
id	uuid	NN																																																															
name	text	NN																																																															
address	text	NN																																																															
note	text																																																																
sort_order	smallint	NN																																																															
is_active	boolean	NN																																																															
created_at	timestampz	NN																																																															
updated_at	timestampz	NN																																																															
condition_type	text	NN																																																															
related_entity_type	text	NN																																																															
related_entity_id	uuid	NN																																																															
is_active	boolean	NN																																																															
occurrence_count	bigint	NN																																																															
title	text	NN																																																															
message	text	NN																																																															
activated_at	timestampz																																																																
resolved_at	timestampz																																																																
last_evaluated_at	timestampz	NN																																																															

Figure 61. Database Schema Detail 2: Authentication, Profiles, Branches, Vehicle Categories, Vehicles, and Payment Methods



Figure 62. Database Schema Detail 3: Bookings, Requirement Sets, Payments, Rentals, and Maintenance

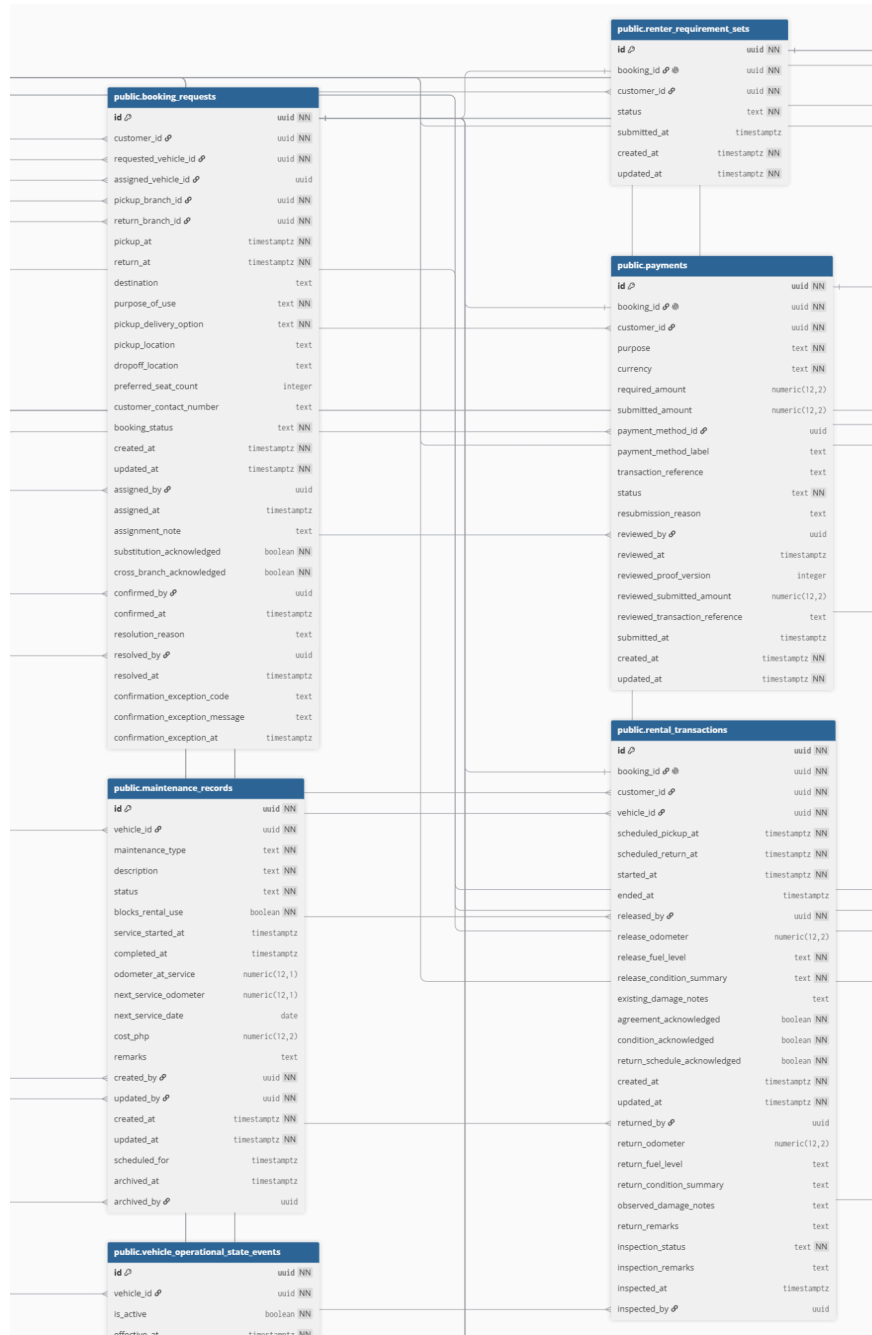


Figure 64. Database Schema Detail 5: Notification Preferences, Audit Events, Email Deliveries, Backup Runs, and Rate Cards (continued)

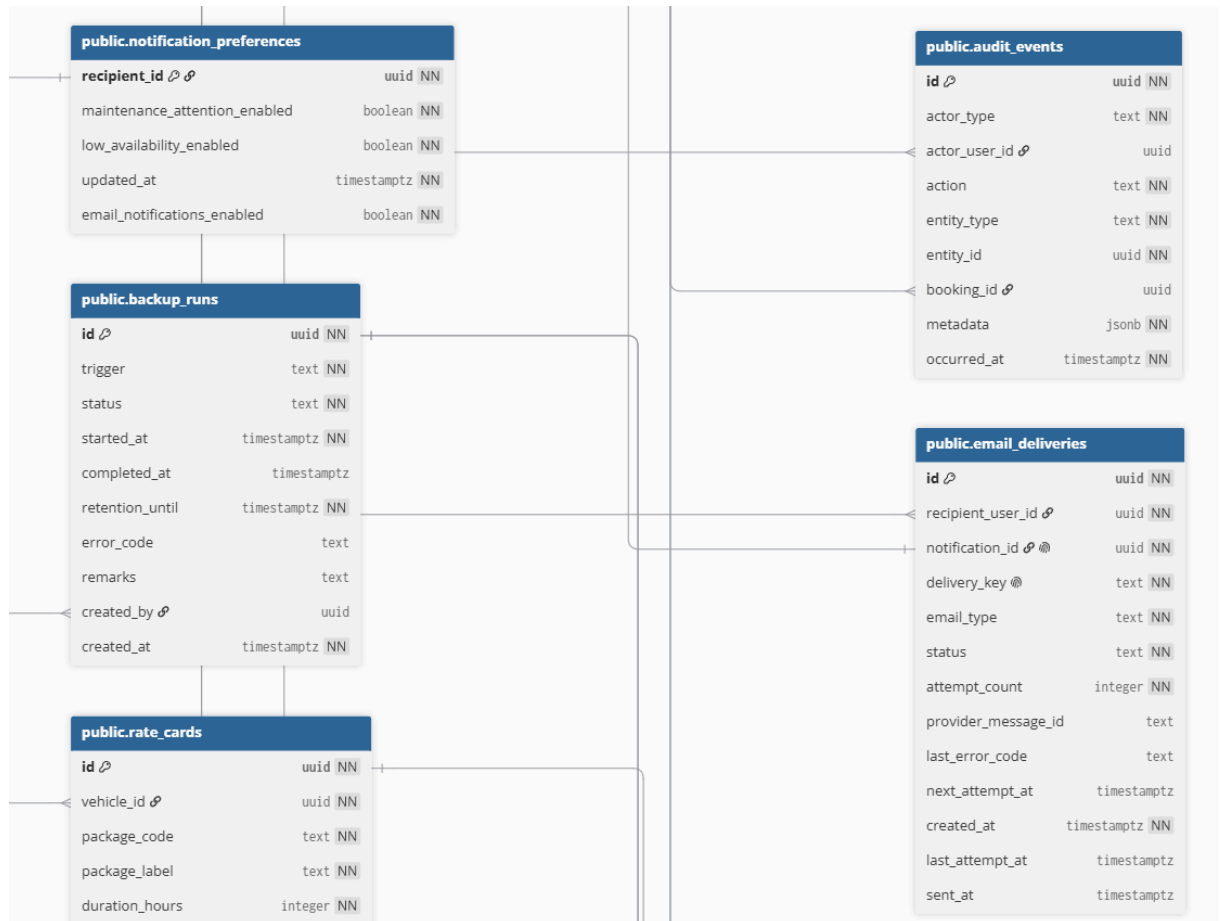


Figure 65. Database Schema Detail 6: Rate Cards, Vehicle Images, Backup Artifacts, Recovery Drills, and Public Contact Settings (continued)

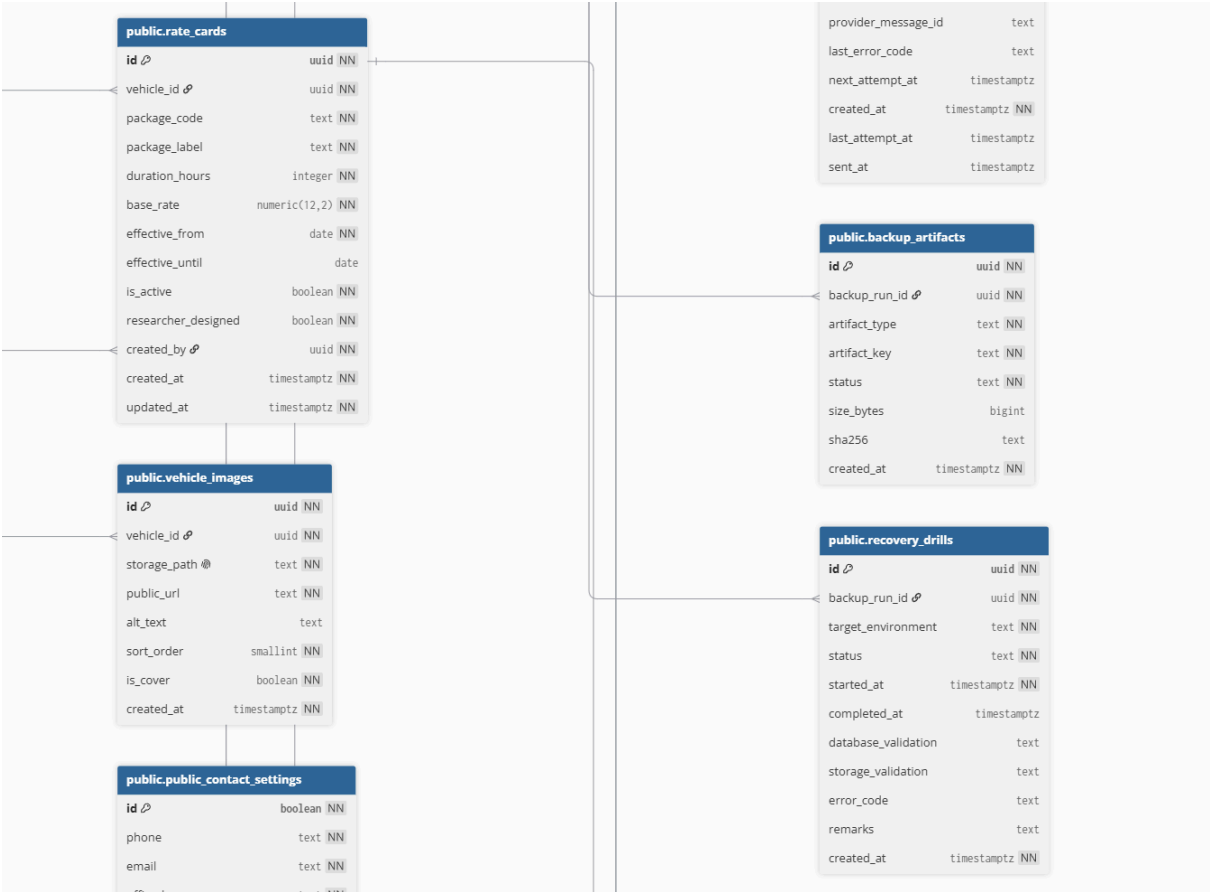


Figure 66. Database Schema Detail 7: Public Contact Settings, Contact Inquiries, and Quotation Records (continued)

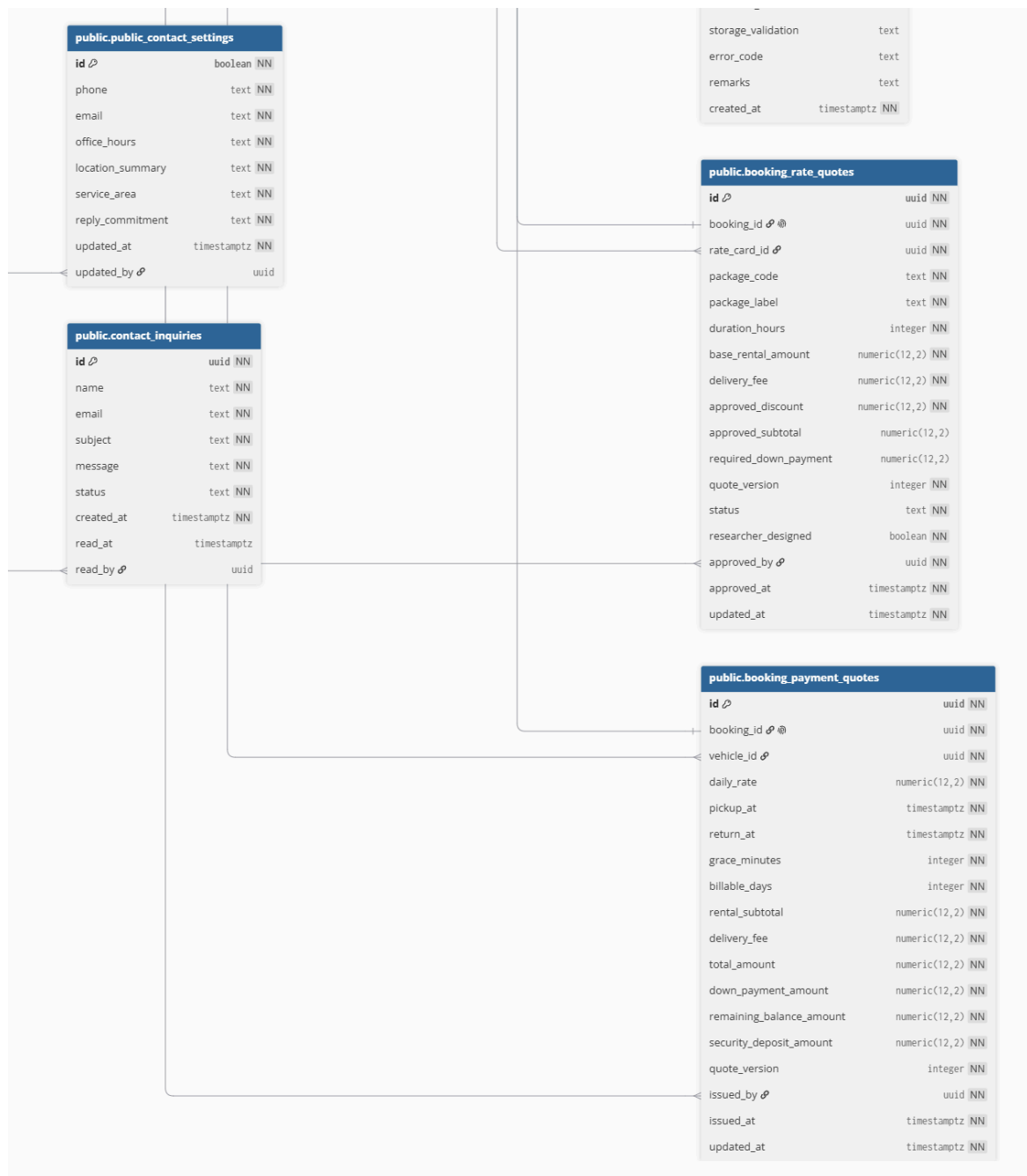


Figure 67. Database Schema Detail 8: Requirement Documents, Payment Proofs, Forecast Inputs, and Supply Evaluations

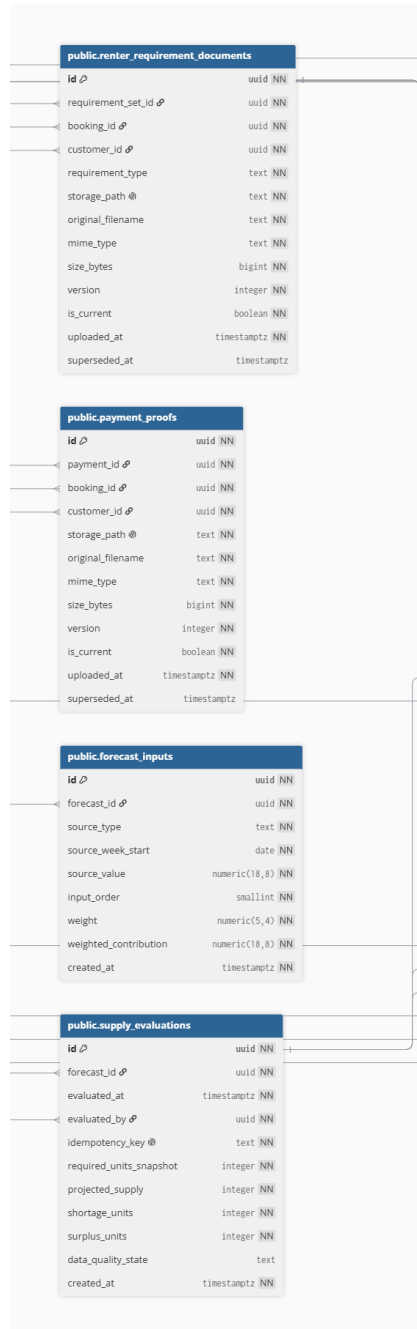
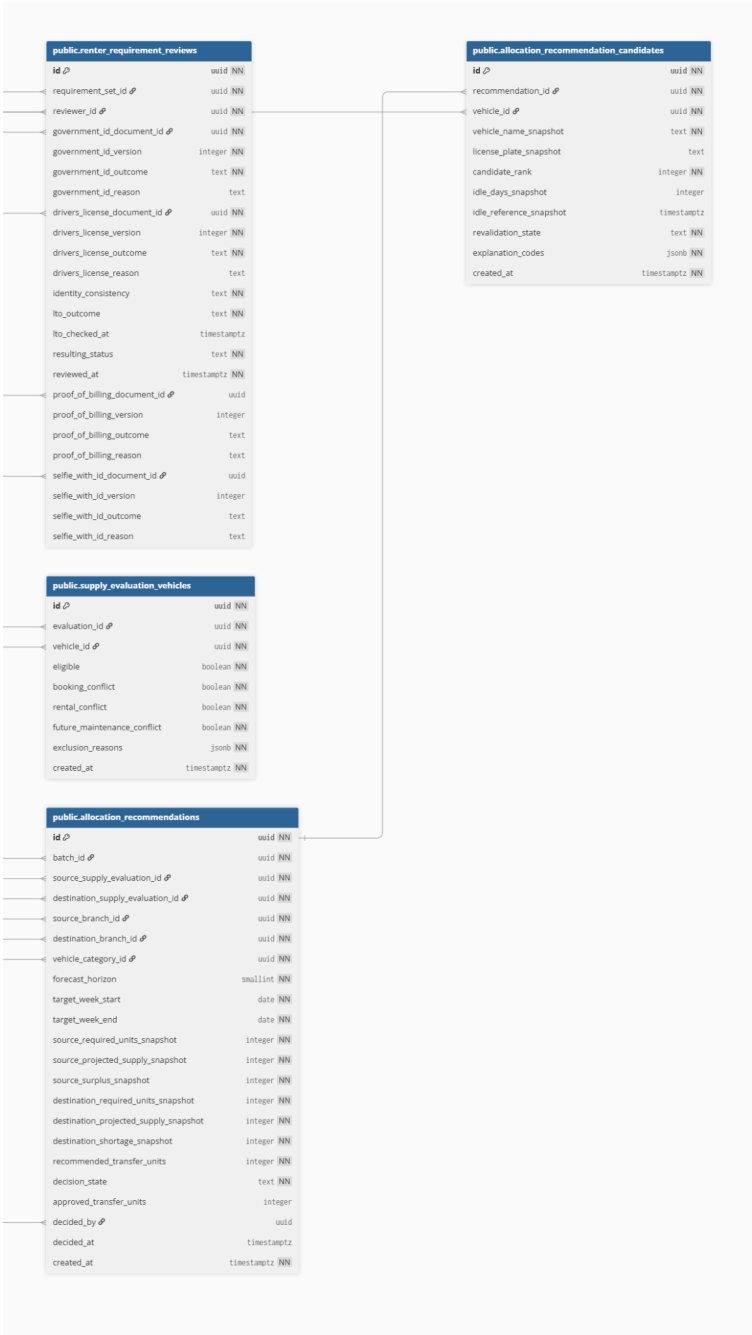


Figure 68. Database Schema Detail 9: Requirement Reviews, Supply Evaluation
Vehicles, and Allocation Recommendations



The detail views continue portions of the same overview. Where a table crosses a view boundary, read the adjacent detail view together with its data dictionary entry.

Data Dictionary

The following data dictionary documents the implemented public database tables that support the rental workflow and decision-support features. Authentication credentials are managed separately by Supabase Auth. Retained earlier quotation structures are identified in their table descriptions.

Field Size identifies the fixed data-type size in bytes where applicable. NUMERIC precision and scale indicate the permitted total digits and decimal places, rather than a fixed byte size. An em dash (—) indicates variable-length data without a declared column-length limit. Required and Optional refer to database nullability; application rules may impose additional conditions.

Table 32

Data Dictionary – User Profiles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. FK to auth.users(id). Required.
email	TEXT	—	Email address associated with the record. Unique when present. Optional.
full_name	TEXT	—	Name displayed for the application user. Required.
phone_number	TEXT	—	Contact telephone number of the user. Optional.
user_type	TEXT	—	Application role used to distinguish administrative, operational, and

Attributes	Data Type	Field Size	Description
			customer access. Allowed values: Owner/Admin, Operations Staff, Customer/Renter. Required.
account_status	TEXT	—	Account-state label; defaults to Active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
street_address	TEXT	—	Street and house or building address. Optional.
barangay	TEXT	—	Barangay portion of the address. Optional.
city_municipality	TEXT	—	City or municipality portion of the address. Optional.
province	TEXT	—	Province portion of the address. Optional.
postal_code	TEXT	—	Postal code portion of the address. Optional.

Table 32 describes the table in the database named profiles, where application profile, contact, address, account-status, and role information for authenticated users is stored. Authentication credentials are managed separately through Supabase Auth. The application roles are Owner/Admin, Operations Staff, and Customer/Renter.

Table 33

Data Dictionary – Branches

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name of the operational branch. Unique when present. Required.
address	TEXT	—	Address of the operational branch. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 33 describes the table in the database named branches, where operational branch names, addresses, and active flags are maintained for fleet assignment, bookings, forecasting, and allocation planning.

Table 34

Data Dictionary – Vehicle Categories

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.

Attributes	Data Type	Field Size	Description
name	TEXT	—	Name of the vehicle category. Unique when present. Required.
description	TEXT	—	Description of the vehicle category. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 34 describes the table in the database named `vehicle_categories`, where vehicle classifications and their descriptions are maintained for fleet organization and category-based demand planning.

Table 35

Data Dictionary – Vehicles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Vehicle name displayed in the fleet and catalog. Required.
category_id	UUID	16 bytes	Vehicle category assigned to the record. FK to <code>public.vehicle_categories(id)</code> . Required.

Attributes	Data Type	Field Size	Description
branch_id	UUID	16 bytes	Operational branch associated with the record. FK to public.branches(id). Required.
license_plate	TEXT	—	Vehicle registration plate; unique when supplied. Unique when present. Optional.
transmission	TEXT	—	Recorded transmission type. Optional.
fuel_type	TEXT	—	Recorded fuel type. Optional.
seat_capacity	INTEGER	4 bytes	Passenger seating capacity; positive when supplied. Optional.
daily_rate	NUMERIC(12,2)	Precision 12, scale 2	Vehicle rental rate per day in Philippine pesos. Optional.
reference_fuel_efficiency_km_per_liter	NUMERIC(8,2)	Precision 8, scale 2	Reference fuel efficiency in kilometers per liter; positive when supplied and used for reference fuel estimates. Optional.
image_url	TEXT	—	Image reference associated with the vehicle. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
current_odometer_km	NUMERIC(12,1)	Precision 12, scale 1	Latest recorded odometer reading in kilometers. Optional.
condition_blocks_rental_use	BOOLEAN	1 byte	Indicates whether the recorded vehicle condition blocks rental use. Required.
large_luggage_capacity	INTEGER	4 bytes	Recorded large-luggage capacity used in vehicle suitability assessment. Optional.

Table 35 describes the table in the database named vehicles, where vehicle specifications, branch assignment, category, reference rental rate, fuel efficiency, luggage capacity, and readiness-related information are maintained. Availability is assessed using operational records and conditions, including active rentals, confirmed bookings, maintenance, and vehicle condition.

Table 36

Data Dictionary – Vehicle Images

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
public_url	TEXT	—	Public image address used to display the gallery image. Required.
alt_text	TEXT	—	Alternative text describing the vehicle image. Optional.
sort_order	SMALLINT	2 bytes	Gallery position from 0 to 4, unique within the vehicle. Required.
is_cover	BOOLEAN	1 byte	Indicates whether the image is the vehicle's cover; at most one cover per vehicle. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 36 describes the table in the database named `vehicle_images`, where ordered vehicle-gallery images and the designated cover image are recorded. Each vehicle may have up to five ordered image positions and at most one cover image.

Table 37

Data Dictionary – Vehicle Operational State Events

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to <code>public.vehicles(id)</code> . Required.
is_active	BOOLEAN	1 byte	Vehicle active state recorded by this event. Required.
effective_at	TIMESTAMP TZ	8 bytes	Date and time from which the recorded active state applies. Required.
recorded_by	UUID	16 bytes	Authenticated user associated with the state recording. FK to <code>auth.users(id)</code> . Optional.
source	TEXT	—	Origin label of the state event; defaults to <code>vehicle_mutation</code> . Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 37 describes the table in the database named `vehicle_operational_state_events`, where changes in a vehicle's active state and their effective times are preserved.

These events support historical operational-state interpretation rather than serving as a complete branch-transfer history.

Table 38

Data Dictionary – Booking Requests

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
requested_vehicle_id	UUID	16 bytes	Vehicle initially requested by the customer. FK to public.vehicles(id). Required.
assigned_vehicle_id	UUID	16 bytes	Vehicle assigned for fulfillment of the booking. FK to public.vehicles(id). Optional.
pickup_branch_id	UUID	16 bytes	Operational branch recorded for pickup. FK to public.branches(id). Required.
return_branch_id	UUID	16 bytes	Operational branch recorded for return. FK to public.branches(id). Required.
pickup_at	TIMESTAMP TZ	8 bytes	Scheduled pickup date and time. Required.
return_at	TIMESTAMP TZ	8 bytes	Scheduled return date and time; later than pickup. Required.
destination	TEXT	—	Destination entered for the planned rental. Optional.
purpose_of_use	TEXT	—	Customer's stated purpose for the rental. Required.

Attributes	Data Type	Field Size	Description
pickup_delivery_option	TEXT	—	Customer's selection of pickup or delivery. Allowed values: pickup, delivery. Required.
pickup_location	TEXT	—	Delivery pickup address; absent for the pickup option. Optional.
dropoff_location	TEXT	—	Delivery drop-off address recorded when delivery is selected. Optional.
preferred_seat_count	INTEGER	4 bytes	Requested seat count; positive when supplied. Optional.
customer_contact_number	TEXT	—	Contact telephone number supplied for the booking. Optional.
booking_status	TEXT	—	Current booking lifecycle state. Allowed values: Draft, Submitted, Confirmed, Rejected, Cancelled. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
assigned_by	UUID	16 bytes	User who recorded the vehicle assignment. FK to public.profiles(id). Optional.
assigned_at	TIMESTAMP TZ	8 bytes	Date and time when the vehicle assignment was recorded. Optional.
assignment_note	TEXT	—	Explanation accompanying vehicle assignment or an acknowledged substitution or cross-branch assignment. Optional.
substitution_acknowledged	BOOLEAN	1 byte	Indicates whether replacement of the requested vehicle was acknowledged. Required.
cross_branch_acknowledged	BOOLEAN	1 byte	Indicates whether assignment from another branch was acknowledged. Required.

Attributes	Data Type	Field Size	Description
confirmed_by	UUID	16 bytes	User who confirmed the booking. FK to public.profiles(id). Optional.
confirmed_at	TIMESTAMP TZ	8 bytes	Date and time when booking confirmation was recorded. Optional.
resolution_reason	TEXT	—	Reason recorded for the booking's resolution. Optional.
resolved_by	UUID	16 bytes	User who recorded the booking resolution. FK to public.profiles(id). Optional.
resolved_at	TIMESTAMP TZ	8 bytes	Date and time of the booking resolution. Optional.
confirmation_exception_code	TEXT	—	Classified condition preventing automatic confirmation after payment verification. Allowed values: assignment_required, vehicle_unavailable, vehicle_maintenance_unready, vehicle_conflict, stale_assignment, assignment_expectation_required, pickup_window_elapsed, substitution_ack_required, cross_branch_ack_required, booking_not_submitted. Optional.
confirmation_exception_message	TEXT	—	Readable explanation of the condition preventing automatic confirmation. Optional.
confirmation_exception_at	TIMESTAMP TZ	8 bytes	Date and time when the confirmation exception was recorded. Optional.

Table 38 describes the table in the database named `booking_requests`, where customer rental requests, requested and assigned vehicles, schedules, locations, assignment acknowledgements, booking decisions, and confirmation exceptions are recorded. Return and pickup branches refer to operational branch records.

Table 39

Data Dictionary – Booking Finder Context

Attributes	Data Type	Field Size	Description
booking_id	UUID	16 bytes	Booking associated with the record. Primary key. FK to public.booking_requests(id). Required.
selected_vehicle_id	UUID	16 bytes	Vehicle selected from the Finder recommendation. FK to public.vehicles(id). Required.
requested_start	TIMESTAMP TZ	8 bytes	Rental start date and time entered in the Finder. Required.
requested_end	TIMESTAMP TZ	8 bytes	Rental end date and time entered in the Finder. Required.
passenger_count	INTEGER	4 bytes	Positive passenger count entered in the Finder. Required.
maximum_budget	NUMERIC	—	Positive maximum budget entered in the Finder. Required.
preferred_category_id	UUID	16 bytes	Optional vehicle-category preference entered in the Finder. FK to public.vehicle_categories(id). Optional.
destination	TEXT	—	Destination entered for the planned rental. Optional.
recommendation_rank	INTEGER	4 bytes	Positive position of the selected vehicle in the recommendation results. Required.
finder_baseline	TEXT	—	Label identifying the Finder recommendation baseline used for this booking. Required.

Attributes	Data Type	Field Size	Description
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
large_bag_count	INTEGER	4 bytes	Requested number of large bags, when supplied. Optional.

Table 39 describes the table in the database named `booking_finder_context`, where the applicable customer inputs, selected vehicle, and recommendation provenance associated with a Finder-assisted booking are preserved. External weather, road, and route responses are retrieved when needed and temporarily cached by the server.

Table 40

Data Dictionary – Booking Creation Idempotency

Attributes	Data Type	Field Size	Description
customer_id	UUID	16 bytes	Customer associated with the record. Part of the composite primary key (customer_id, idempotency_key). FK to public.profiles(id). Required.
idempotency_key	UUID	16 bytes	Request identifier used to prevent duplicate processing of the same operation. Part of the composite primary key (customer_id, idempotency_key). Required.
request_fingerprint	TEXT	—	64-character hexadecimal fingerprint used to compare retried booking requests. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.

Attributes	Data Type	Field Size	Description
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 40 describes the table in the database named booking_creation_idempotency, where customer request identifiers and request fingerprints are associated with the created booking to prevent duplicate booking creation when the same request is retried.

Table 41

Data Dictionary – Renter Requirement Sets

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
status	TEXT	—	Recorded state of the record. Allowed values: Not Submitted, Pending Review, Needs Resubmission, Verified. Required.
submitted_at	TIMESTAMP TZ	8 bytes	Date and time when the submission was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Attributes	Data Type	Field Size	Description
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 41 describes the table in the database named renter_requirement_sets, where the requirement submission and verification state for each booking is maintained. A booking can have at most one requirement set.

Table 42

Data Dictionary – Renter Requirement Documents

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
requirement_set_id	UUID	16 bytes	Requirement set to which this record belongs. FK to public.renter_requirement_sets(id). Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
requirement_type	TEXT	—	Category of the uploaded renter requirement. Allowed values: Valid Government ID, Driver's License, Proof of Billing, Selfie with ID. Required.

Attributes	Data Type	Field Size	Description
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
original_filename	TEXT	—	Original filename provided during upload. Required.
mime_type	TEXT	—	Media type of the uploaded file; image/jpeg, image/png, or application/pdf. Required.
size_bytes	BIGINT	8 bytes	Uploaded file size; greater than zero and no more than 10,485,760 bytes. Required.
version	INTEGER	4 bytes	Positive version number identifying this file revision. Required.
is_current	BOOLEAN	1 byte	Indicates whether this file is the current version. Required.
uploaded_at	TIMESTAMP TZ	8 bytes	Date and time when the file was uploaded. Required.
superseded_at	TIMESTAMP TZ	8 bytes	Date and time when a newer file version replaced this file. Optional.

Table 42 describes the table in the database named `renter_requirement_documents`, where uploaded requirement-file references, document categories, and replacement versions are maintained. Previous versions remain distinguishable from the current file for each requirement category.

Table 43

Data Dictionary – Renter Requirement Reviews

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
requirement_set_id	UUID	16 bytes	Requirement set to which this record belongs. FK to public.renter_requirement_sets(id). Required.
reviewer_id	UUID	16 bytes	User who performed the document review. FK to public.profiles(id). Required.
government_id_document_id	UUID	16 bytes	Government id file examined in this review. FK to public.renter_requirement_documents(id). Required.
government_id_version	INTEGER	4 bytes	Version of the government ID file examined. Required.
government_id_outcome	TEXT	—	Review outcome for the government ID file. Allowed values: Accepted, Needs Replacement. Required.
government_id_reason	TEXT	—	Reason recorded when the government ID file needs replacement. Optional.
drivers_license_document_id	UUID	16 bytes	Driver's license file examined in this review. FK to public.renter_requirement_documents(id). Required.
drivers_license_version	INTEGER	4 bytes	Version of the driver's license file examined. Required.
drivers_license_outcome	TEXT	—	Review outcome for the driver's license file. Allowed values: Accepted, Needs Replacement. Required.
drivers_license_reason	TEXT	—	Reason recorded when the driver's license file needs replacement. Optional.
identity_consistency	TEXT	—	Recorded consistency between the reviewed identity documents.

Attributes	Data Type	Field Size	Description
			Allowed values: Consistent, Concern. Required.
lto_outcome	TEXT	—	Retained license-check outcome; the current review endpoint records Not Checked and does not require an LTO check. Allowed values: Not Checked, Clear, Concern, Unavailable. Required.
lto_checked_at	TIMESTAMP TZ	8 bytes	Retained date and time of a recorded LTO check, if present. Optional.
resulting_status	TEXT	—	Requirement-set state resulting from the review. Allowed values: Pending Review, Needs Resubmission, Verified. Required.
reviewed_at	TIMESTAMP TZ	8 bytes	Date and time when the review was recorded. Required.
proof_of_billing_document_id	UUID	16 bytes	Proof of billing file examined in this review. FK to public.renter_requirement_documents(id). Optional.
proof_of_billing_version	INTEGER	4 bytes	Version of the proof of billing file examined. Optional.
proof_of_billing_outcome	TEXT	—	Review outcome for the proof of billing file. Allowed values: Accepted, Needs Replacement. Optional.
proof_of_billing_reason	TEXT	—	Reason recorded when the proof of billing file needs replacement. Optional.
selfie_with_id_document_id	UUID	16 bytes	Selfie with id file examined in this review. FK to public.renter_requirement_documents(id). Optional.
selfie_with_id_version	INTEGER	4 bytes	Version of the selfie with ID file examined. Optional.

Attributes	Data Type	Field Size	Description
selfie_with_id_outcome	TEXT	—	Review outcome for the selfie with ID file. Allowed values: Accepted, Needs Replacement. Optional.
selfie_with_id_reason	TEXT	—	Reason recorded when the selfie with ID file needs replacement. Optional.

Table 43 describes the table in the database named `renter_requirement_reviews`, where review decisions are recorded against the document versions examined, together with identity consistency and the resulting requirement-set status. The retained LTO outcome fields do not establish an automated license-verification integration or a current LTO review gate.

Table 44

Data Dictionary – Booking Payment Quotes

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to <code>public.booking_requests(id)</code> . Unique when present. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to <code>public.vehicles(id)</code> . Required.
daily_rate	NUMERIC(12,2)	Precision 12, scale 2	Vehicle rental rate per day in Philippine pesos. Required.
pickup_at	TIMESTAMP TZ	8 bytes	Scheduled pickup date and time. Required.

Attributes	Data Type	Field Size	Description
return_at	TIMESTAMP TZ	8 bytes	Scheduled return date and time; later than pickup. Required.
grace_minutes	INTEGER	4 bytes	Grace interval used when determining billable rental days; defaults to 60 minutes and is limited to 0–240. Required.
billable_days	INTEGER	4 bytes	Positive number of rental days charged by the quotation. Required.
rental_subtotal	NUMERIC(12,2)	Precision 12, scale 2	Positive vehicle-rental subtotal in Philippine pesos. Required.
delivery_fee	NUMERIC(12,2)	Precision 12, scale 2	Delivery charge in Philippine pesos. Required.
total_amount	NUMERIC(12,2)	Precision 12, scale 2	Rental subtotal plus delivery fee in Philippine pesos. Required.
down_payment_amount	NUMERIC(12,2)	Precision 12, scale 2	Required initial down payment, equal to 50 percent of the total rounded to two decimal places. Required.
remaining_balance_amount	NUMERIC(12,2)	Precision 12, scale 2	Total less the down payment in Philippine pesos. Required.
security_deposit_amount	NUMERIC(12,2)	Precision 12, scale 2	Separately recorded security deposit, fixed at PHP 3,000. Required.
quote_version	INTEGER	4 bytes	Positive revision number of the quotation. Required.
issued_by	UUID	16 bytes	User who issued the current quotation. FK to public.profiles(id). Required.
issued_at	TIMESTAMP TZ	8 bytes	Date and time when the quotation was issued. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 44 describes the table in the database named `booking_payment_quotes`, where the current system-generated quotation for a booking is stored, including the vehicle's

daily rate, rental period, billable days, delivery fee, total amount, 50 percent down payment, remaining balance, and separately recorded PHP 3,000 security deposit.

Table 45

Data Dictionary – Payment Methods

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
code	TEXT	—	Unique code identifying the payment method. Unique when present. Required.
label	TEXT	—	Readable payment-method name. Required.
instructions	TEXT	—	Payment instructions presented to the customer. Required.
is_demo	BOOLEAN	1 byte	Indicates whether the method is designated as a demonstration method. Required.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
recipient_name	TEXT	—	Name of the payment recipient. Optional.
account_number	TEXT	—	Recipient's payment-account reference. Optional.
qr_image_path	TEXT	—	Stored reference to the method's QR-code image. Optional.

Table 45 describes the table in the database named `payment_methods`, where available payment-channel instructions, recipient details, account information, and optional QR-image references are maintained.

Table 46

Data Dictionary – Payments

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to <code>public.booking_requests(id)</code> . Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to <code>public.profiles(id)</code> . Required.
purpose	TEXT	—	Payment purpose, restricted to <code>initial_down_payment</code> . Required.
currency	TEXT	—	Currency code, restricted to PHP. Required.
required_amount	NUMERIC(12,2)	Precision 12, scale 2	Required initial down payment in Philippine pesos. Optional.
submitted_amount	NUMERIC(12,2)	Precision 12, scale 2	Amount submitted by the customer in Philippine pesos. Optional.
payment_method_id	UUID	16 bytes	Selected payment method, when recorded. FK to <code>public.payment_methods(id)</code> . Optional.
payment_method_label	TEXT	—	Recorded payment-method label. Optional.

Attributes	Data Type	Field Size	Description
transaction_reference	TEXT	—	Payment transaction reference supplied by the customer. Optional.
status	TEXT	—	Recorded state of the record. Allowed values: Not Submitted, Pending Verification, Needs Resubmission, Verified. Required.
resubmission_reason	TEXT	—	Reason requiring replacement or correction of the payment submission. Optional.
reviewed_by	UUID	16 bytes	User who reviewed the submission. FK to public.profiles(id). Optional.
reviewed_at	TIMESTAMP TZ	8 bytes	Date and time when the review was recorded. Optional.
reviewed_proof_version	INTEGER	4 bytes	Proof revision examined during the payment review. Optional.
reviewed_submitted_amount	NUMERIC(12,2)	Precision 12, scale 2	Submitted amount preserved with the review decision. Optional.
reviewed_transaction_reference	TEXT	—	Transaction reference preserved with the review decision. Optional.
submitted_at	TIMESTAMP TZ	8 bytes	Date and time when the submission was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 46 describes the table in the database named payments, where the initial down-payment requirement, submitted payment information, review evidence, and verification status for each booking are stored. The record is associated with versioned proof files.

Table 47

Data Dictionary – Payment Proofs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
payment_id	UUID	16 bytes	Payment to which the proof belongs. FK to public.payments(id). Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
original_filename	TEXT	—	Original filename provided during upload. Required.
mime_type	TEXT	—	Media type of the uploaded file; image/jpeg, image/png, or application/pdf. Required.
size_bytes	BIGINT	8 bytes	Uploaded proof size; greater than zero and no more than 10,485,760 bytes. Required.
version	INTEGER	4 bytes	Positive version number identifying this file revision. Required.
is_current	BOOLEAN	1 byte	Indicates whether this file is the current version. Required.
uploaded_at	TIMESTAMP TZ	8 bytes	Date and time when the file was uploaded. Required.

Attributes	Data Type	Field Size	Description
superseded_at	TIMESTAMP TZ	8 bytes	Date and time when a newer file version replaced this file. Optional.

Table 47 describes the table in the database named payment_proofs, where payment evidence-file references and their replacement versions are preserved. Each payment has at most one proof designated as current.

Table 48

Data Dictionary – Rate Cards

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
package_code	TEXT	—	Code identifying the retained rental package. Required.
package_label	TEXT	—	Readable name of the retained rental package. Required.
duration_hours	INTEGER	4 bytes	Rental-package duration in hours. Required.
base_rate	NUMERIC(12,2)	Precision 12, scale 2	Positive base package rate in Philippine pesos. Required.
effective_from	DATE	4 bytes	Beginning date of the retained rate's validity. Required.
effective_until	DATE	4 bytes	Optional end date of the retained rate's validity. Optional.

Attributes	Data Type	Field Size	Description
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
researcher_designed	BOOLEAN	1 byte	Identifies the retained structure as based on the researcher-designed quotation model. Required.
created_by	UUID	16 bytes	User who created the record. FK to public.profiles(id). Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 48 describes the table in the database named `rate_cards`, where versioned vehicle-package rates, effective dates, and researcher-designed indicators are stored as retained earlier structures. These records remain part of the physical database, while `booking_payment_quotes` supports the current quotation and payment workflow.

Table 49

Data Dictionary – Booking Rate Quotes

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.

Attributes	Data Type	Field Size	Description
rate_card_id	UUID	16 bytes	Retained rate card used for the package quotation. FK to public.rate_cards(id). Required.
package_code	TEXT	—	Code identifying the retained rental package. Required.
package_label	TEXT	—	Readable name of the retained rental package. Required.
duration_hours	INTEGER	4 bytes	Rental-package duration in hours. Required.
base_rental_amount	NUMERIC(12,2)	Precision 12, scale 2	Positive base rental amount in Philippine pesos. Required.
delivery_fee	NUMERIC(12,2)	Precision 12, scale 2	Delivery charge in Philippine pesos. Required.
approved_discount	NUMERIC(12,2)	Precision 12, scale 2	Nonnegative approved discount in Philippine pesos. Required.
approved_subtotal	NUMERIC(12,2)	Precision 12, scale 2	Stored generated amount: base rental amount plus delivery fee less approved discount. Optional.
required_down_payment	NUMERIC(12,2)	Precision 12, scale 2	Stored generated amount: 50 percent of the approved subtotal rounded to two decimal places. Optional.
quote_version	INTEGER	4 bytes	Positive revision number of the quotation. Required.
status	TEXT	—	Retained quotation state, restricted to Approved. Allowed values: Approved. Required.
researcher_designed	BOOLEAN	1 byte	Identifies the retained structure as based on the researcher-designed quotation model. Required.
approved_by	UUID	16 bytes	User who approved the retained package quotation. FK to public.profiles(id). Required.
approved_at	TIMESTAMP TZ	8 bytes	Date and time when the retained quotation was approved. Required.

Attributes	Data Type	Field Size	Description
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 49 describes the table in the database named `booking_rate_quotes`, where approved package-based quotation snapshots and their computed subtotal and down-payment values are stored as a retained earlier structure. This table is distinct from the current quotation workflow represented by `booking_payment_quotes`.

Table 50

Data Dictionary – Rental Transactions

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to <code>public.booking_requests(id)</code> . Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to <code>public.profiles(id)</code> . Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to <code>public.vehicles(id)</code> . Required.
scheduled_pickup_at	TIMESTAMP TZ	8 bytes	Pickup schedule preserved when the rental is released. Required.
scheduled_return_at	TIMESTAMP TZ	8 bytes	Return schedule preserved when the rental is released. Required.

Attributes	Data Type	Field Size	Description
started_at	TIMESTAMP TZ	8 bytes	Date and time when the vehicle was released and the rental started. Required.
ended_at	TIMESTAMP TZ	8 bytes	Date and time when the rental was closed after return. Optional.
released_by	UUID	16 bytes	User who recorded vehicle release. FK to public.profiles(id). Required.
release_odometer	NUMERIC(12,2)	Precision 12, scale 2	Odometer reading recorded at release. Optional.
release_fuel_level	TEXT	—	Recorded fuel level at vehicle release. Allowed values: Empty, 1/4, 1/2, 3/4, Full, Other/Unknown. Required.
release_condition_summary	TEXT	—	Summary of the vehicle condition at release. Required.
existing_damage_notes	TEXT	—	Damage observations recorded before release. Optional.
agreement_acknowledged	BOOLEAN	1 byte	Indicates whether the rental agreement was acknowledged. Required.
condition_acknowledged	BOOLEAN	1 byte	Indicates whether the release condition was acknowledged. Required.
return_schedule_acknowledged	BOOLEAN	1 byte	Indicates whether the return schedule was acknowledged. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
returned_by	UUID	16 bytes	User who recorded vehicle return. FK to public.profiles(id). Optional.
return_odometer	NUMERIC(12,2)	Precision 12, scale 2	Odometer reading recorded at return. Optional.

Attributes	Data Type	Field Size	Description
return_fuel_level	TEXT	—	Recorded fuel level at vehicle return. Allowed values: Empty, 1/4, 1/2, 3/4, Full, Other/Unknown. Optional.
return_condition_summary	TEXT	—	Summary of vehicle condition upon return. Optional.
observed_damage_notes	TEXT	—	Damage observations recorded upon return. Optional.
return_remarks	TEXT	—	Additional remarks concerning the return. Optional.
inspection_status	TEXT	—	State of the return inspection recorded within the rental. Allowed values: Not required, Pending, Cleared, Maintenance scheduled. Required.
inspection_remarks	TEXT	—	Remarks concerning the return inspection. Optional.
inspected_at	TIMESTAMP TZ	8 bytes	Date and time of the return inspection. Optional.
inspected_by	UUID	16 bytes	User who recorded the return inspection. FK to public.profiles(id). Optional.

Table 50 describes the table in the database named rental_transactions, where vehicle release, customer acknowledgements, actual rental start and end, return condition, and inspection information are recorded. Return inspection is embedded in this record rather than maintained in a separate inspection table.

Table 51

Data Dictionary – Maintenance Records

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
maintenance_type	TEXT	—	Recorded service or maintenance classification. Required.
description	TEXT	—	Description of the maintenance work. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Scheduled, In Progress, Completed, Overdue, Cancelled. Required.
blocks_rental_use	BOOLEAN	1 byte	Indicates whether this maintenance record blocks rental use. Required.
service_started_at	TIMESTAMP TZ	8 bytes	Date and time when servicing began. Optional.
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
odometer_at_service	NUMERIC(12,1)	Precision 12, scale 1	Odometer reading in kilometers associated with the service. Optional.
next_service_odometer	NUMERIC(12,1)	Precision 12, scale 1	Odometer threshold in kilometers for the next service. Optional.
next_service_date	DATE	4 bytes	Date threshold for the next service. Optional.
cost_php	NUMERIC(12,2)	Precision 12, scale 2	Recorded maintenance cost in Philippine pesos. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_by	UUID	16 bytes	Authenticated user who created the maintenance record. FK to auth.users(id). Required.

Attributes	Data Type	Field Size	Description
updated_by	UUID	16 bytes	Authenticated user who last updated the maintenance record. FK to auth.users(id). Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
scheduled_for	TIMESTAMP TZ	8 bytes	Date and time scheduled for maintenance. Optional.
archived_at	TIMESTAMP TZ	8 bytes	Date and time when the record was archived. Optional.
archived_by	UUID	16 bytes	User who archived the record. FK to public.profiles(id). Optional.

Table 51 describes the table in the database named `maintenance_records`, where scheduled servicing, work in progress, completion, overdue or cancelled work, rental-use restrictions, service readings, and archival information are maintained. Maintenance lifecycle states are Scheduled, In Progress, Completed, Overdue, and Cancelled.

Table 52

Data Dictionary – Forecast Runs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.

Attributes	Data Type	Field Size	Description
generated_at	TIMESTAMP TZ	8 bytes	Date and time when the generation operation was recorded. Required.
generated_by	UUID	16 bytes	User who initiated the generation operation. FK to public.profiles(id). Required.
method	TEXT	—	Forecasting method, restricted to WMA. Required.
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
coverage_start	DATE	4 bytes	Beginning date of historical demand coverage used by the run. Optional.

Table 52 describes the table in the database named `forecast_runs`, where each Weighted Moving Average forecast-generation run, its initiating user, method, and historical coverage boundary are recorded.

Table 53

Data Dictionary – Demand Forecasts

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
run_id	UUID	16 bytes	Forecast-generation run to which the projection belongs. FK to public.forecast_runs(id). Required.

Attributes	Data Type	Field Size	Description
branch_id	UUID	16 bytes	Operational branch associated with the record. FK to public.branches(id). Required.
vehicle_category_id	UUID	16 bytes	Vehicle category for which the record applies. FK to public.vehicle_categories(id). Required.
horizon	SMALLINT	2 bytes	Projection horizon from one to three weeks. Required.
target_week_start	DATE	4 bytes	Beginning date of the forecast or allocation target week. Required.
target_week_end	DATE	4 bytes	Ending boundary of the forecast or allocation target week. Required.
forecasted_demand	NUMERIC(18,8)	Precision 18, scale 8	Decimal weekly vehicle-demand projection produced by WMA. Required.
required_vehicle_units	INTEGER	4 bytes	Whole vehicle units required for planning, obtained by rounding forecast demand upward. Required.
actual_demand	NUMERIC(18,8)	Precision 18, scale 8	Observed demand for the completed target week, when finalized. Optional.
ape	NUMERIC(18,8)	Precision 18, scale 8	Absolute percentage error for the forecast when a valid actual-demand comparison is available. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 53 describes the table in the database named forecasts, where weekly vehicle-demand projections for each branch and vehicle category are stored, together with the forecast horizon, required vehicle units, and subsequent actual-demand and accuracy observations.

Table 54

Data Dictionary – Forecast Inputs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
forecast_id	UUID	16 bytes	Forecast to which this record belongs. FK to public.forecasts(id). Required.
source_type	TEXT	—	Indicates whether the calculation input is an observed actual value or a recursively generated forecast. Allowed values: Actual, Forecast. Required.
source_week_start	DATE	4 bytes	Beginning date of the input week. Required.
source_value	NUMERIC(18,8)	Precision 18, scale 8	Demand value used as a calculation input. Required.
input_order	SMALLINT	2 bytes	Recency position from 1 to 3; position 1 is the most recent input. Required.
weight	NUMERIC(5,4)	Precision 5, scale 4	WMA weight assigned to the input. Required.
weighted_contribution	NUMERIC(18,8)	Precision 18, scale 8	Product of the input demand value and its assigned weight. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 54 describes the table in the database named `forecast_inputs`, where the source observations or recursive forecast values, recency order, weights, and weighted

contributions used by a forecast are preserved. The study applies three-period WMA weights of 0.50, 0.30, and 0.20 from most recent to least recent input.

Table 55

Data Dictionary – Forecast Demand Coverage

Attributes	Data Type	Field Size	Description
id	SMALLINT	2 bytes	Single-row primary-key value, restricted to 1. Primary key. Required.
tracking_started_at	TIMESTAMP TZ	8 bytes	Date and time at which trustworthy demand tracking begins. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 55 describes the table in the database named `forecast_demand_coverage`, where the beginning of trustworthy demand tracking is recorded in a single-row coverage record. This boundary helps distinguish complete zero-demand periods from periods without adequate historical coverage.

Table 56

Data Dictionary – Supply Evaluations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
forecast_id	UUID	16 bytes	Forecast to which this record belongs. FK to public.forecasts(id). Required.
evaluated_at	TIMESTAMP TZ	8 bytes	Date and time when projected supply was assessed. Required.
evaluated_by	UUID	16 bytes	User who initiated the supply assessment. FK to public.profiles(id). Required.
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
required_units_snaps hot	INTEGER	4 bytes	Required vehicle units preserved from the evaluated forecast. Required.
projected_supply	INTEGER	4 bytes	Number of vehicles projected to be eligible for the target period. Required.
shortage_units	INTEGER	4 bytes	Nonnegative shortfall of projected supply against required units. Required.
surplus_units	INTEGER	4 bytes	Nonnegative excess of projected supply over required units. Required.
data_quality_state	TEXT	—	Recorded qualification concerning the reliability or completeness of evaluation inputs. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 56 describes the table in the database named `supply_evaluations`, where a forecast's required units are compared with projected eligible vehicle supply, preserving shortage, surplus, and data-quality observations at the time of evaluation.

Table 57

Data Dictionary – Supply Evaluation Vehicles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
evaluation_id	UUID	16 bytes	Supply evaluation to which this vehicle assessment belongs. FK to public.supply_evaluations(id). Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
eligible	BOOLEAN	1 byte	Indicates whether the vehicle was eligible during the supply assessment. Required.
booking_conflict	BOOLEAN	1 byte	Indicates whether a booking conflict was identified. Required.
rental_conflict	BOOLEAN	1 byte	Indicates whether an active-rental conflict was identified. Required.
future_maintenance_conflict	BOOLEAN	1 byte	Indicates whether future maintenance conflicts with the target period. Required.
exclusion_reasons	JSONB	—	Structured explanations of conditions excluding the vehicle from projected supply. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 57 describes the table in the database named `supply_evaluation_vehicles`, where vehicle-level eligibility, booking and rental conflicts, future maintenance conflicts, and exclusion explanations are captured for each supply evaluation.

Table 58

Data Dictionary – Allocation Recommendation Batches

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
generated_by	UUID	16 bytes	User who initiated the generation operation. FK to public.profiles(id). Required.
generated_at	TIMESTAMP TZ	8 bytes	Date and time when the generation operation was recorded. Required.
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
generation_context_fingerprint	TEXT	—	Fingerprint of the evaluation context used to generate the recommendation batch. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 58 describes the table in the database named `allocation_recommendation_batches`, where each allocation-recommendation generation event and its context fingerprint are recorded, allowing the resulting recommendations to be traced to a common evaluation context.

Table 59

Data Dictionary – Allocation Recommendations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
batch_id	UUID	16 bytes	Generation batch to which the recommendation belongs. FK to public.allocation_recommendation_batches(id). Required.
source_supply_evaluation_id	UUID	16 bytes	Supply evaluation supporting the source branch's surplus. FK to public.supply_evaluations(id). Required.
destination_supply_evaluation_id	UUID	16 bytes	Supply evaluation supporting the destination branch's shortage. FK to public.supply_evaluations(id). Required.
source_branch_id	UUID	16 bytes	Branch proposed as the source of vehicle units. FK to public.branches(id). Required.
destination_branch_id	UUID	16 bytes	Different branch proposed as the recipient of vehicle units. FK to public.branches(id). Required.
vehicle_category_id	UUID	16 bytes	Vehicle category for which the record applies. FK to public.vehicle_categories(id). Required.
forecast_horizon	SMALLINT	2 bytes	Forecast horizon, expressed as one, two, or three weeks. Required.
target_week_start	DATE	4 bytes	Beginning date of the forecast or allocation target week. Required.

Attributes	Data Type	Field Size	Description
target_week_end	DATE	4 bytes	Ending boundary of the forecast or allocation target week. Required.
source_required_units_snapshot	INTEGER	4 bytes	Source branch's required vehicle units at generation. Required.
source_projected_supply_snapshot	INTEGER	4 bytes	Source branch's projected eligible supply at generation. Required.
source_surplus_snapshot	INTEGER	4 bytes	Positive source surplus preserved at generation. Required.
destination_required_units_snapshot	INTEGER	4 bytes	Destination branch's required vehicle units at generation. Required.
destination_projected_supply_snapshot	INTEGER	4 bytes	Destination branch's projected eligible supply at generation. Required.
destination_shortage_snapshot	INTEGER	4 bytes	Positive destination shortage preserved at generation. Required.
recommended_transfer_units	INTEGER	4 bytes	Suggested positive unit count, limited by the preserved source surplus and destination shortage. Required.
decision_state	TEXT	—	Owner/Admin review state of the recommendation. Allowed values: Pending, Approved, Rejected. Required.
approved_transfer_units	INTEGER	4 bytes	Approved positive unit count, no greater than the recommended count; recorded for an approved recommendation. Optional.
decided_by	UUID	16 bytes	User who approved or rejected the recommendation. FK to public.profiles(id). Optional.
decided_at	TIMESTAMP TZ	8 bytes	Date and time when the review decision was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 59 describes the table in the database named `allocation_recommendations`, where suggested units for movement between source and destination branches are stored with supply snapshots and Owner/Admin review decisions. Approval records a decision and does not automatically transfer vehicles or change their branch assignments.

Table 60

Data Dictionary – Allocation Recommendation Candidates

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recommendation_id	UUID	16 bytes	Allocation recommendation to which the candidate belongs. FK to <code>public.allocation_recommendations(id)</code> . Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to <code>public.vehicles(id)</code> . Required.
vehicle_name_snapshot	TEXT	—	Vehicle name preserved at candidate generation. Required.
license_plate_snapshot	TEXT	—	Vehicle registration plate preserved at candidate generation. Optional.
candidate_rank	INTEGER	4 bytes	Positive candidate position, unique within the recommendation. Required.
idle_days_snapshot	INTEGER	4 bytes	Recorded nonnegative idle-day count, when determinable. Optional.
idle_reference_snapshot	TIMESTAMP TZ	8 bytes	Reference time used to interpret the recorded idle state. Optional.

Attributes	Data Type	Field Size	Description
revalidation_state	TEXT	—	Eligibility classification recorded at generation, restricted to EligibleAtGeneration. Allowed values: EligibleAtGeneration. Required.
explanation_codes	JSONB	—	Structured list of explanations accompanying candidate selection. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 60 describes the table in the database named `allocation_recommendation_candidates`, where ranked vehicle candidates and the eligibility and idle-state information observed when an allocation recommendation was generated are preserved.

Table 61

Data Dictionary – Notifications

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recipient_id	UUID	16 bytes	User to whom the notification or preference belongs. FK to <code>public.profiles(id)</code> . Required.
notification_type	TEXT	—	Canonical business-event or reminder category of the notification. Allowed values: <code>requirements_needs_resubmission</code> , <code>requirements_verified</code> ,

Attributes	Data Type	Field Size	Description
			payment_needs_resubmission, payment_verified, booking_confirmed, new_booking_request, requirements_submitted, payment_proof_submitted, upcoming_pickup, upcoming_return, rental_overdue, maintenance_attention, low_availability, backup_attention. Required.
title	TEXT	—	Title of the notification or operational alert. Required.
message	TEXT	—	Text of the message. Required.
related_entity_type	TEXT	—	Type of business record associated with the message or condition. Allowed values: booking, requirements, payment, rental, vehicle, branch, backup_run. Required.
related_entity_id	UUID	16 bytes	Identifier of the associated business record; interpreted using its type and not declared as a foreign key. Required.
event_key	TEXT	—	Event identifier used with the recipient to prevent duplicate notifications. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
read_at	TIMESTAMP TZ	8 bytes	Date and time when the message was marked as read. Optional.

Table 61 describes the table in the database named notifications, where recipient-specific in-application messages, event identifiers, related-entity references, and read times are stored. Related-entity references identify different business-record types and are not declared foreign keys.

Table 62

Data Dictionary – Notification Preferences

Attributes	Data Type	Field Size	Description
recipient_id	UUID	16 bytes	User to whom the notification or preference belongs. Primary key. FK to public.profiles(id). Required.
maintenance_attention_enabled	BOOLEAN	1 byte	Indicates whether maintenance-attention notifications are enabled. Required.
low_availability_enabled	BOOLEAN	1 byte	Indicates whether low-availability notifications are enabled. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
email_notifications_enabled	BOOLEAN	1 byte	Indicates whether email notifications are enabled. Required.

Table 62 describes the table in the database named notification_preferences, where each user's choices for maintenance alerts, low-availability alerts, and email notifications are maintained.

Table 63

Data Dictionary – Operational Notification Conditions

Attributes	Data Type	Field Size	Description
condition_type	TEXT	—	Operational condition being tracked. Part of the composite primary key (condition_type, related_entity_id). Allowed values: maintenance_attention, low_availability. Required.
related_entity_type	TEXT	—	Type of business record associated with the message or condition. Allowed values: vehicle, branch. Required.
related_entity_id	UUID	16 bytes	Identifier of the associated business record; interpreted using its type and not declared as a foreign key. Part of the composite primary key (condition_type, related_entity_id). Required.
is_active	BOOLEAN	1 byte	Indicates whether the operational condition is currently active. Required.
occurrence_count	BIGINT	8 bytes	Nonnegative count of condition occurrences. Required.
title	TEXT	—	Title of the notification or operational alert. Required.
message	TEXT	—	Text of the message. Required.
activated_at	TIMESTAMP TZ	8 bytes	Date and time when the condition became active. Optional.
resolved_at	TIMESTAMP TZ	8 bytes	Date and time when the condition was resolved. Optional.
last_evaluated_at	TIMESTAMP TZ	8 bytes	Date and time of the most recent condition assessment. Required.

Table 63 describes the table in the database named operational_notification_conditions, where the activation and resolution of maintenance-attention and branch low-availability conditions are tracked to support

controlled alert generation. The associated entity identifier is interpreted according to the condition type.

Table 64

Data Dictionary – Email Deliveries

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recipient_user_id	UUID	16 bytes	User to whom the notification email is addressed. FK to public.profiles(id). Required.
notification_id	UUID	16 bytes	In-application notification associated with the email; unique per notification. FK to public.notifications(id). Unique when present. Required.
delivery_key	TEXT	—	Unique identifier preventing duplicate email delivery. Unique when present. Required.
email_type	TEXT	—	Canonical business-event or reminder category eligible for email delivery. Allowed values: requirements_needs_resubmission, requirements_verified, payment_needs_resubmission, payment_verified, booking_confirmed, upcoming_pickup, upcoming_return, rental_overdue. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Pending, Processing, Sent, Failed, Skipped. Required.

Attributes	Data Type	Field Size	Description
attempt_count	INTEGER	4 bytes	Number of delivery attempts, limited to 0–4. Required.
provider_message_id	TEXT	—	Message reference returned by the email provider. Optional.
last_error_code	TEXT	—	Classified result of the latest delivery error. Allowed values: ConfigurationError, RecipientUnavailable, RecipientInvalid, PreferenceDisabled, RateLimited, ProviderUnavailable, ProviderRejected, NetworkError, UnknownProviderError. Optional.
next_attempt_at	TIMESTAMP TZ	8 bytes	Scheduled date and time of the next delivery attempt. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
last_attempt_at	TIMESTAMP TZ	8 bytes	Date and time of the latest delivery attempt. Optional.
sent_at	TIMESTAMP TZ	8 bytes	Date and time when successful sending was recorded. Optional.

Table 64 describes the table in the database named email_deliveries, where the delivery state, retry attempts, provider reference, and classified errors for an eligible notification email are tracked. Each notification can be associated with at most one email-delivery record.

Table 65

Data Dictionary – Audit Events

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
actor_type	TEXT	—	Indicates whether the recorded actor is a User or the System. Allowed values: User, System. Required.
actor_user_id	UUID	16 bytes	Acting application user; present for User events and absent for System events. FK to public.profiles(id). Optional.
action	TEXT	—	Canonical business-event action recorded in the audit trail. Allowed values: booking.created, booking.edited, booking.vehicle_assigned, booking.confirmed, booking.cancelled, booking.rejected, booking.withdrawn, booking.location_reconciled, requirements.submitted, requirements.resubmitted, requirements.needs_resubmission, requirements.verified, payment.quote_issued, payment.submitted, payment.resubmitted, payment.needs_resubmission, payment.verified, payment.requirement_set, rental.released, rental.returned, maintenance.created, maintenance.started, maintenance.completed, maintenance.cancelled. Required.
entity_type	TEXT	—	Type of business record affected by the event. Allowed values: booking, requirements, payment, rental, maintenance. Required.
entity_id	UUID	16 bytes	Identifier interpreted using entity_type; not declared as a foreign key. Required.

Attributes	Data Type	Field Size	Description
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Optional.
metadata	JSONB	—	Structured object containing contextual details of the recorded event. Required.
occurred_at	TIMESTAMP TZ	8 bytes	Date and time when the event occurred. Required.

Table 65 describes the table in the database named audit_events, where user-initiated or system-initiated business events are preserved with their action, entity reference, optional booking association, metadata, and occurrence time. This event-based record does not use a generic old-value and new-value model.

Table 66

Data Dictionary – Backup Runs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
trigger	TEXT	—	Indicates whether the backup was Scheduled or Manual. Allowed values: Scheduled, Manual. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Running, Completed, Partial, Failed. Required.
started_at	TIMESTAMP TZ	8 bytes	Date and time when the operation started. Required.

Attributes	Data Type	Field Size	Description
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
retention_until	TIMESTAMP TZ	8 bytes	Date and time through which the backup is scheduled for retention. Required.
error_code	TEXT	—	Classified error associated with the operation. Allowed values: ConfigurationError, DatabaseDumpFailed, StorageEnumerationFailed, StorageObjectReadFailed, ArtifactUploadFailed, IntegrityValidationFailed, RetentionCleanupFailed, UnknownBackupError. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_by	UUID	16 bytes	User who created the record. FK to public.profiles(id). Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 66 describes the table in the database named backup_runs, where scheduled or manual backup attempts, their completion states, retention dates, and classified errors are recorded. Completed, Partial, and Failed attempts remain distinguishable.

Table 67

Data Dictionary – Backup Artifacts

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
backup_run_id	UUID	16 bytes	Backup attempt associated with the artifact or recovery drill. FK to public.backup_runs(id). Required.
artifact_type	TEXT	—	Indicates whether the recovery asset protects Database or Storage content. Allowed values: Database, Storage. Required.
artifact_key	TEXT	—	Logical recovery-asset reference without an access credential or signed access URL. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Completed, Failed. Required.
size_bytes	BIGINT	8 bytes	Size of the file or artifact in bytes. Optional.
sha256	TEXT	—	64-character hexadecimal SHA-256 integrity checksum for a completed artifact. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 67 describes the table in the database named backup_artifacts, where database and file-storage recovery assets belonging to a backup attempt are identified, together with their completion state, byte size, and integrity checksum. Logical artifact references do not contain access credentials.

Table 68

Data Dictionary – Recovery Drills

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
backup_run_id	UUID	16 bytes	Backup attempt associated with the artifact or recovery drill. FK to public.backup_runs(id). Required.
target_environment	TEXT	—	Recovery-test environment, restricted to NonProduction. Allowed values: NonProduction. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Running, Passed, Failed. Required.
started_at	TIMESTAMP TZ	8 bytes	Date and time when the operation started. Required.
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
database_validation	TEXT	—	Recorded database-restoration validation outcome. Allowed values: Pending, Passed, Failed. Optional.
storage_validation	TEXT	—	Recorded file-storage restoration validation outcome. Allowed values: Pending, Passed, Failed. Optional.
error_code	TEXT	—	Classified error associated with the operation. Allowed values: ConfigurationError, IntegrityValidationFailed, RestoreFailed, ValidationFailed, UnknownBackupError. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 68 describes the table in the database named `recovery_drills`, where non-production recovery-validation attempts are recorded separately from backup creation. A passed drill requires successful database and storage validation.

Table 69

Data Dictionary – Public Contact Settings

Attributes	Data Type	Field Size	Description
id	BOOLEAN	1 byte	Single-row primary-key value, restricted to true. Primary key. Required.
phone	TEXT	—	Published contact telephone number. Required.
email	TEXT	—	Published email address of the rental business. Required.
office_hours	TEXT	—	Published business or contact hours. Required.
location_summary	TEXT	—	Readable summary of the public contact locations. Required.
service_area	TEXT	—	Published description of the geographic service area. Required.
reply_commitment	TEXT	—	Published statement concerning replies to inquiries. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
updated_by	UUID	16 bytes	User who last updated the record. FK to <code>public.profiles(id)</code> . Optional.

Table 69 describes the table in the database named `public_contact_settings`, where the published contact details, office hours, service-area information, and reply commitment are maintained in a single settings record.

Table 70

Data Dictionary – Public Contact Locations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name of the public-facing contact location. Required.
address	TEXT	—	Published address of the contact location. Required.
note	TEXT	—	Optional explanatory note concerning the location. Optional.
sort_order	SMALLINT	2 bytes	Presentation position from 0 to 99; unique across public contact locations. Unique when present. Required.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 70 describes the table in the database named `public_contact_locations`, where ordered public-facing contact locations are maintained separately from operational fleet

branches. This distinction prevents public contact addresses from being treated as internal allocation records.

Table 71

Data Dictionary – Contact Inquiries

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name supplied by the inquiry sender. Required.
email	TEXT	—	Reply email address supplied by the inquiry sender. Required.
subject	TEXT	—	Subject supplied for the inquiry. Required.
message	TEXT	—	Inquiry message supplied by the sender. Required.
status	TEXT	—	Recorded state of the record. Allowed values: New, Read, Closed. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
read_at	TIMESTAMP TZ	8 bytes	Date and time when the message was marked as read. Optional.
read_by	UUID	16 bytes	User who marked the inquiry as read. FK to public.profiles(id). Optional.

Table 71 describes the table in the database named `contact_inquiries`, where public inquiry messages, sender details, review status, and the user who marked a message as read are recorded.

Operational and Decision-Support Logic

This section presents the operational rules, computational procedures, and decision-support logic used by the proposed system. These mechanisms define how the system transforms booking, vehicle, maintenance, forecasting, and contextual data into analytical results and recommendations. The proposed system uses deterministic business rules, statistical computation, and rule-based decision support appropriate to the operational requirements of a small-scale multi-branch car rental business. Rule-based decision-support approaches are suitable for operational environments where decision criteria must remain explicit, interpretable, and auditable rather than dependent on complex or opaque models (Haber et al., 2026; Mavrin & Makarova, 2026; Meira et al., 2022; Mohamad et al., 2026). The generated outputs are intended to support decision-making and do not replace the judgment of the Owner/Admin. Final decisions regarding booking approval, payment verification, vehicle assignment, vehicle transfer, branch allocation, and other critical operational actions remain under authorized personnel.

A. Demand Definition and Weekly Data Aggregation

Historical rental and booking information can be used to identify demand patterns and support short-term planning and vehicle-allocation decisions in transportation and rental-related operations (Suganya et al., 2026; Zhao et al., 2021;

Martin et al., 2021; Huang et al., 2021). In this study, vehicle demand shall be represented by the number of qualifying rental bookings recorded for a specific branch and vehicle category during a calendar week.

A booking shall contribute to historical demand when it has been accepted or confirmed as a valid rental request for the corresponding branch, vehicle category, and rental schedule. Rejected requests, duplicate requests, and customer-cancelled requests that did not proceed as valid rental transactions shall not be included in the demand count.

Weekly aggregation is adopted because the forecasting component is intended to support short-term operational planning rather than long-term strategic forecasting. Each weekly demand value shall count qualifying confirmed bookings whose scheduled rental start falls within that week, grouped by booking branch and requested vehicle category. Weeks shall run from Monday 00:00 to the following Monday 00:00 in Asia/Manila time. A booking is counted once; its duration does not multiply the demand count. This measure represents observed confirmed booking starts, not unmet demand, concurrent occupancy, or vehicle-days.

Weekly demand for branch b , vehicle category c , and week t is expressed as:

$D_{b,c,t}$ *Number of qualifying bookings for branch b and category c during week t*

Demand values shall be grouped separately according to branch and vehicle category so that differences in customer demand across locations and vehicle types can be reflected in the forecasting and allocation process.

B. Weighted Moving Average Demand Forecasting

The proposed system shall use the Weighted Moving Average (WMA) method to generate short-term weekly demand forecasts for each branch and vehicle category. WMA assigns different weights to recent observations and gives greater influence to more recent demand information. The reviewed literature supports WMA as a practical, interpretable, and computationally lightweight method for short-term forecasting, particularly where historical data and computational resources are limited (Rahayu et al., 2022; Sariat et al., 2026; Paramitha & Saifuddin, 2023; Khalid, 2024).

Based on this principle, the study adopts a three-period WMA using fixed descending weights of 0.50, 0.30, and 0.20. The system shall maintain a rolling forecast horizon covering up to three succeeding weekly periods for each branch and vehicle category.

The first forecasted week shall be calculated entirely from the three most recently completed actual weekly demand observations:

$$F_{t+1} = 0.50D_t + 0.30D_{t-1} + 0.20D_{t-2}$$

where:

F_{t+1} = forecasted demand for the first succeeding week;

D_t = actual demand during the most recently completed week;

D_{t-1} = actual demand during the second-most recent week; and

D_{t-2} = actual demand during the third-most recent week.

The second forecasted week shall be generated recursively by incorporating the first forecasted value together with the two most recent applicable actual observations:

$$F_{t+2} = 0.50F_{t+1} + 0.30D_t + 0.20D_{t-1}$$

The third forecasted week shall likewise be generated recursively using the most recently available forecasted and actual values:

$$F_{t+3} = 0.50F_{t+2} + 0.30F_{t+1} + 0.20D_t$$

At least three complete, consecutive historical weekly observations with trustworthy recording coverage shall be required before the system generates the initial three-week forecast horizon. A covered week with no qualifying bookings is a valid zero; an unrecorded or partially covered week is missing data and shall not be treated as zero. The incomplete current week shall not be included in the historical input. If fewer than three complete observations are available for a particular branch and vehicle category, the system shall indicate insufficient historical data rather than generate a forecast using incomplete observations.

Each forecast value may contain a decimal because it represents an estimated demand level. Decimal values shall be retained for analytical reporting, historical comparison, and forecast visualization. When a forecasted weekly value is used to determine the corresponding vehicle requirement for branch-allocation planning, it shall be rounded upward to the nearest whole vehicle:

$$RequiredVehicle_{t+h} = \lceil Ft + h \rceil, h \in \{1, 2, 3\}$$

At each weekly rollover, the system shall generate the three-week horizon from the three preceding completed weeks. Week +1 denotes the week immediately following the latest completed input week; Weeks +2 and +3 follow it. Each run shall retain its generation time, input weeks, input values, target weeks, and forecast values. A run generated after a target week has begun shall be identified as an in-period planning update and shall not be represented as a forecast available before that week began. Consequently, the dashboard may display historical Actual Demand and Forecasted Demand together and extend the most recent Forecasted Demand series through the next three weekly periods. The first-week forecast is based entirely on observed demand, while the second- and third-week forecasts progressively incorporate forecasted values and shall therefore be treated as longer-horizon planning projections rather than guaranteed demand.

All WMA outputs shall remain advisory and shall not automatically trigger vehicle transfer, branch reassignment, or any other operational action.

C. Forecast Evaluation Using Mean Absolute Percentage Error

Mean Absolute Percentage Error (MAPE) shall be used to evaluate the difference between forecasted demand and the corresponding actual demand after the forecast period. MAPE expresses the average absolute forecasting error as a percentage of actual observed demand and is commonly used in the evaluation of forecasting models (Rahayu et al., 2022).

MAPE shall be computed as:

$$MAPE = \frac{100}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|$$

where:

A_t = actual demand during the evaluated period;

F_t = forecasted demand for the corresponding period; and

n = number of valid forecast observations included in the calculation.

MAPE shall only be calculated for observations where actual demand is greater than zero because percentage error is undefined when the actual value is zero. Periods with zero actual demand shall therefore be identified separately and excluded from the MAPE divisor.

The resulting MAPE shall be reported as the observed average percentage forecasting error. Lower MAPE values indicate smaller forecasting errors. The study shall not automatically assign qualitative classifications such as “Highly Accurate,” “Good,” or “Poor” unless a separately verified interpretation basis is formally adopted.

All WMA and MAPE examples and results reported in the current study are derived from the labeled synthetic dataset and shall be presented as illustrative or functional-test computations. They shall not be presented as evidence of actual client demand, empirical predictive accuracy, seasonal validity, or long-term forecasting effectiveness.

For consistency in forecast-error evaluation, the primary MAPE calculation shall use the original Week +1 forecast for the matching branch, vehicle category, and target week. The forecast snapshot shall be frozen at the weekly boundary before the target week's realized synthetic observations are incorporated, and later in-period runs shall be excluded. These timing rules allow the system's computation to be checked using a

consistent forecast horizon. Because the current study has no genuine client history, this procedure constitutes synthetic functional backtesting and shall not be described as prospective real-world performance or an empirical historical backtest. If sufficient verified client or post-implementation history becomes available later, it may be evaluated separately using only the three completed weeks preceding each target. The second- and third-week forecasts may be retained for planning and visualization but shall not be combined with Week +1 errors unless performance is reported separately by horizon.

D. Vehicle Utilization Analysis

Vehicle utilization monitoring supports fleet planning by helping administrators identify frequently used, underutilized, and idle fleet units and by providing information useful for fleet deployment decisions. Studies in shared mobility and fleet management likewise show that historical vehicle usage and utilization indicators can support resource planning and operational decision-making (Abouelela et al., 2023; Eliyan & Kerbach, 2024).

In the proposed system, vehicle utilization shall measure the proportion of eligible operational days within a selected reporting period during which a vehicle was actually used for rental operations.

Vehicle utilization is computed as:

$$VehicleUtilization(\%) = \frac{RentalDays}{EligibleOperationalDays} \times 100$$

Rental Days refer to the number of calendar days within the reporting period during which the vehicle was associated with an active rental transaction. Eligible

Operational Days refer to days during which the vehicle could reasonably have been used for rental operations.

Days when the vehicle was inactive or unavailable because of maintenance shall be excluded from Eligible Operational Days because the vehicle could not have been offered for rental during those periods. A reservation that does not proceed into an actual rental shall not be counted as vehicle utilization.

The administrative dashboard may use the most recent 30-day period as the default monitoring interval, while reports may allow a selected date range. The resulting utilization percentage shall serve as an analytical indicator and shall not independently determine vehicle transfer or branch allocation.

E. Idle Vehicle Detection

Idle-vehicle monitoring is relevant to fleet management because prolonged underutilization may indicate that a vehicle is positioned in a location where available supply exceeds current operational use. Research on fleet relocation and vehicle rebalancing shows that demand, availability, and idle-vehicle information can support improved fleet-distribution decisions (Eliyan & Kerbach, 2024; Zhu et al., 2024; Skrzynski et al., 2026).

In this study, idle vehicle detection shall identify vehicles that remain operationally available but have recorded no rental activity for a defined period.

A vehicle shall only be eligible for idle classification when it is active, rental-ready, not currently rented, not unavailable because of maintenance, and not otherwise prevented from being offered for rental.

Idle duration shall be computed as:

$$IdleDays = CurrentDate - LastRentalEndDate$$

For a vehicle with no previous rental record, the date when the vehicle became operationally available may be used as the reference date.

For purposes of the proposed system, an eligible vehicle shall be classified as idle when it has recorded no rental activity for at least fourteen consecutive days:

$$IdleVehicle = EligibleForRental \wedge IdleDays \geq 14$$

The fourteen-day threshold is adopted by the study as an operational monitoring rule representing two consecutive weekly periods. It is intended to distinguish short temporary inactivity from more meaningful underutilization. Idle classification shall serve only as an administrative indicator. It shall not automatically cause vehicle transfer, but idle vehicles may be prioritized for review when another branch has a forecasted shortage.

F. Branch Demand Analysis

Multi-location fleet operations may experience uneven demand and vehicle imbalance when one location receives greater rental activity while another retains unused or underutilized vehicles (Boysen et al., 2024; Eliyan & Kerbache, 2024; Brandizzi et al., 2022). To provide descriptive branch-level analytics, the proposed system may calculate the proportion of qualifying booking activity associated with each branch.

Branch share shall be computed as:

$$BranchShare(\%) = \frac{BranchBookings}{TotalBookings} \times 100$$

This metric provides a descriptive comparison of booking activity between branches. Branch share shall be used for reporting and dashboard presentation and shall not independently determine whether vehicles should be transferred between branches. Branch-allocation decisions shall instead use the forecasted requirement, projected supply, shortage, and surplus calculations defined in the succeeding subsections.

G. Maintenance Readiness Determination

Maintenance status, mileage, vehicle condition, preventive maintenance schedules, and historical service information are important fleet-management factors because a vehicle may appear available in the booking schedule while still being unsuitable for rental operation. Studies on vehicle maintenance and fleet management support the use of mileage, maintenance records, service schedules, and vehicle condition as factors in determining fleet readiness and preventing operational use of vehicles requiring maintenance (Arena et al., 2022; Chen et al., 2024; Crespo del Castillo & Parlikad, 2024; Chen et al., 2025).

In the proposed system, maintenance information shall be treated as an operational eligibility condition rather than as an arbitrary numerical score. Before a vehicle is included in a customer recommendation, vehicle assignment, or branch-allocation recommendation, the system shall determine whether the vehicle is maintenance-ready.

A vehicle shall be treated as not maintenance-ready when at least one of the following conditions exists: an active maintenance activity remains in progress; required preventive maintenance is due or overdue; the recorded vehicle condition indicates that the vehicle is unsafe or unsuitable for rental; or an unresolved maintenance or repair concern prevents operational use.

When preventive maintenance is monitored using date and mileage criteria, a vehicle may be considered due for maintenance when:

$$CurrentOdometer \geq NextServiceOdometer$$

or

$$CurrentDate \geq NextServiceDate$$

depending on the applicable maintenance criteria recorded for the vehicle.

A vehicle classified as not maintenance-ready shall be excluded from Customer-Side Vehicle Recommendation results and from candidate vehicles considered for branch transfer until the relevant maintenance concern has been completed or resolved.

H. Travel Distance, Reference Fuel Efficiency, and Estimated Fuel Consumption

Fuel efficiency and estimated fuel consumption may serve as supporting factors in transportation and vehicle-selection decisions because distance and vehicle fuel characteristics influence projected trip fuel requirements and cost-related considerations (Vyas et al., 2022; Moawad et al., 2024; Şimşek et al., 2025; Alrashdi et al., 2025).

In this study, fuel efficiency shall be treated as estimated or reference vehicle information and shall be expressed in kilometers per liter (km/L). The reference value may be based on manufacturer specifications or owner-provided vehicle information. The system does not claim to continuously measure actual real-world fuel efficiency.

When travel-distance information is available, estimated fuel consumption shall be calculated as:

$$EstimatedFuelConsumption = \frac{TravelDistance_{km}}{ReferenceFuelEfficiency_{km/L}}$$

For example, if an estimated trip distance is 250 kilometers and the vehicle's recorded reference fuel efficiency is 10 km/L, the estimated fuel consumption is:

$$250 \div 10 = 25 \text{ liters}$$

The calculated value represents only an operational estimate. The displayed distance shall identify whether it covers a one-way or explicitly defined return journey. Fuel estimates require a usable distance and a positive reference km/L value; missing inputs shall produce an unavailable estimate rather than zero fuel consumption. The estimate is expressed in liters and shall not be described as a monetary cost unless an explicit fuel-price input and calculation are also provided. Actual fuel consumption may vary because of traffic, driver behavior, passenger or cargo load, terrain, weather, road condition, vehicle condition, and other factors. Fuel-related information shall therefore serve only as supporting decision information and shall not independently determine final vehicle assignment or branch allocation.

I. Customer-Side Vehicle Recommendation Logic

Recommendation systems can support user decision-making by matching available options with user preferences or requirements, while rule-based approaches are appropriate when the recommendation criteria must remain simple, transparent, and interpretable (Sabet et al., 2022; Boteju & Munasinghe, 2020; Prabowol et al., 2019; Haber et al., 2026).

In the proposed system, the Customer-Side Vehicle Recommendation shall use rule-based eligibility filtering followed by deterministic ranking. It shall not use an unsupported numerical match percentage or arbitrary 0–100 recommendation score.

The recommendation process shall consist of two stages: eligibility filtering and ranking.

During eligibility filtering, the system shall exclude vehicles that are unavailable for the requested rental period, inactive, not maintenance-ready, or affected by a conflicting booking or rental schedule.

Passenger capacity shall operate as a hard requirement:

$$VehicleCapacity \geq RequiredPassengerCount$$

Vehicles with passenger capacity below the customer's requirement shall be excluded.

The customer's stated total rental budget is compared with a provisional daily-rate reference estimate for the requested duration. RentalDuration in the formula below means the number of started 24-hour periods, rounded upward with a minimum of one. This convention is used only for recommendation filtering; it does not encode 12-hour offers, promotional or long-term packages, overtime, delivery charges, deposits, or the

final amount due. The budget is a screening input for this feature; customers shall still obtain final rate confirmation from the participating car rental business. Where commercial-policy boundaries remain unverified, the prototype uses a researcher-designed policy baseline for controlled testing; it does not claim current rate, refund, or penalty terms. The reference estimate is computed as:

$$\textit{EstimatedRentalCost} = \textit{DailyRentalRate} \times \textit{RentalDuration}$$

A vehicle whose estimated base rental cost exceeds the customer's stated budget shall be excluded. Charges that cannot yet be determined during the recommendation stage, such as penalties or other conditional charges, shall not be included in the base estimate.

After eligibility filtering, the remaining vehicles shall be ranked using predefined preference rules. A vehicle matching the customer's preferred vehicle category shall be ranked ahead of an eligible non-matching category. Where several vehicles satisfy the same preference, the vehicle with the closest adequate passenger capacity shall be preferred. Large-luggage capacity shall also satisfy the stated large-bag count. Among otherwise equally ranked vehicles, the closest adequate luggage capacity shall be preferred, followed by the lower reference rental estimate, vehicle name, and a stable vehicle identifier. Fuel efficiency, fuel estimates, and destination context shall not affect this customer ranking.

An optional destination or travel area shall be retained for administrative review of the booking. Any route, weather, road, or fuel context obtained for that review belongs to the administrative decision-support process and shall not be presented as a factor used in customer recommendation ranking.

Instead of an unsupported match score, the system shall provide reasons for recommending a vehicle, such as matching the preferred category, satisfying the passenger requirement, remaining within the stated budget, being available for the selected dates, and being maintenance-ready.

Customer-Side Vehicle Recommendation results shall serve only as vehicle-selection guidance before booking submission. They shall not constitute booking approval or guarantee final vehicle assignment.

J. Branch Vehicle Requirement, Projected Supply, Shortage, and Surplus

Fleet-allocation studies emphasize that vehicle-distribution decisions are strengthened when expected demand and available supply are assessed together rather than independently (Favier et al., 2024; Eliyan & Kerbache, 2024; Skrzynski et al., 2026). In the proposed system, the branch-allocation process shall therefore compare the forecasted vehicle requirement with projected operational supply for the same branch, vehicle category, and evaluated weekly period within the three-week forecast horizon.

For this study, the rounded-up weekly booking-start forecast shall be used as a conservative planning proxy for vehicle requirements. It is not a direct estimate of simultaneous occupancy: one vehicle may serve several non-overlapping bookings, while a multi-day rental may occupy a vehicle across weeks. Shortage and surplus outputs shall therefore be labeled as planning indicators and checked against actual schedules before an operational decision. The planning requirement shall be represented as:

$$R_{b,c,h} = \lceil F_{b,c,h} \rceil$$

where:

$R_{b,c,h}$ = required number of vehicles for branch b , vehicle category c , and forecast horizon h ;

$F_{b,c,h}$ = forecasted demand for branch b , vehicle category c , and forecast horizon h ; and

$h \in \{1, 2, 3\}$, representing Week +1, Week +2, and Week +3, respectively.

Projected available supply, represented as $S_{b,c,h}$ shall include vehicles assigned to the evaluated branch and vehicle category that are active, maintenance-ready, and not committed to conflicting reservations or rentals during the corresponding forecasted weekly period h .

A branch shortage shall be computed as:

$$Shortage_{b,c,h} = \max(0, R_{b,c,h} - S_{b,c,h})$$

A branch surplus shall be computed as:

$$Surplus_{b,c,h} = \max(0, S_{b,c,h} - R_{b,c,h})$$

A shortage therefore indicates that projected operational supply is below the forecasted requirement, while a surplus indicates that supply exceeds the forecasted

requirement. When supply equals the forecasted requirement, neither a shortage nor a surplus shall be recorded.

K. Rule-Based Branch Allocation Recommendation

Rule-based decision-support systems translate operational knowledge and constraints into explicit decision conditions, allowing recommendations to remain traceable and understandable to the decision-maker (Haber et al., 2026; Mavrin & Makarova, 2026; Meira et al., 2022; Mohamad et al., 2026). This approach is appropriate for the proposed system because branch-allocation outputs are intended to assist rather than replace administrative judgment.

The Branch Allocation Recommendation shall therefore use explicit operational rules instead of an arbitrary urgency or confidence score. Its purpose is to determine whether an expected shortage at one branch may be supported by an available surplus at another branch.

A potential branch-transfer recommendation may be produced when the destination branch has a forecasted shortage for a vehicle category and another branch has a forecasted surplus for the corresponding category.

The maximum recommended transfer quantity shall be limited by the destination shortage, the source surplus after reserving its own forecast-based planning requirement, and the number of revalidated eligible candidate vehicles. The shortage-and-surplus bound is:

$$RecommendedTransferUnits_{b_s, b_d, c, h} = \min(Shortage_{b_d, c, h}, Surplus_{b_s, c, h})$$

For example, if the destination has a shortage of two SUVs and the source has a surplus of three SUVs after reserving its own planning requirement, up to two SUVs may be recommended if at least two candidates pass all eligibility checks. If only one candidate qualifies, the quantity is limited to one. Remaining shortage and surplus balances shall be updated within a recommendation batch, and a candidate shall not be counted in more than one conflicting proposed movement.

Candidate vehicles from the source branch shall be active, operationally available, maintenance-ready, and free of conflicting booking or rental commitments for the period in which transfer is being considered. Eligible candidates shall be ordered by longest known idle duration, with unknown idle duration placed after known values and a stable vehicle identifier used to resolve remaining ties. Unknown idle duration shall not be treated as zero or as evidence of prolonged inactivity.

The resulting recommendation shall identify the source branch, destination branch, vehicle category, forecast requirement, projected supply, shortage, surplus, recommended transfer quantity, applicable candidate vehicle or vehicles, and the operational reasons supporting the recommendation.

The system shall not automatically change a vehicle's branch assignment. The Owner/Admin shall review the proposed quantity, eligible candidates, supporting reasons, and available contextual information before approving, rejecting, or reducing the proposed quantity. The system shall retain the recommendation snapshot, review outcome, reviewer, timestamp, and decision remarks. Approval records an advisory decision; it does not execute a transfer or update a vehicle's branch. Eligibility and scheduling conflicts shall be rechecked before any subsequent operational reassignment.

L. Context-Aware Operational Assessment

Context-aware transportation research shows that road condition, route feasibility, weather, travel distance, and related environmental or operational information can provide useful supporting information for transport decisions (Liu et al., 2022; Dubey et al., 2025; Gabbar et al., 2023). Weather-aware transportation studies also demonstrate that precipitation, visibility, and adverse weather conditions may affect route suitability and operational decision-making (Tobin et al., 2024; Hussein et al., 2021; Čelar et al., 2024; Pradeeppraja et al., 2025). These contextual variables can be integrated through external APIs or other supplementary data sources where reliable and feasible data are available (Penumala & Naganaboina, 2025; Sandhip et al., 2025; Joshna et al., 2026; Vdovic et al., 2021).

In the proposed system, contextual information shall serve as supporting information after the system has evaluated its primary internal operational data, including forecasted demand, projected supply, vehicle availability, and maintenance readiness.

Applicable contextual information may include weather, incident-based road information, route availability and accessibility, travel distance, reference fuel efficiency, and estimated fuel consumption. For allocation review, the route shall connect the source and destination branches; for booking assignment, it shall use the relevant branch and recorded destination. The review shall identify the location or route assessed, retrieval or recording time, data source, and applicable time coverage. Current conditions shall not be presented as a reliable forecast for a future transfer date beyond the source's supported horizon.

External API Selection and Fallback Strategy

The proposed system will use selected external Application Programming Interfaces (APIs) to obtain applicable weather, location, routing, and incident-based road information for administrative context-aware decision support. The API providers were selected based on the contextual information required by the proposed system, documented service capabilities, geographic coverage applicable to the Philippines, implementation feasibility, and the availability of a comparable secondary provider that can serve as a fallback. External API information will remain supporting information and will not independently determine vehicle assignment, branch allocation, or other final operational actions.

To reduce dependence on a single external service, the proposed system adopts a primary-and-fallback provider strategy. A designated primary provider will normally be used for each external contextual category. The corresponding fallback provider will only be requested when the primary provider is unavailable, times out, exceeds an applicable service limitation, lacks applicable coverage, or returns unusable or insufficiently current information. A valid adverse result returned by the primary provider, such as a road closure or adverse weather condition, shall not be treated as provider failure and shall not cause the system to request the fallback provider merely to obtain a different result.

Table X

Selected Primary and Fallback APIs for Context-Aware Decision Support

Contextual Information	Primary Provider	Fallback Provider	Intended Use	System
------------------------	------------------	-------------------	--------------	--------

Weather Condition	Open-Meteo Forecast API	OpenWeather One Call API 3.0	Obtain applicable current or forecast weather information that will be normalized into Normal, Caution, Severe, or Unavailable.
Destination Location / Coordinates	Geoapify Geocoding API	LocationIQ Geocoding API	Convert textual destinations, addresses, or travel areas into geographic coordinates for routing and contextual assessment.
Travel Distance and Estimated Travel Time	TomTom Orbis Routing API	HERE Routing API v8	Calculate an applicable driving route and obtain route distance and estimated travel time.
Route Feasibility	TomTom Orbis Routing API with TomTom Traffic Incidents information	HERE Routing API v8 with HERE Traffic API v7	Provide routing and incident information from which the system derives whether a route is Feasible, Feasible with Caution, Not Feasible, or Unavailable.
Road Condition	TomTom Traffic Incidents API	HERE Traffic API v7	Obtain incident-based operational road information such as road closures, lane closures, road works, flooding, dangerous conditions, accidents, and other relevant disruptions.

Route Accessibility	TomTom Orbis Routing API with TomTom Traffic Incidents information	HERE Routing API v8 with HERE Traffic API v7	Provide routing, restriction, and incident information from which the system derives whether an applicable route is Accessible, Limited, Closed/Restricted, or Unknown.
Reference Fuel Efficiency	Internal Reference data	Not API-dependent	Use a vehicle-specific reference value expressed in kilometers per liter (km/L) based on manufacturer specifications or owner-provided information.
Estimated Fuel Consumption	Internally calculated	Not API-dependent	Calculate estimated fuel consumption using applicable route distance and the vehicle's recorded reference fuel-efficiency value.

Open-Meteo is selected as the primary weather-data provider. Its Forecast API accepts geographic coordinates and provides weather information such as weather codes, precipitation, visibility, wind, and other forecast variables that may support operational weather assessment. The service supports forecasts of up to sixteen days. OpenWeather One Call API 3.0 is designated as the fallback weather provider and may be requested when the primary provider cannot provide sufficiently usable information. Because external weather-forecast availability does not necessarily cover the complete

three-week demand-forecast horizon, the system shall only evaluate weather for periods where sufficiently current provider data are available. Weather information shall not be fabricated or extrapolated merely to cover all three forecasted demand periods.

Geoapify is selected as the primary geocoding provider, with LocationIQ as fallback. TomTom routing services are selected as the primary source for route information. TomTom's routing services provide route-related information including route distance and estimated travel time, while its documented routing and real-time traffic coverage includes the Philippines. TomTom Traffic Incidents information may additionally identify operational disruptions such as road closures, lane closures, road works, flooding, dangerous conditions, and other incidents that may affect route movement. Corresponding HERE Technologies services are designated as fallback providers. HERE Routing API v8 may provide route calculations and related information, while HERE Traffic API v7 may provide incident information when the corresponding primary service cannot provide usable data. Location-coordinate fallback remains assigned to LocationIQ.

Route feasibility, route accessibility, road-condition classification, and weather classification are not treated as direct provider-generated decision labels. External providers return provider-specific weather, route, restriction, and incident information, which the proposed system will normalize using predefined study-adopted operational rules. This allows the system to use consistent classifications even when primary and fallback providers use different response structures. Each classification shall retain its supporting condition and source. A returned route or absence of reported incidents shall not be treated as proof of safe travel, complete road coverage, or vehicle-specific

permission to use the route. API errors and missing coverage shall remain unavailable information rather than a confirmed road closure or a favorable condition.

Reference fuel efficiency will not depend on an external API. The system will use a reference vehicle fuel-efficiency value expressed in kilometers per liter (km/L), based on manufacturer specifications or owner-provided information. When applicable travel-distance information is available, estimated fuel consumption will be calculated internally as:

$$EstimatedFuelConsumption = \frac{TravelDistance_{km}}{ReferenceFuelEfficiency_{km/L}}$$

If the designated primary external provider fails, the system shall attempt to obtain comparable information from the designated fallback provider. If neither provider supplies sufficiently usable information, authorized manual contextual information may be recorded when available. Manually supplied information shall be identified separately from API-derived information. If no reliable API-derived or manual information is available, the affected contextual factor shall remain Unavailable or Unknown. Internal demand forecasting, projected-supply analysis, maintenance-readiness checking, shortage and surplus calculations, vehicle-eligibility rules, and other available internal processes shall continue without assuming a favorable contextual condition.

External contextual information shall remain advisory. API-derived or manually recorded context shall not independently create a branch shortage or surplus, automatically select a vehicle, automatically transfer a vehicle between branches, or override internal availability and maintenance-readiness requirements. Final

administrative vehicle-assignment and branch-allocation decisions shall remain under the Owner/Admin.

Context Classification and Normalization Rules

External API providers return raw weather, routing, restriction, and incident information using provider-specific formats. To ensure that contextual information is interpreted consistently within the proposed system, applicable external and authorized manual information shall be normalized into predefined operational classifications before being presented as administrative decision-support information. These classifications are adopted by the study for system implementation and do not represent classifications directly produced by the external providers.

Weather Classification. Available weather information shall be normalized as **Normal, Caution, Severe, or Unavailable**. Normal refers to generally clear, fair, cloudy, overcast, mist, or other available conditions without a significant adverse-weather indication affecting normal travel. Caution refers to rain, showers, drizzle, fog, thunderstorms without an identified severe condition, or another reported condition that may affect visibility, traction, or normal road travel but does not independently indicate that movement should not proceed. Severe refers to severe thunderstorms, very heavy precipitation, tropical-cyclone-related severe weather, an applicable official severe-weather warning or alert, or another reported condition that may materially affect safe travel. Unavailable shall be used when sufficiently current and usable weather information cannot be obtained from either the primary or fallback provider and no reliable authorized manual information is available.

Road-Condition Classification. For purposes of the proposed system, road condition refers to the **incident-based operational state** of an applicable route rather than continuous measurement of pavement roughness, potholes, or physical road-surface quality. Road condition shall be represented as **Open, Caution, Closed/Impassable, or Unknown**. Open shall indicate that an applicable route is available and no relevant blocking or cautionary road incident has been identified. Caution shall indicate that the route remains traversable but applicable information identifies road works, lane closures, non-blocking flooding, accidents affecting movement, dangerous conditions, or another relevant non-blocking disruption. Closed/Impassable shall indicate an explicit road closure, impassable flooding, blocking restriction, or another condition preventing the applicable road movement. Unknown shall be used when sufficiently reliable incident or road-condition information cannot be obtained.

Route-Feasibility Classification. Route feasibility shall be represented as **Feasible, Feasible with Caution, Not Feasible, or Unavailable**. Feasible shall indicate that a valid applicable driving route is available and no blocking or significant cautionary condition has been identified. Feasible with Caution shall indicate that a valid route remains available but applicable weather, road, restriction, or incident information requires administrative attention. Not Feasible shall indicate that available information confirms that the proposed movement cannot presently proceed because of an applicable closure, restriction, or impassable route. Unavailable shall indicate that sufficient routing or contextual information cannot be obtained to determine feasibility.

Route-Accessibility Classification. Route accessibility shall be represented as **Accessible, Limited, Closed/Restricted, or Unknown**. Accessible shall indicate that an applicable route is available without a material restriction affecting normal vehicle

movement. Limited shall indicate that the route remains available but one or more applicable restrictions or incidents may limit normal movement. Closed/Restricted shall indicate that the applicable route or relevant road segment is closed, blocked, or otherwise restricted against the proposed movement. Unknown shall indicate that sufficient accessibility information cannot be obtained.

Context classifications shall supplement rather than replace the system's internal operational analysis. When weather is Normal, road condition is Open, route feasibility is Feasible, and route accessibility is Accessible, an applicable allocation recommendation may be presented without an additional contextual warning. When one or more applicable conditions are classified as Caution, Severe, Limited, or Feasible with Caution, the underlying operational recommendation may remain available but shall include an administrative warning requiring review.

When the route is classified as Not Feasible, or when the applicable road or accessibility information indicates a condition that prevents movement, the system shall retain the underlying demand, supply, shortage, surplus, and allocation-recommendation information but shall indicate that the proposed movement is ****not presently feasible**** under the recorded conditions. The existence of a forecasted shortage shall therefore remain distinguishable from the current feasibility of performing the recommended vehicle transfer.

When applicable contextual information cannot be reliably evaluated, the system shall indicate that the corresponding context is ****Unavailable**** or ****Unknown****.

Internal operational analysis may continue using available system records, but no unavailable contextual factor shall be silently interpreted as Normal, Open, Accessible, or Feasible.

For presentation purposes, the system may summarize the normalized context using the following non-numerical administrative states: ****No Context Warning****, ****Review Recommended****, ****Movement Not Presently Feasible****, or ****Context Unavailable****. These states shall not represent confidence scores, urgency scores, prediction probabilities, or numerical suitability ratings. They serve only as concise representations of the underlying normalized contextual information.

The fallback provider shall only be requested when the designated primary provider is unavailable, times out, exceeds an applicable service limitation, lacks applicable coverage, or returns unusable or insufficiently current information. A valid adverse condition returned by the primary provider shall not constitute provider failure. Therefore, the system shall not request a fallback provider merely because the primary provider reports adverse weather, flooding, road closure, or another unfavorable condition.

When neither the primary nor fallback provider supplies usable information, authorized manual contextual information may be recorded where available. Manual information shall be clearly identified as manually supplied and shall include the

applicable recording time and authorized user. If no reliable API-derived or manual information is available, the corresponding contextual factor shall remain Unavailable or Unknown. The system shall not fabricate, extrapolate, or assume favorable contextual conditions.

Contextual conditions shall not independently create a branch shortage or surplus. The primary basis for a branch-allocation recommendation shall remain the relationship between forecasted requirement and projected available supply. Context shall instead assist the Owner/Admin in determining whether an otherwise valid recommendation is practical under the available operational conditions.

Route feasibility may be represented as **Feasible**, **Feasible with Caution**, **Not Feasible**, or **Unavailable**. A route may be considered Not Feasible when available information identifies a confirmed closure, impassable route, or another condition that prevents the proposed movement. A route may be classified as Feasible with Caution when a route remains available but relevant conditions require administrative attention. When reliable information cannot be obtained, the system shall report the contextual factor as Unavailable instead of assuming a favorable condition.

Road condition may be represented as **Open**, **Caution**, **Closed/Impassable**, or **Unknown**, based on the available source or authorized administrative input.

Weather information obtained from the selected source may be translated into operational presentation categories such as **Normal**, **Caution**, **Severe**, or

Unavailable. These categories shall serve as advisory indicators and shall not independently choose or assign a vehicle.

When severe weather, cautionary road conditions, or another relevant contextual concern is identified, the system may retain the underlying operational recommendation while attaching a warning requiring administrative review. When the route is determined to be Not Feasible, the system shall indicate that the proposed movement should not proceed under the recorded condition.

If an external API or contextual data source is unavailable, the system shall continue evaluating the internal operational factors that remain available and shall clearly indicate that the unavailable contextual factor was not considered. Missing contextual information shall not be silently replaced using assumed values.

M. Human Decision Control and Recommendation Transparency

Interpretability and traceability are important advantages of rule-based decision-support approaches because recommendations can be linked directly to the conditions that produced them (Haber et al., 2026; Mohamad et al., 2026). Accordingly, forecasting, vehicle utilization, idle-vehicle detection, maintenance readiness, Customer-Side Vehicle Recommendation, and Branch Allocation Recommendation outputs shall be treated as decision-support information.

The proposed system shall not use unsupported confidence classifications, arbitrary urgency values, or unexplained 0–100 recommendation scores. Instead, recommendation results shall present traceable reasons based on the rules and data that produced the recommendation.

For example, a Branch Allocation Recommendation may indicate that the destination branch has a forecasted shortage of two vehicles, the source branch has a corresponding surplus of three vehicles, the proposed vehicle is maintenance-ready, has no conflicting booking, and has been idle for the defined monitoring period.

This reason-oriented approach is intended to make system-generated recommendations transparent and interpretable while preserving final operational authority under the Owner/Admin.

Development Methodology

This study will use a developmental and descriptive research design in the development and evaluation of the proposed web-based car rental management system with demand forecasting and context-aware fleet allocation. The study is developmental because it focuses on designing, developing, testing, and evaluating a functional system that addresses the identified operational problems of the participating car rental business. It is also descriptive because it examines the existing business workflow and evaluates the developed system based on selected quality and acceptability criteria.

For the software development process, the study will use the Iterative Waterfall Model. This model was selected because the proposed system requires clear documentation, organized development phases, structured system design outputs, and systematic testing before evaluation. At the same time, its iterative nature allows the researchers to revisit earlier phases when revisions are needed based on adviser feedback, client validation, prototype review, testing results, or evaluation findings.

The use of a structured development methodology is appropriate for this study because the proposed system includes several connected modules, such as booking management, customer and requirement verification, payment verification, vehicle and fleet management, rental transaction management, maintenance monitoring, report generation, demand forecasting, rule-based recommendation, context-aware decision support, and role-based access control. Related studies show that manual and semi-manual car rental operations commonly experience delays, fragmented records, overlapping reservations, and poor vehicle availability monitoring (Dhami et al., 2024; Mhatre et al., 2026; Rahman & Mohd Rusli, 2023). In addition, prior studies support the use of historical operational data for demand forecasting and the use of rule-based decision-support approaches for improving vehicle allocation and operational planning (Yu et al., 2020; Vateekul et al., 2021; Liu et al., 2021; You & Hsieh, 2025; Labib et al., 2026).

The development and evaluation of the system will also consider security, usability, reliability, and overall software quality. Since the system will handle customer records, uploaded requirements, proof of payment, booking records, fleet information, and operational reports, secure authentication and role-based access control will be implemented to protect sensitive information and restrict access based on user roles such as administrator, owner, and staff (Dedhia, 2026; Lu et al., 2024). The system will also include role-based access restrictions to ensure that only authorized users can access modules related to booking approval, payment verification, reports generation, user management, and operational monitoring.

The completed system will undergo unit testing, integration testing, security and access control testing, system testing, and user acceptance testing to verify whether

the developed modules function correctly both individually and as an integrated system. Unit testing will separately evaluate the customer web application, admin or owner web application, and staff web application to ensure that each module performs according to its intended functionality. Integration testing will verify proper interaction between connected modules, including role-based access behavior across different user accounts and permissions. Security and access control testing will assess authentication, authorization, protected routes, and user permission restrictions to ensure that unauthorized access to sensitive records and modules is prevented. User acceptance testing will also be conducted with selected users to determine whether the developed system aligns with the operational workflow and requirements of the business.

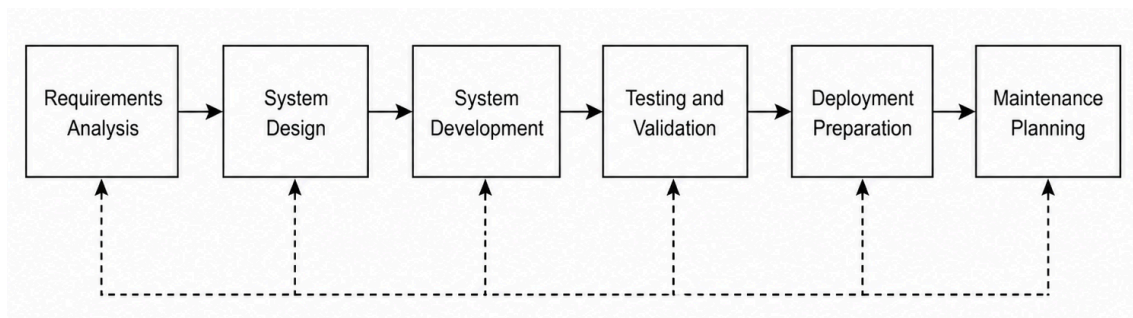
The completed system will be evaluated using selected ISO/IEC 25010 software quality characteristics, which provide a structured basis for assessing web-based and multi-module information systems (Monteverde et al., 2023; Bondoc, 2023; Mendoza et al., 2025; Adyaputra et al., 2025).

Process Model

The proponents will use the Iterative Waterfall Model as the process model for the development of the proposed system. The model follows a structured sequence of phases, namely requirements analysis, system design, implementation, testing and validation, deployment preparation, and maintenance planning. Unlike the strictly linear waterfall approach, the iterative waterfall model allows the researchers to return to earlier phases when improvements or corrections are needed. This makes the model

suitable for the study because the system may require refinement based on adviser feedback, client validation, prototype review, testing results, and evaluation findings.

Figure 69. Iterative Waterfall Model



The first phase is requirements analysis. In this phase, the researchers will gather and analyze information from client interviews, workflow validation, observation, and document review to identify the functional and non-functional requirements of the system.

The second phase is system design. In this phase, the researchers will prepare the system architecture, use case diagrams, activity diagrams, database schema, data dictionary, and user interface designs.

The third phase is implementation. In this phase, the system modules will be developed, including booking management, requirement verification, payment verification, vehicle and fleet management, rental monitoring, maintenance monitoring, reports generation, demand forecasting, rule-based recommendation, and context-aware decision support.

The fourth phase is testing and validation. In this phase, the developed modules will be checked through unit testing, integration testing, and system testing to determine whether they meet the identified requirements.

The fifth phase is deployment preparation. In this phase, the system will be prepared for pilot use by configuring user roles, database records, access permissions, and basic user guidance.

The final phase is maintenance planning. In this phase, possible improvements, error corrections, backup procedures, security updates, and future enhancements will be identified.

Development Tools

Table 72

Development Tools

TYPE	SOFTWARE	SPECIFICS	DESCRIPTION
Frontend Development	Frontend Framework	React.js with TypeScript	Used for developing the frontend interface of the web-based car rental management system with improved maintainability, modularity, and type safety.
Frontend Development	Build Tool	Vite	Used for local development, project bundling, and optimized frontend builds.

Frontend Development	Routing Library	TanStack Router	Used for managing navigation and routing between customer, staff, administrator, and owner pages.
Frontend Development	Styling Framework	Tailwind CSS	Used for responsive and utility-based interface styling across desktop and mobile browsers.
Frontend Development	UI Component Library	shadcn/ui and Radix UI	Used for reusable and accessible interface components such as forms, dialogs, buttons, tables, and navigation elements.
Backend Development	Runtime Environment	Node.js	Used for handling backend operations, server-side logic, and system workflows.
Backend Development	Server Framework	TanStack Start Server Functions and Server Routes	Used for processing backend requests, server-side operations, and frontend-backend communication.

Continuation of **Table 72. Development Tools**

TYPE	SOFTWARE	SPECIFICS	DESCRIPTION
Database Management	Relational Database Management System	PostgreSQL	Used for storing and managing structured system records such as bookings, rentals, users, vehicles, payments, maintenance records, and reports.
Database Management	Backend Service Platform	Supabase	Used for hosted database services, backend management, authentication support, and storage services.
Authentication and Access Control	Authentication Service	Supabase Auth	Used for login authentication, account verification, password reset, and role-based access control for customer, staff,

			administrator, and owner accounts.
File Storage	Cloud Storage Service	Supabase Storage	Used for storing uploaded customer requirements, valid IDs, driver's licenses, and proof-of-payment files.
Payment Processing	Manual Payment Verification	Proof-of-Payment Upload and Administrator Verification	Used for recording and manually verifying customer payment proof and transaction references before payment confirmation.
Notification Services	Email Delivery Service	Brevo transactional email	Used for sending booking updates, verification results, payment reminders, and rental-related notifications.
Notification Services	SMTP Service	Custom SMTP for Supabase Auth Emails	Used for account confirmation emails and password reset notifications.

Continuation of **Table 72. Development Tools**

TYPE	SOFTWARE	SPECIFICS	DESCRIPTION
Decision-Support Development	Forecasting and Recommendation Logic	TypeScript-Based Weighted Moving Average and Rule-Based Recommendation Logic	Used for implementing demand forecasting, utilization analysis, idle vehicle detection, vehicle recommendation, and branch allocation recommendation.
Context-Aware Decision Support	Weather Data APIs	Open-Meteo Forecast API; OpenWeather One Call API 3.0	Open-Meteo serves as the primary weather-data provider, while OpenWeather serves as the fallback provider for applicable weather information used by the administrative context-aware decision-support component.

Context-Aware Decision Support	Location, Routing, and Road-Incident APIs	Geoapify; LocationIQ; TomTom Orbis APIs; HERE Technologies APIs	Geoapify with LocationIQ fallback provides destination geocoding. TomTom Orbis Routing with HERE Routing fallback provides route distance and travel time. TomTom Traffic with HERE Traffic fallback provides incident information. Provider availability and location-quality checks determine whether a factor is usable.
UI/UX Design	Design and Prototyping Tool	Figma	Used for interface prototyping, layout planning, and user interface design.
System Diagramming	Diagramming Tool	Lucidchart	Used for creating use case diagrams, activity diagrams, data flow diagrams, and other system diagrams.
Testing and Quality Assurance	Unit Testing Framework	Node.js built-in test runner (node:test)	Used for automated tests of functions, server logic, and decision-support rules.
Testing and Quality Assurance	Component Testing Tool	React Testing Library (planned; not included in the current test suite)	Planned for component behavior and interaction tests; no React Testing Library suite is currently included in the repository.
Testing and Quality Assurance	End-to-End Testing Tool	Playwright	Used for testing complete workflows such as login, booking, payment upload, and role-based access behavior.
Testing and Quality Assurance	API Testing Tool	Postman	Used for testing backend routes, requests, responses, and API integration behavior.

Continuation of **Table 72. Development Toolss**

TYPE	SOFTWARE	SPECIFICS	DESCRIPTION
Testing and Quality Assurance	Debugging Tool	Browser Developer Tools	Used for frontend debugging, responsive testing, and runtime inspection.
Version Control	Source Code Management Platform	Git and GitHub	Used for version control, collaboration, source code management, and tracking development changes.
Deployment and Hosting	Local Development Environment	Localhost	Used during development and defense demonstration of the proposed system.
Deployment and Hosting	Web Deployment Platform	Vercel	Used for deploying and hosting the web application when online deployment is required.
Deployment and Hosting	Hosted Backend Services	Supabase Hosted Services	Used for hosted database, authentication, storage, and backend-related services.

The development tools were selected based on the technical requirements of the proposed web-based car rental management system. React.js with TypeScript, Vite, TanStack Router, Tailwind CSS, shadcn/ui, and Radix UI will be utilized to develop a responsive and user-friendly interface for customers, staff, administrators, and owners. These technologies will support modular interface development, reusable components, responsive layouts, and browser-based accessibility across desktop and mobile devices.

For backend and data management, Node.js together with TanStack Start server functions and server routes will be utilized to handle server-side operations, backend workflows, and frontend-backend communication. PostgreSQL through Supabase will serve as the main database management platform for handling customer

records, booking transactions, vehicle records, rental information, maintenance records, payment validation records, notifications, and operational reports. Supabase Auth will be implemented for authentication and role-based access control, while Supabase Storage will be utilized for storing uploaded customer requirements, valid identification cards, driver's licenses, and proof-of-payment files.

The system will also incorporate notification and communication support through Brevo transactional email, while Supabase Auth emails such as account confirmation and password reset will use custom SMTP services. For payment processing, the system will support proof-of-payment upload, transaction-reference recording, payment-status tracking, and manual verification by an authorized owner or administrator against the business's external bank or e-wallet records. The system will not integrate an automated payment gateway within the current scope of the study. Context-aware decision support will obtain applicable weather information primarily through the Open-Meteo Forecast API, with the OpenWeather One Call API 3.0 serving as the fallback weather provider. Geoapify with LocationIQ fallback will provide destination geocoding. TomTom Orbis Routing with HERE Routing fallback will provide route distance and travel-time estimates, while TomTom Traffic with HERE Traffic fallback will provide incident information. Geocoding quality, source freshness, and coverage will be checked before these outputs support administrative decisions. When neither the primary nor fallback provider can supply sufficiently usable information, authorized manual contextual information may be recorded where available; otherwise, the affected contextual factor will be marked as Unavailable or Unknown. Simulated contextual data may be used during functional testing but shall not be represented as live API-derived operational information.

For the decision-support components, TypeScript-based logic will be implemented for Weighted Moving Average demand forecasting, vehicle utilization analysis, idle vehicle detection, Customer-Side Vehicle Recommendation, and rule-based Branch Allocation Recommendation. Figma will be utilized for interface design and prototyping, while Lucidchart will be used for creating system diagrams. Testing and quality assurance will be supported through Node.js built-in test runner (node:test), React Testing Library (planned; not included in the current test suite), Playwright, Postman, and Browser Developer Tools to verify system functionality, component behavior, role-based workflows, server routes, and user interaction processes. Git and GitHub will be used for version control and collaborative development. During development and defense, the system may be demonstrated through a localhost environment, while Vercel and Supabase hosted services may be used for online deployment and backend hosting.

Test Methodology/Procedures

To ensure the quality, functionality, reliability, security, and usability of the proposed Web-based Car Rental Management System, several testing procedures will be conducted throughout the development process. These procedures will determine whether the system meets the identified functional and non-functional requirements and whether it supports the actual workflow of the participating car rental business. The testing process will help the researchers identify errors, validate system outputs, check database transactions, verify access restrictions, and confirm that the system performs the required processes for customer account management, booking management, requirement verification, payment submission and verification, vehicle and fleet

management, rental monitoring, reports generation, demand forecasting, and rule-based recommendation.

Test Case Description

Test cases will be prepared to guide the testing of the system's major modules and workflows. Each test case will include the module or feature to be tested, test objective, input data, expected output, actual output, result status, and remarks. The test cases will be based on the identified system requirements, use case reports, activity diagrams, and system workflows.

The test cases will cover both valid and invalid inputs to determine whether the system can process correct data, reject incomplete or incorrect entries, display appropriate messages, and store records accurately. The test cases will also include role-based scenarios to verify whether customers, staff, administrators, and owners can only access the features and records assigned to their roles.

Unit Testing

Unit testing will be conducted to evaluate the individual components, modules, and functions of the system before they are integrated with other parts of the platform. This procedure will help determine whether each module accepts valid inputs, rejects invalid inputs, processes data correctly, stores records accurately, and produces the expected outputs.

For the Customer or Renter Web Application, unit testing will focus on account registration, login, vehicle catalog viewing, booking request submission, requirement upload, proof-of-payment upload, booking status tracking, rental policy viewing, and notification viewing. These tests will ensure that customers can properly enter booking details, submit required documents, upload payment proof, and monitor the status of their reservations.

For the Staff Web Application, unit testing will focus on the modules assigned to staff users, such as booking schedule viewing, reservation coordination, customer communication support, calendar monitoring, vehicle availability viewing, and operational status updating based on the access permissions granted to staff accounts. These tests will ensure that staff users can perform their intended operational tasks without accessing restricted owner-level information.

For the Admin or Owner Web Application, unit testing will focus on booking approval, customer and requirement verification, payment verification, vehicle and fleet management, rental transaction management, maintenance monitoring, reports generation, user and role management, demand forecasting, rule-based vehicle recommendation, and branch allocation recommendation. These tests will ensure that owner and administrator accounts can properly manage operational records, verify transactions, generate reports, and access decision-support features according to their authorized roles.

The decision-support features will also undergo unit testing. WMA tests shall verify confirmed-booking-start aggregation by branch, category, and Asia/Manila week; fixed weights; recursive horizons; valid zero-demand weeks; incomplete coverage; and preservation of forecast snapshots. Hand-calculated expected values shall be

compared with system outputs. Allocation tests shall cover shortage and surplus calculations, source-demand protection, candidate-count limits, conflicting bookings and maintenance, duplicate candidate prevention, no feasible source, and no automatic branch change after approval. Customer recommendation tests shall use its separate capacity, luggage, budget, availability, and preference rules. Context tests shall cover valid primary responses, fallback after provider failure, adverse primary responses without favorable-result substitution, stale or missing data, supported time horizons, and distance-to-fuel calculations. Utilization and idle-duration tests shall verify their defined operational periods and eligibility conditions.

Integration Testing

Integration testing will be conducted after unit testing to verify whether the different modules of the system work together properly. Since the proposed system contains interconnected features, this testing phase will focus on checking the accuracy of data flow between the customer web application, staff web application, admin or owner web application, database, authentication service, file storage, payment records, vehicle records, rental records, notification services, reporting module, and decision-support features.

For the booking workflow, integration testing will validate the connection between customer registration, booking request submission, requirement upload, requirement verification, quotation or booking review, proof-of-payment upload, payment verification, vehicle assignment, booking confirmation, and customer notification. This will ensure that the booking process is properly connected from the customer's initial request up to administrative approval and confirmation.

For the rental and fleet workflow, integration testing will verify the connection between confirmed bookings, vehicle records, rental records, vehicle release details, rental status updates, return processing, maintenance records, and generated reports. This will ensure that changes in booking and rental status are reflected in related records such as vehicle availability, branch assignment, payment records, maintenance status, and operational reports.

For the decision-support workflow, integration testing will check whether booking history, rental records, vehicle records, branch data, and selected context inputs are correctly used by the forecasting and recommendation modules. The test sequence shall trace a known booking-history dataset through the WMA forecast, projected-supply evaluation, shortage or surplus output, eligible candidate list, contextual review, and recorded administrative decision. Expected results and observed results shall be recorded at each step. Approval shall leave vehicle branch assignments unchanged, and missing external context shall remain visible. These tests establish conformance to the stated rules, not guaranteed real-world forecasting accuracy.

Role-based access testing will also be included during integration testing. This will verify whether each user role can access only the modules and records assigned to that role. Customer accounts should only access their own booking details and submitted requirements. Staff accounts should only access permitted booking, calendar, and operational coordination features. Admin or owner accounts should have access to approval, payment verification, reports, user management, and decision-support features based on the defined access rules.

System Testing

System testing will be conducted to evaluate the entire Web-based Car Rental Management System as a complete web-based platform. This testing will determine whether all major features function according to the system requirements and whether the system supports the actual business process from booking request to rental completion.

The researchers will test the complete workflow beginning with customer registration, booking request submission, requirement upload, staff or admin review, payment proof submission, payment verification, vehicle assignment, booking confirmation, vehicle release, rental monitoring, physical vehicle return, operational inspection, and report generation. Financial settlement is outside the current workflow. The purpose of this testing is to determine whether the system can complete the full operational workflow without errors, missing data, or inconsistent record updates.

System testing will also include checking database accuracy, record updating, status changes, file uploads, notifications, and transaction consistency. For example, when a booking is confirmed, the assigned vehicle status should update properly. When payment proof is uploaded, the payment record should be stored and made available only to authorized users. When a rental is completed, related reports and dashboard summaries should reflect the updated records.

User Acceptance Testing

User Acceptance Testing will be conducted to determine whether the system is acceptable and usable from the perspective of its intended users. Selected users, including customer or renter participants, staff users, administrators, and owner representatives, will be asked to perform common system tasks based on their assigned roles.

Customer or renter participants will test tasks such as account registration, vehicle browsing, booking request submission, requirement upload, proof-of-payment upload, booking status tracking, and notification viewing. Staff users will test tasks such as viewing booking schedules, monitoring calendar-related information, checking customer and booking records based on their access level, and updating permitted operational statuses. Admin or owner users will test tasks such as approving bookings, verifying requirements, checking payment records, managing vehicles, monitoring fleet status, generating reports, managing user roles, and reviewing decision-support outputs.

Feedback from users will be gathered to identify concerns related to ease of use, clarity of interface, accuracy of displayed information, mobile accessibility, access restrictions, and overall usefulness of the system. The results of User Acceptance Testing will help the researchers determine whether the system meets the operational needs of the participating car rental business and whether revisions are needed before final implementation.

Performance Testing

Performance testing will be conducted to determine whether the system responds within an acceptable time during common user activities. This testing will focus on practical system performance rather than large-scale stress testing. The researchers will check the response time of major pages and workflows such as login, dashboard loading, vehicle catalog viewing, booking request submission, file upload, report generation, and decision-support output generation.

The forecasting and recommendation features will be tested using a known, labeled synthetic dataset to determine whether the system follows the defined computation and allocation rules and produces results within a reasonable time. The test documentation will identify the dataset as synthetic and record the expected and observed outputs. Performance results from these records will demonstrate processing behavior only and will not be interpreted as empirical forecasting accuracy. Since the system will be used through desktop and mobile browsers, performance testing will help ensure that users can complete common tasks without unnecessary delays during development, demonstration, and evaluation.

Security and Access Control Testing

Security and access control testing will be conducted to verify whether the system properly protects sensitive records and restricts access based on user roles. This testing will focus on authentication, password reset, account verification, protected routes, role-based access control, file access restrictions, and unauthorized access prevention.

The researchers will test whether customers, staff, administrators, and owners are redirected only to the pages assigned to their roles. Attempts to access restricted pages or records will also be tested to ensure that unauthorized users cannot view or modify sensitive information. This includes customer requirements, valid IDs, driver's licenses, proof-of-payment files, payment verification records, reports, user management functions, and decision-support outputs.

The testing will also check whether uploaded files are stored properly and can only be accessed by authorized users. Since the system will handle customer documents, payment proof, booking records, and operational reports, access control must be verified to prevent unauthorized viewing, editing, or downloading of confidential information.

Compatibility and Responsiveness Testing

Compatibility and responsiveness testing will be conducted to ensure that the system can be accessed through supported desktop, tablet, and mobile browsers. Since the participating car rental business uses laptops, iPads, iPhones, and mobile data during operations, the system will be tested on common browsers such as Google Chrome, Microsoft Edge, and Safari.

This testing will check whether pages, forms, tables, buttons, dialogs, calendars, uploaded files, dashboards, and notifications display and function properly across different screen sizes. It will also verify whether customer-side, staff-side, and admin or owner-side pages remain readable, responsive, and usable on mobile devices, tablets, and desktop screens.

Usability Testing

Usability testing will be conducted to determine whether the system is easy to understand, navigate, and operate for its intended users. This testing will focus on the clarity of labels, arrangement of menus, readability of information, ease of completing tasks, and overall user experience.

The researchers will observe whether users can complete common tasks such as submitting a booking request, uploading requirements, checking booking status, reviewing booking schedules, verifying payments, managing vehicles, generating reports, and reviewing dashboard information with minimal confusion. Feedback gathered from usability testing will be used to identify areas for improvement in interface design, navigation flow, form layout, and user guidance.

Test Documentation

The results of the testing procedures will be documented through test cases, test results, screenshots, issue logs, and tester remarks. Each test case will include the module tested, test objective, input data, expected output, actual output, result status, and remarks. Any errors, inconsistencies, access control issues, performance concerns, compatibility problems, or usability concerns found during testing will be recorded and corrected before proceeding to final evaluation and implementation.

Overall, the testing procedures will help ensure that the proposed Web-based Car Rental Management System functions as a reliable, secure, responsive, and

usable web-based platform. Through test case preparation, unit testing, integration testing, system testing, user acceptance testing, performance testing, security and access control testing, compatibility and responsiveness testing, usability testing, and test documentation, the researchers will be able to verify whether the system supports the operational needs of the business and improves the management of bookings, requirement verification, payment processing, vehicle records, fleet monitoring, rental transactions, reports, forecasting, and rule-based recommendations.

System Requirements

The proposed the participating car rental business Management System is a web-based platform designed to run through standard browsers on staff and renter devices. Since the current operations rely on laptops, iPads, iPhones, home internet, and mobile data, the system requirements focus on browser compatibility, mobile accessibility, stable connectivity, lightweight performance, and secure data handling for booking, payment, vehicle, rental, and reporting processes.

Table 73

Desktop and Tablet System Requirements for Staff and Admin

COMPONENT	MINIMUM REQUIREMENT	RECOMMENDED REQUIREMENT
Operating System	Windows 10 / macOS Monterey / iPadOS 15 or later	Latest Windows 11 / macOS Ventura or later / latest iPadOS
Browser	Google Chrome / Microsoft Edge / Safari	Latest version of Google Chrome or Safari

Internet	Stable Wi-Fi or mobile data connection	High-speed broadband, Wi-Fi, or 5G mobile data
----------	--	--

Continuation of **Table 73. Desktop and Tablet System Requirements for**

Staff and Admin

COMPONENT	MINIMUM REQUIREMENT	RECOMMENDED REQUIREMENT
RAM	4 GB	8 GB or higher
Processor	Dual-core CPU or equivalent	Quad-core CPU or better
Display	1366 x 768 resolution or iPad screen equivalent	1920 x 1080 resolution or larger tablet screen
Device Type	Laptop, desktop, or iPad	Laptop with updated browser or latest iPad model

Table 73 presents the minimum and recommended desktop and tablet system requirements for the internal users of the participating car rental business Management System including Operations Staff and Owner/Admin users. These requirements include the operating system, browser, internet connection, memory, processor, display, and device type needed to manage booking records, renter details, payment verification, vehicle availability, rental transactions, reports, and dashboard functions. Since the business currently uses laptops, iPads, and iPhones for daily operations, the system is designed to be accessible through standard web browsers without requiring separate software installation.

Table 74

Mobile System Requirements for Renter

COMPONENT	MINIMUM REQUIREMENT	RECOMMENDED REQUIREMENT
Operating System	Android 10 or iOS 14 and later	Android 12 or later / latest iOS version
Browser	Google Chrome / Safari / mobile browser	Latest version of Google Chrome or Safari
Storage	At least 300 MB of free space	500 MB or more free space
RAM	3 GB	4 GB or higher
Internet	Stable mobile data or Wi-Fi	High-speed Wi-Fi or 5G mobile data
Screen Size	5.0 inches or larger	6.0 inches or larger
Camera / File Upload	Built-in camera or file upload support	Clear camera and updated file upload support

Table 74 presents the minimum and recommended mobile system requirements for renters using the participating car rental business Management System. These requirements include the operating system, browser, storage, memory, internet connection, screen size, and camera or file upload support needed to access the vehicle catalog, submit booking requests, upload renter requirements, upload proof of payment, view booking status, and receive system notifications. Since renters may use

different mobile devices, the system supports both Android and iOS devices through a browser-based interface.

Quality Plan

Throughout the development of the project, several quality assurance activities will be implemented to ensure that the system is reliable, functional, secure, and easy to use. The researchers will conduct system testing, review system functions, validate user workflows, and collect evaluation feedback from selected respondents. These activities will help identify errors, improve system usability, and ensure that the developed system meets its intended requirements.

The system will be evaluated based on selected ISO/IEC 25010 software quality characteristics, specifically functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety. These characteristics will serve as the basis for assessing the overall quality and acceptability of the developed system.

The evaluation will focus on the system's ability to support customer-side and administrative-side functions. Customer-side functions include account registration, vehicle catalog viewing, Customer-Side Vehicle Recommendation, booking request submission, requirement upload, proof-of-payment upload, booking status tracking, notifications, and rental policy viewing. Administrative-side functions include booking management, customer verification, payment validation, vehicle records management,

fleet monitoring, maintenance monitoring, demand forecasting, branch allocation recommendation, reports, and role-based access control.

The ISO/IEC 25010 standard will serve as the primary framework for the system evaluation and questionnaire used in this study. The research instrument will be adapted from an existing ISO/IEC 25010-based evaluation questionnaire, particularly the modified survey questionnaire used by Mission (2020) in evaluating a system based on software quality criteria. Since Mission's instrument was based on an earlier ISO/IEC 25010 model, the researchers will not copy the instrument directly. Instead, the questionnaire structure and item construction will be adapted and revised according to the selected ISO/IEC 25010 characteristics, the system features, and the respondent groups of the present study.

A recent ISO/IEC 25010 based application evaluation study by Nugraha and Rachman (2024) will also be used as a supporting basis for aligning the instrument with the updated model. Prior to data gathering, the adapted questionnaire will be reviewed by qualified validators to ensure that the items are clear, relevant, and aligned with the selected software quality characteristics.

Implementation Plan

The implementation of the proposed web-based car rental management and decision-support system will be carried out in a controlled and systematic manner to ensure proper functionality, usability, and reliability. The system will be prepared for the participating car rental business, specifically for its Manila and Antipolo operations, with

features intended for customers or renters, owners or administrators, and operational staff.

The customer-side system will be accessible through standard desktop and mobile web browsers. Customers will be able to register an account, view available vehicles, use the Customer-Side Vehicle Recommendation, submit booking requests, upload rental requirements, upload proof of payment, track booking status, view rental policies, and receive notifications. The Customer-Side Vehicle Recommendation will be tested to ensure that it properly accepts customer inputs and displays suitable vehicle suggestions based on predefined rule-based logic.

The administrative system will be implemented through a web-based platform accessible to authorized users. Owners, administrators, and staff will be given access according to their assigned roles. The administrative features will include booking management, customer verification, payment validation, vehicle and fleet management, maintenance monitoring, scheduling, demand forecasting, branch allocation recommendation, report generation, notification management, and user role management.

The implementation process will be conducted in phases. First, the researchers will configure the system and verify that all major modules are functional. Second, system testing will be conducted to check whether the customer-side and administrative-side features are working as intended. Third, selected users will be allowed to test the system by performing common tasks such as using the Customer-Side Vehicle Recommendation, submitting booking requests, uploading requirements, verifying payments, updating vehicle records, recording maintenance

information, generating demand forecasts, and reviewing branch allocation recommendations.

After system testing, necessary revisions will be made based on observed issues, user feedback, and adviser or panel recommendations. Once the system has been configured, tested, revised, and validated through user feedback, it will be prepared for user acceptance testing. This implementation approach ensures that the system is introduced in a practical and manageable manner while allowing the researchers to assess whether the system supports its intended requirements.

Evaluation Plan

In order to assess the system's performance, usability, technical quality, and overall level of user satisfaction, the study will employ purposive sampling and total enumeration depending on the respondent group. This approach ensures that data will be gathered from respondents who are directly involved in or have actual experience with the use of the system, thereby providing relevant and reliable evaluation results.

The evaluation will involve 50 to 80 customers or renters, who will assess the system based on their booking experience and use of the Customer-Side Vehicle Recommendation for vehicle selection. In addition, 1 to 3 owners/administrators, and staff will be included through total enumeration or actual available population and will evaluate the system from an operational management perspective, particularly in relation to booking management, fleet monitoring, maintenance tracking, demand forecasting, and branch allocation recommendation.

Furthermore, 3 to 5 IT expert evaluators will be selected purposely based on their professional background in information technology, software development, systems evaluation, database management, or related fields. They will assess the

technical quality of the system in accordance with selected ISO/IEC 25010 software quality characteristics, including functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety.

The survey questionnaire will be distributed through printed or digital copies depending on the availability and accessibility of the participants. The questionnaire will be based on the ISO/IEC 25010 standard and will be aligned with the metrics used in the User Acceptance Testing phase.

This study will utilize three separate survey questionnaires to evaluate the proposed Web-based Car Rental Management System in accordance with the ISO/IEC 25010 System and Software Quality Model. The questionnaires will be designed for three distinct user groups: customer or renter respondents, owner/administrator/staff respondents, and IT expert evaluators. Although the instruments are grounded in the same quality framework, each questionnaire focuses on quality characteristics relevant to the respondents' roles and level of system interaction.

Table 75

Respondent Profile

Respondent Group	Target Respondents	Sampling Technique	Evaluation Focus
Customer/Renter Respondents	Actual or potential renters of the participating car rental business	Slovin's formula or purposive sampling, depending on available	Customer-side features, Customer-Side Vehicle Recommendation,

		customer population	booking request, requirement upload, proof-of-payment upload, booking tracking, notifications, rental policy viewing, and ease of use
Owner/Administrator/ Staff Respondents	Owner, administrator, and operational staff of the participating car rental business	Total enumeration, if all business users are available	Booking management, payment validation, customer verification, vehicle records, maintenance monitoring, fleet monitoring, reports, demand forecasting, and branch allocation recommendation

IT Expert Evaluators	Three to five IT experts	Purposive sampling	Functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, safety, and technical correctness
----------------------	--------------------------	--------------------	---

Table 75 presents the profile of respondents who will participate in the system evaluation. The respondents are grouped according to their role in the system and the specific features they are expected to evaluate.

Table 76

Evaluation Respondent Matrix

System Feature / Evaluation Area	Customer/Renter	Owner/Admin/Staff	IT Experts
Account Registration and Login	Yes	Yes	Yes

Vehicle Catalog Viewing	Yes	Yes	Yes
Customer-Side Vehicle Recommendation	Yes	No	Yes
Booking Request Submission	Yes	Yes	Yes
Requirement Upload and Verification	Yes	Yes	Yes
Proof-of-Payment Upload and Payment Validation	Yes	Yes	Yes
Booking Status Tracking and Notifications	Yes	Yes	Yes
Rental Policy Viewing	Yes	Yes	Yes
Vehicle and Fleet Management	No	Yes	Yes
Maintenance Monitoring	No	Yes	Yes

Demand Forecasting using WMA	No	Yes	Yes
Branch Allocation Recommendation	No	Yes	Yes
Reports and Dashboard	No	Yes	Yes
Role-Based Access Control	No	Yes	Yes
Security, Reliability, and Maintainability	No	No	Yes

Table 76 shows the Evaluation Respondent Matrix of the study. It identifies which respondent group will evaluate each system feature or evaluation area. This ensures that each feature will be assessed by the respondents who are most familiar with its intended use.

Customer/Renter Survey Questionnaire

The Customer/Renter Survey Questionnaire was designed to assess user-facing quality characteristics of the system based on ISO/IEC 25010. It focuses on Functional Suitability, Performance Efficiency, Compatibility, Interaction Capability, Reliability, Security, Maintainability, Flexibility, and Safety, which are critical in evaluating customers' experience in using the booking and vehicle selection features of the system.

A. Functional Suitability

In the ISO/IEC 25010 definition, functional suitability refers to how well a product or system provides the right functions to meet user needs under specified conditions, ensuring that all required functions are present, work correctly, and effectively support users in achieving their intended tasks and objectives.

The following statements were used as part of the survey instrument:

- The system provides all the features needed for submitting a car rental booking request.
- The vehicle catalog allows customers to view available vehicles clearly.
- The Customer-Side Vehicle Recommendation allows customers to input trip requirements before booking.
- The Customer-Side Vehicle Recommendation provides vehicle suggestions that match the customer's passenger capacity, destination or travel area, budget, rental duration, and vehicle preference.
- The system allows customers to upload rental requirements and proof of payment properly.
- The system allows customers to track their booking status.

These questions specifically address whether the system offers complete and appropriate customer-side features, covers essential booking functions, and supports vehicle selection before booking. By directly targeting these functional areas, the questions together measure the system's functional suitability from the customer perspective.

B. Performance Efficiency

In the ISO/IEC 25010 definition, performance efficiency refers to the degree to which a product or system performs its functions within specified time and throughput parameters while efficiently utilizing resources under specified conditions.

The following statements were used as part of the survey instrument:

- The system loads pages and forms within an acceptable time.
- The Customer-Side Vehicle Recommendation processes customer inputs without unnecessary delay.
- The system allows customers to submit booking requests efficiently.
- The system allows customers to upload requirements and proof of payment without noticeable delay.

These questions include response time, processing speed, booking submission, and file upload performance. These elements help assess the efficiency of the customer-side system from the perspective of the renter.

C. Compatibility

In the ISO/IEC 25010 definition, compatibility refers to the degree to which a product, system, or component can perform its required functions while sharing a common environment and resources.

The following statements were used as part of the survey instrument:

- The system works properly on the device being used.
- The system can be accessed through common web browsers.
- The system interface remains usable on desktop and mobile browsers.
- The customer-side features work properly across supported devices.

These questions check whether the system works correctly across different user devices and common browsers. They help measure how compatible the system is for customers who may use laptops, tablets, or mobile phones.

D. Interaction Capability

In the ISO/IEC 25010 definition, interaction capability refers to the degree to which a product or system can be effectively interacted with by specified users through its user interface to exchange information and complete tasks in various contexts of use.

The following statements were used as part of the survey instrument:

- The system is easy to understand and operate.
- The labels, buttons, and forms are clear.
- The Customer-Side Vehicle Recommendation is easy to use when selecting a suitable vehicle.
- Booking-related tasks can be completed without confusion.
- The system provides clear messages, prompts, and notifications.

These questions assess ease of use, independent task completion, clarity of design, and the ability of customers to complete system tasks without confusion.

E. Reliability

According to ISO/IEC 25010, reliability refers to the degree to which a system performs specified functions consistently and accurately under stated conditions for a specified period of time.

The following statements were used as part of the survey instrument:

- The system performs its customer-side functions consistently.

- The system accurately records booking requests.
- The system properly handles incomplete or incorrect customer inputs.
- The system provides accurate booking status updates.

These questions assess whether the system can consistently support customer transactions and provide correct outputs during customer use.

F. Security

According to ISO/IEC 25010, security is the degree to which a system protects information and data so that unauthorized persons or systems cannot access, modify, or misuse them.

The following statements were used as part of the survey instrument:

- The system requires customers to log in before accessing their booking records.
- The system protects customer information and uploaded requirements.
- The system protects proof-of-payment records from unauthorized access.
- The system separates customer access from administrative access.

These questions are justified under this characteristic because the proposed system will handle sensitive customer information, uploaded requirements, proof of payment, and booking records.

G. Maintainability

According to ISO/IEC 25010, maintainability refers to the ease with which a system can be modified, supported, and kept operational over time.

The following statements were used as part of the survey instrument:

- The system appears organized and properly maintained.

- The system provides updated information about bookings and vehicle availability.
- The system allows customer-side information to be updated when necessary.

These questions help assess whether the system appears manageable and capable of supporting future improvements from the customer perspective.

H. Flexibility

According to ISO/IEC 25010, flexibility refers to the ability of a system to be adapted to different requirements, environments, and user needs.

The following statements were used as part of the survey instrument:

- The system supports different customer booking needs.
- The Customer-Side Vehicle Recommendation allows customers to enter different trip requirements.
- The system can support different vehicle preferences and rental durations.
- The system can accommodate changes in customer booking details before submission.

These questions are justified under flexibility because they assess whether the system can support varied customer requirements and rental situations.

I. Safety

According to ISO/IEC 25010, safety concerns the degree to which a system reduces the risk of harm or negative consequences during its operation and use.

The following statements were used as part of the survey instrument:

- The system provides clear instructions that support safe and proper use.

- The system helps prevent customers from selecting vehicles that are unavailable.
- The system shows booking-related information clearly before submission.
- The system ensures that final booking approval remains under authorized personnel.

These questions are justified under safety because they assess whether the system supports safe, controlled, and properly reviewed rental transactions.

Owner/Administrator/Staff Survey Questionnaire

The Owner/Administrator/Staff Survey Questionnaire was developed to evaluate system quality from an administrative and operational perspective, following the ISO/IEC 25010 quality model. This focuses on Functional Suitability, Performance Efficiency, Compatibility, Interaction Capability, Reliability, Security, Maintainability, Flexibility, and Safety related to booking management, vehicle records, payment verification, maintenance monitoring, demand forecasting, branch allocation recommendation, reports, and access control.

A. Functional Suitability

ISO/IEC 25010 defines functional appropriateness as how well a system's functions meet both stated and unstated requirements. For administrators and staff, this means checking if the system offers enough tools for managing, monitoring, and running operations.

The following statements were used as part of the survey instrument:

- The system provides the necessary functions for managing booking requests.
- The system supports customer requirement verification.

- The system supports manual proof-of-payment verification.
- The system allows administrators to manage vehicle and fleet records.
- The system supports maintenance monitoring and vehicle condition recording.
- The system provides demand forecasting using the Weighted Moving Average method.
- The system provides branch allocation recommendations for administrative review.
- The system generates useful reports and dashboard summaries.

These questions check if the system's administrative features are complete and suitable. They look at whether administrators have the right tools to manage rental operations, track records, and support decision-making effectively.

B. Performance Efficiency

According to ISO/IEC 25010, performance efficiency refers to system performance relative to the number of resources used under stated conditions. In the admin context, this characteristic emphasizes timely task execution and efficient generation of reports and system outputs.

The following statements were used as part of the survey instrument:

- The system processes administrative records efficiently.
- The system loads booking, customer, vehicle, and maintenance records within an acceptable time.
- The system generates reports and dashboard summaries without unnecessary delay.
- The system processes demand forecasting and branch allocation recommendation efficiently.

These questions assess performance efficiency by focusing on response time and processing speed. Efficient performance ensures that administrators can carry out tasks without delays, supporting smooth rental operations.

C. Compatibility

According to ISO/IEC 25010, compatibility refers to the degree to which a system can operate effectively while sharing an environment with other systems and devices.

The following statements were used as part of the survey instrument:

- The administrative system works properly on common web browsers.
- The system can be accessed using the client's existing devices.
- The system supports the browser-based workflow of the business.
- The system interface remains usable across supported desktop and mobile browsers.

These questions assess whether the system can operate within the client's existing technical environment and browser-accessible workflow.

D. Interaction Capability

ISO/IEC 25010 says interaction capability is about how easily users can use a system to reach their goals. For administrators and staff, this means being able to finish tasks on their own, with easy navigation and a clear interface.

The following statements were used as part of the survey instrument:

- The administrative interface is easy to understand and navigate.
- The system forms and labels are clear.
- Booking, payment, vehicle, and maintenance records are easy to manage.

- Reports and dashboard summaries are easy to interpret.
- The system provides clear prompts, alerts, and notifications.

These questions assess ease of use and clarity of the administrative system interface, especially for tasks related to booking, payments, vehicles, maintenance, and reports.

E. Reliability

According to ISO/IEC 25010, reliability refers to the degree to which a system performs specified functions consistently and accurately under stated conditions.

The following statements were used as part of the survey instrument:

- The system consistently records booking and rental transactions.
- The system accurately stores customer, vehicle, payment, and maintenance records.
- The system properly handles incomplete or incorrect inputs.
- The system maintains accurate vehicle availability and rental status information.
- The system supports backup and recovery procedures to reduce the risk of data loss.

These questions evaluate whether the system can support accurate and consistent operational recordkeeping.

F. Security

According to ISO/IEC 25010, security is the degree to which a system protects information and data from unauthorized access, modification, or misuse.

The following statements were used as part of the survey instrument:

- The system restricts access based on assigned user roles.

- The system protects customer documents and uploaded requirements.
- The system protects payment information and proof-of-payment records.
- The system limits sensitive administrative functions to authorized users.
- The system records important user actions through audit logs or activity records.

These questions assess whether the system provides access control and protects sensitive customer, payment, and operational records.

G. Maintainability

According to ISO/IEC 25010, maintainability refers to the ease with which a system can be modified, corrected, updated, and supported over time.

The following statements were used as part of the survey instrument:

- The system allows authorized users to update customer, vehicle, booking, and maintenance records.
- The system supports maintenance history and audit records.
- The system structure allows future updates and corrections.
- The system can support future enhancements such as payment gateway integration or GPS integration.

These questions are included because the system must remain manageable, correctable, and capable of future improvement after development.

H. Flexibility

According to ISO/IEC 25010, flexibility refers to the ability of a system to adapt to different operational needs, changes in requirements, and possible future enhancements.

The following statements were used as part of the survey instrument:

- The system can support changes in vehicle records, rental policies, and user roles.
- The system can accommodate additional vehicles, users, and booking records.
- The system can support optional context inputs depending on data availability.
- The system can be expanded for future operational requirements.

These questions assess whether the system can adjust to changing business needs and operational requirements.

I. Safety

According to ISO/IEC 25010, safety evaluates whether the system helps reduce operational risks and supports controlled decision-making.

The following statements were used as part of the survey instrument:

- The system helps prevent unavailable vehicles from being recommended or assigned.
- The system helps identify vehicles that are under maintenance or not ready for rental.
- The system supports safer operational decisions by showing vehicle condition, maintenance status, and monitoring notes.
- The system ensures that final booking approval, payment validation, and vehicle assignment remain under authorized human control.

These questions assess whether the system helps reduce operational risks in vehicle recommendation, booking approval, and rental processing.

IT Expert Evaluation Questionnaire

The IT Expert Evaluation Questionnaire was developed to assess the technical quality of the system based on the ISO/IEC 25010 quality model. This focuses on Functional Suitability, Performance Efficiency, Compatibility, Interaction Capability, Reliability, Security, Maintainability, Flexibility, and Safety related to system correctness, technical behavior, access control, maintainability, and future system improvement.

A. Functional Suitability

The following statements were used as part of the IT expert evaluation instrument:

- The system functions are aligned with the stated requirements.
- The Customer-Side Vehicle Recommendation performs according to its intended rule-based logic.
- The demand forecasting feature follows the Weighted Moving Average method.
- The branch allocation recommendation feature provides advisory outputs based on defined rules.
- The system modules support the intended car rental management process.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

B. Performance Efficiency

The following statements were used as part of the IT expert evaluation instrument:

- The system responds within an acceptable time during normal use.
- The system processes database records efficiently.
- The system generates reports, forecasts, and recommendations without unnecessary delay.

- The system uses resources efficiently under expected operating conditions.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

C. Compatibility

The following statements were used as part of the IT expert evaluation instrument:

- The system works properly on supported browsers.
- The system can be accessed using supported desktop and mobile devices.
- The system modules work properly within the same web-based environment.
- The system supports the client's browser-accessible workflow.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

D. Interaction Capability

The following statements were used as part of the IT expert evaluation instrument:

- The system interface is clear and understandable.
- The system provides appropriate prompts, messages, and feedback.
- The user interface supports efficient task completion.
- The system design is appropriate for both customer-side and administrative-side users.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

E. Reliability

The following statements were used as part of the IT expert evaluation instrument:

- The system performs its intended functions consistently.
- The system handles input errors properly.
- The system maintains accurate system records.
- The system supports data backup and recovery procedures.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

F. Security

The following statements were used as part of the IT expert evaluation instrument:

- The system implements secure authentication.
- The system applies role-based access control.
- The system protects sensitive customer, payment, and operational records.
- The system records important user actions through logs or audit records.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

G. Maintainability

The following statements were used as part of the IT expert evaluation instrument:

- The system structure supports future modification.
- The system modules are organized and manageable.
- The system can be corrected or updated without affecting unrelated functions.
- The database structure supports proper record management.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

H. Flexibility

The following statements were used as part of the IT expert evaluation instrument:

- The system can support additional users, vehicles, and booking records.
- The system can support future enhancements.
- The system can accommodate changes in rules, policies, and operational data.
- The system can be adapted to changes in data source availability.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

I. Safety

The following statements were used as part of the IT expert evaluation instrument:

- The system helps reduce risks caused by incorrect vehicle recommendation.
- The system prevents unauthorized users from accessing restricted functions.
- The system supports human-controlled approval for booking, payment, and vehicle assignment.
- The system provides system behavior that supports safe and controlled operations.

These questions are included to determine whether the system meets the expected technical quality requirements under this ISO/IEC 25010 characteristic.

Validity and Reliability of the Instrument

The evaluation instrument will undergo validation before it is administered to the final respondents. Since the study will use an existing ISO/IEC 25010-based evaluation instrument as a guide, the researchers will adapt the items according to the selected quality characteristics, system features, and respondent groups. The adapted questionnaire will be reviewed by qualified validators to determine whether the items

are clear, relevant, aligned with the selected ISO/IEC 25010 characteristics, and appropriate for the intended respondents.

The validators may include an IT expert, a research or statistics validator, and a language or grammar validator. The IT expert may review the technical relevance of the items, the research or statistics validator may review the appropriateness of the instrument and scale, and the language or grammar validator may review the clarity and wording of the questionnaire. Revisions will be made based on their comments and suggestions before the final administration of the instrument.

To establish reliability, the revised questionnaire may be pilot-tested with respondents who are not part of the final evaluation group. The responses from the pilot test may be analyzed using Cronbach's Alpha to determine the internal consistency of the instrument. If the reliability result does not meet the acceptable threshold required by the adviser or panel, the questionnaire items will be reviewed, revised, or removed before final administration.

Ethical Considerations

Ethical considerations will be observed throughout the conduct of the study to ensure responsible, respectful, and lawful handling of data during requirements gathering, system development, testing, and evaluation. Since the proposed system will involve customer information, booking records, uploaded requirements, proof of payment, vehicle records, rental transactions, maintenance records, and operational reports, the proponents will ensure that data privacy and confidentiality are maintained.

Before conducting interviews, surveys, testing, or evaluation, the participants will be informed about the purpose of the study and their role in the research. Participation will be voluntary, and respondents will have the right to refuse or withdraw

from the activity without any penalty. The information gathered from respondents will be used only for academic and system development purposes.

The study will also observe the principles of the Data Privacy Act of 2012, or Republic Act No. 10173. No real client booking or rental records are used in the current development and evaluation dataset. The synthetic records will not contain real customer identities, contact information, identification documents, proof of payment, or other personal client data. If verified client records are obtained for a later authorized evaluation, the proponents will minimize, mask, or anonymize identifiable information and analyze those records separately from the synthetic dataset. Current synthetic results will remain clearly labeled and will not be presented as measured real-world forecasting performance.

The proposed system will also be designed to protect data through secure authentication, role-based access control, and restricted access to sensitive records. Customers will only be allowed to access their own account, booking details, uploaded requirements, and payment proof. Administrators will be allowed to manage operational records based on their assigned responsibilities. Owners will have access to financial, reporting, and decision-support functions.

The study will not include unauthorized tracking of vehicles or individuals. Although the client uses existing tools such as AKSH GPS, Apple Find My, and AirTags, the proposed system will not directly integrate live GPS tracking. The system will only record vehicle status, monitoring notes, rental status, condition records, and maintenance-related updates entered by authorized users. This ensures that the system remains aligned with its declared scope and avoids privacy concerns related to live location monitoring.

The proponents will also maintain academic integrity by properly citing all related literature, studies, systems, frameworks, and standards used in the manuscript. The proponents will not fabricate data, manipulate evaluation results, or claim features that are not included in the developed system.

Data Analysis (Procedure and Treatment)

After the system evaluation, the collected questionnaire responses will be checked, tabulated, and analyzed according to respondent group and ISO/IEC 25010 quality characteristics. The responses of customer or renter respondents will be used to evaluate customer-side features such as account registration, Customer-Side Vehicle Recommendation, vehicle browsing, booking request submission, requirement upload, proof-of-payment submission, booking status tracking, notifications, rental policy viewing, and ease of use.

The responses of owner, administrator, and staff respondents will be used to evaluate administrative and operational features such as booking management, customer verification, payment validation, vehicle records, maintenance monitoring, fleet monitoring, reports, demand forecasting, branch allocation recommendation, and role-based access control. The responses of IT expert evaluators will be used to assess technical quality characteristics such as functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety.

The data gathered from the evaluation questionnaire will be analyzed using frequency count, percentage, weighted mean, ranking, and verbal interpretation. Frequency count and percentage will be used to summarize respondent profiles and response distribution. Weighted means will be used to determine the level of

acceptability of the system based on each ISO/IEC 25010 quality characteristic. Ranking will be used to identify the highest and lowest-rated quality characteristics or system features.

Because no real historical client records are currently available, Mean Absolute Percentage Error will be used only to verify forecast-error computation within the labeled synthetic time series. The calculation will match original Week +1 forecasts with the realized synthetic qualifying-demand values for the same branch, vehicle category, and target week and will report the synthetic coverage period, valid observation count, and excluded zero-demand periods. The resulting MAPE will be described as a functional-test result, not as evidence of empirical predictive accuracy, seasonal performance, or long-term operational effectiveness. A separate real-world accuracy evaluation may only be conducted if sufficient verified client or post-implementation observations later become available.

Statistical Treatments

Table 77

Likert Scale Interpretation Table

Likert-Scale Description	Likert-Scale Point	Weighted Mean Range
Strongly Agree	4	3.26 - 4.00
Agree	3	2.51 - 3.25
Disagree	2	1.76 - 2.50
Strongly Disagree	1	1.00 - 1.75

Table 77 presents the four-point Likert scale interpretation that will be used to determine the level of acceptability of the proposed system based on the computed weighted mean.

Table 78

Statistical Technique Used for Data Analysis

Data to be Analyzed	Statistical Technique
Number of responses per rating	Frequency Count
Distribution of respondent answers	Percentage
Acceptability of each ISO/IEC 25010 criterion	Weighted Mean
Comparison of quality characteristics	Ranking
Synthetic Week +1 forecast-error computation	Mean Absolute Percentage Error
Respondents' comments and suggestions	Descriptive Analysis

Table 78 presents the statistical techniques that will be used to analyze and interpret the data gathered in the study.

Frequency Count

Frequency count will be used to determine the number of respondents who selected each rating in the evaluation questionnaire. This will help identify how many respondents rated each statement as strongly agree, agree, disagree, or strongly disagree.

Formula:

$$f = \text{number of responses per rating}$$

Where:

f = frequency or number of responses

Percentage

Percentage will be used to determine the proportion of responses for each rating in relation to the total number of respondents.

Formula:

$$P = (f / N) \times 100$$

Where:

P = percentage

f = frequency of responses

N = total number of respondents

100 = constant value

Weighted Mean

Weighted mean will be used to determine the average rating for each evaluation indicator and each ISO/IEC 25010 quality characteristic.

Formula:

$$WM = \Sigma fx / N$$

Where:

WM = weighted mean

Σ = summation symbol

f = frequency of responses

x = assigned numerical value or weight of each rating

N = total number of respondents

The computed weighted mean will be interpreted using the four-point Likert scale interpretation table.

Ranking

Ranking will be used to identify which ISO/IEC 25010 quality characteristics received the highest and lowest ratings. The quality characteristic with the highest

weighted mean will be ranked first, while the characteristic with the lowest weighted mean will be ranked last. This will help the proponents determine the strongest and weakest areas of the system based on the evaluation results.

Mean Absolute Percentage Error

Mean Absolute Percentage Error (MAPE) will be used to verify the observed error calculation of the Weighted Moving Average forecasts within the labeled synthetic test dataset. In this evaluation, actual demand means the realized qualifying booking count for a completed week in the synthetic time series and does not represent empirical client demand. Forecast and realized values shall refer to the same branch, vehicle category, and weekly period, and the forecast used shall be the value originally generated when the target period was one week ahead. The evaluation shall follow the snapshot and timing rules in the forecasting methodology and report the evaluated branch-category series, synthetic coverage period, number of valid observations, and excluded zero-demand weeks. MAPE shall include only observations where the realized synthetic demand is greater than zero because percentage error is undefined when the denominator is zero. Zero-demand periods shall be reported separately and excluded from the MAPE divisor. Lower MAPE values indicate smaller errors within the synthetic test series, but the result shall be reported only as evidence of computational behavior and rule conformance. It shall not be interpreted as real-world predictive accuracy, seasonal validity, or long-term operational effectiveness, and no qualitative accuracy classification shall be assigned.

Formula:

$$\text{MAPE} = (1/n) \sum |(Actual Value - Forecasted Value) / Actual Value| \times 100$$

Where:

n = number of observations

Actual Value = actual demand value

Forecasted Value = demand value estimated by the system

References

- A Real-Time Traffic Congestion Prediction System Using Weather and Temporal Data. (2026). Iconic Research and Engineering Journals. <https://doi.org/10.64388/IREV9I9-1715492>
- A. et al. (2023). A Smart Car Rental Management System using Data Mining. International Research Journal of Computer Science. <https://doi.org/10.26562/irjcs.2023.v1005.10>
- Adhyapak, R. (2020). Scheduling and short-term demand forecasting in an on-demand ride service industry. <https://doi.org/10.1504/ijssom.2020.10028881>
- Adishesha & Moula. (2026). Online Car Rental System. International Research Journal of Modernization in Engineering Technology & Science. <https://doi.org/10.56726/irjmets87436>
- Adyaputra, R., Imtihan, K., & Saleh, M. (2025). Assessing user satisfaction of local government websites through ISO 25010 and Technology Acceptance Model (TAM). Journal of Information and Organizational Sciences, 49(2). <https://doi.org/10.31341/jios.49.2.8>
- Agarwal, R., & Shinde, D. (2022). Vehicle allotment and route optimization for transportation services using logistics techniques in supply chain management. Proceedings of the International Conference on Industrial Engineering and Operations Management. <https://doi.org/10.46254/IN02.20220034>
- Akay, B. (2024). Bibliometric analysis of carsharing and car rental research in the field of urban transportation and tourism transportation. Journal of Multidisciplinary Academic Tourism, 9(2), 113–126. <https://doi.org/10.31822/jomat.2024-9-2-113>
- Alencar, V. A., Pessamilio, L. R., Rooke, F., Bernardino, H., & Vieira, A. (2021). Forecasting the carsharing service demand using uni and multivariable models.

Journal of Internet Services and Applications.

<https://doi.org/10.1186/s13174-021-00137-8>

Alrashdi, I., Ali, A. M., Sallam, K. M., & Abdel-Basset, M. (2025). Intelligent decision support framework for assessment of alternative vehicle technologies in transportation system: A sustainable approach toward environmental remedy. *Sustainable Futures*. <https://doi.org/10.1016/j.sftr.2025.100472>

Alzaidi, A., Shakya, S., & Khargharia, H. (2022). Investigating prediction models for vehicle demand in a service industry. *International Joint Conference on Computational Intelligence*. <https://doi.org/10.5220/0011527400003332>

Ariantini, M. S., & Supartha, I. K. D. G. (2022). Implementasi metode Weighted Moving Average (WMA) pada peramalan jumlah kunjungan. <https://doi.org/10.36002/jutik.v8i2.1592>

Bakale, S. C., Naware, A. M., Parab, S. V., Saxena, S. A., & Anjaria, S. A. (2023). MERN stack-based car rental website development. *International Journal of Advanced Research in Computer Science*, 14(2). <https://doi.org/10.26483/ijarcs.v14i2.6971>

Bhale & Singh. (2025). Comparative Analysis of Regression Techniques for Weather-Based Route Optimization. *2025 World Skills Conference on Universal Data Analytics and Sciences (WorldSUAS)*. <https://doi.org/10.1109/WorldSUAS66815.2025.11198976>

Bondoc, B. C. (2023). Web-based smart attendance monitoring system using QR code and GSM module for SMS notification. *World Journal of Advanced Research and Reviews*, 20(3). <https://doi.org/10.30574/wjarr.2023.20.3.2644>

- Boysen, N., Briskorn, D., & Emde, S. (2024). Matching vs. individual choice: How to counter regional imbalance of carsharing demand. *Transportation Science*. <https://doi.org/10.1287/trsc.2022.0067>
- Brandizzi, N., Russo, S., Galati, G., & Napoli, C. (2022). Addressing vehicle sharing through behavioral analysis: A solution to user clustering using recency-frequency-monetary and vehicle relocation based on neighborhood splits. *Information*, 13(11), 511. <https://doi.org/10.3390/info13110511>
- Brar, A. S., Kasture, P., & Su, R. (2022). Supply-demand balancing model for EV rental fleet. 2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC). <https://doi.org/10.1109/ITSC55140.2022.9921775>
- Cardos, N. C., Empal, Z. K., Gabriel, C. E., & Prudente, R. S. (2025). Evaluating the impact of role-based access control and data privacy measures on user satisfaction and security compliance in the SEAIT OJT evaluation and feedback system. *International Journal of Innovative Science and Research Technology*. <https://doi.org/10.38124/ijisrt/25may565>
- Chavhan, S., Gupta, D., Nagaraju, C., A., R., Khanna, A., & Rodrigues, J. (2022). An efficient context-aware vehicle incidents route service management for intelligent transportation systems. <https://doi.org/10.1109/JSYST.2021.3066776>
- Dedhia, N. (2026). Secure web authentication using JWT and role-based access control. *International Journal of Scientific Research in Engineering and Management*. <https://doi.org/10.55041/ijrem60274>
- Dhami, A. S., Rout, H., Talom, Y., & Sakshi. (2024). Online vehicle rental management system. *International Research Journal of Modernization in Engineering Technology and Science*, 6(4), 6692–6698. <https://doi.org/10.56726/IRJMET53564>

- Dubey, S., Esakkiammal, U., Ugli, M. N. N., et al. (2025). On-the-fly path replanning using real-time road condition data. 2025 International Conference on Intelligent Systems and Pioneering Innovations in Robotics and Electric Mobility (INSPIRE). <https://doi.org/10.1109/INSPIRE67328.2025.11300571>
- Eliyan, A. F., & Kerbach, L. (2024). Vehicle relocation in one-way carsharing: A review. *Sustainability*, 16(3), 1014. <https://doi.org/10.3390/su16031014>
- Fajar, M., Atmaja, A. P., Hafidhoh, N., Ismar, M. R., & Ivansyah, M. N. (2026). Implementation of a rule-based expert system in the web-based SMK boarding and semi boarding admission system. *Andalasian International Journal of Applied Science Engineering and Technology*.
- Fauzi, N. A., Sampurno, Y. G., Sudarwanto, & Wahyudi, K. S. (2025). Development of a centralized vehicle maintenance information system. <https://doi.org/10.21831/jvars.v2i2.2527>
- Favier, J.-C., Ganguly, S., & Shalpegin, T. (2024). Optimal fleet policy of rental vehicles with relocation: A simulation study. *Journal of Advanced Transportation*, 2024, Article 7764326. <https://doi.org/10.1155/2024/7764326>
- Fernández, C. M., Freitas, A. A., Morais, A. N., Lima, T., & Gaspar, P. (2020). Fleet management optimization in car rental industry: Decision aid models for management of information systems. <https://doi.org/10.1109/DASA51403.2020.9317221>
- Gabbar, H. A., Chahid, A., Isham, M. U., et al. (2023). HAIS: Highways automated-inspection system. *Technologies*. https://www.researchgate.net/publication/369771890_HAIS_Highways_Automated-Inspection_System

- Ganeshkar, V., Moze, A., Surwade, S., & Sonawane, S. (2021). An online car rental management system. *International Journal of Innovative Research in Science, Engineering and Technology*, 10(6), 7392–7396.
https://www.ijirset.com/upload/2021/june/248_AN_NC.pdf
- Gurusinghe, B. M. I. (2021). Automated fleet management system for CIT operations.
<https://www.semanticscholar.org/paper/f4875181581628673f0952a1d76cc3e84f653f95>)
- Haber et al. (2026). Rule-Based Expert System for Resource Planning in Liquid Transportation. *Sustainability*. <https://doi.org/10.3390/su18063156>
- Harissa, F. V. (2025). Design and prototype implementation of a web-based car rental information system. <https://doi.org/10.35314/wfaayr38>
- Heidari et al. (2024). A decision support system for resilient vehicle route planning using mathematical modeling and artificial neural networks: a case study. *Kybernetes*. <https://doi.org/10.1108/K-10-2024-2935>
- Huang, K., An, K., Correia, G., Rich, J., & Ma, W. (2021). An innovative approach to solve the carsharing demand-supply imbalance problem under demand uncertainty. *Transportation Research Part C: Emerging Technologies*.
<https://doi.org/10.1016/j.trc.2021.103369>
- Hussein et al. (2021). Incorporating weather variables with probabilistic approach for trip planning. *Case Studies on Transport Policy*.
<https://doi.org/10.1016/j.cstp.2021.08.005>
- K. S., S. (2025). Online car rental management system.
<https://doi.org/10.55041/ijrsrem42114>

- Khan, A. (2020). Car rental management system. <https://doi.org/10.22214/ijraset.2020.6185>
- Lopez, S. K. R., Tipon, J. J. D., Visbal, J., & Castro, A. (2025). IoT-based fleet monitoring system with driver logging and OBD-II diagnostics. <https://doi.org/10.1145/3787330.3787339>
- Maghfur, M. A., & Tranggono, T. (2025). Analysis of forecasting demand for wheel loader unit rental using the ARIMA method. <https://doi.org/10.38035/rrj.v7i3.1368>
- Moein, E., & Awasthi, A. (2020). Carsharing customer demand forecasting using causal, time series and neural networks methods. <https://doi.org/10.1504/ijssom.2020.104333>
- Naim, S. W., Nangraj, S. S., Khan, D., Hassan, S. M., & N. S. (2025). Reengineering logistics and fleet management: Integration of GPS, AI, and predictive analytics. <https://doi.org/10.62019/qxdzeq92>
- Pawestri, C. R. R., & Putro, O. S. (2024). Sistem aplikasi rental mobil berbasis web untuk optimalisasi armada. <https://doi.org/10.46808/informa.v9i2.257>
- Şimşek, A. İ., Gür, Y. E., & Ünal, E. (2025). Innovative MCDM-ML algorithms-based decision-support system for electric vehicle selection. *Environment, Development and Sustainability*. <https://doi.org/10.1007/s10668-025-06476-x>
- Islam, M., Imran, R., & Hosain, S. (2021). The evaluation of enterprise resource planning using ISO 25010 based quality model. 2021 2nd International Informatics and Software Engineering Conference (IISEC). <https://doi.org/10.1109/iisec54230.2021.9672349>

- Ismail, H., Andayani, D. D., & Kaswar, A. (2025). Development of a web-based information system for food availability and production at the Makassar City Food Security Agency. *Journal of Embedded Systems, Security and Intelligent Systems*, 6(1). <https://doi.org/10.59562/jessi.v6i1.6824>
- Joshna et al. (2026). Real-Time Intelligent Travel Planning and Cost Management Using API-Driven Architecture. *International Journal of Science and Research Archive*. <https://doi.org/10.30574/ijsra.2026.18.2.0304>
- Jyothi, P. C., & Sheethal, P. P. (2025). Car rental management system. *International Journal of Sciences and Innovation Engineering*, 2(12), 265–271. <https://ijsci.com/index.php/home/article/view/1056>
- Kanagamalliga, S., Nath, S. S., Ganapathy, K., & Salaudeen, A. (2026). A robust access control and logging mechanism for sensitive information systems. 2026 International Conference on Visual Analytics and Data Visualization (ICVADV). <https://doi.org/10.1109/ICVADV67766.2026.11470325>
- Karche, P., Kapare, K., Chapale, N., & Shirsath, P. (2025). Car rental booking system using MERN stack. *International Research Journal of Modernization in Engineering Technology and Science*, 7(10), 3470–3472. https://www.irjmets.com/upload_newfiles/irjmets71000102163/paper_file/irjmets71000102163.pdf
- Khalid. (2024). Short-term and long-term product demand forecasting with time series models. *Journal of Trends in Financial and Economics*. <https://doi.org/10.61784/jtfe3022>
- Ko, J., & Byun, Y. (2023). Analyzing factors affecting micro-mobility and predicting micro-mobility demand using ensemble voting regressor. *Electronics*, 12(21), 4410. <https://doi.org/10.3390/electronics12214410>

- Kotsialos, A., & Vassilakopoulou, P. (2023). Fleet management enterprise systems and traffic control synergies: A literature review and research agenda. *Procedia Computer Science*, 219, 529–536. <https://doi.org/10.1016/j.procs.2023.01.321>
- Kumar, R., & Singh, S. K. (2024). Online car rental system. *International Journal for Technological Research in Engineering*, 11(11), 33–42. <https://ijtre.com/wp-content/uploads/2024/07/2024111105.pdf>
- Labib, A., Tănăsuică (Zotic), C., Seecharan, T. S., & Roman, M.-D. (2026). Data-driven fleet optimization using ML algorithms and a decision-making grid framework. *Applied System Innovation*, 9(3), 63. <https://doi.org/10.3390/asi9030063>
- Lamouchi, N. (2021). Introduction to the monolithic architecture. *Pro Java Microservices with Quarkus and Kubernetes*. https://doi.org/10.1007/978-1-4842-7170-4_2
- Laudza, N. A. S., & Sofyan, H. (2024). Evaluasi kualitas sistem informasi akademik dengan standar ISO/IEC 25010 (Studi kasus: Universitas ABC). *Telematika*, 21(2). <https://doi.org/10.31315/telematika.v21i2.12406>
- Alhwety, R. N. H., & Elfadil, N. (2024). Vehicle tracking system approaches: A systematic literature review. *International Journal of Computer Science and Mobile Computing*, 13(8), 23–31. <https://doi.org/10.47760/ijcsmc.2024.v13i08.003>
- Yu, E. Y. M., Hlaing, N. W., & Aung, P. P. (2026). Vehicle location and status tracking system using GPS, GSM/GPRS, and OBD technologies. In V. Bhateja, A. L. S. Hoong, J. D. Kong, & S. Urooj (Eds.), *Smart computing paradigms: Intelligence and network applications: Proceedings of Seventh International Conference on*

Smart Computing and Informatics (SCI 2025), Volume 1 (pp. 213–225). Springer. https://doi.org/10.1007/978-3-032-08240-4_19

Ariningsih, P., & Muhammad, A. H. (2024). Quality evaluation of Ticketing Management System using ISO/IEC25010:2023 standards and AHP method. *Intechno Journal: Information Technology Journal*, 6(2). <https://doi.org/10.24076/intechnojournal.2024v6i2.1870>

International Organization for Standardization, & International Electrotechnical Commission. (2023). ISO/IEC 25010:2023: Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model. International Organization for Standardization. <https://www.iso.org/standard/78176.html>

Nugraha, U., & Rachman, M. A. N. (2024). Evaluation of Ixitask Application Quality Based on ISO/IEC 25010:2023 in Marketing Division. In *Proceedings of the Widyatama International Conference on Engineering 2024 (WICOENG 2024)* (pp. 139–147). Atlantis Press. https://doi.org/10.2991/978-94-6463-618-5_15

Kankaraš, M., & Capecchi, S. (2025). Neither agree nor disagree: Use and misuse of the neutral response category in Likert-type scales. *METRON*, 83, 111–140. <https://doi.org/10.1007/s40300-024-00276-5>

Koo, M., & Yang, S.-W. (2025). Likert-type scale. *Encyclopedia*, 5(1), Article 18. <https://doi.org/10.3390/encyclopedia5010018>

Mariano, L. T., Phillips, A., Estes, K., & Kilburn, M. R. (2024). Should survey Likert scales include neutral response categories? Evidence from a randomized

- school climate survey (Working Paper No. WR-A3135-2). RAND Corporation.
<https://doi.org/10.7249/WRA3135-2>
- Linghuraj, P. R., Aravind, K., & Giritharan, R. (2024). Car rental management system. *International Journal of Novel Research and Development*, 9(3), 200–215.
<https://www.ijnrd.org/papers/IJNRD2403623.pdf>
- Liu, D., Lu, J., & Ma, W. (2021). Real-time return demand prediction based on multisource data of one-way carsharing systems. *Journal of Advanced Transportation*, 2021, 1–14. <https://doi.org/10.1155/2021/6654909>
- Liu, H., Tong, Y., Han, J., et al. (2022). Incorporating multi-source urban data for personalized and context-aware multi-modal transportation recommendation. *IEEE Transactions on Knowledge and Data Engineering*.
<https://doi.org/10.1109/TKDE.2020.2985954>
- Liu, L., & Wang, T. (2025). Optimizing urban last mile delivery efficiency through dynamic vehicle routing heuristics and traffic flow analysis. *Scientific Reports*.
<https://doi.org/10.1038/s41598-025-29861-y>
- Lope et al. (2023). Fast and Efficient Very Short-Term Load Forecasting Using Analogue and Moving Average Tools. *IEEE Latin America Transactions*.
<https://doi.org/10.1109/TLA.2023.10251808>
- Lu, Y., Yang, M., Guo, K., Zhang, N., Lu, X., & Li, B. (2024). Fusion of differential privacy algorithm and advanced AI: Innovating customer information protection in transaction data. *International Conference on Computer Science and Artificial Intelligence*. <https://doi.org/10.1145/3709026.3709051>
- Manoj, Rajya, L. M., Venkata, S. B. K., & Karthikeya, K. (2025). Access control mechanisms for securing data during transmissions. *i-manager's Journal on Software Engineering*, 19(4). <https://doi.org/10.26634/jse.19.4.21781>

- Marafo, M. P., Antonio, J. S., Palay-en, B. A., Toribio, C. M., & Guaki, G. S. (2025). Arkila: A progressive web app car rental management system for Kataguan Rides in La Trinidad, Benguet. *Southeast Asian Journal of Science and Technology*, 10(1).
- Martin, G., Donain, M., Fromont, É., Guns, T., Rozé, L., & Termier, A. (2021). Prediction-based fleet relocation for free floating car sharing services. *IEEE International Conference on Tools with Artificial Intelligence*. <https://doi.org/10.1109/ICTAI52525.2021.00187>
- Mavrin & Makarova. (2026). Developing a Decision Support System to Improve the Waste Transportation Process. *Logistics*. <https://doi.org/10.3390/logistics10040078>
- Meira et al. (2022). Route Optimization For A Beer Deliver Decision Support System. *Proceedings of the IV Workshop on Disruptive Information and Communication Technologies for Innovation and Digital Transformation: 18th June 2021 Online*. <https://doi.org/10.14201/0AQ00AQ020213125969>
- Mendoz, L. Y., Geraldo, M. K., Francisco, H. D., Domingo, M. L., Regencia, J. A., & Kagakit, M. (2025). WOMB: A web-based system for maternal support and infant health tracking with integrated data analytics and smart algorithms. *International Journal of Latest Technology in Engineering Management & Applied Science*. <https://doi.org/10.51583/ijltemas.2025.1410000025>
- Mhatre, S., Patil, S., & Mhatre, M. (2026). Vehicle rental system. *International Journal of Advanced Research in Science, Communication and Technology*, 6(6), 637–640. <https://ijarsct.co.in/Paper32086.pdf>

- Mizik, T. (2021). Economic aspects and sustainability of ethanol production—A systematic literature review. *Energies*, 14(19), 6137. <https://doi.org/10.3390/en14196137>
- Moawad, A., Li, Z., Pancorbo, I., et al. (2024). A real-time energy and cost efficient vehicle route assignment neural recommender system. *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswa.2024.125634>
- Mohamad et al. (2026). Predictive Analytics for Vehicle Utilization and Fleet Decision Support in Higher Education. 2026 International Conference on Cognitive Systems and Computer Interaction (ICoSCI). <https://doi.org/10.1109/ICoSCI66700.2026.11448071>
- Mohamad, S., Isa, N., & Mahmud, S. (2026). Predictive analytics for vehicle utilization and fleet decision support in higher education. 2026 International Conference on Cognitive Systems and Computer Interaction (ICoSCI). <https://doi.org/10.1109/ICoSCI66700.2026.11448071>
- Monteverde, A. L., Maderazo, J. J. S., Cruz, K. C. M., & Magnaye, N. A. (2023). Web-based rental house smart finder using rapid application development basis for evaluation of ISO 25010. *International Journal of Metaverse*, 1(1). <https://doi.org/10.54536/ijm.v1i1.1464>
- Mozafari, H., Soltanpour, A., Jazlan, F., Vosoughinia, S., Ghamami, M., & Zockaie, A. (2025). Leveraging alternative fuel vehicles in operation and asset management strategies to reduce fleet economic and societal impacts. *Journal of Transportation Technologies*. <https://doi.org/10.4236/jtts.2025.153016>
- Nasser & Simon. (2025). Traffic and Weather Data Fusion for Traffic Prediction in Sustainable Cities. *Journal of Advanced Transportation*. <https://doi.org/10.1155/atr/1580010>

- Niroshan et al. (2025). Design and implementation of a data sharing API for supporting urban governance schemes in environmental and traffic monitoring. *MethodsX*. <https://doi.org/10.1016/j.mex.2025.103458>
- Nuzula, M. I. F., & Rochimah, S. (2023). Evaluation of service quality in human resource information systems using the ISO/IEC 25010. 2023 International Seminar on Application for Technology of Information and Communication (iSemantic). <https://doi.org/10.1109/iSemantic59612.2023.10295365>
- Ogbiti, J. T., & Aaron, W. (2024). Development of a web-based car rental management system. *Science World Journal*, 19(3), 797–807. <https://doi.org/10.4314/swj.v19i3.27>
- Paramitha & Z.S. (2023). FORECASTING ANALYSIS OF CAR WHEEL RIM DEMAND AT PT. XX TO REDUCE WASTE USING POM-QM SOFTWARE. *Barometer*. <https://doi.org/10.35261/barometer.v8i2.7702>
- Park, C. R., Heo, H., Suh, C. H., & Shim, W. H. (2025). Uncover this tech term: Application programming interface for large language models. *Korean Journal of Radiology*. <https://doi.org/10.3348/kjr.2025.0360>
- Peng, H., Zhou, J., Li, Z., et al. (2025). Aviation refueling strategies based on neural networks and optimization algorithms. *International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery*. <https://doi.org/10.1109/ICNC-FSKD67701.2025.11198108>
- Penumala & Naganaboina. (2025). Contextually Aware Navigation Systems using Environmental and Interest-Based Data. *IEEE India Conference*. <https://doi.org/10.1109/INDICON68490.2025.11392971>
- Pradeeppraja et al. (2025). WeatherSync-ITS: An Adaptive Meteorological Intelligence Framework for Proactive Urban Traffic Management. 2025 5th International

- Conference on Evolutionary Computing and Mobile Sustainable Networks (ICECMSN). <https://doi.org/10.1109/ICECMSN68058.2025.11383260>
- Purba, D. A., & Hidayasari, N. (2026). Implementation of role-based access control (RBAC) in a drug stock management information system. *SISTEMASI*, 15(2). <https://doi.org/10.32520/stmsi.v15i2.5958>
- R et al. (2025). Intelligent Traffic Monitoring and Autonomous Navigation System with Real-Time Weather and Incident Alerts. 2025 International Conference on Data Science and Business Systems (ICDSBS). <https://doi.org/10.1109/ICDSmoBS63635.2025.11031650>
- Rahayu et al. (2022). ANALYSIS OF PRODUCT STOCK INVENTORY FORECASTING USING WEIGHTED MOVING AVERAGE METHOD. *Jurnal Teknik Informatika (Jutif)*. <https://doi.org/10.20884/1.jutif.2022.3.6.421>
- Rahman, W. F. W. A., & Mohd Rusli, N. I. N. B. (2023). The development of a web-based car rental business management system. *Journal of Mathematics and Computing Science*, 9(2), 101–108. <https://doi.org/10.24191/jmcs.v9i2.513>
- Rangasamy, T. (2026). Contextual computing and AI integration: Adaptive decision systems for enterprise environments. *International Journal of Computational and Experimental Science and Engineering*.
- Reddy, Y. R., & Sharanya. (2025). Analysis of COVID-2019 by using weighted moving averages. *Advanced International Journal for Research*. <https://doi.org/10.63363/aijfr.2025.v06i04.1195>
- Romadhan, H., & Mansur, M. (2026). Web-based educational payment information system using role-based access control security. *SISTEMASI*, 15(2). <https://doi.org/10.32520/stmsi.v15i2.5920>

- Routaray & Kumar. (2024). Auto Mobilize Pro. International Journal of Advanced Research in Science, Communication and Technology. <https://doi.org/10.48175/ijarsct-22510>
- Saderova, J., Cerevkova, K., Petrovska, I., & Kleinova, L. (2023). Evaluation of the use of transport means based on time and performance indicators. CLC Conference Proceedings.
- Sangaraju, H. R., Balakrishnan, S., Subramanian, S. S., Chakrabarti, T., Chakrabarti, P., & Margala, M. (2024). An effective resource allocation and revenue generation for rental vehicles. International Journal of Transport Development and Integration, 8(3), 393–401. <https://doi.org/10.18280/ijtdi.080303>
- Sariati et al. (2026). Implementation of Simple and Weighted Moving Average for Forecasting Tela-Tela Production in MSME X. RIGGS: Journal of Artificial Intelligence and Digital Business. <https://doi.org/10.31004/riggs.v4i4.5205>
- Saritha, P., Srinivasu, N., Subhani, S. M., Reddy, P. S. K., & Rambabu, V. (2026). Distributed car rental reservation, fleet allocation and customer billing management. International Journal of Advanced Research in Computer and Communication Engineering. <https://ijarcce.com/papers/distributed-car-rental-reservation-fleet-allocation-and-customer-billing-managem/>
- Schroer, K., Ketter, W., Lee, T., et al. (2021). Data-driven competitor-aware positioning in on-demand vehicle rental networks. Transportation Science. <https://doi.org/10.2139/ssrn.3915466>
- Suganya, N. C., Csbs, B. T., Suganthan, M., Vishnu, T., Csbs, K. B. T., & Ragul, R. (2026). An integrated AI-driven forecasting and optimization framework for intelligent vehicle rental management using RentOptima. International

Conferences on Information Science and System.

<https://doi.org/10.1109/ICISS67859.2026.11453753>

Suganya, N. C., Suganthan, M., et al. (2026). An integrated AI-driven forecasting and optimization framework for intelligent vehicle rental management using RentOptima. International Conference on Information Science and System. <https://doi.org/10.1109/ICISS67859.2026.11453753>

Tobin et al. (2024). Towards the development of an impact-based decision support tool for surface-transportation hazards. Part I: Tying Weather Variables to Road Hazards and Quantifying Impacts. Weather and Forecasting. <https://doi.org/10.1175/waf-d-23-0219.1>

Vateekul, P., Sri-iesaranusorn, P., Aiemvaravutigul, P., Chanakitkarnchok, A., & Rojviboonchai, K. (2021). Recurrent neural-based vehicle demand forecasting and relocation optimization for car-sharing system: A real use case in Thailand. Journal of Advanced Transportation, 2021, 1–16. <https://doi.org/10.1155/2021/8885671>

Vdovic et al. (2021). An Application Programming Interface for Advanced Analytics of Contextually Enriched Automotive Data. International Conference on Telecommunications. <https://doi.org/10.23919/ConTEL52528.2021.9495964>

Vyas, J., Das, D., & Chaudhury, S. (2022). DriveBFR: Driver behavior and fuel-efficiency-based recommendation system. IEEE Transactions on Computational Social Systems. <https://doi.org/10.1109/TCSS.2021.3112076>

You, P.-S., & Hsieh, Y.-C. (2025). Dynamic allocation strategy for the car rental industry with multiple rental channels. International Journal of Engineering and Technology Innovation, 15(1), 68–84. <https://doi.org/10.46604/ijeti.2024.13827>

- Yuliani, R., Handayani, T., & Desyanti, D. (2022). Sistem forecasting untuk pengadaan material biji plastik menggunakan metode Weighted Moving Average. <https://doi.org/10.52072/jutekinf.v10i1.359>
- Yusuf, A., Afriansa, M., Annisa, A., et al. (2026). Penerapan teknologi web untuk smart transportation dan manajemen armada di era IoT: Systematic literature review 2020–2025. *Jurnal Teknologi dan Bisnis Cerdas*. <https://doi.org/10.64476/jtbc.v2i1.22>
- Zhao, F., Wang, W., Sun, H., Yang, H., & Wu, J. (2021). Station-level short-term demand forecast of carsharing system via station-embedding-based hybrid neural network. *Transportmetrica B: Transport Dynamics*. <https://doi.org/10.1080/21680566.2021.1951>

Database Schema

The database schema represents 40 public tables supporting application profiles, branches, vehicles, bookings, requirement and payment verification, rental release and return inspection, maintenance, forecasting, supply evaluation, allocation recommendations, notifications, audit events, backup metadata, and public contact records. Supabase Auth is shown through its external auth.users reference. The overview and continued detail views show table attributes, keys, and relationships; database checks, functions, triggers, and access policies are documented in the implementation rather than reproduced in the diagram.

Figure 59. Database Schema Overview

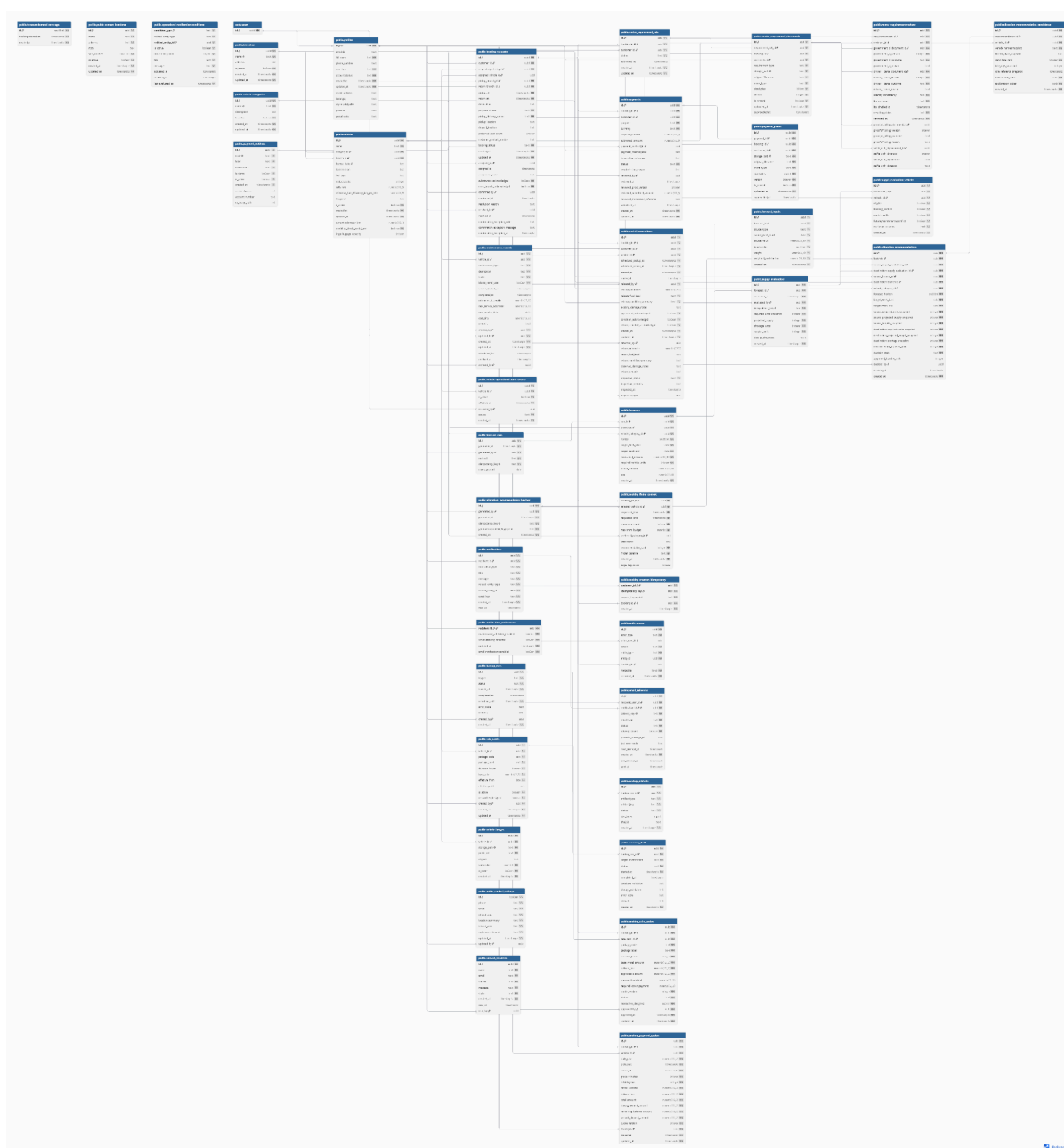


Figure 60. Database Schema Detail 1: Demand Coverage, Public Contact Locations, and Notification Conditions

public.forecast_demand_coverage	public.public_contact_locations	public.operational_notification_conditions
id ⓘ smallint NN	id ⓘ uuid NN	condition_type ⓘ text NN
tracking_started_at timestampz NN	name text NN	related_entity_type text NN
created_at timestampz NN	address text NN	related_entity_id ⓘ uuid NN
	note text	is_active boolean NN
	sort_order ⓘ smallint NN	occurrence_count bigint NN
	is_active boolean NN	title text NN
	created_at timestampz NN	message text NN
	updated_at timestampz NN	activated_at timestampz
		resolved_at timestampz
		last_evaluated_at timestampz NN

Figure 61. Database Schema Detail 2: Authentication, Profiles, Branches, Vehicle Categories, Vehicles, and Payment Methods



Figure 62. Database Schema Detail 3: Bookings, Requirement Sets, Payments, Rentals, and Maintenance

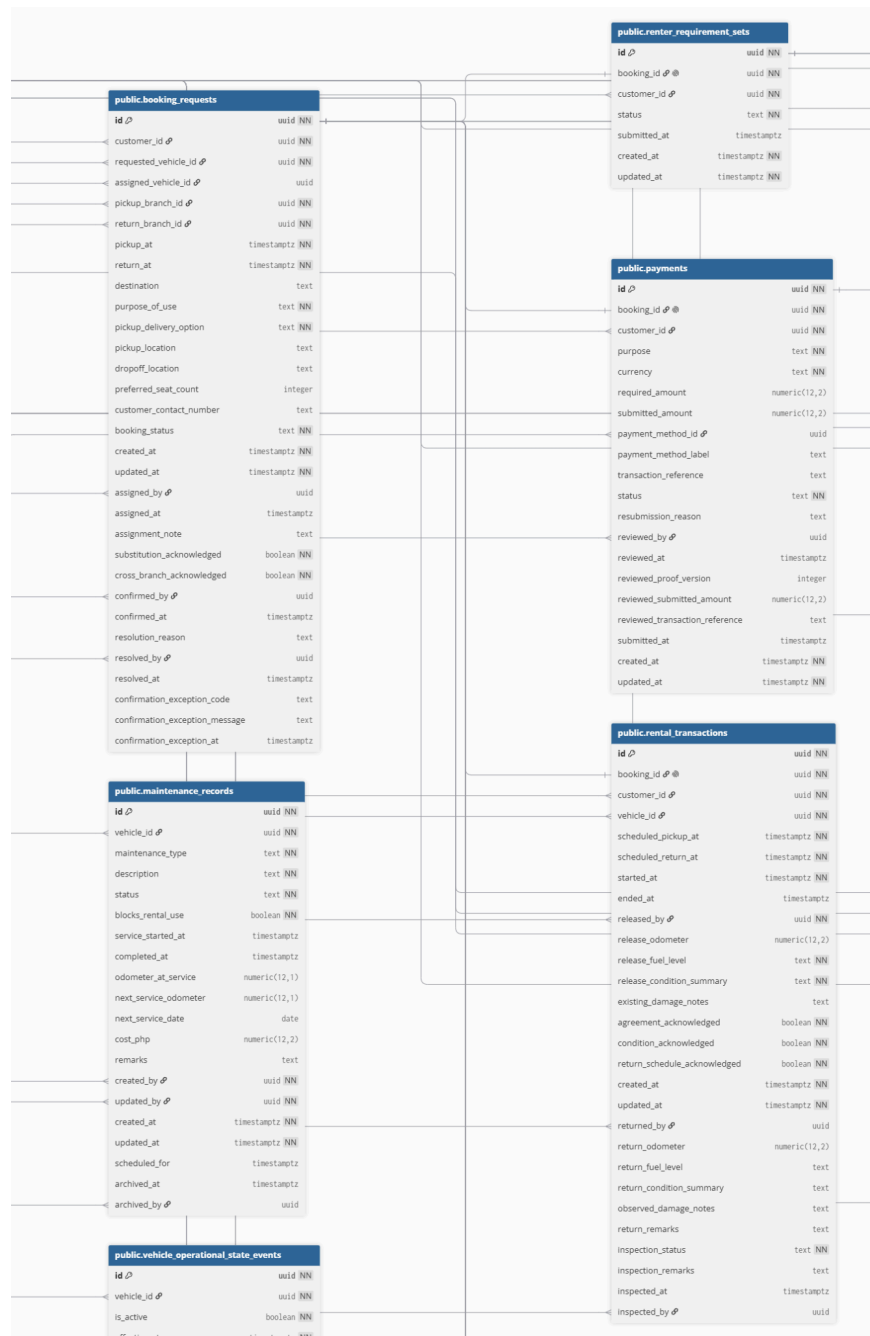


Figure 63. Database Schema Detail 4: Operational State Events, Forecast Runs, Forecasts, Finder Context, and Booking Idempotency (continued)

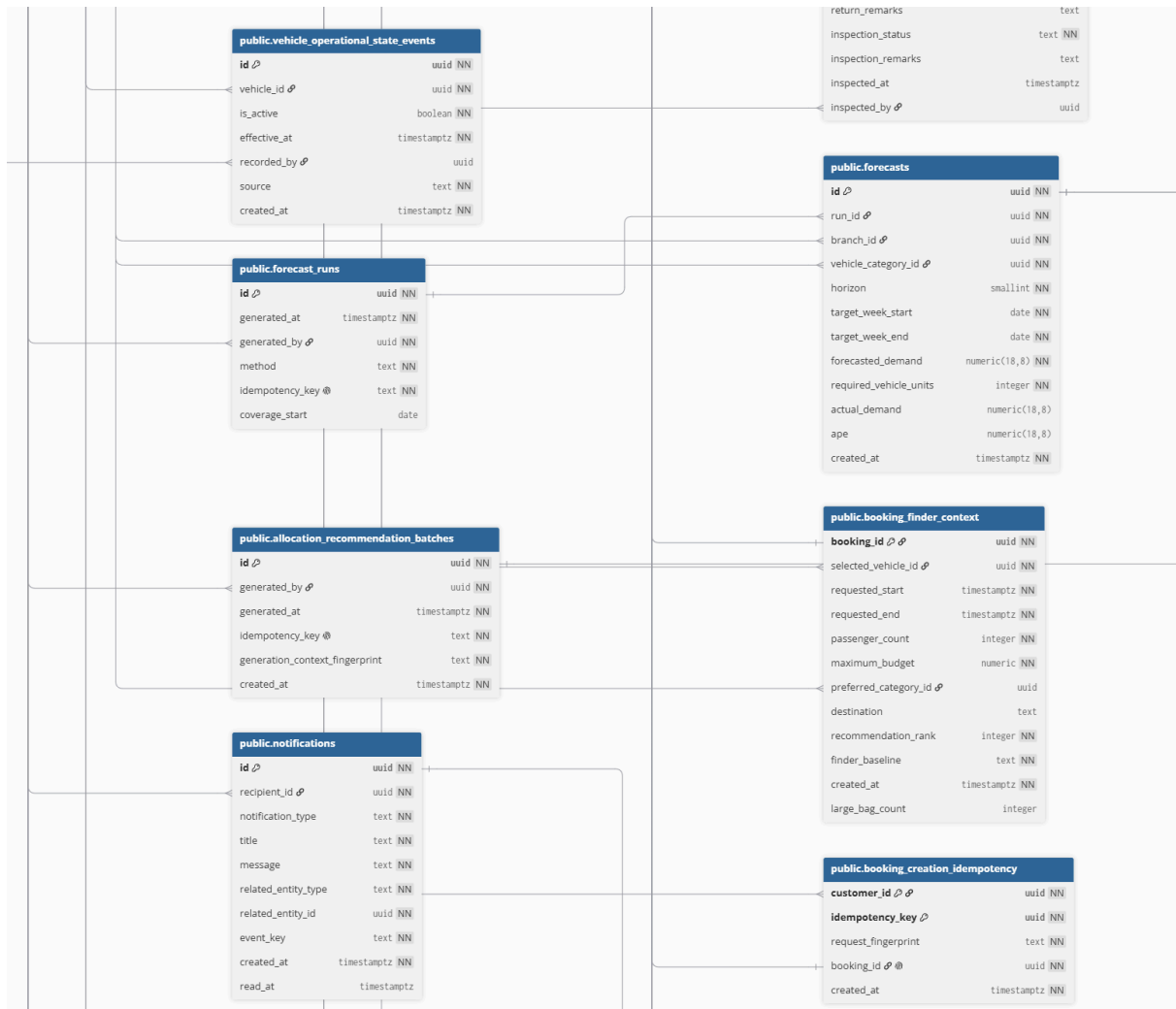


Figure 64. Database Schema Detail 5: Notification Preferences, Audit Events, Email Deliveries, Backup Runs, and Rate Cards (continued)

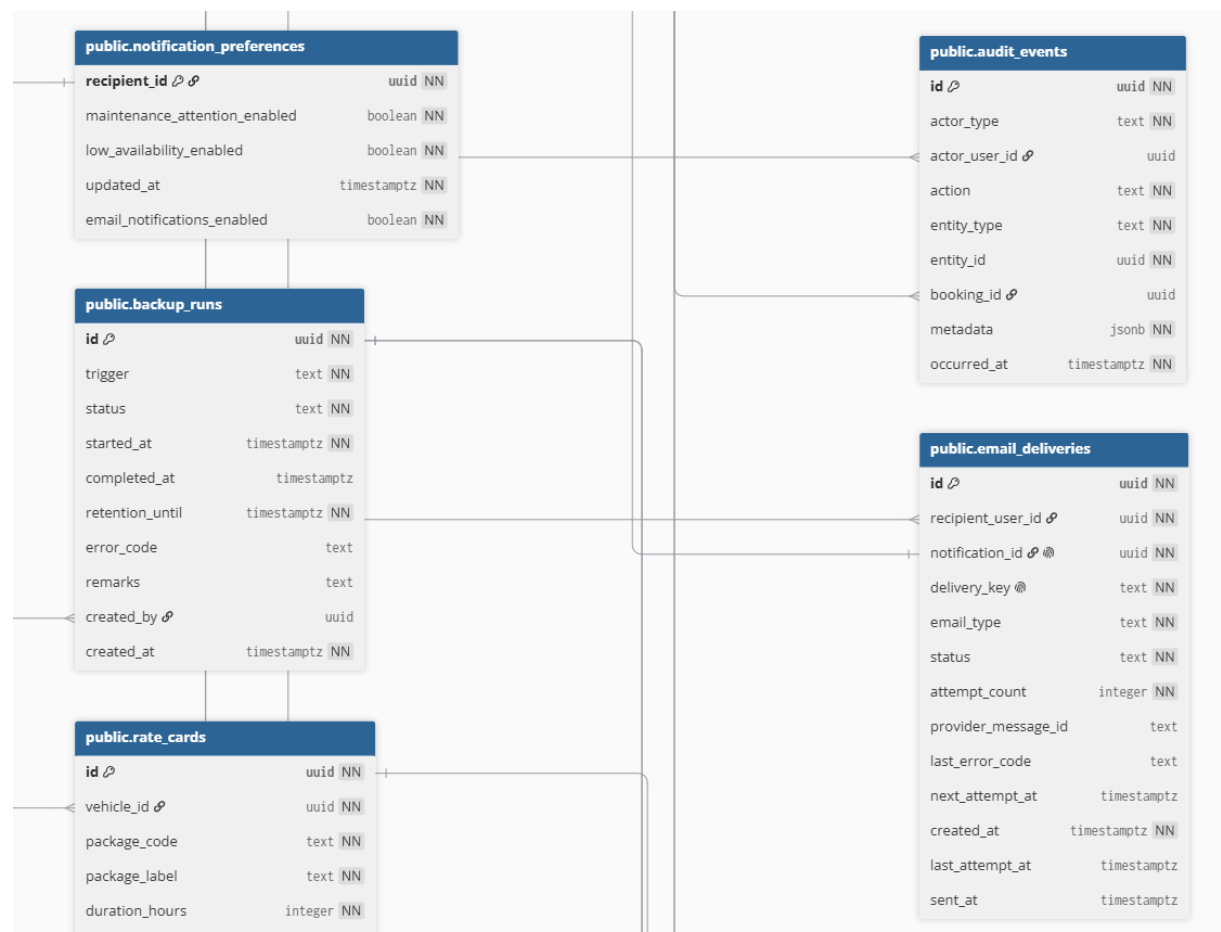


Figure 65. Database Schema Detail 6: Rate Cards, Vehicle Images, Backup Artifacts, Recovery Drills, and Public Contact Settings (continued)

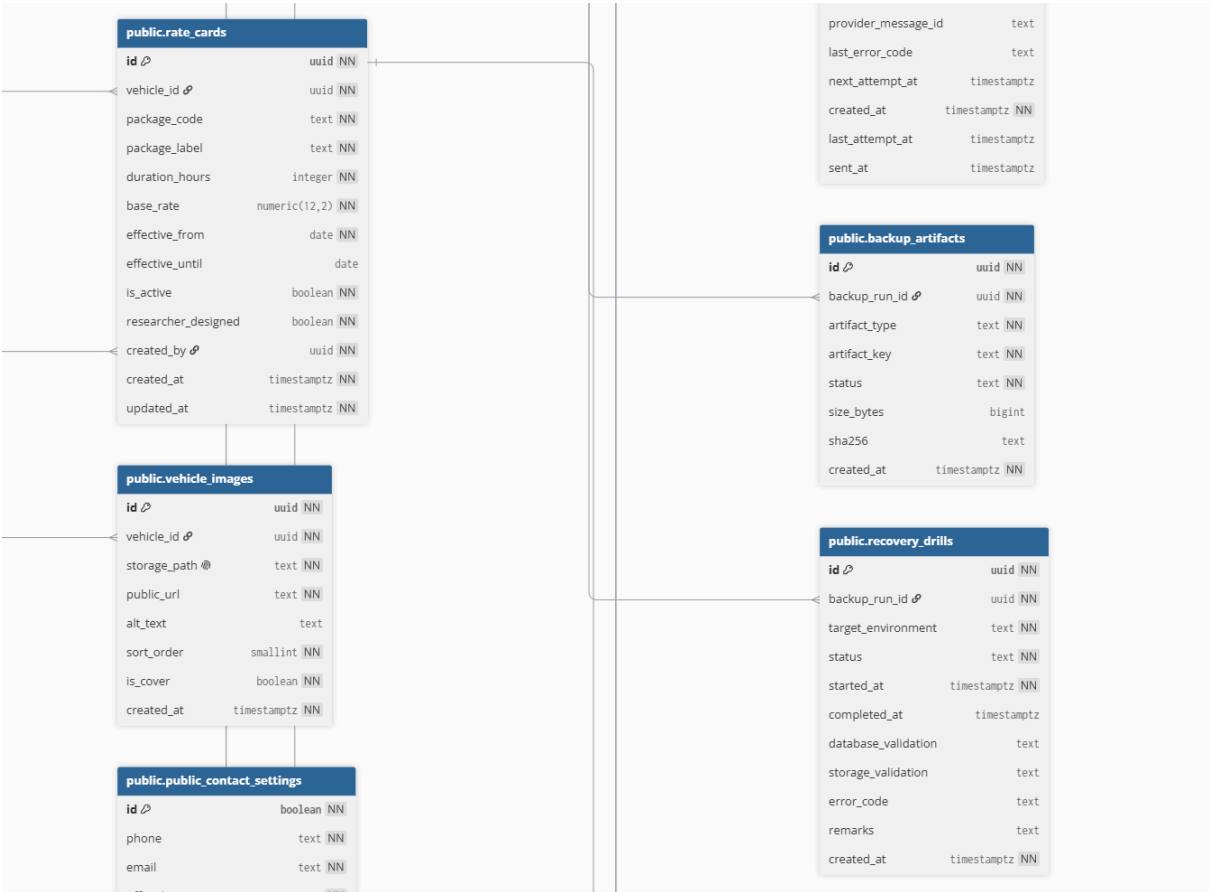


Figure 66. Database Schema Detail 7: Public Contact Settings, Contact Inquiries, and Quotation Records (continued)

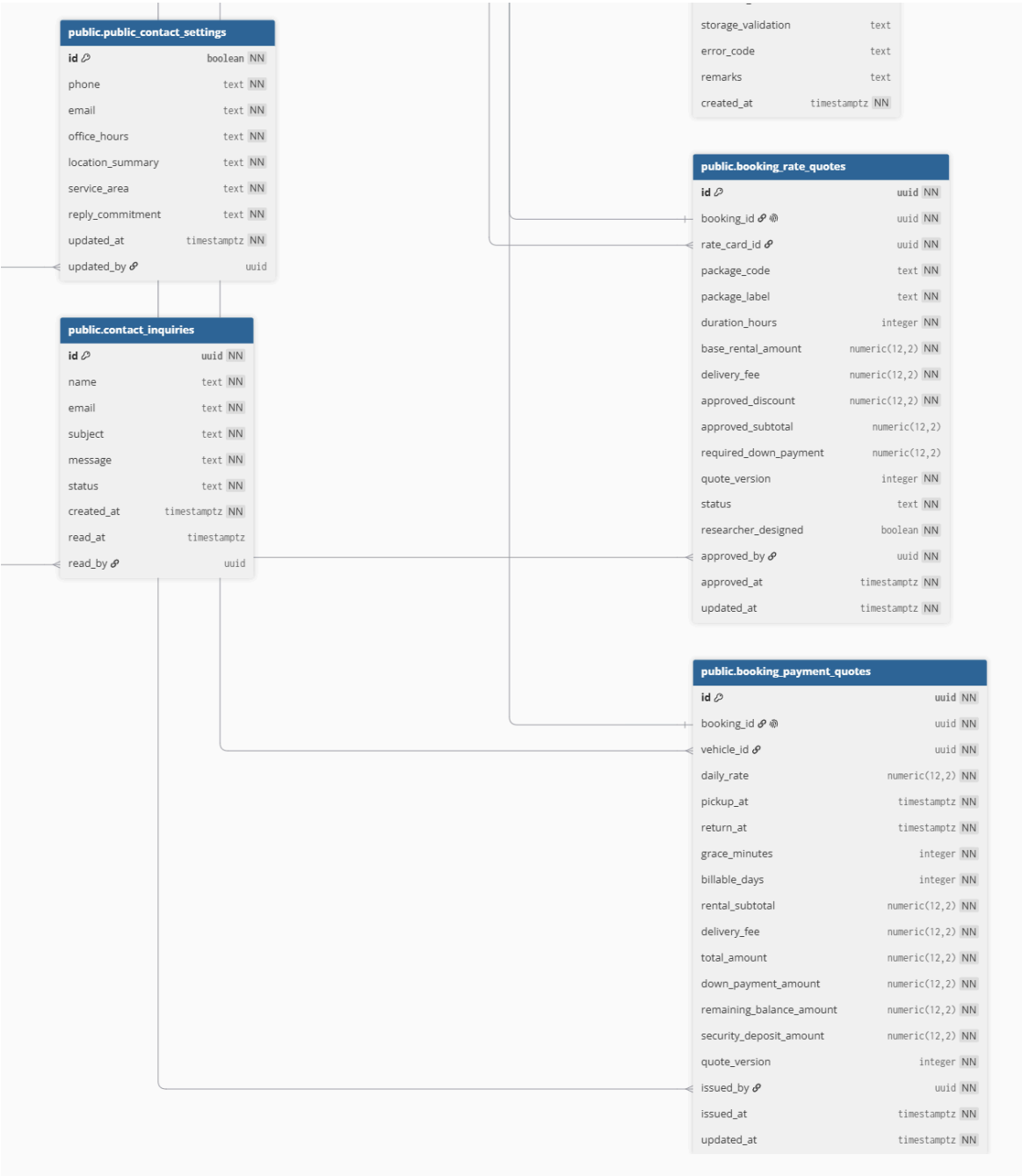


Figure 67. Database Schema Detail 8: Requirement Documents, Payment Proofs, Forecast Inputs, and Supply Evaluations

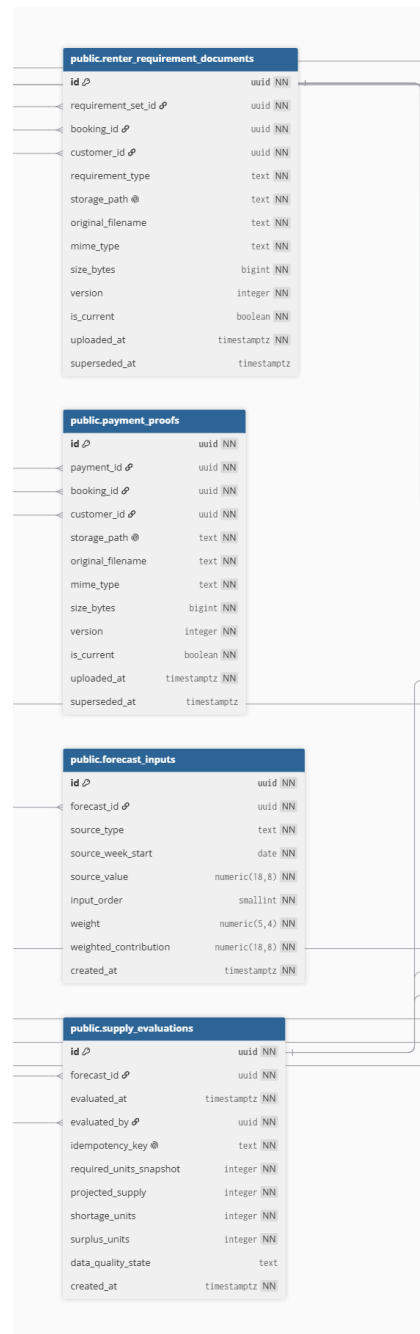
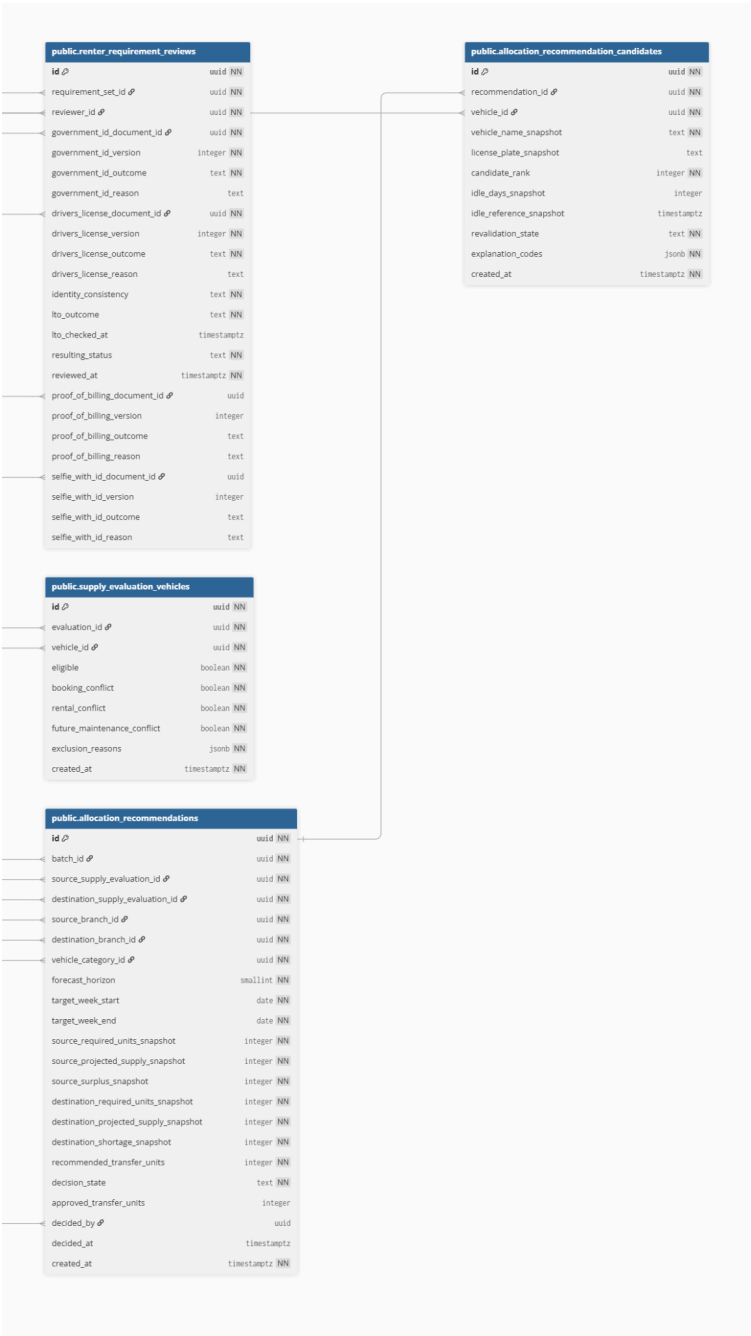


Figure 68. Database Schema Detail 9: Requirement Reviews, Supply Evaluation Vehicles, and Allocation Recommendations



The detail views continue portions of the same overview. Where a table crosses a view boundary, read the adjacent detail view together with its data dictionary entry.

Data Dictionary

The following data dictionary documents the implemented public database tables that support the rental workflow and decision-support features. Authentication credentials are managed separately by Supabase Auth. Retained earlier quotation structures are identified in their table descriptions.

Field Size identifies the fixed data-type size in bytes where applicable. NUMERIC precision and scale indicate the permitted total digits and decimal places, rather than a fixed byte size. An em dash (—) indicates variable-length data without a declared column-length limit. Required and Optional refer to database nullability; application rules may impose additional conditions.

Table 32

Data Dictionary – User Profiles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. FK to auth.users(id). Required.
email	TEXT	—	Email address associated with the record. Unique when present. Optional.
full_name	TEXT	—	Name displayed for the application user. Required.
phone_number	TEXT	—	Contact telephone number of the user. Optional.
user_type	TEXT	—	Application role used to distinguish administrative, operational, and customer access. Allowed values: Owner/Admin, Operations Staff, Customer/Renter. Required.
account_status	TEXT	—	Account-state label; defaults to Active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
street_address	TEXT	—	Street and house or building address. Optional.
barangay	TEXT	—	Barangay portion of the address. Optional.
city_municipality	TEXT	—	City or municipality portion of the address. Optional.
province	TEXT	—	Province portion of the address. Optional.
postal_code	TEXT	—	Postal code portion of the address. Optional.

Table 32 describes the table in the database named profiles, where application profile, contact, address, account-status, and role information for authenticated users is stored. Authentication

credentials are managed separately through Supabase Auth. The application roles are Owner/Admin, Operations Staff, and Customer/Renter.

Table 33

Data Dictionary – Branches

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name of the operational branch. Unique when present. Required.
address	TEXT	—	Address of the operational branch. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 33 describes the table in the database named branches, where operational branch names, addresses, and active flags are maintained for fleet assignment, bookings, forecasting, and allocation planning.

Table 34

Data Dictionary – Vehicle Categories

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name of the vehicle category. Unique when present. Required.
description	TEXT	—	Description of the vehicle category. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 34 describes the table in the database named vehicle_categories, where vehicle classifications and their descriptions are maintained for fleet organization and category-based demand planning.

Table 35

Data Dictionary – Vehicles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Vehicle name displayed in the fleet and catalog. Required.
category_id	UUID	16 bytes	Vehicle category assigned to the record. FK to public.vehicle_categories(id). Required.
branch_id	UUID	16 bytes	Operational branch associated with the record. FK to public.branches(id). Required.
license_plate	TEXT	—	Vehicle registration plate; unique when supplied. Unique when present. Optional.
transmission	TEXT	—	Recorded transmission type. Optional.
fuel_type	TEXT	—	Recorded fuel type. Optional.
seat_capacity	INTEGER	4 bytes	Passenger seating capacity; positive when supplied. Optional.
daily_rate	NUMERIC(12,2)	Precision 12, scale 2	Vehicle rental rate per day in Philippine pesos. Optional.
reference_fuel_efficiency_km_per_liter	NUMERIC(8,2)	Precision 8, scale 2	Reference fuel efficiency in kilometers per liter; positive when supplied and used for reference fuel estimates. Optional.
image_url	TEXT	—	Image reference associated with the vehicle. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
current_odometer_km	NUMERIC(12,1)	Precision 12, scale 1	Latest recorded odometer reading in kilometers. Optional.
condition_blocks_rental_use	BOOLEAN	1 byte	Indicates whether the recorded vehicle condition blocks rental use. Required.
large_luggage_capacity	INTEGER	4 bytes	Recorded large-luggage capacity used in vehicle suitability assessment. Optional.

Table 35 describes the table in the database named vehicles, where vehicle specifications, branch assignment, category, reference rental rate, fuel efficiency, luggage capacity, and readiness-related information are maintained. Availability is assessed using operational records and conditions, including active rentals, confirmed bookings, maintenance, and vehicle condition.

Table 36

Data Dictionary – Vehicle Images

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
public_url	TEXT	—	Public image address used to display the gallery image. Required.
alt_text	TEXT	—	Alternative text describing the vehicle image. Optional.
sort_order	SMALLINT	2 bytes	Gallery position from 0 to 4, unique within the vehicle. Required.
is_cover	BOOLEAN	1 byte	Indicates whether the image is the vehicle's cover; at most one cover per vehicle. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 36 describes the table in the database named vehicle_images, where ordered vehicle-gallery images and the designated cover image are recorded. Each vehicle may have up to five ordered image positions and at most one cover image.

Table 37

Data Dictionary – Vehicle Operational State Events

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
is_active	BOOLEAN	1 byte	Vehicle active state recorded by this event. Required.
effective_at	TIMESTAMP TZ	8 bytes	Date and time from which the recorded active state applies. Required.
recorded_by	UUID	16 bytes	Authenticated user associated with the state recording. FK to auth.users(id). Optional.
source	TEXT	—	Origin label of the state event; defaults to vehicle_mutation. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 37 describes the table in the database named vehicle_operational_state_events, where changes in a vehicle's active state and their effective times are preserved. These events support

historical operational-state interpretation rather than serving as a complete branch-transfer history.

Table 38

Data Dictionary – Booking Requests

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
requested_vehicle_id	UUID	16 bytes	Vehicle initially requested by the customer. FK to public.vehicles(id). Required.
assigned_vehicle_id	UUID	16 bytes	Vehicle assigned for fulfillment of the booking. FK to public.vehicles(id). Optional.
pickup_branch_id	UUID	16 bytes	Operational branch recorded for pickup. FK to public.branches(id). Required.
return_branch_id	UUID	16 bytes	Operational branch recorded for return. FK to public.branches(id). Required.
pickup_at	TIMESTAMP TZ	8 bytes	Scheduled pickup date and time. Required.
return_at	TIMESTAMP TZ	8 bytes	Scheduled return date and time; later than pickup. Required.
destination	TEXT	—	Destination entered for the planned rental. Optional.
purpose_of_use	TEXT	—	Customer's stated purpose for the rental. Required.
pickup_delivery_option	TEXT	—	Customer's selection of pickup or delivery. Allowed values: pickup, delivery. Required.
pickup_location	TEXT	—	Delivery pickup address; absent for the pickup option. Optional.
dropoff_location	TEXT	—	Delivery drop-off address recorded when delivery is selected. Optional.
preferred_seat_count	INTEGER	4 bytes	Requested seat count; positive when supplied. Optional.
customer_contact_number	TEXT	—	Contact telephone number supplied for the booking. Optional.
booking_status	TEXT	—	Current booking lifecycle state. Allowed values: Draft, Submitted, Confirmed, Rejected, Cancelled. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Attributes	Data Type	Field Size	Description
assigned_by	UUID	16 bytes	User who recorded the vehicle assignment. FK to public.profiles(id). Optional.
assigned_at	TIMESTAMP TZ	8 bytes	Date and time when the vehicle assignment was recorded. Optional.
assignment_note	TEXT	—	Explanation accompanying vehicle assignment or an acknowledged substitution or cross-branch assignment. Optional.
substitution_acknowledged	BOOLEAN	1 byte	Indicates whether replacement of the requested vehicle was acknowledged. Required.
cross_branch_acknowledged	BOOLEAN	1 byte	Indicates whether assignment from another branch was acknowledged. Required.
confirmed_by	UUID	16 bytes	User who confirmed the booking. FK to public.profiles(id). Optional.
confirmed_at	TIMESTAMP TZ	8 bytes	Date and time when booking confirmation was recorded. Optional.
resolution_reason	TEXT	—	Reason recorded for the booking's resolution. Optional.
resolved_by	UUID	16 bytes	User who recorded the booking resolution. FK to public.profiles(id). Optional.
resolved_at	TIMESTAMP TZ	8 bytes	Date and time of the booking resolution. Optional.
confirmation_exception_code	TEXT	—	Classified condition preventing automatic confirmation after payment verification. Allowed values: assignment_required, vehicle_unavailable, vehicle_maintenance_unready, vehicle_conflict, stale_assignment, assignment_expectation_required, pickup_window_elapsed, substitution_ack_required, cross_branch_ack_required, booking_not_submitted. Optional.
confirmation_exception_message	TEXT	—	Readable explanation of the condition preventing automatic confirmation. Optional.
confirmation_exception_at	TIMESTAMP TZ	8 bytes	Date and time when the confirmation exception was recorded. Optional.

Table 38 describes the table in the database named booking_requests, where customer rental requests, requested and assigned vehicles, schedules, locations, assignment acknowledgements, booking decisions, and confirmation exceptions are recorded. Return and pickup branches refer to operational branch records.

Table 39

Data Dictionary – Booking Finder Context

Attributes	Data Type	Field Size	Description
booking_id	UUID	16 bytes	Booking associated with the record. Primary key. FK to public.booking_requests(id). Required.
selected_vehicle_id	UUID	16 bytes	Vehicle selected from the Finder recommendation. FK to public.vehicles(id). Required.
requested_start	TIMESTAMP TZ	8 bytes	Rental start date and time entered in the Finder. Required.
requested_end	TIMESTAMP TZ	8 bytes	Rental end date and time entered in the Finder. Required.
passenger_count	INTEGER	4 bytes	Positive passenger count entered in the Finder. Required.
maximum_budget	NUMERIC	—	Positive maximum budget entered in the Finder. Required.
preferred_category_id	UUID	16 bytes	Optional vehicle-category preference entered in the Finder. FK to public.vehicle_categories(id). Optional.
destination	TEXT	—	Destination entered for the planned rental. Optional.
recommendation_rank	INTEGER	4 bytes	Positive position of the selected vehicle in the recommendation results. Required.
finder_baseline	TEXT	—	Label identifying the Finder recommendation baseline used for this booking. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
large_bag_count	INTEGER	4 bytes	Requested number of large bags, when supplied. Optional.

Table 39 describes the table in the database named booking_finder_context, where the applicable customer inputs, selected vehicle, and recommendation provenance associated with a Finder-assisted booking are preserved. External weather, road, and route responses are retrieved when needed and temporarily cached by the server.

Table 40

Data Dictionary – Booking Creation Idempotency

Attributes	Data Type	Field Size	Description
customer_id	UUID	16 bytes	Customer associated with the record. Part of the composite primary key (customer_id, idempotency_key). FK to public.profiles(id). Required.

Attributes	Data Type	Field Size	Description
idempotency_key	UUID	16 bytes	Request identifier used to prevent duplicate processing of the same operation. Part of the composite primary key (customer_id, idempotency_key). Required.
request_fingerprint	TEXT	—	64-character hexadecimal fingerprint used to compare retried booking requests. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 40 describes the table in the database named booking_creation_idempotency, where customer request identifiers and request fingerprints are associated with the created booking to prevent duplicate booking creation when the same request is retried.

Table 41

Data Dictionary – Renter Requirement Sets

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
status	TEXT	—	Recorded state of the record. Allowed values: Not Submitted, Pending Review, Needs Resubmission, Verified. Required.
submitted_at	TIMESTAMP TZ	8 bytes	Date and time when the submission was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 41 describes the table in the database named renter_requirement_sets, where the requirement submission and verification state for each booking is maintained. A booking can have at most one requirement set.

Table 42

Data Dictionary – Renter Requirement Documents

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
requirement_set_id	UUID	16 bytes	Requirement set to which this record belongs. FK to public.renter_requirement_sets(id). Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
requirement_type	TEXT	—	Category of the uploaded renter requirement. Allowed values: Valid Government ID, Driver's License, Proof of Billing, Selfie with ID. Required.
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
original_filename	TEXT	—	Original filename provided during upload. Required.
mime_type	TEXT	—	Media type of the uploaded file; image/jpeg, image/png, or application/pdf. Required.
size_bytes	BIGINT	8 bytes	Uploaded file size; greater than zero and no more than 10,485,760 bytes. Required.
version	INTEGER	4 bytes	Positive version number identifying this file revision. Required.
is_current	BOOLEAN	1 byte	Indicates whether this file is the current version. Required.
uploaded_at	TIMESTAMP TZ	8 bytes	Date and time when the file was uploaded. Required.
superseded_at	TIMESTAMP TZ	8 bytes	Date and time when a newer file version replaced this file. Optional.

Table 42 describes the table in the database named `renter_requirement_documents`, where uploaded requirement-file references, document categories, and replacement versions are maintained. Previous versions remain distinguishable from the current file for each requirement category.

Table 43

Data Dictionary – Renter Requirement Reviews

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
requirement_set_id	UUID	16 bytes	Requirement set to which this record belongs. FK to public.renter_requirement_sets(id). Required.
reviewer_id	UUID	16 bytes	User who performed the document review. FK to public.profiles(id). Required.
government_id_document_id	UUID	16 bytes	Government id file examined in this review. FK to public.renter_requirement_documents(id). Required.
government_id_version	INTEGER	4 bytes	Version of the government ID file examined. Required.
government_id_outcome	TEXT	—	Review outcome for the government ID file. Allowed values: Accepted, Needs Replacement. Required.
government_id_reason	TEXT	—	Reason recorded when the government ID file needs replacement. Optional.
drivers_license_document_id	UUID	16 bytes	Driver's license file examined in this review. FK to public.renter_requirement_documents(id). Required.
drivers_license_version	INTEGER	4 bytes	Version of the driver's license file examined. Required.
drivers_license_outcome	TEXT	—	Review outcome for the driver's license file. Allowed values: Accepted, Needs Replacement. Required.
drivers_license_reason	TEXT	—	Reason recorded when the driver's license file needs replacement. Optional.
identity_consistency	TEXT	—	Recorded consistency between the reviewed identity documents. Allowed values: Consistent, Concern. Required.
lto_outcome	TEXT	—	Retained license-check outcome; the current review endpoint records Not Checked and does not require an LTO check. Allowed values: Not Checked, Clear, Concern, Unavailable. Required.
lto_checked_at	TIMESTAMP TZ	8 bytes	Retained date and time of a recorded LTO check, if present. Optional.

Attributes	Data Type	Field Size	Description
resulting_status	TEXT	—	Requirement-set state resulting from the review. Allowed values: Pending Review, Needs Resubmission, Verified. Required.
reviewed_at	TIMESTAMP TZ	8 bytes	Date and time when the review was recorded. Required.
proof_of_billing_document_id	UUID	16 bytes	Proof of billing file examined in this review. FK to public.renter_requirement_documents(id). Optional.
proof_of_billing_version	INTEGER	4 bytes	Version of the proof of billing file examined. Optional.
proof_of_billing_outcome	TEXT	—	Review outcome for the proof of billing file. Allowed values: Accepted, Needs Replacement. Optional.
proof_of_billing_reason	TEXT	—	Reason recorded when the proof of billing file needs replacement. Optional.
selfie_with_id_document_id	UUID	16 bytes	Selfie with id file examined in this review. FK to public.renter_requirement_documents(id). Optional.
selfie_with_id_version	INTEGER	4 bytes	Version of the selfie with ID file examined. Optional.
selfie_with_id_outcome	TEXT	—	Review outcome for the selfie with ID file. Allowed values: Accepted, Needs Replacement. Optional.
selfie_with_id_reason	TEXT	—	Reason recorded when the selfie with ID file needs replacement. Optional.

Table 43 describes the table in the database named renter_requirement_reviews, where review decisions are recorded against the document versions examined, together with identity consistency and the resulting requirement-set status. The retained LTO outcome fields do not establish an automated license-verification integration or a current LTO review gate.

Table 44

Data Dictionary – Booking Payment Quotes

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
daily_rate	NUMERIC(12,2)	Precision 12, scale 2	Vehicle rental rate per day in Philippine pesos. Required.

Attributes	Data Type	Field Size	Description
pickup_at	TIMESTAMP TZ	8 bytes	Scheduled pickup date and time. Required.
return_at	TIMESTAMP TZ	8 bytes	Scheduled return date and time; later than pickup. Required.
grace_minutes	INTEGER	4 bytes	Grace interval used when determining billable rental days; defaults to 60 minutes and is limited to 0–240. Required.
billable_days	INTEGER	4 bytes	Positive number of rental days charged by the quotation. Required.
rental_subtotal	NUMERIC(12,2)	Precision 12, scale 2	Positive vehicle-rental subtotal in Philippine pesos. Required.
delivery_fee	NUMERIC(12,2)	Precision 12, scale 2	Delivery charge in Philippine pesos. Required.
total_amount	NUMERIC(12,2)	Precision 12, scale 2	Rental subtotal plus delivery fee in Philippine pesos. Required.
down_payment_amount	NUMERIC(12,2)	Precision 12, scale 2	Required initial down payment, equal to 50 percent of the total rounded to two decimal places. Required.
remaining_balance_amount	NUMERIC(12,2)	Precision 12, scale 2	Total less the down payment in Philippine pesos. Required.
security_deposit_amount	NUMERIC(12,2)	Precision 12, scale 2	Separately recorded security deposit, fixed at PHP 3,000. Required.
quote_version	INTEGER	4 bytes	Positive revision number of the quotation. Required.
issued_by	UUID	16 bytes	User who issued the current quotation. FK to public.profiles(id). Required.
issued_at	TIMESTAMP TZ	8 bytes	Date and time when the quotation was issued. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 44 describes the table in the database named `booking_payment_quotes`, where the current system-generated quotation for a booking is stored, including the vehicle's daily rate, rental period, billable days, delivery fee, total amount, 50 percent down payment, remaining balance, and separately recorded PHP 3,000 security deposit.

Table 45

Data Dictionary – Payment Methods

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
code	TEXT	—	Unique code identifying the payment method. Unique when present. Required.
label	TEXT	—	Readable payment-method name. Required.

Attributes	Data Type	Field Size	Description
instructions	TEXT	—	Payment instructions presented to the customer. Required.
is_demo	BOOLEAN	1 byte	Indicates whether the method is designated as a demonstration method. Required.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
recipient_name	TEXT	—	Name of the payment recipient. Optional.
account_number	TEXT	—	Recipient's payment-account reference. Optional.
qr_image_path	TEXT	—	Stored reference to the method's QR-code image. Optional.

Table 45 describes the table in the database named `payment_methods`, where available payment-channel instructions, recipient details, account information, and optional QR-image references are maintained.

Table 46

Data Dictionary – Payments

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to <code>public.booking_requests(id)</code> . Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to <code>public.profiles(id)</code> . Required.
purpose	TEXT	—	Payment purpose, restricted to <code>initial_down_payment</code> . Required.
currency	TEXT	—	Currency code, restricted to PHP. Required.
required_amount	NUMERIC(12,2)	Precision 12, scale 2	Required initial down payment in Philippine pesos. Optional.
submitted_amount	NUMERIC(12,2)	Precision 12, scale 2	Amount submitted by the customer in Philippine pesos. Optional.
payment_method_id	UUID	16 bytes	Selected payment method, when recorded. FK to <code>public.payment_methods(id)</code> . Optional.
payment_method_label	TEXT	—	Recorded payment-method label. Optional.
transaction_reference	TEXT	—	Payment transaction reference supplied by the customer. Optional.

Attributes	Data Type	Field Size	Description
status	TEXT	—	Recorded state of the record. Allowed values: Not Submitted, Pending Verification, Needs Resubmission, Verified. Required.
resubmission_reason	TEXT	—	Reason requiring replacement or correction of the payment submission. Optional.
reviewed_by	UUID	16 bytes	User who reviewed the submission. FK to public.profiles(id). Optional.
reviewed_at	TIMESTAMP TZ	8 bytes	Date and time when the review was recorded. Optional.
reviewed_proof_version	INTEGER	4 bytes	Proof revision examined during the payment review. Optional.
reviewed_submitted_amount	NUMERIC(12,2)	Precision 12, scale 2	Submitted amount preserved with the review decision. Optional.
reviewed_transaction_reference	TEXT	—	Transaction reference preserved with the review decision. Optional.
submitted_at	TIMESTAMP TZ	8 bytes	Date and time when the submission was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 46 describes the table in the database named payments, where the initial down-payment requirement, submitted payment information, review evidence, and verification status for each booking are stored. The record is associated with versioned proof files.

Table 47

Data Dictionary – Payment Proofs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
payment_id	UUID	16 bytes	Payment to which the proof belongs. FK to public.payments(id). Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
storage_path	TEXT	—	Stored object reference identifying the uploaded file. Unique when present. Required.
original_filename	TEXT	—	Original filename provided during upload. Required.

Attributes	Data Type	Field Size	Description
mime_type	TEXT	—	Media type of the uploaded file; image/jpeg, image/png, or application/pdf. Required.
size_bytes	BIGINT	8 bytes	Uploaded proof size; greater than zero and no more than 10,485,760 bytes. Required.
version	INTEGER	4 bytes	Positive version number identifying this file revision. Required.
is_current	BOOLEAN	1 byte	Indicates whether this file is the current version. Required.
uploaded_at	TIMESTAMP TZ	8 bytes	Date and time when the file was uploaded. Required.
superseded_at	TIMESTAMP TZ	8 bytes	Date and time when a newer file version replaced this file. Optional.

Table 47 describes the table in the database named payment_proofs, where payment evidence-file references and their replacement versions are preserved. Each payment has at most one proof designated as current.

Table 48

Data Dictionary – Rate Cards

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
package_code	TEXT	—	Code identifying the retained rental package. Required.
package_label	TEXT	—	Readable name of the retained rental package. Required.
duration_hours	INTEGER	4 bytes	Rental-package duration in hours. Required.
base_rate	NUMERIC(12,2)	Precision 12, scale 2	Positive base package rate in Philippine pesos. Required.
effective_from	DATE	4 bytes	Beginning date of the retained rate's validity. Required.
effective_until	DATE	4 bytes	Optional end date of the retained rate's validity. Optional.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
researcher_designed	BOOLEAN	1 byte	Identifies the retained structure as based on the researcher-designed quotation model. Required.
created_by	UUID	16 bytes	User who created the record. FK to public.profiles(id). Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Attributes	Data Type	Field Size	Description
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 48 describes the table in the database named rate_cards, where versioned vehicle-package rates, effective dates, and researcher-designed indicators are stored as retained earlier structures. These records remain part of the physical database, while booking_payment_quotes supports the current quotation and payment workflow.

Table 49

Data Dictionary – Booking Rate Quotes

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
rate_card_id	UUID	16 bytes	Retained rate card used for the package quotation. FK to public.rate_cards(id). Required.
package_code	TEXT	—	Code identifying the retained rental package. Required.
package_label	TEXT	—	Readable name of the retained rental package. Required.
duration_hours	INTEGER	4 bytes	Rental-package duration in hours. Required.
base_rental_amount	NUMERIC(12,2)	Precision 12, scale 2	Positive base rental amount in Philippine pesos. Required.
delivery_fee	NUMERIC(12,2)	Precision 12, scale 2	Delivery charge in Philippine pesos. Required.
approved_discount	NUMERIC(12,2)	Precision 12, scale 2	Nonnegative approved discount in Philippine pesos. Required.
approved_subtotal	NUMERIC(12,2)	Precision 12, scale 2	Stored generated amount: base rental amount plus delivery fee less approved discount. Optional.
required_down_payment	NUMERIC(12,2)	Precision 12, scale 2	Stored generated amount: 50 percent of the approved subtotal rounded to two decimal places. Optional.
quote_version	INTEGER	4 bytes	Positive revision number of the quotation. Required.
status	TEXT	—	Retained quotation state, restricted to Approved. Allowed values: Approved. Required.
researcher_designed	BOOLEAN	1 byte	Identifies the retained structure as based on the researcher-designed quotation model. Required.

Attributes	Data Type	Field Size	Description
approved_by	UUID	16 bytes	User who approved the retained package quotation. FK to public.profiles(id). Required.
approved_at	TIMESTAMP TZ	8 bytes	Date and time when the retained quotation was approved. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

Table 49 describes the table in the database named booking_rate_quotes, where approved package-based quotation snapshots and their computed subtotal and down-payment values are stored as a retained earlier structure. This table is distinct from the current quotation workflow represented by booking_payment_quotes.

Table 50

Data Dictionary – Rental Transactions

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Unique when present. Required.
customer_id	UUID	16 bytes	Customer associated with the record. FK to public.profiles(id). Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
scheduled_pickup_at	TIMESTAMP TZ	8 bytes	Pickup schedule preserved when the rental is released. Required.
scheduled_return_at	TIMESTAMP TZ	8 bytes	Return schedule preserved when the rental is released. Required.
started_at	TIMESTAMP TZ	8 bytes	Date and time when the vehicle was released and the rental started. Required.
ended_at	TIMESTAMP TZ	8 bytes	Date and time when the rental was closed after return. Optional.
released_by	UUID	16 bytes	User who recorded vehicle release. FK to public.profiles(id). Required.
release_odometer	NUMERIC(12,2)	Precision 12, scale 2	Odometer reading recorded at release. Optional.
release_fuel_level	TEXT	—	Recorded fuel level at vehicle release. Allowed values: Empty, 1/4, 1/2, 3/4, Full, Other/Unknown. Required.
release_condition_summary	TEXT	—	Summary of the vehicle condition at release. Required.
existing_damage_notes	TEXT	—	Damage observations recorded before release. Optional.
agreement_acknowledged	BOOLEAN	1 byte	Indicates whether the rental agreement was acknowledged. Required.

Attributes	Data Type	Field Size	Description
condition_acked	BOOLEAN	1 byte	Indicates whether the release condition was acknowledged. Required.
return_schedule_acked	BOOLEAN	1 byte	Indicates whether the return schedule was acknowledged. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
returned_by	UUID	16 bytes	User who recorded vehicle return. FK to public.profiles(id). Optional.
return_odometer	NUMERIC(12,2)	Precision 12, scale 2	Odometer reading recorded at return. Optional.
return_fuel_level	TEXT	—	Recorded fuel level at vehicle return. Allowed values: Empty, 1/4, 1/2, 3/4, Full, Other/Unknown. Optional.
return_condition_summary	TEXT	—	Summary of vehicle condition upon return. Optional.
observed_damage_notes	TEXT	—	Damage observations recorded upon return. Optional.
return_remarks	TEXT	—	Additional remarks concerning the return. Optional.
inspection_status	TEXT	—	State of the return inspection recorded within the rental. Allowed values: Not required, Pending, Cleared, Maintenance scheduled. Required.
inspection_remarks	TEXT	—	Remarks concerning the return inspection. Optional.
inspected_at	TIMESTAMP TZ	8 bytes	Date and time of the return inspection. Optional.
inspected_by	UUID	16 bytes	User who recorded the return inspection. FK to public.profiles(id). Optional.

Table 50 describes the table in the database named rental_transactions, where vehicle release, customer acknowledgements, actual rental start and end, return condition, and inspection information are recorded. Return inspection is embedded in this record rather than maintained in a separate inspection table.

Table 51

Data Dictionary – Maintenance Records

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
maintenance_type	TEXT	—	Recorded service or maintenance classification. Required.

Attributes	Data Type	Field Size	Description
description	TEXT	—	Description of the maintenance work. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Scheduled, In Progress, Completed, Overdue, Cancelled. Required.
blocks_rental_use	BOOLEAN	1 byte	Indicates whether this maintenance record blocks rental use. Required.
service_started_at	TIMESTAMP TZ	8 bytes	Date and time when servicing began. Optional.
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
odometer_at_service	NUMERIC(12,1)	Precision 12, scale 1	Odometer reading in kilometers associated with the service. Optional.
next_service_odometer	NUMERIC(12,1)	Precision 12, scale 1	Odometer threshold in kilometers for the next service. Optional.
next_service_date	DATE	4 bytes	Date threshold for the next service. Optional.
cost_php	NUMERIC(12,2)	Precision 12, scale 2	Recorded maintenance cost in Philippine pesos. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_by	UUID	16 bytes	Authenticated user who created the maintenance record. FK to auth.users(id). Required.
updated_by	UUID	16 bytes	Authenticated user who last updated the maintenance record. FK to auth.users(id). Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
scheduled_for	TIMESTAMP TZ	8 bytes	Date and time scheduled for maintenance. Optional.
archived_at	TIMESTAMP TZ	8 bytes	Date and time when the record was archived. Optional.
archived_by	UUID	16 bytes	User who archived the record. FK to public.profiles(id). Optional.

Table 51 describes the table in the database named `maintenance_records`, where scheduled servicing, work in progress, completion, overdue or cancelled work, rental-use restrictions, service readings, and archival information are maintained. Maintenance lifecycle states are Scheduled, In Progress, Completed, Overdue, and Cancelled.

Table 52

Data Dictionary – Forecast Runs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
generated_at	TIMESTAMP TZ	8 bytes	Date and time when the generation operation was recorded. Required.
generated_by	UUID	16 bytes	User who initiated the generation operation. FK to public.profiles(id). Required.
method	TEXT	—	Forecasting method, restricted to WMA. Required.
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
coverage_start	DATE	4 bytes	Beginning date of historical demand coverage used by the run. Optional.

Table 52 describes the table in the database named `forecast_runs`, where each Weighted Moving Average forecast-generation run, its initiating user, method, and historical coverage boundary are recorded.

Table 53

Data Dictionary – Demand Forecasts

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
run_id	UUID	16 bytes	Forecast-generation run to which the projection belongs. FK to <code>public.forecast_runs(id)</code> . Required.
branch_id	UUID	16 bytes	Operational branch associated with the record. FK to <code>public.branches(id)</code> . Required.
vehicle_category_id	UUID	16 bytes	Vehicle category for which the record applies. FK to <code>public.vehicle_categories(id)</code> . Required.
horizon	SMALLINT	2 bytes	Projection horizon from one to three weeks. Required.
target_week_start	DATE	4 bytes	Beginning date of the forecast or allocation target week. Required.
target_week_end	DATE	4 bytes	Ending boundary of the forecast or allocation target week. Required.
forecasted_demand	NUMERIC(18,8)	Precision 18, scale 8	Decimal weekly vehicle-demand projection produced by WMA. Required.

Attributes	Data Type	Field Size	Description
required_vehicle_units	INTEGER	4 bytes	Whole vehicle units required for planning, obtained by rounding forecast demand upward. Required.
actual_demand	NUMERIC(18,8)	Precision 18, scale 8	Observed demand for the completed target week, when finalized. Optional.
ape	NUMERIC(18,8)	Precision 18, scale 8	Absolute percentage error for the forecast when a valid actual-demand comparison is available. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 53 describes the table in the database named forecasts, where weekly vehicle-demand projections for each branch and vehicle category are stored, together with the forecast horizon, required vehicle units, and subsequent actual-demand and accuracy observations.

Table 54

Data Dictionary – Forecast Inputs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
forecast_id	UUID	16 bytes	Forecast to which this record belongs. FK to public.forecasts(id). Required.
source_type	TEXT	—	Indicates whether the calculation input is an observed actual value or a recursively generated forecast. Allowed values: Actual, Forecast. Required.
source_week_start	DATE	4 bytes	Beginning date of the input week. Required.
source_value	NUMERIC(18,8)	Precision 18, scale 8	Demand value used as a calculation input. Required.
input_order	SMALLINT	2 bytes	Recency position from 1 to 3; position 1 is the most recent input. Required.
weight	NUMERIC(5,4)	Precision 5, scale 4	WMA weight assigned to the input. Required.
weighted_contribution	NUMERIC(18,8)	Precision 18, scale 8	Product of the input demand value and its assigned weight. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 54 describes the table in the database named forecast_inputs, where the source observations or recursive forecast values, recency order, weights, and weighted contributions used by a forecast are preserved. The study applies three-period WMA weights of 0.50, 0.30, and 0.20 from most recent to least recent input.

Table 55

Data Dictionary – Forecast Demand Coverage

Attributes	Data Type	Field Size	Description
id	SMALLINT	2 bytes	Single-row primary-key value, restricted to 1. Primary key. Required.
tracking_started_at	TIMESTAMP TZ	8 bytes	Date and time at which trustworthy demand tracking begins. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 55 describes the table in the database named forecast_demand_coverage, where the beginning of trustworthy demand tracking is recorded in a single-row coverage record. This boundary helps distinguish complete zero-demand periods from periods without adequate historical coverage.

Table 56

Data Dictionary – Supply Evaluations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
forecast_id	UUID	16 bytes	Forecast to which this record belongs. FK to public.forecasts(id). Required.
evaluated_at	TIMESTAMP TZ	8 bytes	Date and time when projected supply was assessed. Required.
evaluated_by	UUID	16 bytes	User who initiated the supply assessment. FK to public.profiles(id). Required.
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
required_units_snaps hot	INTEGER	4 bytes	Required vehicle units preserved from the evaluated forecast. Required.
projected_supply	INTEGER	4 bytes	Number of vehicles projected to be eligible for the target period. Required.
shortage_units	INTEGER	4 bytes	Nonnegative shortfall of projected supply against required units. Required.
surplus_units	INTEGER	4 bytes	Nonnegative excess of projected supply over required units. Required.
data_quality_state	TEXT	—	Recorded qualification concerning the reliability or completeness of evaluation inputs. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 56 describes the table in the database named `supply_evaluations`, where a forecast's required units are compared with projected eligible vehicle supply, preserving shortage, surplus, and data-quality observations at the time of evaluation.

Table 57

Data Dictionary – Supply Evaluation Vehicles

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
evaluation_id	UUID	16 bytes	Supply evaluation to which this vehicle assessment belongs. FK to <code>public.supply_evaluations(id)</code> . Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to <code>public.vehicles(id)</code> . Required.
eligible	BOOLEAN	1 byte	Indicates whether the vehicle was eligible during the supply assessment. Required.
booking_conflict	BOOLEAN	1 byte	Indicates whether a booking conflict was identified. Required.
rental_conflict	BOOLEAN	1 byte	Indicates whether an active-rental conflict was identified. Required.
future_maintenance_conflict	BOOLEAN	1 byte	Indicates whether future maintenance conflicts with the target period. Required.
exclusion_reasons	JSONB	—	Structured explanations of conditions excluding the vehicle from projected supply. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 57 describes the table in the database named `supply_evaluation_vehicles`, where vehicle-level eligibility, booking and rental conflicts, future maintenance conflicts, and exclusion explanations are captured for each supply evaluation.

Table 58

Data Dictionary – Allocation Recommendation Batches

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
generated_by	UUID	16 bytes	User who initiated the generation operation. FK to <code>public.profiles(id)</code> . Required.
generated_at	TIMESTAMP TZ	8 bytes	Date and time when the generation operation was recorded. Required.

Attributes	Data Type	Field Size	Description
idempotency_key	TEXT	—	Request identifier used to prevent duplicate processing of the same operation. Unique when present. Required.
generation_context_fingerprint	TEXT	—	Fingerprint of the evaluation context used to generate the recommendation batch. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 58 describes the table in the database named `allocation_recommendation_batches`, where each allocation-recommendation generation event and its context fingerprint are recorded, allowing the resulting recommendations to be traced to a common evaluation context.

Table 59

Data Dictionary – Allocation Recommendations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
batch_id	UUID	16 bytes	Generation batch to which the recommendation belongs. FK to <code>public.allocation_recommendation_batches(id)</code> . Required.
source_supply_evaluation_id	UUID	16 bytes	Supply evaluation supporting the source branch's surplus. FK to <code>public.supply_evaluations(id)</code> . Required.
destination_supply_evaluation_id	UUID	16 bytes	Supply evaluation supporting the destination branch's shortage. FK to <code>public.supply_evaluations(id)</code> . Required.
source_branch_id	UUID	16 bytes	Branch proposed as the source of vehicle units. FK to <code>public.branches(id)</code> . Required.
destination_branch_id	UUID	16 bytes	Different branch proposed as the recipient of vehicle units. FK to <code>public.branches(id)</code> . Required.
vehicle_category_id	UUID	16 bytes	Vehicle category for which the record applies. FK to <code>public.vehicle_categories(id)</code> . Required.
forecast_horizon	SMALLINT	2 bytes	Forecast horizon, expressed as one, two, or three weeks. Required.
target_week_start	DATE	4 bytes	Beginning date of the forecast or allocation target week. Required.
target_week_end	DATE	4 bytes	Ending boundary of the forecast or allocation target week. Required.
source_required_units_snapshot	INTEGER	4 bytes	Source branch's required vehicle units at generation. Required.

Attributes	Data Type	Field Size	Description
source_projected_supply_snapshot	INTEGER	4 bytes	Source branch's projected eligible supply at generation. Required.
source_surplus_snapshot	INTEGER	4 bytes	Positive source surplus preserved at generation. Required.
destination_required_units_snapshot	INTEGER	4 bytes	Destination branch's required vehicle units at generation. Required.
destination_projected_supply_snapshot	INTEGER	4 bytes	Destination branch's projected eligible supply at generation. Required.
destination_shortage_snapshot	INTEGER	4 bytes	Positive destination shortage preserved at generation. Required.
recommended_transfer_units	INTEGER	4 bytes	Suggested positive unit count, limited by the preserved source surplus and destination shortage. Required.
decision_state	TEXT	—	Owner/Admin review state of the recommendation. Allowed values: Pending, Approved, Rejected. Required.
approved_transfer_units	INTEGER	4 bytes	Approved positive unit count, no greater than the recommended count; recorded for an approved recommendation. Optional.
decided_by	UUID	16 bytes	User who approved or rejected the recommendation. FK to public.profiles(id). Optional.
decided_at	TIMESTAMP TZ	8 bytes	Date and time when the review decision was recorded. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 59 describes the table in the database named `allocation_recommendations`, where suggested units for movement between source and destination branches are stored with supply snapshots and Owner/Admin review decisions. Approval records a decision and does not automatically transfer vehicles or change their branch assignments.

Table 60

Data Dictionary – Allocation Recommendation Candidates

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recommendation_id	UUID	16 bytes	Allocation recommendation to which the candidate belongs. FK to public.allocation_recommendations(id). Required.
vehicle_id	UUID	16 bytes	Vehicle associated with the record. FK to public.vehicles(id). Required.
vehicle_name_snapshot	TEXT	—	Vehicle name preserved at candidate generation. Required.

Attributes	Data Type	Field Size	Description
license_plate_snapshot	TEXT	—	Vehicle registration plate preserved at candidate generation. Optional.
candidate_rank	INTEGER	4 bytes	Positive candidate position, unique within the recommendation. Required.
idle_days_snapshot	INTEGER	4 bytes	Recorded nonnegative idle-day count, when determinable. Optional.
idle_reference_snapshot	TIMESTAMP TZ	8 bytes	Reference time used to interpret the recorded idle state. Optional.
revalidation_state	TEXT	—	Eligibility classification recorded at generation, restricted to EligibleAtGeneration. Allowed values: EligibleAtGeneration. Required.
explanation_codes	JSONB	—	Structured list of explanations accompanying candidate selection. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 60 describes the table in the database named `allocation_recommendation_candidates`, where ranked vehicle candidates and the eligibility and idle-state information observed when an allocation recommendation was generated are preserved.

Table 61

Data Dictionary – Notifications

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recipient_id	UUID	16 bytes	User to whom the notification or preference belongs. FK to <code>public.profiles(id)</code> . Required.
notification_type	TEXT	—	Canonical business-event or reminder category of the notification. Allowed values: <code>requirements_needs_resubmission</code> , <code>requirements_verified</code> , <code>payment_needs_resubmission</code> , <code>payment_verified</code> , <code>booking_confirmed</code> , <code>new_booking_request</code> , <code>requirements_submitted</code> , <code>payment_proof_submitted</code> , <code>upcoming_pickup</code> , <code>upcoming_return</code> , <code>rental_overdue</code> , <code>maintenance_attention</code> , <code>low_availability</code> , <code>backup_attention</code> . Required.
title	TEXT	—	Title of the notification or operational alert. Required.
message	TEXT	—	Text of the message. Required.

Attributes	Data Type	Field Size	Description
related_entity_type	TEXT	—	Type of business record associated with the message or condition. Allowed values: booking, requirements, payment, rental, vehicle, branch, backup_run. Required.
related_entity_id	UUID	16 bytes	Identifier of the associated business record; interpreted using its type and not declared as a foreign key. Required.
event_key	TEXT	—	Event identifier used with the recipient to prevent duplicate notifications. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
read_at	TIMESTAMP TZ	8 bytes	Date and time when the message was marked as read. Optional.

Table 61 describes the table in the database named notifications, where recipient-specific in-application messages, event identifiers, related-entity references, and read times are stored. Related-entity references identify different business-record types and are not declared foreign keys.

Table 62

Data Dictionary – Notification Preferences

Attributes	Data Type	Field Size	Description
recipient_id	UUID	16 bytes	User to whom the notification or preference belongs. Primary key. FK to public.profiles(id). Required.
maintenance_attention_enabled	BOOLEAN	1 byte	Indicates whether maintenance-attention notifications are enabled. Required.
low_availability_enabled	BOOLEAN	1 byte	Indicates whether low-availability notifications are enabled. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
email_notifications_enabled	BOOLEAN	1 byte	Indicates whether email notifications are enabled. Required.

Table 62 describes the table in the database named notification_preferences, where each user's choices for maintenance alerts, low-availability alerts, and email notifications are maintained.

Table 63

Data Dictionary – Operational Notification Conditions

Attributes	Data Type	Field Size	Description
condition_type	TEXT	—	Operational condition being tracked. Part of the composite primary key (condition_type, related_entity_id). Allowed values: maintenance_attention, low_availability. Required.
related_entity_type	TEXT	—	Type of business record associated with the message or condition. Allowed values: vehicle, branch. Required.
related_entity_id	UUID	16 bytes	Identifier of the associated business record; interpreted using its type and not declared as a foreign key. Part of the composite primary key (condition_type, related_entity_id). Required.
is_active	BOOLEAN	1 byte	Indicates whether the operational condition is currently active. Required.
occurrence_count	BIGINT	8 bytes	Nonnegative count of condition occurrences. Required.
title	TEXT	—	Title of the notification or operational alert. Required.
message	TEXT	—	Text of the message. Required.
activated_at	TIMESTAMP TZ	8 bytes	Date and time when the condition became active. Optional.
resolved_at	TIMESTAMP TZ	8 bytes	Date and time when the condition was resolved. Optional.
last_evaluated_at	TIMESTAMP TZ	8 bytes	Date and time of the most recent condition assessment. Required.

Table 63 describes the table in the database named operational_notification_conditions, where the activation and resolution of maintenance-attention and branch low-availability conditions are tracked to support controlled alert generation. The associated entity identifier is interpreted according to the condition type.

Table 64

Data Dictionary – Email Deliveries

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
recipient_user_id	UUID	16 bytes	User to whom the notification email is addressed. FK to public.profiles(id). Required.

Attributes	Data Type	Field Size	Description
notification_id	UUID	16 bytes	In-application notification associated with the email; unique per notification. FK to public.notifications(id). Unique when present. Required.
delivery_key	TEXT	—	Unique identifier preventing duplicate email delivery. Unique when present. Required.
email_type	TEXT	—	Canonical business-event or reminder category eligible for email delivery. Allowed values: requirements_needs_resubmission, requirements_verified, payment_needs_resubmission, payment_verified, booking_confirmed, upcoming_pickup, upcoming_return, rental_overdue. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Pending, Processing, Sent, Failed, Skipped. Required.
attempt_count	INTEGER	4 bytes	Number of delivery attempts, limited to 0–4. Required.
provider_message_id	TEXT	—	Message reference returned by the email provider. Optional.
last_error_code	TEXT	—	Classified result of the latest delivery error. Allowed values: ConfigurationError, RecipientUnavailable, RecipientInvalid, PreferenceDisabled, RateLimited, ProviderUnavailable, ProviderRejected, NetworkError, UnknownProviderError. Optional.
next_attempt_at	TIMESTAMP TZ	8 bytes	Scheduled date and time of the next delivery attempt. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
last_attempt_at	TIMESTAMP TZ	8 bytes	Date and time of the latest delivery attempt. Optional.
sent_at	TIMESTAMP TZ	8 bytes	Date and time when successful sending was recorded. Optional.

Table 64 describes the table in the database named `email_deliveries`, where the delivery state, retry attempts, provider reference, and classified errors for an eligible notification email are tracked. Each notification can be associated with at most one email-delivery record.

Table 65

Data Dictionary – Audit Events

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
actor_type	TEXT	—	Indicates whether the recorded actor is a User or the System. Allowed values: User, System. Required.
actor_user_id	UUID	16 bytes	Acting application user; present for User events and absent for System events. FK to public.profiles(id). Optional.
action	TEXT	—	Canonical business-event action recorded in the audit trail. Allowed values: booking.created, booking.edited, booking.vehicle_assigned, booking.confirmed, booking.cancelled, booking.rejected, booking.withdrawn, booking.location_reconciled, requirements.submitted, requirements.resubmitted, requirements.needs_resubmission, requirements.verified, payment.quote_issued, payment.submitted, payment.resubmitted, payment.needs_resubmission, payment.verified, payment.requirement_set, rental.released, rental.returned, maintenance.created, maintenance.started, maintenance.completed, maintenance.cancelled. Required.
entity_type	TEXT	—	Type of business record affected by the event. Allowed values: booking, requirements, payment, rental, maintenance. Required.
entity_id	UUID	16 bytes	Identifier interpreted using entity_type; not declared as a foreign key. Required.
booking_id	UUID	16 bytes	Booking associated with the record. FK to public.booking_requests(id). Optional.
metadata	JSONB	—	Structured object containing contextual details of the recorded event. Required.
occurred_at	TIMESTAMP TZ	8 bytes	Date and time when the event occurred. Required.

Table 65 describes the table in the database named `audit_events`, where user-initiated or system-initiated business events are preserved with their action, entity reference, optional booking association, metadata, and occurrence time. This event-based record does not use a generic old-value and new-value model.

Table 66

Data Dictionary – Backup Runs

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
trigger	TEXT	—	Indicates whether the backup was Scheduled or Manual. Allowed values: Scheduled, Manual. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Running, Completed, Partial, Failed. Required.
started_at	TIMESTAMP TZ	8 bytes	Date and time when the operation started. Required.
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
retention_until	TIMESTAMP TZ	8 bytes	Date and time through which the backup is scheduled for retention. Required.
error_code	TEXT	—	Classified error associated with the operation. Allowed values: ConfigurationError, DatabaseDumpFailed, StorageEnumerationFailed, StorageObjectReadFailed, ArtifactUploadFailed, IntegrityValidationFailed, RetentionCleanupFailed, UnknownBackupError. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_by	UUID	16 bytes	User who created the record. FK to <code>public.profiles(id)</code> . Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 66 describes the table in the database named `backup_runs`, where scheduled or manual backup attempts, their completion states, retention dates, and classified errors are recorded. Completed, Partial, and Failed attempts remain distinguishable.

Table 67

Data Dictionary – Backup Artifacts

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
backup_run_id	UUID	16 bytes	Backup attempt associated with the artifact or recovery drill. FK to public.backup_runs(id). Required.
artifact_type	TEXT	—	Indicates whether the recovery asset protects Database or Storage content. Allowed values: Database, Storage. Required.
artifact_key	TEXT	—	Logical recovery-asset reference without an access credential or signed access URL. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Completed, Failed. Required.
size_bytes	BIGINT	8 bytes	Size of the file or artifact in bytes. Optional.
sha256	TEXT	—	64-character hexadecimal SHA-256 integrity checksum for a completed artifact. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 67 describes the table in the database named backup_artifacts, where database and file-storage recovery assets belonging to a backup attempt are identified, together with their completion state, byte size, and integrity checksum. Logical artifact references do not contain access credentials.

Table 68

Data Dictionary – Recovery Drills

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
backup_run_id	UUID	16 bytes	Backup attempt associated with the artifact or recovery drill. FK to public.backup_runs(id). Required.
target_environment	TEXT	—	Recovery-test environment, restricted to NonProduction. Allowed values: NonProduction. Required.
status	TEXT	—	Recorded state of the record. Allowed values: Running, Passed, Failed. Required.
started_at	TIMESTAMP TZ	8 bytes	Date and time when the operation started. Required.

Attributes	Data Type	Field Size	Description
completed_at	TIMESTAMP TZ	8 bytes	Date and time when the operation completed. Optional.
database_validation	TEXT	—	Recorded database-restoration validation outcome. Allowed values: Pending, Passed, Failed. Optional.
storage_validation	TEXT	—	Recorded file-storage restoration validation outcome. Allowed values: Pending, Passed, Failed. Optional.
error_code	TEXT	—	Classified error associated with the operation. Allowed values: ConfigurationError, IntegrityValidationFailed, RestoreFailed, ValidationFailed, UnknownBackupError. Optional.
remarks	TEXT	—	Additional remarks associated with the record. Optional.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.

Table 68 describes the table in the database named `recovery_drills`, where non-production recovery-validation attempts are recorded separately from backup creation. A passed drill requires successful database and storage validation.

Table 69

Data Dictionary – Public Contact Settings

Attributes	Data Type	Field Size	Description
id	BOOLEAN	1 byte	Single-row primary-key value, restricted to true. Primary key. Required.
phone	TEXT	—	Published contact telephone number. Required.
email	TEXT	—	Published email address of the rental business. Required.
office_hours	TEXT	—	Published business or contact hours. Required.
location_summary	TEXT	—	Readable summary of the public contact locations. Required.
service_area	TEXT	—	Published description of the geographic service area. Required.
reply_commitment	TEXT	—	Published statement concerning replies to inquiries. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.
updated_by	UUID	16 bytes	User who last updated the record. FK to <code>public.profiles(id)</code> . Optional.

Table 69 describes the table in the database named `public_contact_settings`, where the published contact details, office hours, service-area information, and reply commitment are maintained in a single settings record.

Table 70

Data Dictionary – Public Contact Locations

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name of the public-facing contact location. Required.
address	TEXT	—	Published address of the contact location. Required.
note	TEXT	—	Optional explanatory note concerning the location. Optional.
sort_order	SMALLINT	2 bytes	Presentation position from 0 to 99; unique across public contact locations. Unique when present. Required.
is_active	BOOLEAN	1 byte	Indicates whether the record is active. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
updated_at	TIMESTAMP TZ	8 bytes	Date and time of the latest recorded update. Required.

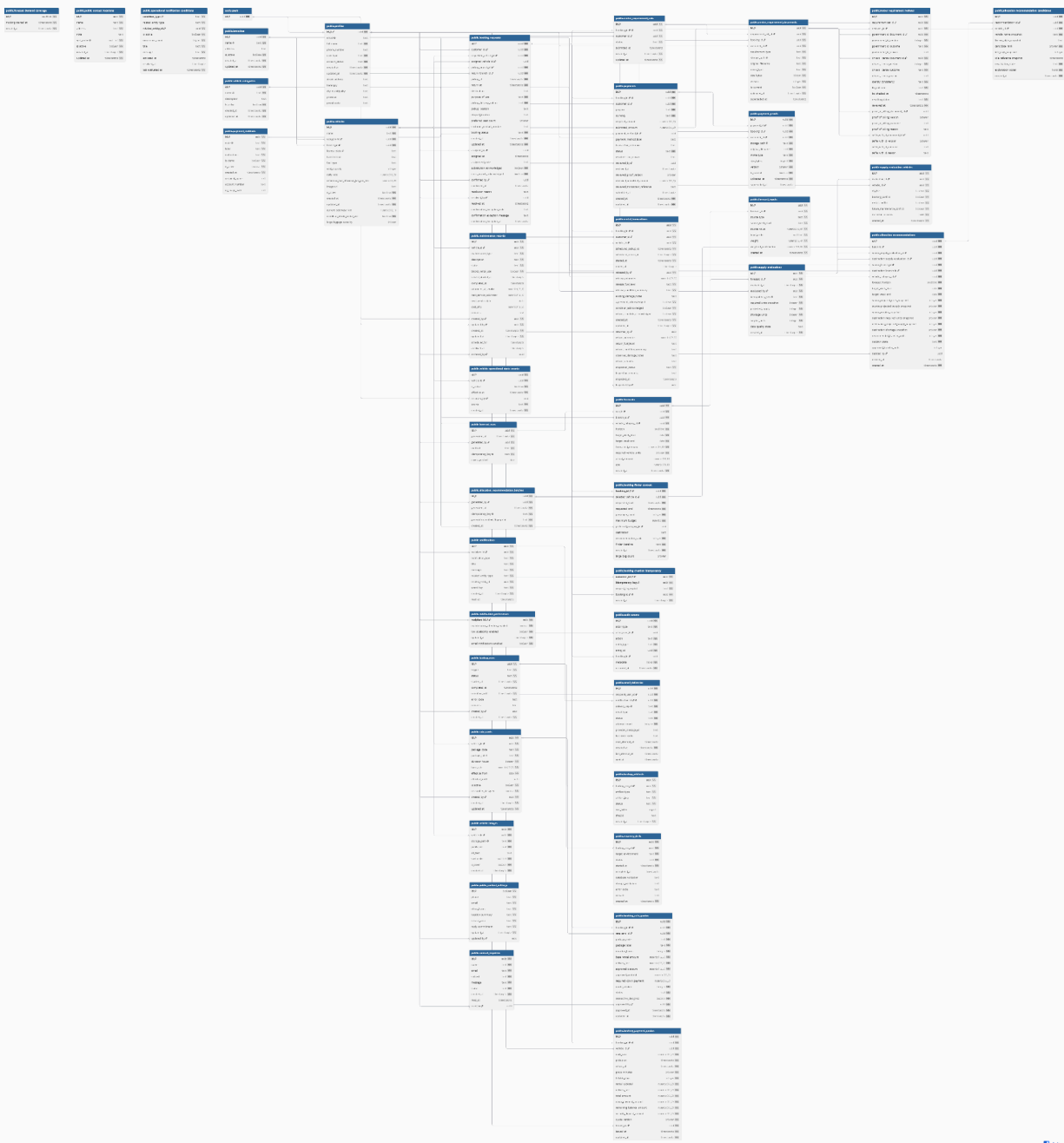
Table 70 describes the table in the database named public_contact_locations, where ordered public-facing contact locations are maintained separately from operational fleet branches. This distinction prevents public contact addresses from being treated as internal allocation records.

Table 71

Data Dictionary – Contact Inquiries

Attributes	Data Type	Field Size	Description
id	UUID	16 bytes	Unique identifier for the record. Primary key. Required.
name	TEXT	—	Name supplied by the inquiry sender. Required.
email	TEXT	—	Reply email address supplied by the inquiry sender. Required.
subject	TEXT	—	Subject supplied for the inquiry. Required.
message	TEXT	—	Inquiry message supplied by the sender. Required.
status	TEXT	—	Recorded state of the record. Allowed values: New, Read, Closed. Required.
created_at	TIMESTAMP TZ	8 bytes	Date and time when the record was created. Required.
read_at	TIMESTAMP TZ	8 bytes	Date and time when the message was marked as read. Optional.
read_by	UUID	16 bytes	User who marked the inquiry as read. FK to public.profiles(id). Optional.

Table 71 describes the table in the database named `contact_inquiries`, where public inquiry messages, sender details, review status, and the user who marked a message as read are recorded.



public.forecast_demand_coverage		
id 	smallint	NN
tracking_started_at	timestampz	NN
created_at	timestampz	NN

public.public_contact_locations		
id 	uuid	NN
name	text	NN
address	text	NN
note	text	
sort_order 	smallint	NN
is_active	boolean	NN
created_at	timestampz	NN
updated_at	timestampz	NN

public.operational_notification_conditions		
condition_type 	text	NN
related_entity_type	text	NN
related_entity_id 	uuid	NN
is_active	boolean	NN
occurrence_count	bigint	NN
title	text	NN
message	text	NN
activated_at	timestampz	
resolved_at	timestampz	
last_evaluated_at	timestampz	NN

auth.users	
id	uuid NN

public.branches	
id	uuid NN
name	text NN
address	text
is_active	boolean NN
created_at	timestampz NN
updated_at	timestampz NN

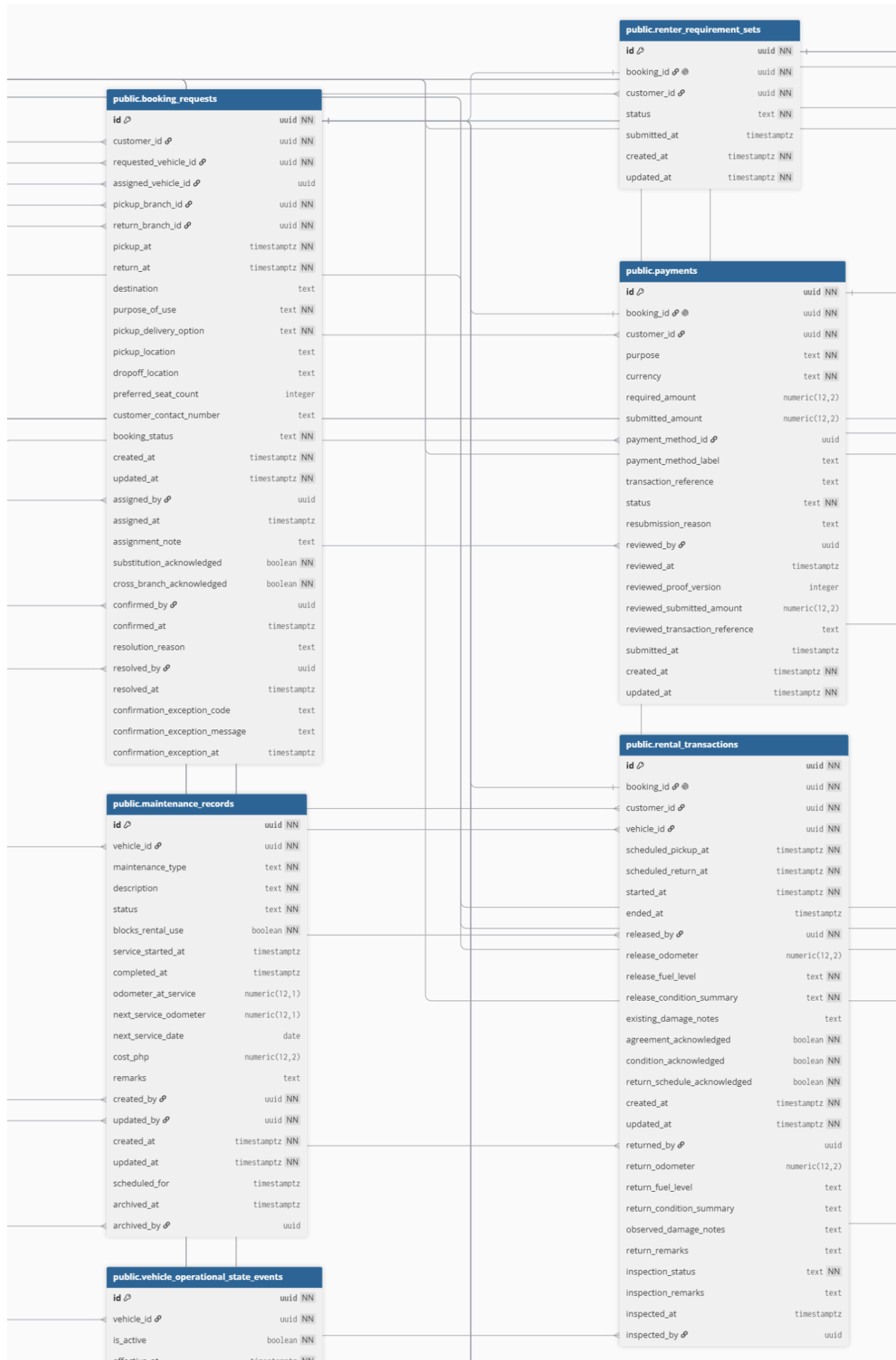
public.vehicle_categories	
id	uuid NN
name	text NN
description	text
is_active	boolean NN
created_at	timestampz NN
updated_at	timestampz NN

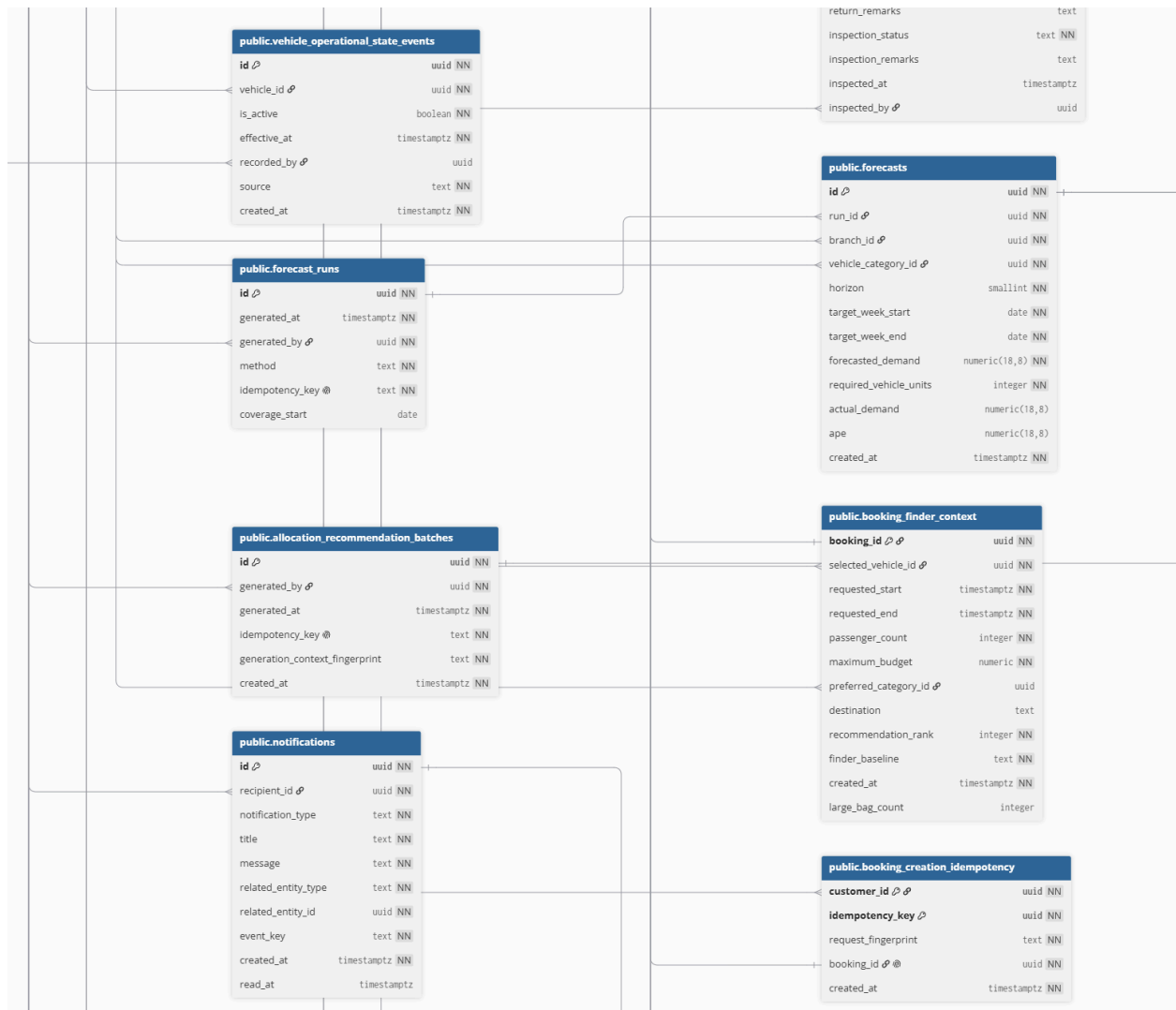
public.payment_methods	
id	uuid NN
code	text NN
label	text NN
instructions	text NN
is_demo	boolean NN
is_active	boolean NN
created_at	timestampz NN
recipient_name	text
account_number	text
qr_image_path	text

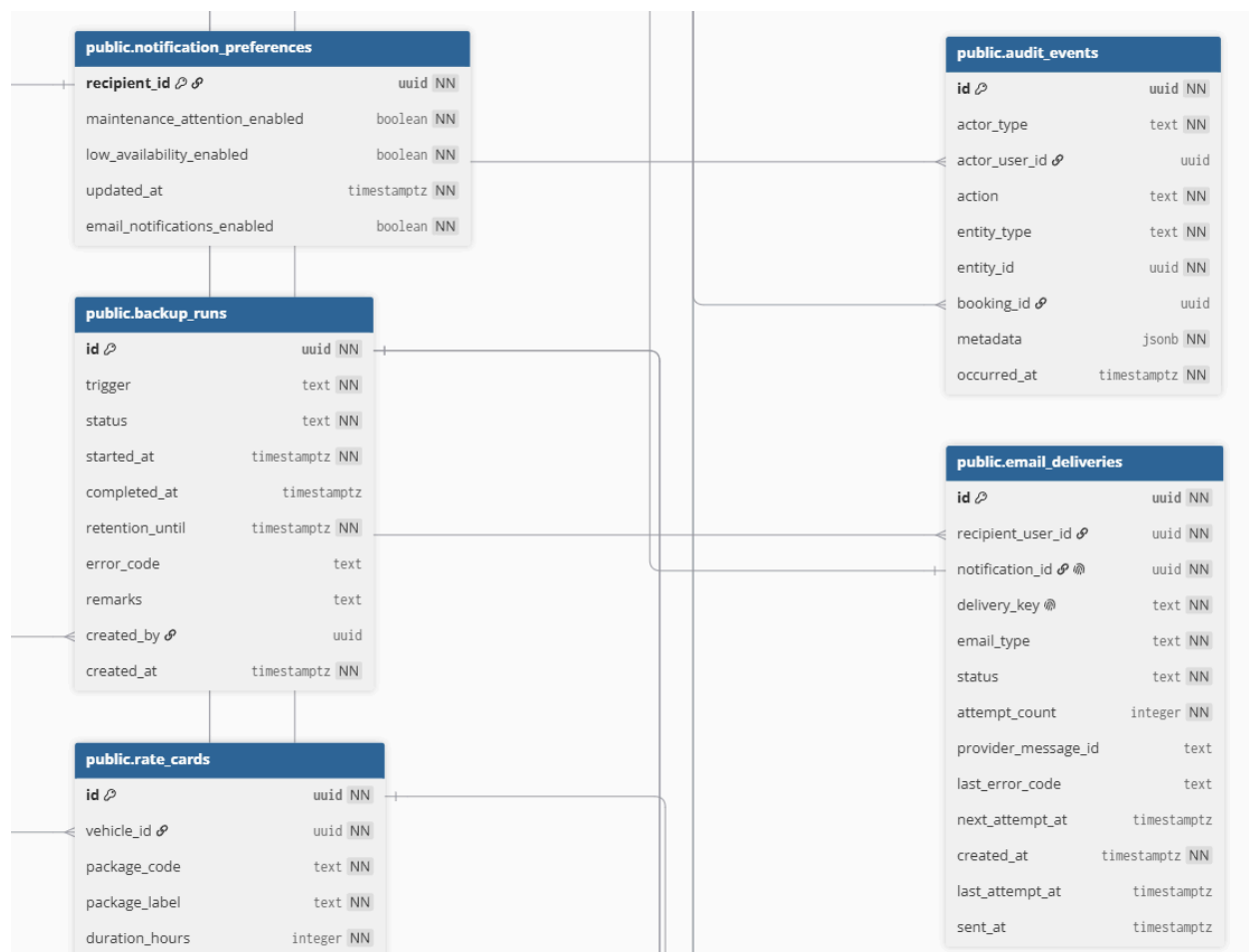
public.profiles	
id	uuid NN
email	text
full_name	text NN
phone_number	text
user_type	text NN
account_status	text NN
created_at	timestampz NN
updated_at	timestampz NN
street_address	text
barangay	text
city_municipality	text
province	text
postal_code	text

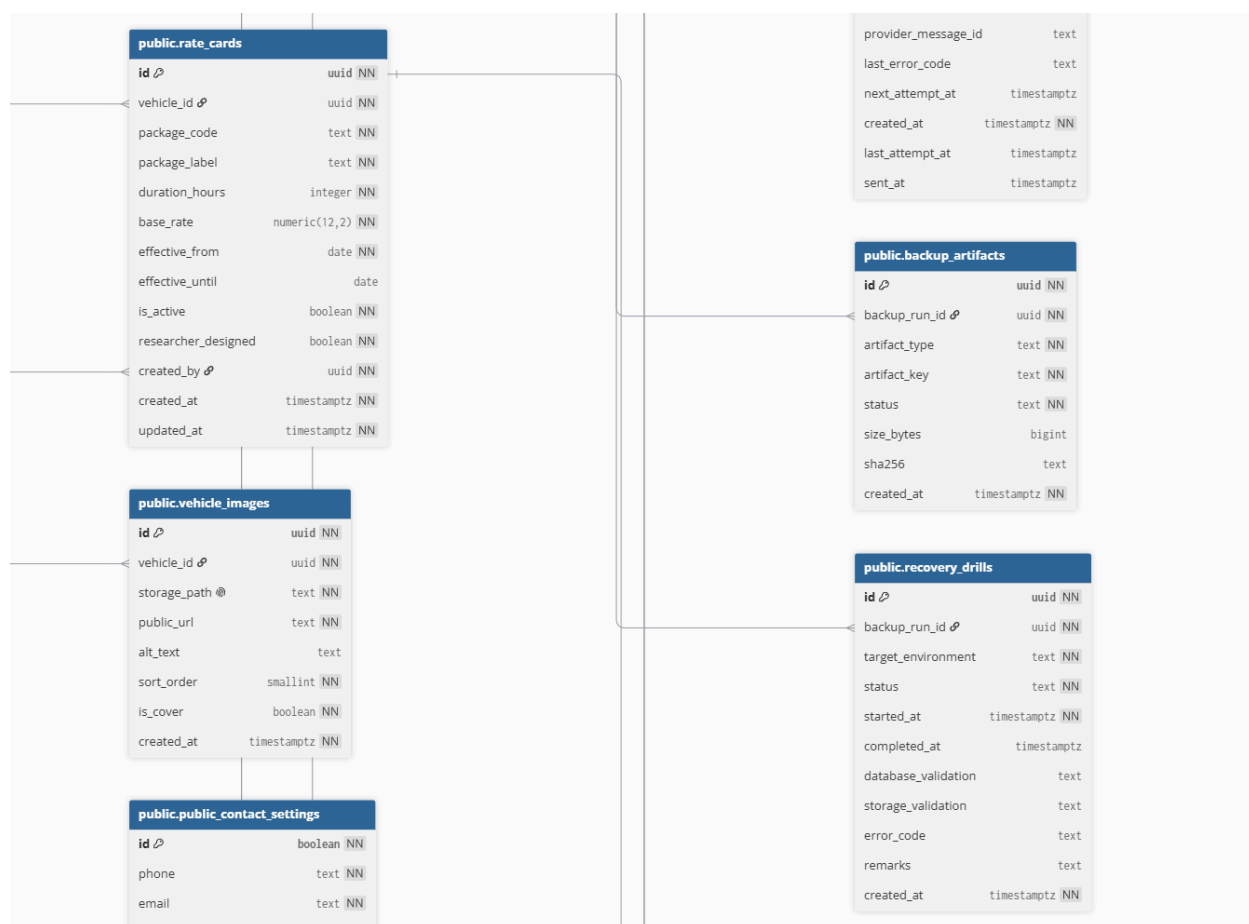
public.vehicles	
id	uuid NN
name	text NN
category_id	uuid NN
branch_id	uuid NN
license_plate	text
transmission	text
fuel_type	text
seat_capacity	integer
daily_rate	numeric(12,2)
reference_fuel_efficiency_km_per_liter	numeric(8,2)
image_url	text
is_active	boolean NN
created_at	timestampz NN
updated_at	timestampz NN
current_odometer_km	numeric(12,1)
condition_blocks_rental_use	boolean NN
large_luggage_capacity	integer

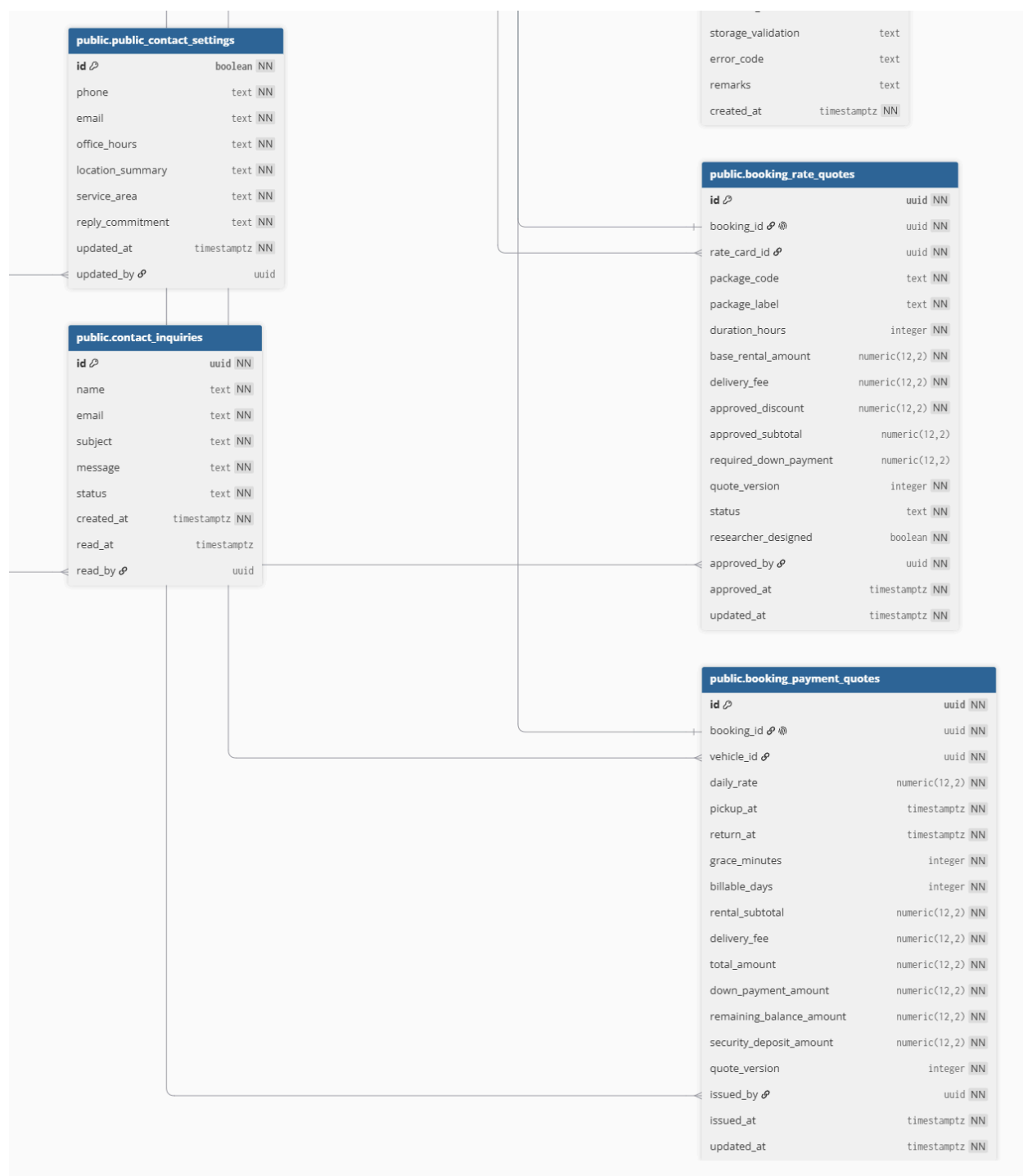


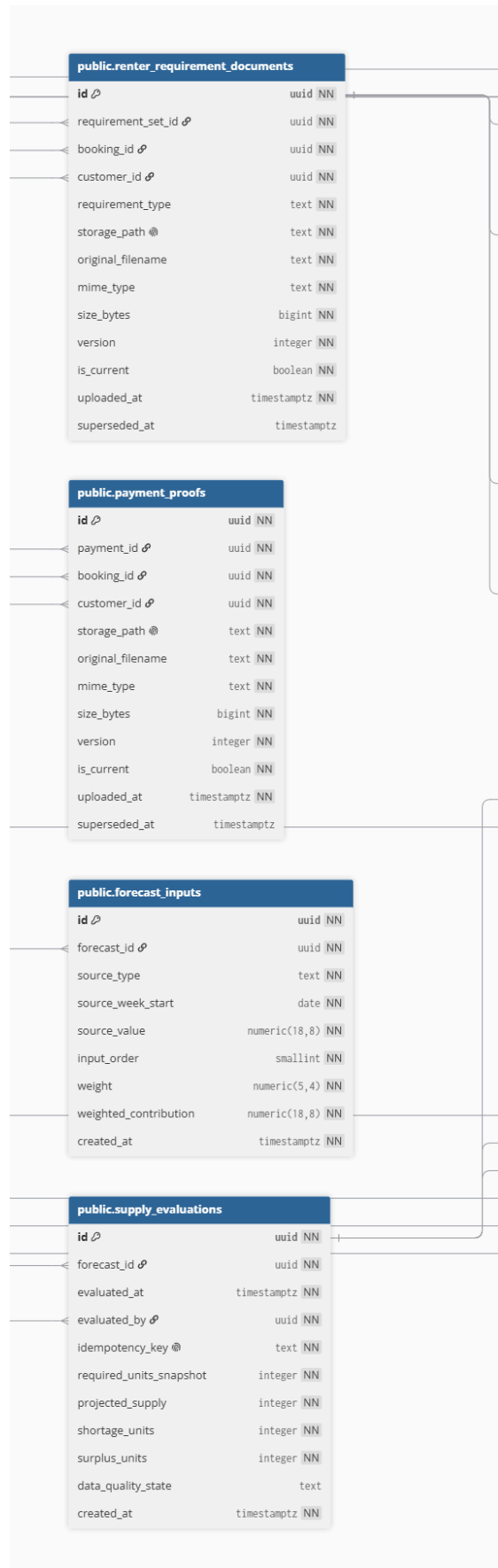


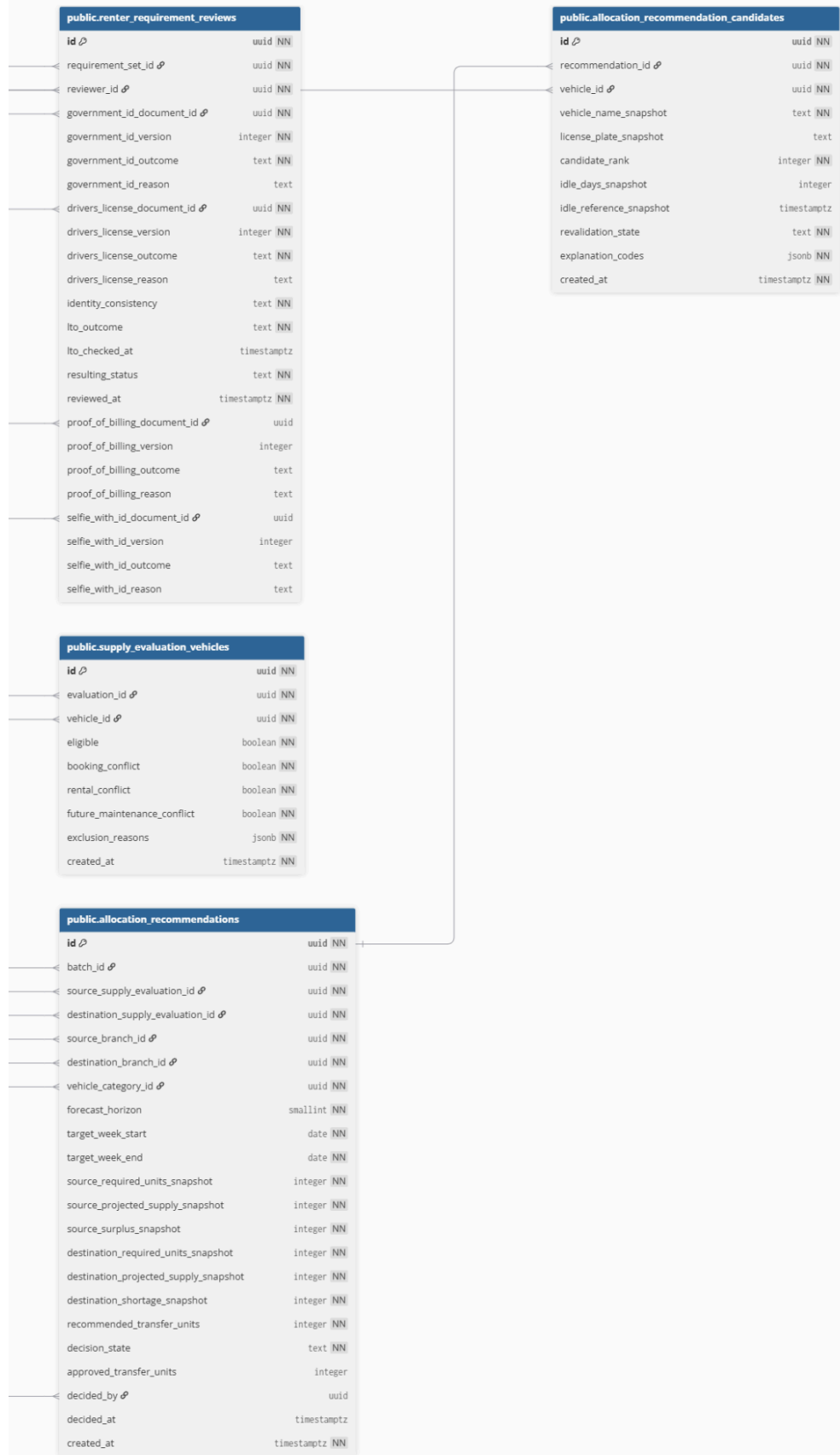












// Paste this entire file into the dbdiagram.io DBML editor.

// Repository baseline: 02705878169422ae1d7a01d937a9a9680c99bf95

// Diagram representation: defaults, generated expressions, checks,

// partial indexes, triggers, functions, and RLS are omitted.

Project briah_car_rental {

 database_type: 'PostgreSQL'

 Note: 'Database schema for the Web-Based Car Rental Management System for Briah Car Rental. Forty public tables; auth.users is an external authentication reference.'

}

Table auth.users {

 id uuid [pk, not null]

 Note: 'External authentication reference. Only the referenced primary key is shown.'

}

Table public.profiles {

 id uuid [pk, not null]

 email text [null, unique]

 full_name text [not null]

 phone_number text [null]

 user_type text [not null]

 account_status text [not null]

 created_at timestamptz [not null]

 updated_at timestamptz [not null]

 street_address text [null]

```
barangay text [null]
city_municipality text [null]
province text [null]
postal_code text [null]
}
```

```
Table public.branches {
  id uuid [pk, not null]
  name text [not null, unique]
  address text [null]
  is_active boolean [not null]
  created_at timestamptz [not null]
  updated_at timestamptz [not null]
}
```

```
Table public.vehicle_categories {
  id uuid [pk, not null]
  name text [not null, unique]
  description text [null]
  is_active boolean [not null]
  created_at timestamptz [not null]
  updated_at timestamptz [not null]
}
```

```
Table public.vehicles {
  id uuid [pk, not null]
```



```
name text [not null]
category_id uuid [not null]
branch_id uuid [not null]
license_plate text [null, unique]
transmission text [null]
fuel_type text [null]
seat_capacity integer [null]
daily_rate numeric(12,2) [null]
reference_fuel_efficiency_km_per_liter numeric(8,2) [null]
image_url text [null]
is_active boolean [not null]
created_at timestamptz [not null]
updated_at timestamptz [not null]
current_odometer_km numeric(12,1) [null]
condition_blocks_rental_use boolean [not null]
large_luggage_capacity integer [null]
}
```

```
Table public.booking_requests {
  id uuid [pk, not null]
  customer_id uuid [not null]
  requested_vehicle_id uuid [not null]
  assigned_vehicle_id uuid [null]
  pickup_branch_id uuid [not null]
  return_branch_id uuid [not null]
  pickup_at timestamptz [not null]
```

```
return_at timestampz [not null]
destination text [null]
purpose_of_use text [not null]
pickup_delivery_option text [not null]
pickup_location text [null]
dropoff_location text [null]
preferred_seat_count integer [null]
customer_contact_number text [null]
booking_status text [not null]
created_at timestampz [not null]
updated_at timestampz [not null]
assigned_by uuid [null]
assigned_at timestampz [null]
assignment_note text [null]
substitution_acknowledged boolean [not null]
cross_branch_acknowledged boolean [not null]
confirmed_by uuid [null]
confirmed_at timestampz [null]
resolution_reason text [null]
resolved_by uuid [null]
resolved_at timestampz [null]
confirmation_exception_code text [null]
confirmation_exception_message text [null]
confirmation_exception_at timestampz [null]
}
```

```
Table public.renter_requirement_sets {  
  id uuid [pk, not null]  
  booking_id uuid [not null, unique]  
  customer_id uuid [not null]  
  status text [not null]  
  submitted_at timestamptz [null]  
  created_at timestamptz [not null]  
  updated_at timestamptz [not null]  
}
```

```
Table public.renter_requirement_documents {  
  id uuid [pk, not null]  
  requirement_set_id uuid [not null]  
  booking_id uuid [not null]  
  customer_id uuid [not null]  
  requirement_type text [not null]  
  storage_path text [not null, unique]  
  original_filename text [not null]  
  mime_type text [not null]  
  size_bytes bigint [not null]  
  version integer [not null]  
  is_current boolean [not null]  
  uploaded_at timestamptz [not null]  
  superseded_at timestamptz [null]  
}
```

```
Table public.renter_requirement_reviews {  
  id uuid [pk, not null]  
  requirement_set_id uuid [not null]  
  reviewer_id uuid [not null]  
  government_id_document_id uuid [not null]  
  government_id_version integer [not null]  
  government_id_outcome text [not null]  
  government_id_reason text [null]  
  drivers_license_document_id uuid [not null]  
  drivers_license_version integer [not null]  
  drivers_license_outcome text [not null]  
  drivers_license_reason text [null]  
  identity_consistency text [not null]  
  lto_outcome text [not null]  
  lto_checked_at timestampz [null]  
  resulting_status text [not null]  
  reviewed_at timestampz [not null]  
  proof_of_billing_document_id uuid [null]  
  proof_of_billing_version integer [null]  
  proof_of_billing_outcome text [null]  
  proof_of_billing_reason text [null]  
  selfie_with_id_document_id uuid [null]  
  selfie_with_id_version integer [null]  
  selfie_with_id_outcome text [null]  
  selfie_with_id_reason text [null]  
}
```

```
Table public.payment_methods {  
  id uuid [pk, not null]  
  code text [not null, unique]  
  label text [not null]  
  instructions text [not null]  
  is_demo boolean [not null]  
  is_active boolean [not null]  
  created_at timestamptz [not null]  
  recipient_name text [null]  
  account_number text [null]  
  qr_image_path text [null]  
}
```

```
Table public.payments {  
  id uuid [pk, not null]  
  booking_id uuid [not null, unique]  
  customer_id uuid [not null]  
  purpose text [not null]  
  currency text [not null]  
  required_amount numeric(12,2) [null]  
  submitted_amount numeric(12,2) [null]  
  payment_method_id uuid [null]  
  payment_method_label text [null]  
  transaction_reference text [null]  
  status text [not null]
```

```
resubmission_reason text [null]
reviewed_by uuid [null]
reviewed_at timestampz [null]
reviewed_proof_version integer [null]
reviewed_submitted_amount numeric(12,2) [null]
reviewed_transaction_reference text [null]
submitted_at timestampz [null]
created_at timestampz [not null]
updated_at timestampz [not null]
}
```

```
Table public.payment_proofs {
  id uuid [pk, not null]
  payment_id uuid [not null]
  booking_id uuid [not null]
  customer_id uuid [not null]
  storage_path text [not null, unique]
  original_filename text [not null]
  mime_type text [not null]
  size_bytes bigint [not null]
  version integer [not null]
  is_current boolean [not null]
  uploaded_at timestampz [not null]
  superseded_at timestampz [null]
}
```

```
Table public.rental_transactions {  
  id uuid [pk, not null]  
  booking_id uuid [not null, unique]  
  customer_id uuid [not null]  
  vehicle_id uuid [not null]  
  scheduled_pickup_at timestamptz [not null]  
  scheduled_return_at timestamptz [not null]  
  started_at timestamptz [not null]  
  ended_at timestamptz [null]  
  released_by uuid [not null]  
  release_odometer numeric(12,2) [null]  
  release_fuel_level text [not null]  
  release_condition_summary text [not null]  
  existing_damage_notes text [null]  
  agreement_acknowledged boolean [not null]  
  condition_acknowledged boolean [not null]  
  return_schedule_acknowledged boolean [not null]  
  created_at timestamptz [not null]  
  updated_at timestamptz [not null]  
  returned_by uuid [null]  
  return_odometer numeric(12,2) [null]  
  return_fuel_level text [null]  
  return_condition_summary text [null]  
  observed_damage_notes text [null]  
  return_remarks text [null]  
  inspection_status text [not null]
```

```
inspection_remarks text [null]
inspected_at timestamptz [null]
inspected_by uuid [null]
}
```

```
Table public.maintenance_records {
  id uuid [pk, not null]
  vehicle_id uuid [not null]
  maintenance_type text [not null]
  description text [not null]
  status text [not null]
  blocks_rental_use boolean [not null]
  service_started_at timestamptz [null]
  completed_at timestamptz [null]
  odometer_at_service numeric(12,1) [null]
  next_service_odometer numeric(12,1) [null]
  next_service_date date [null]
  cost_php numeric(12,2) [null]
  remarks text [null]
  created_by uuid [not null]
  updated_by uuid [not null]
  created_at timestamptz [not null]
  updated_at timestamptz [not null]
  scheduled_for timestamptz [null]
  archived_at timestamptz [null]
  archived_by uuid [null]
```



```
}
```

```
Table public.vehicle_operational_state_events {
```

```
  id uuid [pk, not null]
```

```
  vehicle_id uuid [not null]
```

```
  is_active boolean [not null]
```

```
  effective_at timestamptz [not null]
```

```
  recorded_by uuid [null]
```

```
  source text [not null]
```

```
  created_at timestamptz [not null]
```

```
}
```

```
Table public.forecast_runs {
```

```
  id uuid [pk, not null]
```

```
  generated_at timestamptz [not null]
```

```
  generated_by uuid [not null]
```

```
  method text [not null]
```

```
  idempotency_key text [not null, unique]
```

```
  coverage_start date [null]
```

```
}
```

```
Table public.forecasts {
```

```
  id uuid [pk, not null]
```

```
  run_id uuid [not null]
```

```
  branch_id uuid [not null]
```

```
  vehicle_category_id uuid [not null]
```

```
horizon smallint [not null]
target_week_start date [not null]
target_week_end date [not null]
forecasted_demand numeric(18,8) [not null]
required_vehicle_units integer [not null]
actual_demand numeric(18,8) [null]
ape numeric(18,8) [null]
created_at timestamptz [not null]
}
```

```
Table public.forecast_inputs {
  id uuid [pk, not null]
  forecast_id uuid [not null]
  source_type text [not null]
  source_week_start date [not null]
  source_value numeric(18,8) [not null]
  input_order smallint [not null]
  weight numeric(5,4) [not null]
  weighted_contribution numeric(18,8) [not null]
  created_at timestamptz [not null]
}
```

```
Table public.forecast_demand_coverage {
  id smallint [pk, not null]
  tracking_started_at timestamptz [not null]
  created_at timestamptz [not null]
```

```
}
```

```
Table public.supply_evaluations {
```

```
  id uuid [pk, not null]
```

```
  forecast_id uuid [not null]
```

```
  evaluated_at timestamptz [not null]
```

```
  evaluated_by uuid [not null]
```

```
  idempotency_key text [not null, unique]
```

```
  required_units_snapshot integer [not null]
```

```
  projected_supply integer [not null]
```

```
  shortage_units integer [not null]
```

```
  surplus_units integer [not null]
```

```
  data_quality_state text [null]
```

```
  created_at timestamptz [not null]
```

```
}
```

```
Table public.supply_evaluation_vehicles {
```

```
  id uuid [pk, not null]
```

```
  evaluation_id uuid [not null]
```

```
  vehicle_id uuid [not null]
```

```
  eligible boolean [not null]
```

```
  booking_conflict boolean [not null]
```

```
  rental_conflict boolean [not null]
```

```
  future_maintenance_conflict boolean [not null]
```

```
  exclusion_reasons jsonb [not null]
```

```
  created_at timestamptz [not null]
```

```
indexes {  
  (evaluation_id, vehicle_id) [unique]  
}  
}
```

```
Table public.allocation_recommendation_batches {  
  id uuid [pk, not null]  
  generated_by uuid [not null]  
  generated_at timestamptz [not null]  
  idempotency_key text [not null, unique]  
  generation_context_fingerprint text [not null]  
  created_at timestamptz [not null]  
}
```

```
Table public.allocation_recommendations {  
  id uuid [pk, not null]  
  batch_id uuid [not null]  
  source_supply_evaluation_id uuid [not null]  
  destination_supply_evaluation_id uuid [not null]  
  source_branch_id uuid [not null]  
  destination_branch_id uuid [not null]  
  vehicle_category_id uuid [not null]  
  forecast_horizon smallint [not null]  
  target_week_start date [not null]  
  target_week_end date [not null]  
  source_required_units_snapshot integer [not null]
```

```
source_projected_supply_snapshot integer [not null]
source_surplus_snapshot integer [not null]
destination_required_units_snapshot integer [not null]
destination_projected_supply_snapshot integer [not null]
destination_shortage_snapshot integer [not null]
recommended_transfer_units integer [not null]
decision_state text [not null]
approved_transfer_units integer [null]
decided_by uuid [null]
decided_at timestamptz [null]
created_at timestamptz [not null]
indexes {
  (batch_id, source_supply_evaluation_id, destination_supply_evaluation_id) [unique]
}
}
```

Table public.allocation_recommendation_candidates {

```
id uuid [pk, not null]
recommendation_id uuid [not null]
vehicle_id uuid [not null]
vehicle_name_snapshot text [not null]
license_plate_snapshot text [null]
candidate_rank integer [not null]
idle_days_snapshot integer [null]
idle_reference_snapshot timestamptz [null]
revalidation_state text [not null]
```

```
explanation_codes jsonb [not null]
created_at timestamptz [not null]
indexes {
    (recommendation_id, vehicle_id) [unique]
    (recommendation_id, candidate_rank) [unique]
}
}
```

```
Table public.booking_finder_context {
    booking_id uuid [pk, not null]
    selected_vehicle_id uuid [not null]
    requested_start timestamptz [not null]
    requested_end timestamptz [not null]
    passenger_count integer [not null]
    maximum_budget numeric [not null]
    preferred_category_id uuid [null]
    destination text [null]
    recommendation_rank integer [not null]
    finder_baseline text [not null]
    created_at timestamptz [not null]
    large_bag_count integer [null]
}
```

```
Table public.booking_creation_idempotency {
    customer_id uuid [not null]
    idempotency_key uuid [not null]
```

```
request_fingerprint text [not null]
booking_id uuid [not null, unique]
created_at timestamptz [not null]
indexes {
  (customer_id, idempotency_key) [pk]
}
}
```

```
Table public.notifications {
  id uuid [pk, not null]
  recipient_id uuid [not null]
  notification_type text [not null]
  title text [not null]
  message text [not null]
  related_entity_type text [not null]
  related_entity_id uuid [not null]
  event_key text [not null]
  created_at timestamptz [not null]
  read_at timestamptz [null]
  indexes {
    (recipient_id, event_key) [unique]
  }
}
```

```
Table public.audit_events {
  id uuid [pk, not null]
```

```
actor_type text [not null]
actor_user_id uuid [null]
action text [not null]
entity_type text [not null]
entity_id uuid [not null]
booking_id uuid [null]
metadata jsonb [not null]
occurred_at timestamptz [not null]
}
```

```
Table public.notification_preferences {
  recipient_id uuid [pk, not null]
  maintenance_attention_enabled boolean [not null]
  low_availability_enabled boolean [not null]
  updated_at timestamptz [not null]
  email_notifications_enabled boolean [not null]
}
```

```
Table public.operational_notification_conditions {
  condition_type text [not null]
  related_entity_type text [not null]
  related_entity_id uuid [not null]
  is_active boolean [not null]
  occurrence_count bigint [not null]
  title text [not null]
  message text [not null]
```



```
    activated_at timestamptz [null]
    resolved_at timestamptz [null]
    last_evaluated_at timestamptz [not null]
    indexes {
      (condition_type, related_entity_id) [pk]
    }
  }
```

```
Table public.email_deliveries {
  id uuid [pk, not null]
  recipient_user_id uuid [not null]
  notification_id uuid [not null, unique]
  delivery_key text [not null, unique]
  email_type text [not null]
  status text [not null]
  attempt_count integer [not null]
  provider_message_id text [null]
  last_error_code text [null]
  next_attempt_at timestamptz [null]
  created_at timestamptz [not null]
  last_attempt_at timestamptz [null]
  sent_at timestamptz [null]
}
```

```
Table public.backup_runs {
  id uuid [pk, not null]
```

```
trigger text [not null]
status text [not null]
started_at timestamptz [not null]
completed_at timestamptz [null]
retention_until timestamptz [not null]
error_code text [null]
remarks text [null]
created_by uuid [null]
created_at timestamptz [not null]
}
```

```
Table public.backup_artifacts {
  id uuid [pk, not null]
  backup_run_id uuid [not null]
  artifact_type text [not null]
  artifact_key text [not null]
  status text [not null]
  size_bytes bigint [null]
  sha256 text [null]
  created_at timestamptz [not null]
  indexes {
    (backup_run_id, artifact_key) [unique]
  }
}
```

```
Table public.recovery_drills {
```

```
id uuid [pk, not null]
backup_run_id uuid [not null]
target_environment text [not null]
status text [not null]
started_at timestamptz [not null]
completed_at timestamptz [null]
database_validation text [null]
storage_validation text [null]
error_code text [null]
remarks text [null]
created_at timestamptz [not null]
}
```

```
Table public.rate_cards {
  id uuid [pk, not null]
  vehicle_id uuid [not null]
  package_code text [not null]
  package_label text [not null]
  duration_hours integer [not null]
  base_rate numeric(12,2) [not null]
  effective_from date [not null]
  effective_until date [null]
  is_active boolean [not null]
  researcher_designed boolean [not null]
  created_by uuid [not null]
  created_at timestamptz [not null]
```

```
updated_at timestamptz [not null]

indexes {
  (vehicle_id, package_code, effective_from) [unique]
}
}
```

```
Table public.booking_rate_quotes {
  id uuid [pk, not null]
  booking_id uuid [not null, unique]
  rate_card_id uuid [not null]
  package_code text [not null]
  package_label text [not null]
  duration_hours integer [not null]
  base_rental_amount numeric(12,2) [not null]
  delivery_fee numeric(12,2) [not null]
  approved_discount numeric(12,2) [not null]
  approved_subtotal numeric(12,2) [null]
  required_down_payment numeric(12,2) [null]
  quote_version integer [not null]
  status text [not null]
  researcher_designed boolean [not null]
  approved_by uuid [not null]
  approved_at timestamptz [not null]
  updated_at timestamptz [not null]
}
```

```
Table public.vehicle_images {  
  id uuid [pk, not null]  
  vehicle_id uuid [not null]  
  storage_path text [not null, unique]  
  public_url text [not null]  
  alt_text text [null]  
  sort_order smallint [not null]  
  is_cover boolean [not null]  
  created_at timestamptz [not null]  
  indexes {  
    (vehicle_id, sort_order) [unique]  
  }  
}
```

```
Table public.public_contact_settings {  
  id boolean [pk, not null]  
  phone text [not null]  
  email text [not null]  
  office_hours text [not null]  
  location_summary text [not null]  
  service_area text [not null]  
  reply_commitment text [not null]  
  updated_at timestamptz [not null]  
  updated_by uuid [null]  
}
```

```
Table public.public_contact_locations {  
  id uuid [pk, not null]  
  name text [not null]  
  address text [not null]  
  note text [null]  
  sort_order smallint [not null, unique]  
  is_active boolean [not null]  
  created_at timestamptz [not null]  
  updated_at timestamptz [not null]  
}
```

```
Table public.contact_inquiries {  
  id uuid [pk, not null]  
  name text [not null]  
  email text [not null]  
  subject text [not null]  
  message text [not null]  
  status text [not null]  
  created_at timestamptz [not null]  
  read_at timestamptz [null]  
  read_by uuid [null]  
}
```

```
Table public.booking_payment_quotes {  
  id uuid [pk, not null]  
  booking_id uuid [not null, unique]
```

```
vehicle_id uuid [not null]
daily_rate numeric(12,2) [not null]
pickup_at timestamptz [not null]
return_at timestamptz [not null]
grace_minutes integer [not null]
billable_days integer [not null]
rental_subtotal numeric(12,2) [not null]
delivery_fee numeric(12,2) [not null]
total_amount numeric(12,2) [not null]
down_payment_amount numeric(12,2) [not null]
remaining_balance_amount numeric(12,2) [not null]
security_deposit_amount numeric(12,2) [not null]
quote_version integer [not null]
issued_by uuid [not null]
issued_at timestamptz [not null]
updated_at timestamptz [not null]
}
```

```
// Declared foreign-key relationships
```

```
Ref profiles_id_fkey: auth.users.id - public.profiles.id [delete: cascade]
```

```
Ref vehicles_category_id_fkey: public.vehicles.category_id > public.vehicle_categories.id
[delete: no action]
```

```
Ref vehicles_branch_id_fkey: public.vehicles.branch_id > public.branches.id [delete: no action]
```

```
Ref booking_requests_customer_id_fkey: public.booking_requests.customer_id >
public.profiles.id [delete: restrict]
```

Ref booking_requests_requested_vehicle_id_fkey:
public.booking_requests.requested_vehicle_id > public.vehicles.id [delete: restrict]

Ref booking_requests_assigned_vehicle_id_fkey: public.booking_requests.assigned_vehicle_id
> public.vehicles.id [delete: restrict]

Ref booking_requests_pickup_branch_id_fkey: public.booking_requests.pickup_branch_id >
public.branches.id [delete: restrict]

Ref booking_requests_return_branch_id_fkey: public.booking_requests.return_branch_id >
public.branches.id [delete: restrict]

Ref booking_requests_assigned_by_fkey: public.booking_requests.assigned_by >
public.profiles.id [delete: restrict]

Ref booking_requests_confirmed_by_fkey: public.booking_requests.confirmed_by >
public.profiles.id [delete: restrict]

Ref booking_requests_resolved_by_fkey: public.booking_requests.resolved_by >
public.profiles.id [delete: restrict]

Ref renter_requirement_sets_booking_id_fkey: public.booking_requests.id -
public.renter_requirement_sets.booking_id [delete: restrict]

Ref renter_requirement_sets_customer_id_fkey: public.renter_requirement_sets.customer_id >
public.profiles.id [delete: restrict]

Ref renter_requirement_documents_requirement_set_id_fkey:
public.renter_requirement_documents.requirement_set_id > public.renter_requirement_sets.id
[delete: restrict]

Ref renter_requirement_documents_booking_id_fkey:
public.renter_requirement_documents.booking_id > public.booking_requests.id [delete: restrict]

Ref renter_requirement_documents_customer_id_fkey:
public.renter_requirement_documents.customer_id > public.profiles.id [delete: restrict]

Ref payment_proofs_payment_id_fkey: public.payment_proofs.payment_id >
public.payments.id [delete: restrict]

Ref payment_proofs_booking_id_fkey: public.payment_proofs.booking_id >
public.booking_requests.id [delete: restrict]

Ref payment_proofs_customer_id_fkey: public.payment_proofs.customer_id > public.profiles.id
[delete: restrict]

Ref rental_transactions_booking_id_fkey: public.booking_requests.id -
public.rental_transactions.booking_id [delete: restrict]

Ref rental_transactions_customer_id_fkey: public.rental_transactions.customer_id >
public.profiles.id [delete: restrict]

Ref rental_transactions_vehicle_id_fkey: public.rental_transactions.vehicle_id >
public.vehicles.id [delete: restrict]

Ref rental_transactions_released_by_fkey: public.rental_transactions.released_by >
public.profiles.id [delete: restrict]

Ref rental_transactions_returned_by_fkey: public.rental_transactions.returned_by >
public.profiles.id [delete: restrict]

Ref rental_transactions_inspected_by_fkey: public.rental_transactions.inspected_by >
public.profiles.id [delete: restrict]

Ref maintenance_records_vehicle_id_fkey: public.maintenance_records.vehicle_id >
public.vehicles.id [delete: no action]

Ref maintenance_records_created_by_fkey: public.maintenance_records.created_by >
auth.users.id [delete: no action]

Ref maintenance_records_updated_by_fkey: public.maintenance_records.updated_by >
auth.users.id [delete: no action]

Ref maintenance_records_archived_by_fkey: public.maintenance_records.archived_by >
public.profiles.id [delete: restrict]

Ref vehicle_operational_state_events_vehicle_id_fkey:
public.vehicle_operational_state_events.vehicle_id > public.vehicles.id [delete: cascade]

Ref vehicle_operational_state_events_recorded_by_fkey:
public.vehicle_operational_state_events.recorded_by > auth.users.id [delete: no action]

Ref forecast_runs_generated_by_fkey: public.forecast_runs.generated_by > public.profiles.id
[delete: restrict]

Ref forecasts_run_id_fkey: public.forecasts.run_id > public.forecast_runs.id [delete: restrict]

Ref forecasts_branch_id_fkey: public.forecasts.branch_id > public.branches.id [delete: restrict]

Ref forecasts_vehicle_category_id_fkey: public.forecasts.vehicle_category_id >
public.vehicle_categories.id [delete: restrict]

Ref forecast_inputs_forecast_id_fkey: public.forecast_inputs.forecast_id > public.forecasts.id
[delete: restrict]

Ref supply_evaluations_forecast_id_fkey: public.supply_evaluations.forecast_id >
public.forecasts.id [delete: restrict]

Ref supply_evaluations_evaluated_by_fkey: public.supply_evaluations.evaluated_by >
public.profiles.id [delete: restrict]

Ref supply_evaluation_vehicles_evaluation_id_fkey:
public.supply_evaluation_vehicles.evaluation_id > public.supply_evaluations.id [delete: restrict]

Ref supply_evaluation_vehicles_vehicle_id_fkey: public.supply_evaluation_vehicles.vehicle_id
> public.vehicles.id [delete: restrict]

Ref allocation_recommendation_batches_generated_by_fkey:
public.allocation_recommendation_batches.generated_by > public.profiles.id [delete: restrict]

Ref allocation_recommendations_batch_id_fkey: public.allocation_recommendations.batch_id >
public.allocation_recommendation_batches.id [delete: restrict]

Ref allocation_recommendations_source_supply_evaluation_id_fkey:
public.allocation_recommendations.source_supply_evaluation_id > public.supply_evaluations.id
[delete: restrict]

Ref allocation_recommendations_destination_supply_evaluation_id_fkey:
public.allocation_recommendations.destination_supply_evaluation_id >
public.supply_evaluations.id [delete: restrict]

Ref allocation_recommendations_source_branch_id_fkey:
public.allocation_recommendations.source_branch_id > public.branches.id [delete: restrict]

Ref allocation_recommendations_destination_branch_id_fkey:
public.allocation_recommendations.destination_branch_id > public.branches.id [delete: restrict]

Ref allocation_recommendations_vehicle_category_id_fkey:
public.allocation_recommendations.vehicle_category_id > public.vehicle_categories.id [delete:
restrict]

Ref allocation_recommendations_decided_by_fkey:
public.allocation_recommendations.decided_by > public.profiles.id [delete: restrict]

Ref allocation_recommendation_candidates_recommendation_id_fkey:
public.allocation_recommendation_candidates.recommendation_id >
public.allocation_recommendations.id [delete: restrict]

Ref allocation_recommendation_candidates_vehicle_id_fkey:
public.allocation_recommendation_candidates.vehicle_id > public.vehicles.id [delete: restrict]

Ref booking_finder_context_booking_id_fkey: public.booking_requests.id -
public.booking_finder_context.booking_id [delete: restrict]

Ref booking_finder_context_selected_vehicle_id_fkey:
public.booking_finder_context.selected_vehicle_id > public.vehicles.id [delete: restrict]

Ref booking_finder_context_preferred_category_id_fkey:
public.booking_finder_context.preferred_category_id > public.vehicle_categories.id [delete:
restrict]

Ref booking_creation_idempotency_customer_id_fkey:
public.booking_creation_idempotency.customer_id > public.profiles.id [delete: restrict]

Ref booking_creation_idempotency_booking_id_fkey: public.booking_requests.id -
public.booking_creation_idempotency.booking_id [delete: restrict]

Ref notifications_recipient_id_fkey: public.notifications.recipient_id > public.profiles.id [delete:
restrict]

Ref audit_events_actor_user_id_fkey: public.audit_events.actor_user_id > public.profiles.id
[delete: restrict]

Ref audit_events_booking_id_fkey: public.audit_events.booking_id > public.booking_requests.id
[delete: restrict]

Ref notification_preferences_recipient_id_fkey: public.profiles.id -
public.notification_preferences.recipient_id [delete: cascade]

Ref email_deliveries_recipient_user_id_fkey: public.email_deliveries.recipient_user_id >
public.profiles.id [delete: restrict]

Ref email_deliveries_notification_id_fkey: public.notifications.id -
public.email_deliveries.notification_id [delete: restrict]

Ref backup_runs_created_by_fkey: public.backup_runs.created_by > public.profiles.id [delete:
restrict]

Ref backup_artifacts_backup_run_id_fkey: public.backup_artifacts.backup_run_id >
public.backup_runs.id [delete: cascade]

Ref recovery_drills_backup_run_id_fkey: public.recovery_drills.backup_run_id >
public.backup_runs.id [delete: restrict]

Ref rate_cards_vehicle_id_fkey: public.rate_cards.vehicle_id > public.vehicles.id [delete: restrict]

Ref rate_cards_created_by_fkey: public.rate_cards.created_by > public.profiles.id [delete: restrict]

Ref booking_rate_quotes_booking_id_fkey: public.booking_requests.id - public.booking_rate_quotes.booking_id [delete: restrict]

Ref booking_rate_quotes_rate_card_id_fkey: public.booking_rate_quotes.rate_card_id > public.rate_cards.id [delete: restrict]

Ref booking_rate_quotes_approved_by_fkey: public.booking_rate_quotes.approved_by > public.profiles.id [delete: restrict]

Ref vehicle_images_vehicle_id_fkey: public.vehicle_images.vehicle_id > public.vehicles.id [delete: cascade]

Ref public_contact_settings_updated_by_fkey: public.public_contact_settings.updated_by > public.profiles.id [delete: set null]

Ref contact_inquiries_read_by_fkey: public.contact_inquiries.read_by > public.profiles.id [delete: set null]

Ref booking_payment_quotes_booking_id_fkey: public.booking_requests.id - public.booking_payment_quotes.booking_id [delete: restrict]

Ref booking_payment_quotes_vehicle_id_fkey: public.booking_payment_quotes.vehicle_id > public.vehicles.id [delete: restrict]

Ref booking_payment_quotes_issued_by_fkey: public.booking_payment_quotes.issued_by > public.profiles.id [delete: restrict]